

研华“e时代自动化专家之路”系列丛书

数据采集系统 应用与编程

- ☐ PC_Based Control 技术
- ☐ Compact PCI Hot Swap 编程
- ☐ 数据采集 VC 编程详解
- ☐ 数据采集 ActiveX 控件编程
- ☐ 数据采集 LabVIEW 编程
- ☐ 工业通讯卡使用
- ☐ PC_Based 运动控制系统
- ☐ 数据采集系统应用案例
- ☐ 数据采集系统选型指南



Your ePlatform Partner

ADVANTECH

研华（中国）培训中心

 www.advantech.com

快速反应 精确发挥 稳定表现

研华全系列高速采集卡



模拟量输入

PCI-1714 30MS/s 4通道模拟量输入卡

- 30MS/s同步采样
- 32K FIFO/通道
- 自动校准



多功能卡

PCI-1712 1MS/s多功能卡

- 16通道模拟量输入和2通道模拟量输出
- 12位精度, 1MS/s采样速度
- 自动校准



数字量输入/输出

PCI-1755 超高速数字量输入/输出卡

- 最高速度30MS/s
- 模式匹配/启动停止触发
- 支持突发式和0.25%仿真模式
- 模式匹配/状态改变中断检测功能



模拟量输出

PCI-1721 10MHz 4通道模拟量输出卡

- 10MHz最大更新率
- 自动校准功能
- 同步波形输出



计数器/定时器卡

PCI-1780 20MHz 8通道计数器/定时器卡

- 最高20MHz输入频率
- 8通道16位计数器, 8D和0000
- 12个可编程的计数器模式

为关键任务信号测量提供完整解决方案



生产线测试



瞬态信号测量



科研/实验室

Your ePlatform Partner

ADVANTECH
研华科技

研华(中国)公司

北京研华 TEL: (010) 62984346
上海研华 TEL: (021) 63855555
深圳研华 TEL: (0755) 82124222
成都研华 TEL: (028) 85450198

www.advantech.com.cn

西安研华 TEL: (029) 88386187
沈阳研华 TEL: (024) 23263716
广州研华 TEL: (020) 38878420
福州研华 TEL: (0591) 7095751

研华服务专线: 800-810-0345

武汉研华 TEL: (027) 87339504
南京研华 TEL: (025) 83690030/26
杭州研华 TEL: (0571) 88865485
重庆研华 TEL: (023) 68618289
昆明研华 TEL: (0871) 3182769

建立可靠沟通 连接无限可能

研华工业通讯完整解决方案



全面满足工业通讯安全要求

- 数据加密 (Data Encryption)、IP访问控制
- 主机/设备冗余切换
- 信号隔离/浪涌保护/故障处理自动重连
- 设备自诊断/故障报警控制
- 宽电压 (10-30V)/冗余电源
- 坚固安装: DIN导轨安装、面板安装、堆叠安装
- 宽温度范围: -10-70℃



● 工业自动化



● 电力自动化



● 电信通讯



● 金融服务



● 楼宇监控



● 智能交通

Your ePlatform Partner

ADVANTECH

研華科技

研华(中国)公司

北京研华 TEL: (010) 62984346
上海研华 TEL: (021) 63855555
深圳研华 TEL: (0755) 82124222
成都研华 TEL: (028) 85450198

www.advantech.com.cn

西安研华 TEL: (029) 88386187
沈阳研华 TEL: (024) 23263716
广州研华 TEL: (020) 38878420
福州研华 TEL: (0591) 7095751

研华服务专线: 800-810-0345

武汉研华 TEL: (027) 87339504
南京研华 TEL: (025) 83690030/26
杭州研华 TEL: (0571) 88865485
重庆研华 TEL: (023) 68618289
昆明研华 TEL: (0871) 3182769

目 录

第一章 数据采集基本原理.....	7
一 PC_Based Control 技术.....	7
二 数据采集板卡选择参数浅释.....	8
三 Compact PCI 技术.....	9
四 数据采集板卡的安装.....	9
五 数据采集板卡的编程使用简介.....	12
第二章 研华数据采集卡应用.....	14
一、 安装与调试.....	14
二、 应用软件开发.....	24
动态链接库使用方法介绍.....	24
使用 Visual C++.....	25
使用 visual Basic 来创建应用程序.....	28
使用 Borland Dephi.....	31
使用 Borlang C++或 C++ Builder.....	33
三、 Compact PCI Hot Swap.....	36
Hot Swap 介绍.....	36
研华 Compact PCI Hot Swap 编程.....	39
第三章 数据采集卡 VC 编程详解.....	41
一、 软件触发数据采集 Adsoft.....	41
第一步：创建一个 Dialog Based 程序框架。.....	41
第二步：编辑对话框.....	43
第三步：导入外部头文件和函数库。.....	45
第四步：定义变量和函数。.....	45
第五步：编写程序代码。（注：函数的顺序无所谓）.....	48
第六步：运行程序.....	51
二、 应用中断进行数据采集 Adint.....	52
第一步：创建一个 Dialog Based 程序框架。.....	52
第二步：编辑对话框.....	54
第三步：导入外部头文件和函数库。.....	56
第四步：定义变量和函数。.....	56
第五步：编写数据程序代码。（注：函数的顺序无所谓）.....	60
用户线程及其调用函数说明：.....	63
第六步：编写停止采集数据和退出程序按钮。.....	68
第七步：运行程序.....	68
三、 应用 DMA 进行数据采集 addma.....	69
第一步：创建一个 Dialog Based 程序框架。.....	69
第二步：编辑对话框.....	72
第三步：导入外部头文件和函数库。.....	74
第四步：定义变量和函数。.....	74
第五步：编写数据程序代码。（注：函数的顺序无所谓）.....	77

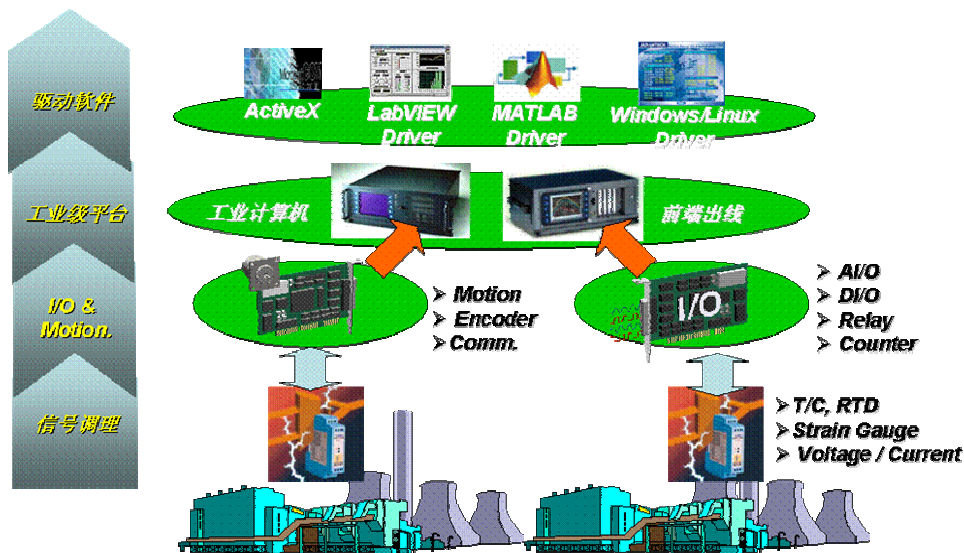
用户线程及其调用函数说明:	81
第六步: 编写停止采集数据和退出程序按钮。	86
第七步: 运行程序	86
第四章 数据采集控制卡 ACTIVEX 控件编程	88
一、 从最简单的开始	88
二、完善程序	91
三、打造自己的第一个综合工程应用程序	92
第五章 数据采集卡 LABVIEW 编程	94
一、研华 LabVIEW 驱动程序的安装	94
二、简单易用, 轻松上手	94
三 更上一层楼, 数据采集结果——显示且存储。	98
四 结论	99
第六章 研华数据采集卡例程使用	100
一、 系列产品介绍、硬件简介: AI, AO, DI, DO	100
二、 例程详解	101
ADSOFT/ADTRIG (软件触发方式例程)	101
ADDMA (DMA 方式进行数据采集的例程) 函数:	102
ADintf (中断方式进行数据采集的例程):	103
DAINT(中断方式模拟量输出)	105
DADMA(DMA 方式模拟量输出)	107
AIEXP (使用扩展板进行模拟量输入的例程):	108
MAIexp(扩展的多通道模拟量输入例程, 软件触发)	109
ALARM (报警例程):	110
DIGOUT (数字量输出):	111
COUNTER (计数程序)	112
QCOUNTER(计数程序)	113
Digin (数字量输入例程)	113
PULSE(脉冲输出例程)	114
PWM(宽度可调的脉冲输出)	115
Pwmin(输入脉冲宽度测量)	115
MAIntf(多通道中断采集例程)	116
CDADINT(看门狗中断数据传输例程)	118
BDADDMA (总线控制 DMA 传输方式)	121
WDADMA(总线控制 DMA 方式模拟波形输出)	123
THERMO(热电偶数据采集)	124
FDIGIN (中断事件数字量输入例程):	125
CSYNCAO (同步/异步电流输出)	126
DiginEx (使用扩展板的数字量输入例程)	127
CommPort (串口通信测试例程)	130
PortIO(端口输入输出例程)	131
第七章 驱动开发基础	132

10.1 虚拟设备初步	132
10.2 虚拟机管理器	132
9.3 虚拟设备驱动程序	133
9.4 VxD 程序之间的通讯	134
第八章 数据采集应用案例	136
一、 压铸机计算机测控系统	136
二、 内燃机车辅助电机综合试验台测控系统	145
三、 PCI-1714 及其在船舶状态监测中的具体应用	149
四、 基于 PCI-1712 的移动式电网谐波分析仪的研制	153
五、 用 VC++ 实现 PCL-812PG 数据采集卡的应用	157
附录一 PCL-813 快速安装使用入门	164
一、 产品介绍	164
二、 安装与测试	165
三、 信号输入连接	174
附录二 工业通讯卡使用入门	176
一、 如何在 Win 95/98/ME 下安装 PCL 系列的通讯卡	176
二、 如何在 Win NT 下安装 PCL 系列的通讯卡	177
三、 如何在 Win 2000 下安装 PCL 系列的通讯卡	179
四、 如何在 Win 95/98/ME 下安装 PCI 系列的通讯卡	179
五、 如何在 Win NT 下安装 PCI 系列的通讯卡	181
六、 如何设置通讯卡上串口的序号	183
七、 如何测试已安装的通讯卡	185
附录三 运动控制卡使用入门	187
一、 PCI-1240 安装	188
二、 PCI-1240 与驱动器接线	190
三、 PCI-1240 测试	196
四、 软件编程	199
附录四 数据采集选型指南	207

第一章 数据采集基本原理

一 PC_Based Control 技术

PC_Based Control 技术是融合 PC 技术、信号量测&分析技术、控制技术、通讯技术于一体的高性能测量与控制技术，用于信号量测、工业过程数据采集&控制、运动控制、通讯控制等。包括工业电脑平台、功能卡和应用软件。通过插入各种功能卡和编写软件，形成功能强大的数据采集系统、通讯控制器和运动控制系统。



近年来，PC_Based Control 技术向更快速、更精确的测控方向发展。PC_Based Control 核心技术包括

1. Compact PCI
 - a) HOT SWAP、紧固式安装、制冷
2. 高速采集技术
 - a) 同步采集、精确信号触发与同步信号、DMA 传输
3. 与传统测试仪器联网：GPIB、ICOM
4. PC_Based 运动控制系统
5. 易于使用的编程方式
 - a) 虚拟仪器编程，控件，组态式开发
6. 易于使用和调试
 - a) Board ID 、通用 PCI、ISA 兼容、高密度

本书主要介绍基本的数据采集编程方法，附录部分列出了工业通讯卡和运动控制卡的使用方法。

数据采集与控制系统的基本任务是物理信号（电压/电流）的产生或测量。但是要使计

计算机系统能够测量物理信号，必须要使用传感器把物理信号转换成电信号（电压或者电流信号）。有时不能把被测信号直接连接到数据采集卡，而必须使用信号调理辅助电路，先将信号进行一定的处理。总之，数据采集与控制系统是在硬件板卡/远程采集模块的基础上借助软件来控制整个系统的工作——包括采集原始数据、分析数据、给出结果等。

二 数据采集板卡选择参数浅释

模拟量：

在使用数据采集卡的时候，操作的对象主要就是模拟量输入/输出，数字量输入/输出。对于模拟量，工程中通常会对如下的一些参数比较关注：输入模式（单端输入或者差分输入）、分辨率、输入范围、采样速率，精度和噪声等。

单端输入以一个共同接地点为参考点。这种方式适用于输入信号为高电平（大于一伏），信号源与采集端之间的距离较短（小于 15 英尺），并且所有输入信号有一个公共接地端。如果不能满足上述条件，则需要使用差分输入。差分输入方式下，每个输入可以有不同的接地参考点。并且，由于消除了共模噪声的误差，所以差分输入的精度较高。

输入范围是指数据采集卡能够量化处理的最大、最小输入电压值。数据采集卡提供了可选择的输入范围，它与分辨率、增益等配合，以获得最佳的测量精度。

分辨率是模/数转换所使用的数字位数。分辨率越高，输入信号的细分程度就越高，能够识别的信号变化量就越小。增益表示输入信号被处理前放大或缩小的倍数。给信号设置一个增益值，就可以实际减小信号的输入范围，使模数转换能尽量地细分输入信号。

总之，输入范围，分辨率以及增益决定了输入信号可识别的最小模拟变化量。此最小模拟变化量对应于数字量的最小位上的变化，通常叫做转换宽度（Code width）。其算式为：输入范围/（增益* $2^{\text{分辨率}}$ ）。

例如：16 位分辨率的板卡，那么该板卡能够分辨的最小单位即为：分辨率=最小分辨率=板卡选择的量程/ 2^{16} 。实际上这个分辨率是板卡上的 A/D 转换芯片的转换精度，并不代表板卡本身实际采集数据是能够达到的精度，通常板卡的采集精度会有另外的说明，例如： $0.03\% \times \text{FSR} \pm 1\text{LSB}$ ——满量程的百分比再加减一个最小分辨率。板卡的精确度的标注实际上更值得工程师注意。

采样率决定了模/数变换的速率。采样率高，则在一定时间内采样点就多，对信号的数字表达就越精确。采样率必须保证一定的数值，如果太低，则精确度就很差。

根据奈奎斯特采样理论，采样频率必须是信号最高频率的两倍以上，采集到的数据才可以有效地复现出原始的采集信号。例如，信号的最高频率为 1M，那么为了采集到的数据能够准确地反映原始信号的频率特性，要求数据采集频率（Sample Rate）至少应该为 2M。

平均化。噪声将会引起输入信号畸变。噪声可以是计算机外部的或者内部的。要抑制外部噪声误差，可以使用适当的信号调理电路，也可以增加采样信号点数，再取这些信号的平均值以抑制噪声误差。

数字量

对于数字量我们通常比较关注的是：数字量的类型：TTL/隔离量，输入/输出速度，并行操作的位数，驱动能力等

数字量的类型决定了数据采集板卡可以接收/侦测的电压范围，例如：TTL 电平 0-0.8V 位逻辑 0，2.4-5V 为逻辑 1，隔离电压逻辑 0/1 依板卡指标来确定。

输入输出速度实际上取决于板卡的总体设计，例如是否支持 Bus Master 工作方式等。研华公司最近推出的 PCI-1755 最高输出速度可达 80Mbyte/S。

并行操作的位数，即同时可以输出/输入的通道的个数，与板卡设计和系统的总线宽度有关。研华公司 PCI-1755 支持 8/16/32 位并行操作。

驱动能力，因为数据采集卡是从计算机总线上取电压电流输出，故驱动能力有限，带大功率负载的时候用户需要自己外加电源驱动设备。

三 Compact PCI 技术

Compact PCI 是将 PCI 电器规范与耐用的欧洲卡机械封装和高性能连接器相结合的产物，不仅保持了与工业 PC 软件标准的兼容性，还提供了热插拔（Hot Swap）、抗震动和强冷却的特性。

在 PICMG 组织的推广下，CompactPCI 产品所拥有的模块化、易使用以及易维护的优点，已广泛地应用于军事武器系统、航天工业、电信网络、CTI 等多种领域。

Compact PCI 的数据采集编程方法与 PCI 相同，所不同的是提供了专门的 Hot Swap 驱动支持。

四 数据采集板卡的安装

ISA 卡是早期推出的工业标准总线，其稳定性备受业界的欢迎因此直到现在仍然在工业控制，数据采集领域占有重要的位置。而 PCI 技术则因为其良好的兼容性，简便的安装和出色的性能而方兴未艾。这里将要讨论的这两类板卡的安装和使用方面的知识，所以对于两者的渊源和特点就不再详述。

PCI 系列数据采集卡，刚才讲过这类卡的安装比较简单，对于研华公司的 PCI 系列数据采集卡来讲，只要先在系统中装好驱动程序，然后关机-插卡-开机，系统就可以自动找到新硬件并自动安装相应的驱动程序。其他公司的 PCI 设备可能会提供相应的.inf 文件，用户可以在安装板卡的时候指定相应的 inf 文件给安装程序。系统自动的为 PCI 设备分配中断和基地址，用户无需关心。

ISA 系列数据采集卡的安装相对于 PCI 数据采集卡略显麻烦了一点，ISA 卡需要手动配置板卡的中断和基地址。这就要求用户首先要了解自己的系统中上述的两类资源中有哪一些是空闲可用的。

例如 Win2000 系统下，鼠标右键单击“我的电脑”选择“属性”菜单项，进入“硬件”属性页，单击“设备管理”按钮，然后选择“查看——>依类型排序资源”菜单，就可以查看哪些中断/基地址是被其他设备占用的了。如下图所示：



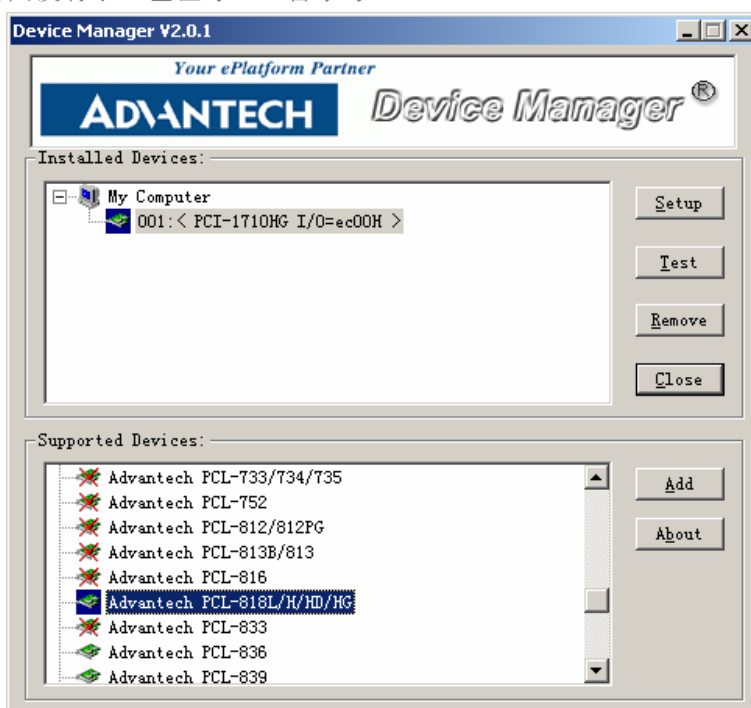
在找到了系统中可用的资源之后,用户就可以根据这些资源的情况通过板卡上的跳线设中断,通过拨码开关设置板卡的基地址。

以研华公司的 PCL-818HD 为例简单讲解:

首先要正确的安装设备驱动程序,然后关机->插卡->开机,使用研华公司提供的 Device Manager 设备管理工具来添加板卡并进行简单的测试。

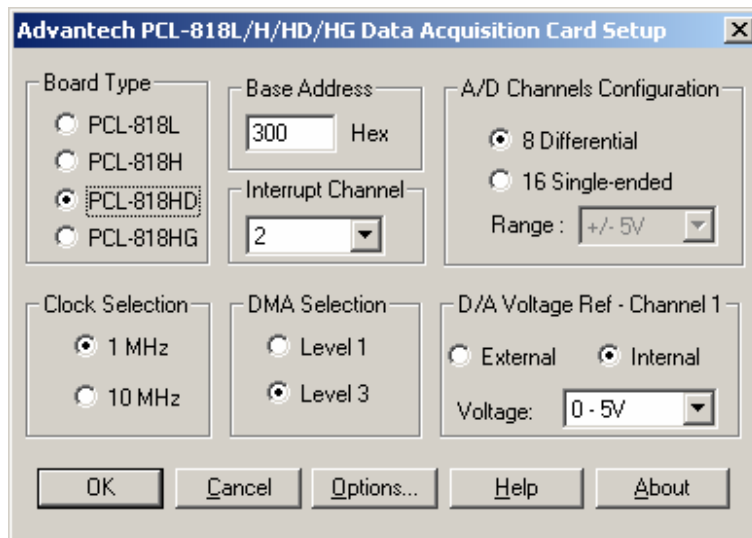
最需要注意的:板卡的基地址,中断的跳线设置,要认真的按照手册上面的说明来做!很多用户常常因为没有看手册,而在这里反复劳动!

1) 打开 Device Manager 工具,在正确的安装了 PCL-818HD 的驱动程序之后,可以看到在 Supported Device 下面的列表框中,Advantech PCL-818L/H/HD/HG 已经处于可用状态(前面没有了红色差号——警示号)



2) 选中 Advantech PCL-818L/H/HD/HG 后,右边的 Add 按钮呈可用状态,单击 Add 按

钮弹出如下的设置对话框：



Board Type: 选择安装的板卡类型，这里计算机上安装的是 PCL-818HD；

Base Address: 基地址，该值必须为系统空闲地址且与板卡上面的拨码开关所设置相同；

Interrupt Channel: 中断设置，该值必须为系统空闲中断号且与板卡上面的跳线设置相同；

A/D Channels Configuration: 通道配置，选择输入的类型，8 路差分/16 路单端，这里也必须和板卡上面的跳线或开关的设置一致；

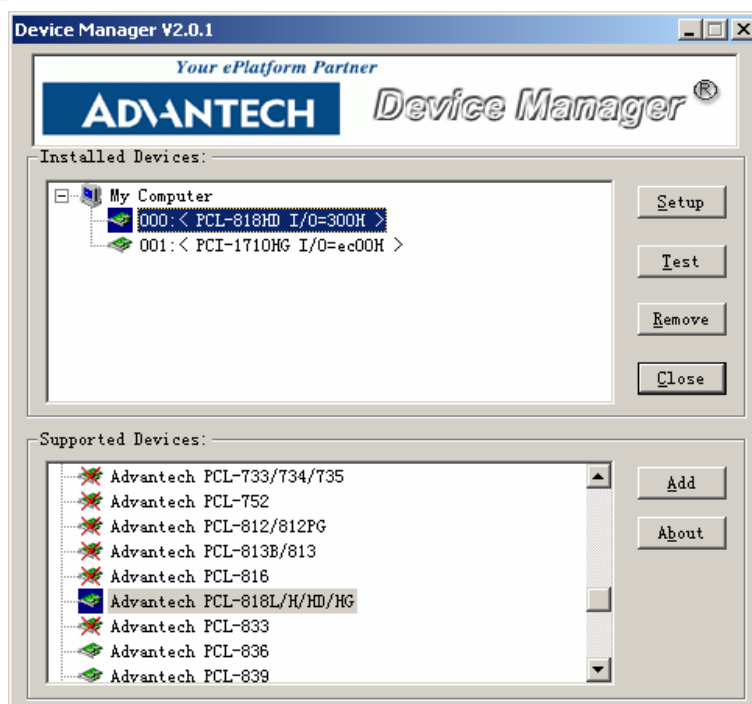
Clock Selection: 时钟选择；

DMA Selection: DMA 通道选择；

D/A Voltage Ref.-Channel1: 通道参考电压选择；

Options 按钮：可以选择连接扩展板，扩展输入通道。

设置确定，单击 OK 按钮，如果设置的基地址/中断号/DMA 与系统资源没有冲突则可以看到 PCL-818HD 已经添加到 My Computer 下面，它在系统中的设备号：000；输入输出地址：I/O=300H，



这表示板卡已经安装成功，这时候可以单击 Test 按钮测试板卡。

五 数据采集板卡的编程使用简介

对数据采集板卡的进行编程使用的方式主要有以下三种：软件触发方式，中断方式，DMA 方式。

软件方式

实际上就是采用系统提供的时钟在毫秒级的精确等级上，通过对寄存器的查询来实现数据采集，由于其采集速度比较慢，因此多用于低速数据采集场合。

中断传输方式

使用中断传输方式，你需要编写中断服务程序（ISR），将板卡上的数据传输到预先定义好的内存变量中，每次 A/D 转换结束后，EOC 信号都会产生一个硬件中断，然后由中断服务程序(ISR)完成数据传输。在使用中断传输方式时，必须制定中断级别。

DMA 数据传输方式

尽管应用比较复杂，但由于不需要 CPU 的参与，DMA(Direct Memory Access)方式特别适合应用于大量数据的高速采集。同中断方式一样，在使用 DMA 方式传输方式时必须指定 DMA 级别，需要对板卡上的 DMA 控制寄存器操作，并且对 Intel 8237DMA 控制其进行操作，因此，强烈建议您使用驱动来实现这种方式。

DMA 方式将板卡上的数据不通过 CPU 数据就传输到内存中，一般板卡上会提供单 DMA(Single Channel)或者双 DMA(Dual Channel)方式，双 DMA 方式允许你在传输数据的同时采集。双 DMA 方式使用两个缓冲区和两个 DMA 通道，板卡首先通过 DMA 通道 6 拷贝到两个缓冲区。你的应用程序，可以从第一个缓冲区传输数据，当第二个缓冲区变满时，硬件回切换到第一个缓冲区。应用程序又可以从第二个缓冲区传输数据。然后不断循环下去。

关于 FIFO

设计 FIFO 的目的

为了防止在高速数据采集时丢失数据，特别是在像 Windows 这样的多任务操作系统下。通常板卡完成 A/D 转换后，将数据写入到数据输出寄存器中，接着使用 DMA 或中断服务功能将数据传输到 CPU/内存。

如果没有 FIFO 功能，每次硬件完成 A/D 转换后，会改写保存在数据寄存器中的值，如果上次 A/D 的数据在新数据到来之前未被传输到 CPU，那这个数据就丢失了。

如果使用 FIFO 功能，新数据仅仅被添加到 FIFO 缓冲区的第二个位置上，而不会覆盖原先的数据。随后的数据会依次排列到缓冲区中。当你想从 FIFO 缓冲区中搬移数据时仅仅需要从数据寄存器种读取一个数据即可，这样将会将最初的数据取出，FIFO 中的下以个位置的数据会取代数据寄存器中的值，你可以在任何时候传输来自 FIFO 缓冲区中的数据，当你在传输旧的数据时，硬件会将最新的数据保存在 FIFO 中，从而防止数据丢失。你也可以在 FIFO 半满或全满时，一次性的传输数据。由于这样减少了 CPU 的时间，因此非常适合于大量的高速数据传输。

怎样合理的使用 FIFO

软件查询方式下如何使用 FIFO

可以通过查询状态寄存器的状态未来判断 FIFO 缓冲区的状态，分别判断空标志位（Empty flag）;半满标志位（Half full flag）以及全满标志位（full flag）的状态。

中断状态下如何使用 FIFO

为何说 FIFO 可以防止数据丢失呢？通常情况下（没有 FIFO），硬件完成一次 A/D 转换后就会向系统产生一次中断请求，此时中断服务程序会去传输数据，现在问题产生了：如果

此时有一个优先级高的系统中断请求，你的中断服务程序会等待系统中断服务结束才会继续响应。如果等待的时间过长，硬件会进行下一次 A/D 转换（如果采用硬件触发方式，A/D 转的速度会很快），并且在你的中断服务程序传输旧的数据之前改写数据寄存器中的值。新的 A/D 转换会有启动你的中断服务程序，而传输的数据是最新的采样值。旧的数据已经被新的数据覆盖（丢失了）。

FIFO 功能解决了上述问题。FIFO 缓冲区会存储连续采样的数据，如果中断服务程序还未传输最初的采样值，FIFO 会将下一次采样值放入到 FIFO 中的下一个位置。

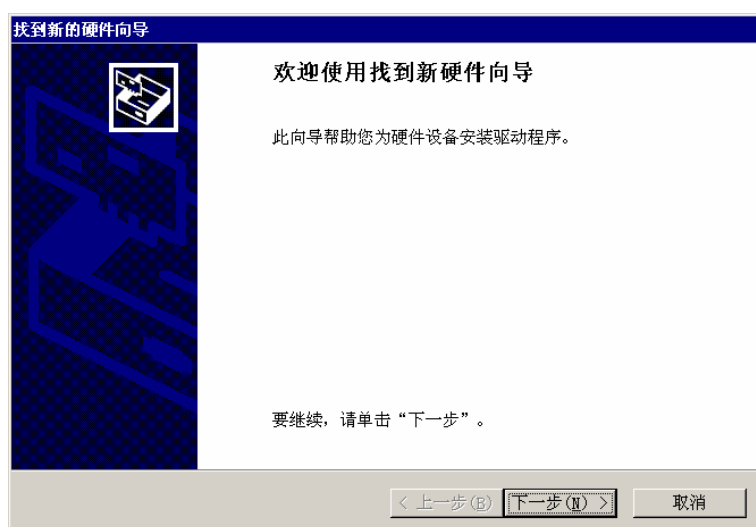
中断方式下使用 FIFO 的方法与软件查寻方法类似。

第二章 研华数据采集卡应用

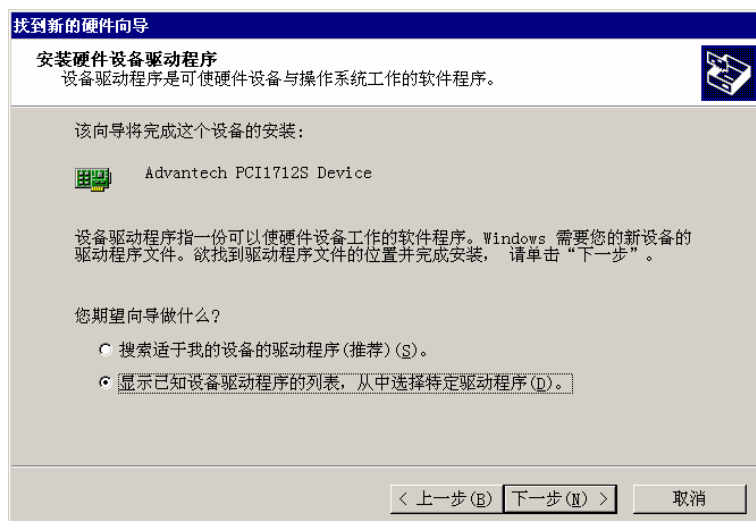
一、 安装与调试

1) PCI 系列（以 PCI-1712 为例，其他型号的 PCI 板卡安装过程基本类似，只是型号显示有所不同而以）：

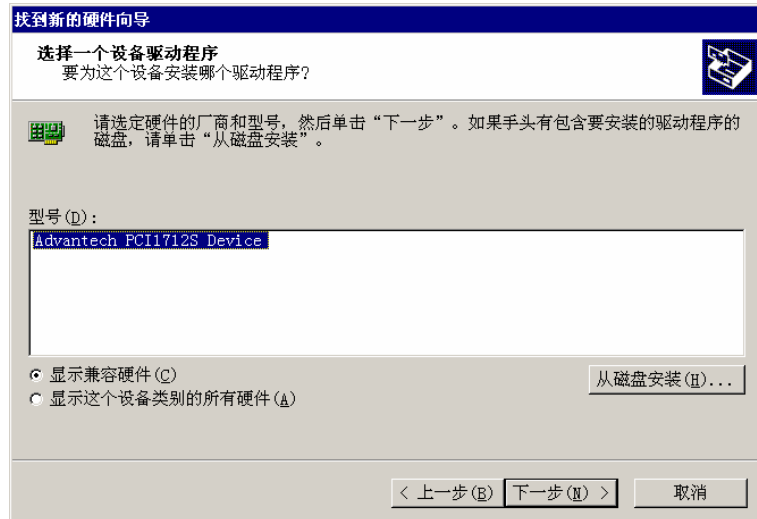
PCI 系列板卡需要先安装驱动程序，然后关机插卡重起，会出现找到新硬件向导：



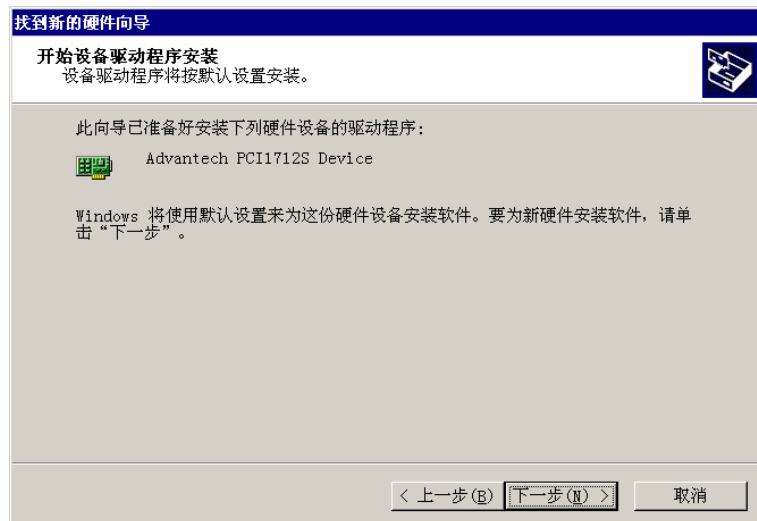
选择下一步，显示安装设备驱动程序对话框，选择“显示已知设备…… (D)”



选择下一步，出现“选择设备驱动程序”对话框



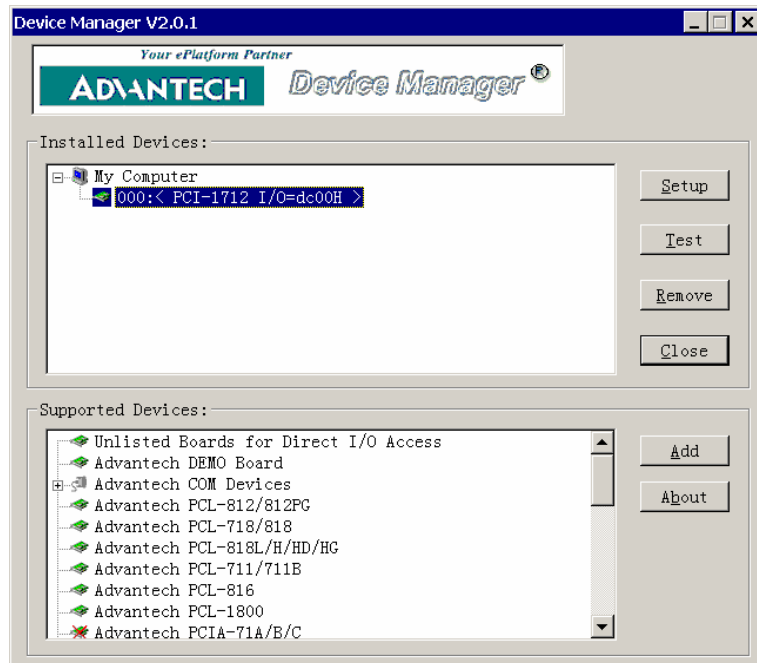
选择下一步，出现“开始设备驱动程序安装”对话框



选择下一步，“完成找到硬件向导”，“完成”设备安装



这时在 Device Manger 中就可以看到 PCI-1712:



同时在设备管理器中也可以看到安装得 PCI-1712 卡：



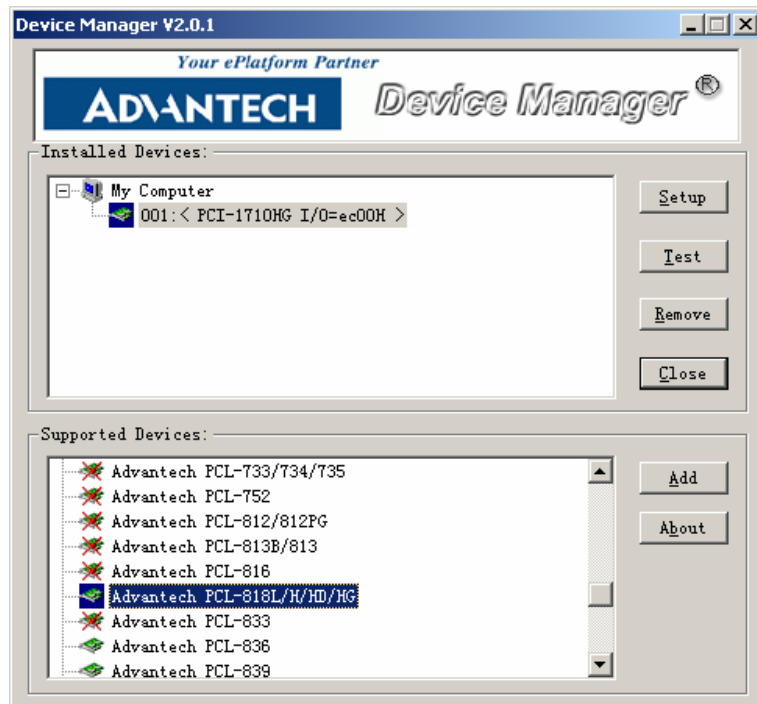
测试的方法在下面的 PCL-818HD 安装例程中详细介绍。

2) PCL 系列（以 PCL-818HD 为例，其他型号的 PCL 板卡安装过程基本类似，只是型号显示有所不同而以）：

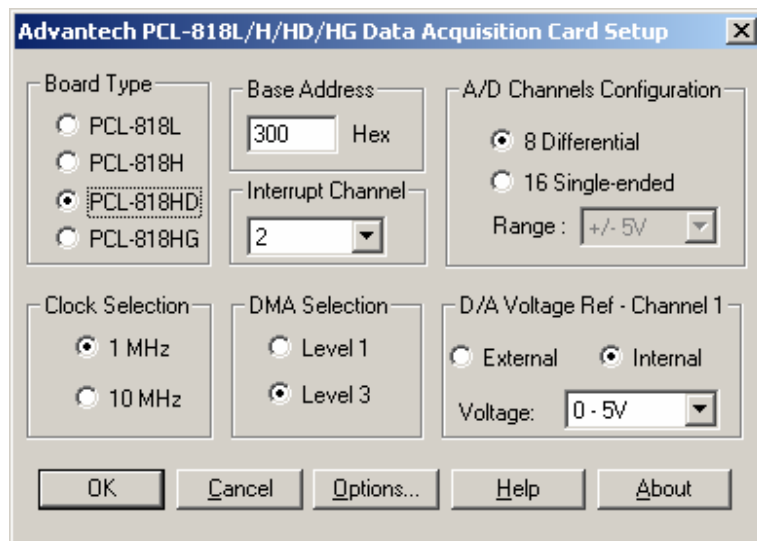
正确安装了设备驱动程序之后，关机->插卡->开机，使用 Device Manager 工具来添加板卡并进行简单的测试，这里以 PCL-818HD 为例。

注：PCL-832/PCL-839/PCL-841 是功能比较特殊的板卡，研华公司专门编写了针对这些板卡的测试程序，在安装了这些板卡的驱动之后会在相应的目录安装与之对应的测试程序，例如：PCL832/839DEMO/PCL841。

1) 打开 Device Manager 工具可以看到正确的安装了 PCL-818HD 的驱动程序之后，可以看到在 Supported Device 下面的列表框中，Advantech PCL-818L/H/HD/HG 已经处于可用状态（前面没有了红色差号——警示号）



2) 选中 Advantech PCL-818L/H/HD/HG 后，右边的 Add 按钮呈可用状态，单击 Add 按钮弹出如下的设置对话框：



Board Type: 选择安装的板卡类型，这里计算机上安装的是 PCL-818HD；

Base Address: 基地址，该值必须为系统空闲地址且与板卡上面的拨码开关所设置相同；

Interrupt Channel: 中断设置，该值必须为系统空闲中断号且与板卡上面的设置相同；

A/D Channels Configuration: 通道配置，选择输入的类型，8 路差分/16 路单端，这里也必须和板卡上面的跳线或开关的设置一致；

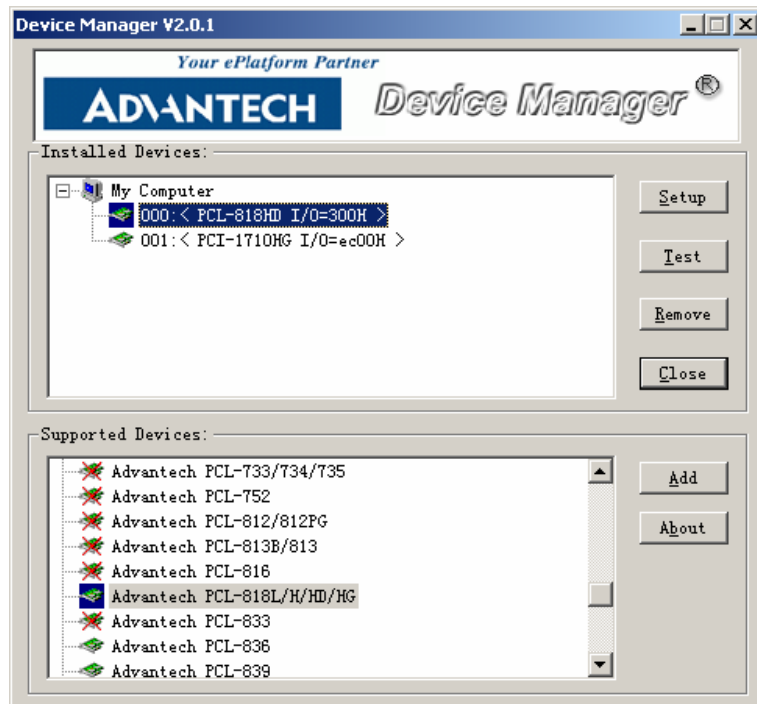
Clock Selection: 时钟选择；

DMA Selection: DMA 通道选择；

D/A Voltage Ref.-Channel1: 通道参考电压选择；

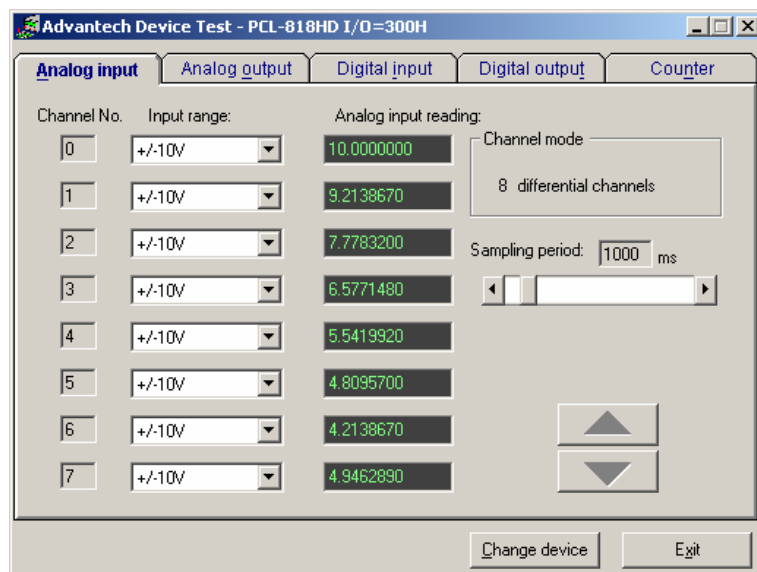
Options 按钮: 可以选择连接扩展板，扩展输入通道。

设置确定，单击 OK 按钮，如果设置的基地址/中断号/DMA 与系统资源没有冲突则可以看到 PCL-818HD 已经添加到 My Computer 下面，它在系统中的设备号：000；输入输出地址：I/O=300H，



这表示板卡已经安装成功，这时候可以单击 Test 按钮测试板卡，对于板卡不支持的功能测试页会显示 Function not support 信息。

注：PCL-832/PCL-839/PCL-841 是功能特殊的板卡，研华为这些板卡专门编写了测试程序，安装之后在其相应的文件夹中可以找到，例如：PCL832/839DEMO/PCL841。

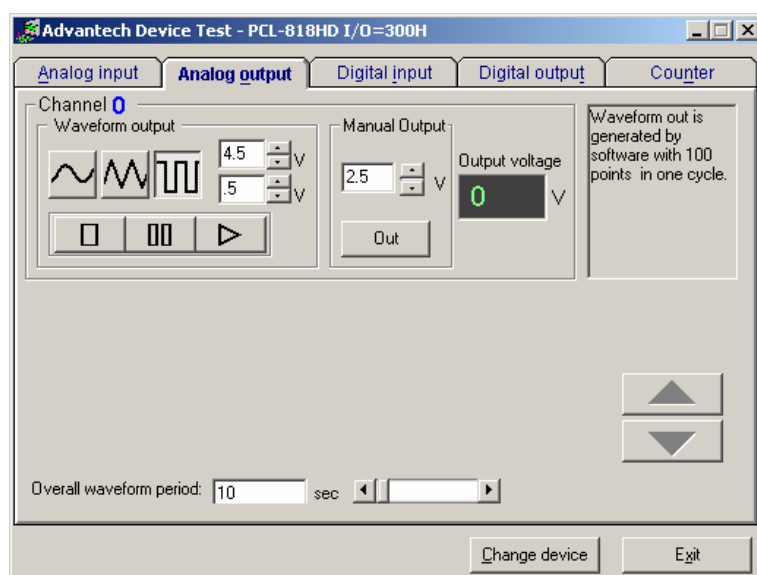


这是对模拟量输出测试的页面：

依次可以看到 Channel No: 通道号；Input range: 输入范围选择； Analog input reading: 模拟量输入通道读取的数值； Channel mode: 通道设定模式； Sample period: 采样时间间隔；

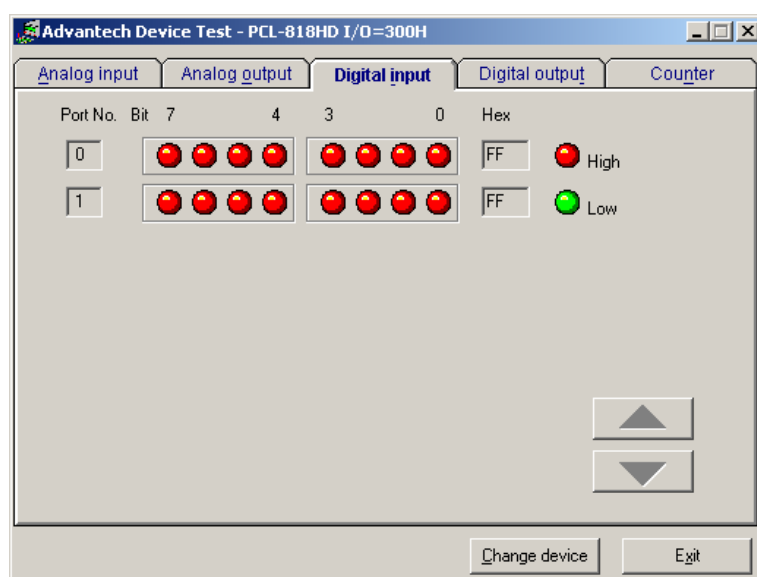
注：在不接任何信号的时候，程序也会读到一个随机的述值，当接入实际电压的时候就会恢复正常；在使用板卡的时候建议将不用的通道短接。

选择 Analog Output 属性页，如下所示：



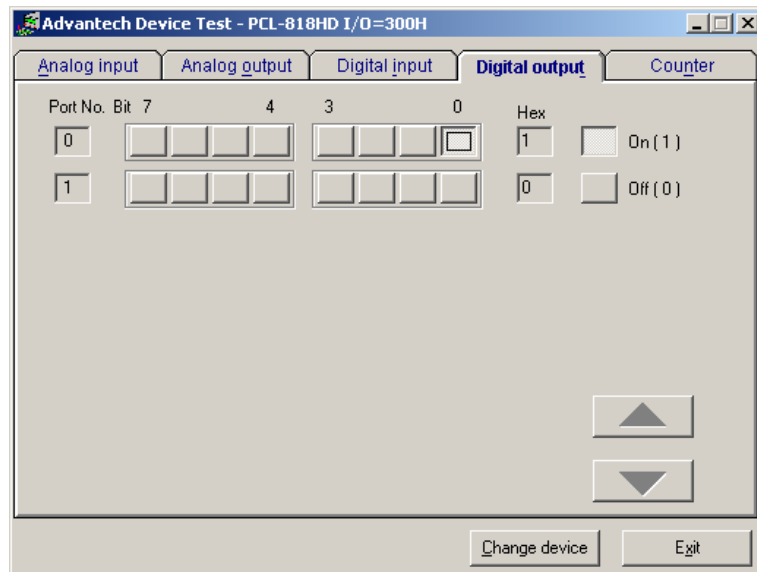
可以看到 PCL-818HD 只有一个模拟量输出通道，可以选择波形输出进行测试，三种可选的波形，可以调节输出波形的峰值/谷值，每个周期输出 100 个点，输出时间可有下面的 Overall waveform period 选择；也可以选择输出常量进行测试。

数字量输入页面如下所示：



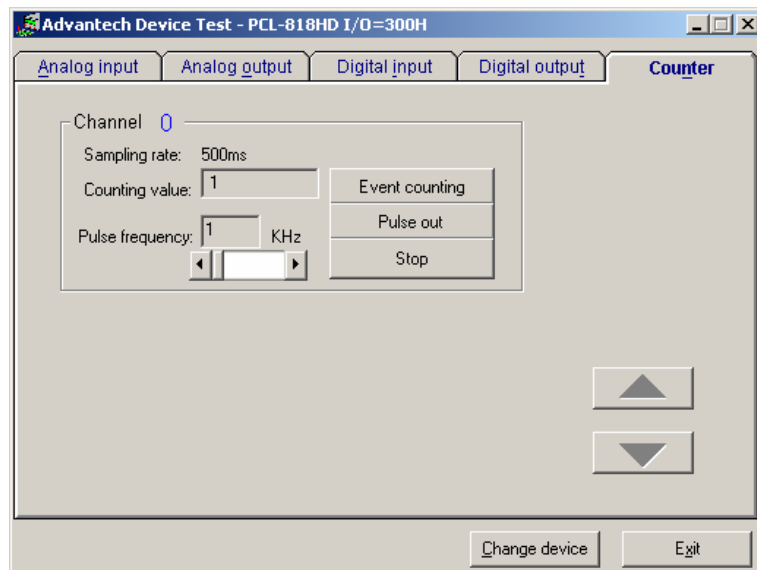
没有连接任何信号的时候，多数板卡把这种浮空的状态作为高有效信号，将数字量输入管脚同地短接时，可以看到对应的数据位变绿（低电平信号）。

数字量输出页面如下所示：



按下一个数据为对应的按钮则在相应的数据位输出一个有效的高电平信号，通常高电平为 5V，低电平为：0 伏。

计数器页面如下：



可以选择 Event counting：事件计数 Pulse out ：脉冲输出； Stop：停止；

Sample rate：采样时间间隔； Counting value：计数值显示；

Pulse frequency：输出脉冲频率；

经过上面的测试我们就可以对板卡的功能有大体的了解，仔细的阅读过板卡的手册和与 Device Manager 一起安装的 Device Driver User Manual 中的函数说明之后，您就可以参照我们的例子程序开始您自己的程序设计了！

您可以参考我们提供的例程开始编程，每一块板卡支持的例程及其含义如下面的两个表所示：

板卡支持的例程列表	
PCI-1710	Adintf, Adsoft, Adexp, Counter, Dasoft, Digin, Digout, Freq, Madintf, Madsoft, Pulse
PCI-1710L	Adintf, Adsoft, Adexp, Counter, Dasoft, Digin, Digout, Freq, Madintf, Madsoft, Pulse
PCI-1710GHL	Adintf, Adsoft, Adexp, Counter, Dasoft, Digin, Digout, Freq, Madintf, Madsoft, Pulse
PCI-1710HG	Adintf, Adsoft, Adexp, Counter, Dasoft, Digin, Digout, Freq, Madintf, Madsoft, Pulse , Thermo, , ,
PCI-1711	Adintf, Adsoft, Adexp, Counter, Dasoft, Digin, Digout, Freq, Madintf, Madsoft, Pulse
PCI-1711L	Adintf, Adsoft, Adexp, Counter, Dasoft, Digin, Digout, Freq, Madintf, Madsoft, Pulse
PCI-1712	Adbmdma, Adsoft, Counter, Dabmdma , Dasoft , Digin, Digout, Freq, Madsoft, Pulse, Pwmin
PCI-1712L	Adbmdma, Adsoft, Counter, Dabmdma , Dasoft , Digin, Digout, Freq, Madsoft, Pulse, Pwmin
PCI-1713	Adintf, Adsoft, Adexp, Madintf, Madsoft
PCI-1716	Adint, Adintf, Adsoft, Dasoft, Counter, Digin, Digout, Freq
PCI-1716L	Adint, Adintf, Adsoft, Dasoft, Counter, Digin, Digout, Freq
PCI-1720	Dasoft, Dasyncc, Dasyncv,
PCI-1721	Dabmdma, Dasoft, Digin, Digout, Counter, Wadbmdma
PCI-1723	Dasoft, Digin, Digout
PCI-1730	Digin, Digout, Diint
PCI-1731	Adintf, Adsoft, Adexp, Counter, Pulse, Digin, Digout, Freq, Madintf, Madsoft,
PCI-1733	Digin, Diint,
PCI-1734	Digout,
PCI-1750	Digin, Digout, TmCntSet,
PCI-1751	Digin, Digout, TmCntSet,
PCI-1752	Digout,
PCI-1753	Digin, Digout, 1753evt,
PCI-1754	Digin, Latchint,
PCI-1756	Digin, Digout, Latchint,
PCI-1760	Digin, Digout, Diint, Dipattn,
PCI-1761	Digin, Digout, Diint,
PCI-1762	Digin, Digout, Diint,
PCI-1784	Digin, Digout, Counter, Pulse, Timer ,

PCL-711	Adintf, Adint, Adexp, Adsoft, Dasoft, Digin, Digout, Madint, Madintf, Madsoft,
PCL-720	Digin, Digout, Counter, Pulse, Freq
PCL-722	Digin, Digout, Diint,
PCL-724	Digin, Digout, Diint,
PCL-725	Digin, Digout,
PCL-726	Digin, Digout, Dasoft, ,
PCL-727	Digin, Digout, Dasoft, ,
PCL-728	Dasoft,
PCL-731	Digin, Digout, Diint,
PCL-733	Digin, Diint,
PCL-734	Digout,
PCL-735	Digout,
PCL-812PG	Addma, Adintf, Adint, Adexp, Adsoft, Counter, Dasoft, Digin, Digout, Freq, Madint, Madintf, Pulse, Maddma,
PCL-813B	Adsoft,
PCL-816	Addma, Adint, Adexp, Adsoft, Counter, Digin, Digout, Freq , Madint, Madsoft, Pulse
PCL-816-DA	Dadma, Dasoft ,
PCL-818L	Addma, Adint, Adexp, Adsoft, Counter, Dasoft, Digin, Digout, Freq, Madint, Madsoft , Pulse, Maddma,
PCL-818H	Addma, Adint, Adexp, Adsoft, Counter, Dasoft, Digin, Digout, Freq, Madint, Madsoft , Pulse, Maddma,
PCL-818HD	Addma, Adintf, Adint, Adexp, Adsoft, Counter, Dasoft, Digin, Digout, Freq, Madint , Madintf, Madsoft, Pulse, Maddma, , , ,
PCL-818HG	Addma, Adintf, Adint, Adexp, Adsoft, Counter, Dasoft, Digin, Digout, Freq, Madint, Madintf, Madsoft, Pulse, Maddma, Thermo,
PCL-833	Digin, QCounter
PCL-836	Digin, Digout, Pulse, Counter, Freq, Pulsepwm,
PCL-1800	Addma, Adintf, Adint, Adexp, Adsoft, Counter, Daddma, Dadma, Daint, Dasoft, Digin, Digout, Freq, Maddma, Madint, Madintf, Madsoft, Pulse, Caddma, Cadint,
PCM-3725	Digin, Digout, Diint,

Examples' Name	Description
1753evt	PCI-1753 中断事件例程
839demo	839 使用例程
Adbmdma	总线控制 DMA 方式数据采集例程
Addma	DMA 方式单通道数据采集
Adexp	带扩展板的单通道数据采集例程
Adint	中断方式单通道数据采集例程（固定 1K FIFO）
Adintf	中断方式单通道数据采集例程（FIFO 大小可调）
Adsoft	软件触发模式数据采集例程
ALARM	报警例程
Caddma	使用看门狗的 DMA 模式多通道数据采集例程
Cadint	使用看门狗的中断模式多通道数据采集例程
COMM	串口测试程序
COUNTER	计数例程
DabmDma	总线控制 DMA 模式模拟量输出例程
Daddma	双 BUFFER 多通道,DMA 模式数据采集例程
Dadma	模拟量（电压）DMA 模式输出例程
Daint	模拟量（电压）中断模式输出例程
Dasoft	模拟量（电压）软件触发模式输出例程
Dasyncc	同步/异步电流输出例程
Dasyncv	同步/异步电压输出例程
Digin	数字量输入例程
Digout	数字量输出例程
Diint	中断方式数字量输入例程
Dipattn	带模式匹配，状态改变，计数匹配，计数溢出功能的数字量输入例程
Freq	频率测量例程
Latchint	中断方式数字量输入例程
Maddma	多通道 DMA 模式数据采集例程
Madint	多通道中断模式数据采集例程（固定 1K FIFO）
Madintf	多通道中断模式数据采集例程（可调 FIFO）
Pulse	脉冲输出例程
Pulsepwm	宽度可调脉冲输出例程
PwmIn	宽度可调脉冲输入例程
Qcounter	减计数例程
Thermo	热电偶使用例程
Tmcntset	Demo runtime change timer divisor. 时钟/中断数字量输入
WDabmdma	DMA 模式波形输出例程
DaCOut	电流输出例程

二、 应用软件开发

动态链接库使用方法介绍

这一部分包括了 创建 DA&C 应用的基本信息，介绍了设备驱动程序文件的特点，讲解了基本的使用方法。

- Visual C++ 5.0 或更高版本
- Visual Basic5.0 或更高版本
- Delphi4.0 或更高版本
- Borland C++ 5.0 或 C++ builder 1.0 或更高版本

如果您正在使用的开发工具不是上面所述的几种，请仔细查阅您所使用的工具的文档，了解该工具如何调用动态链接库来创建应用程序。

Windows 95/98/NT/2000 的 32 位 DLL 的函数库是动态连接，这就是说动态连接库并不直接编译到应用程序的可执行文件中，而只是将动态连接库的路径信息保存在可执行文件中。设备驱动程序只是在可执行程序执行过程中动态连接库被调用的时候才被连接到应用程序。

导入库 (*.LIB) 包含了它们的动态连接库定义的外部函数。它们隐含了动态连接库的位置。如何为您的应用程序引入动态连接库的路径信息，例如：是通过导入函数库还是通过函数声明的方法，取决于您所使用的编程工具。

使用函数原型是良好的编程习惯，这也是为什么设备驱动程序总是被打包成函数原型形式。安装工具为您选择的的开发工具拷贝正确的的原型文件。如果你不打算使用研华驱动程序支持的开发工具，那么您需要创建您自己的函数原型。

按照下面的步骤开始对研华的硬件设备编程应用：

这一部分我们推荐您采用研华的设备驱动程序结合微软的可视化平台来开发您的系统！研华设备驱动程序支持 Visual C++ 5.0 及其以上的版本。

要使用 DA&C 函数，需要按照下面的步骤使用研华驱动程序提供的 DLL (动态连接库，Dynamic Linked Library)

1. 象编写一般的 Windows 应用程序一样创建您的应用程序代码，在这些代码中可以象典型的函数调用一样调用研华驱动程序中提供的动态连接库中（DLL）的函数。

2. 声明在您的程序中用到的函数，即：包含动态连接库（DLL）的头文件，该函数声明了所有的动态连接库中的函数，如下所示：

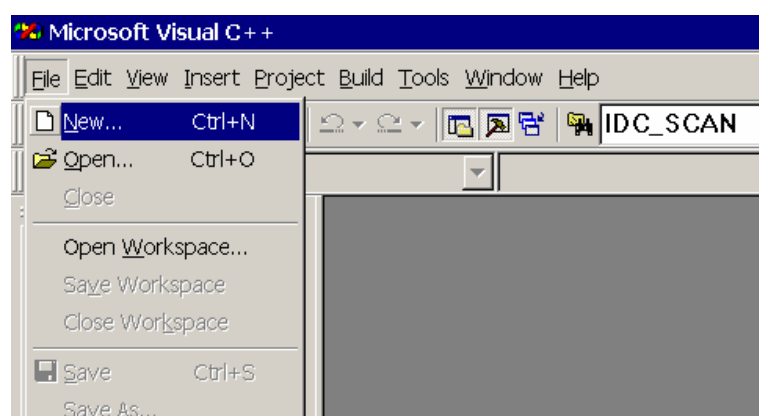
```
#include "driver.h"
```

3. 将动态连接库的（DLL）的导入库（例如，“ADSAPI32.LIB”）加到工程模块中。

使用 Visual C++

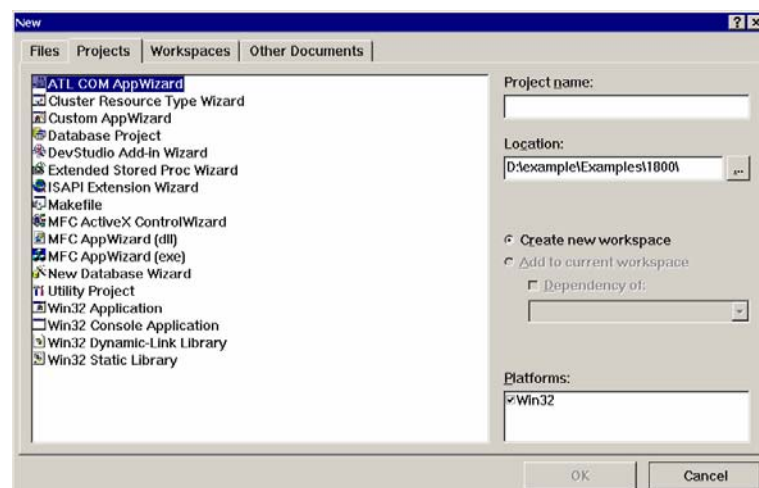
依照下面的步骤即可编写一个简单的 Visual C++ Windows 程序：

1. 点击 File|New 菜单，创建一个新工程



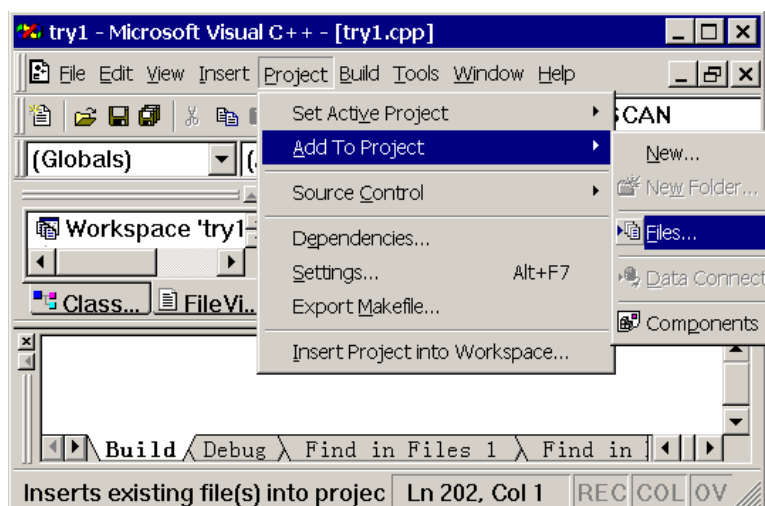
图（1）创建 Visual C++应用程序

2. 选择新工程的类型为“Win32 Application”，确定应用平台为“Win32”，并为其指定一个文件目录。



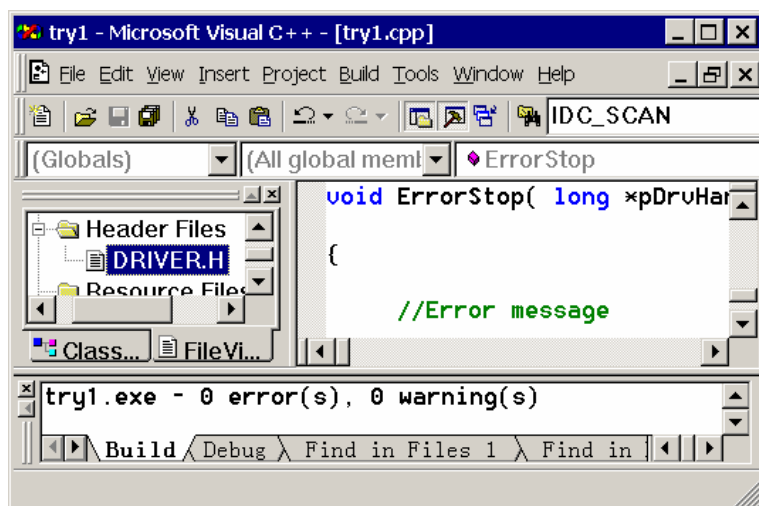
图（2）：确定应用程序类型和指定一个工程目录

3. 要使用研华提供的设备驱动程序及其库函数来开发应用程序，必须首先在 Visual C++ 添加头文件，单击图中所示的 Files 菜单项，找到 driver.h 文件并加入到工程中



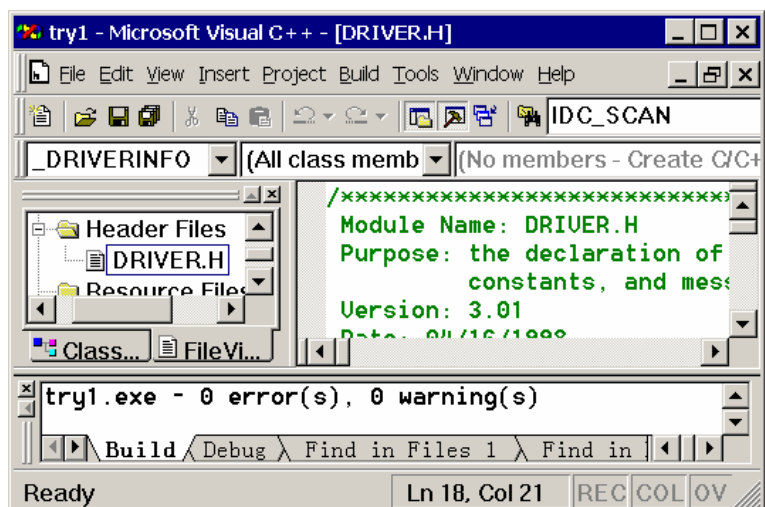
图（3）在您的工程中加入驱动程序的头函数

4. 加入 driver.h 文件之后，就可以在 VC 的工作区中看到它。



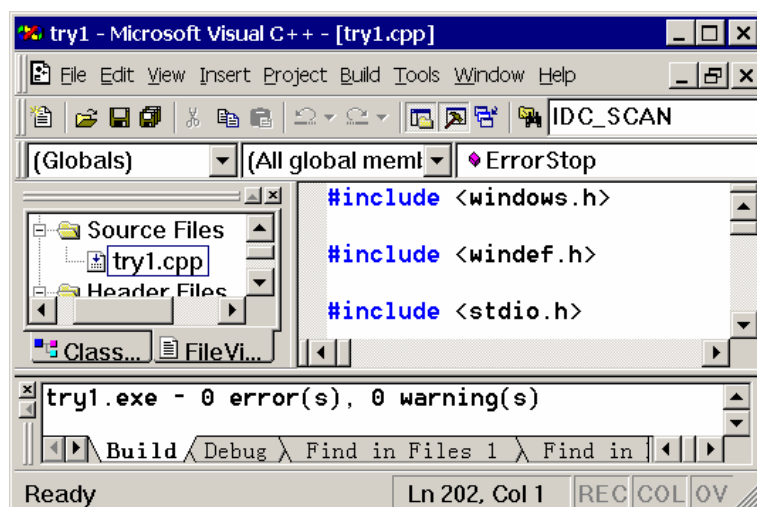
图（4）将头文件引入工程目录后

5. 在加入头文件之后，这个文件定义了研华在动态连接库中要用到的函数和变量，这些变量和函数都可以在这个新创建的工程中使用。



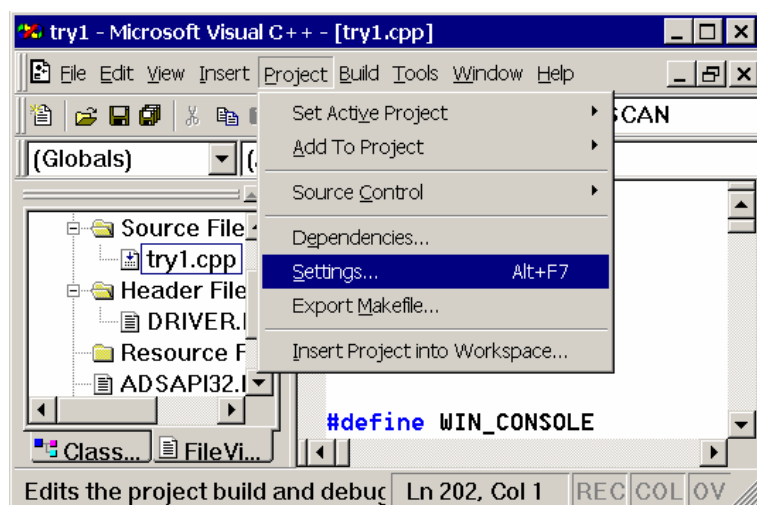
图（5）在工程中的头函数

6. 在工作区的 ClassView 中，就可以编辑自己所需要的代码，其中使用到具体的研华提供的库函数的时候可以参照 Device Driver User Manual。



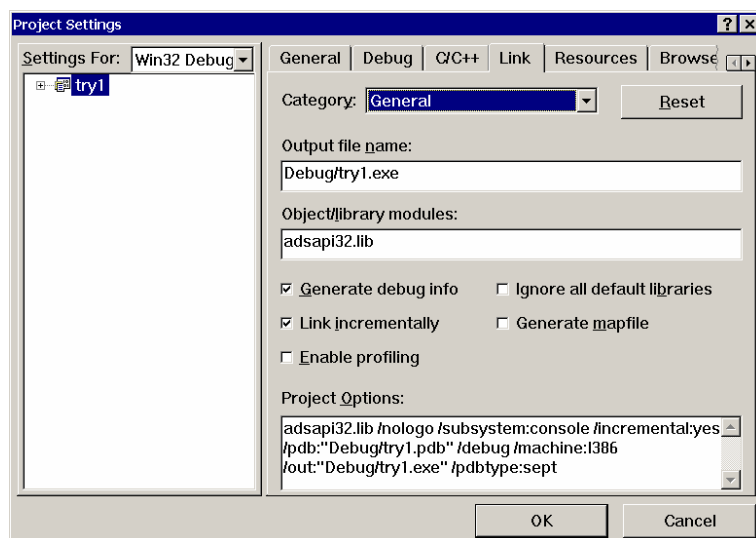
图（6）创建你自己的应用程序

7. 为了能够调用所有的动态连接库，我们需要在工程导入 ADSAPI32.LIB，同加入 driver.h 的方法类似，我们通过 Project->Setting 选项来进行设置，具体过程如下：



图（7）将动态连接库导入工程

8. 单击 Setting 菜单后弹出 Project Setting 对话框，选择其中的 Link 属性页，在其中的 Object/Library modules 框里面输入 adsapi32.lib 的路径，然后点击 OK 按钮退出该对话框，这样编译程序的时候就可以把 adsapi32.lib 函数库编译并连接到执行文件中。



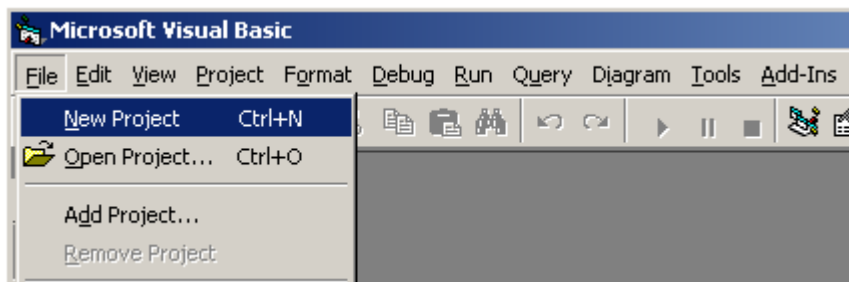
图(8) 将动态连接库与工程相连接

在 VC 下面加入动态连接库比较简单的办法是：直接将所需的头文件和库函数拷贝到您的工程文件夹下面，然后再依照上面的方法将头文件和库函数导入您的工程。

使用 visual Basic 来创建应用程序:

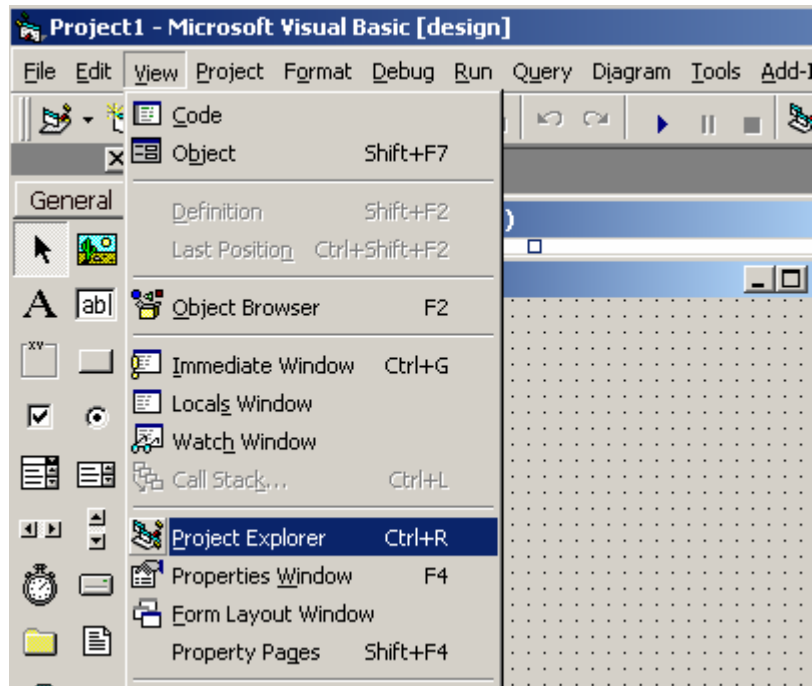
要正确的使用 DA&C 函数，你必须按照下面的步骤来使用 DA&C 的动态连接库

1. 择 File | New Project 菜单创建工程，这一点同其他的类型的程序创建是完全一样的。

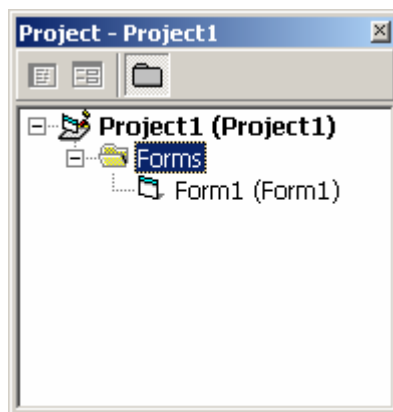


图（9）创建新的 VB 工程

为了能够使用研华的的驱动程序来开发数据采集工程，首先需要将研华驱动程序的头文件包含到工程中，具体方法是选择 View | Project Explorer 菜单

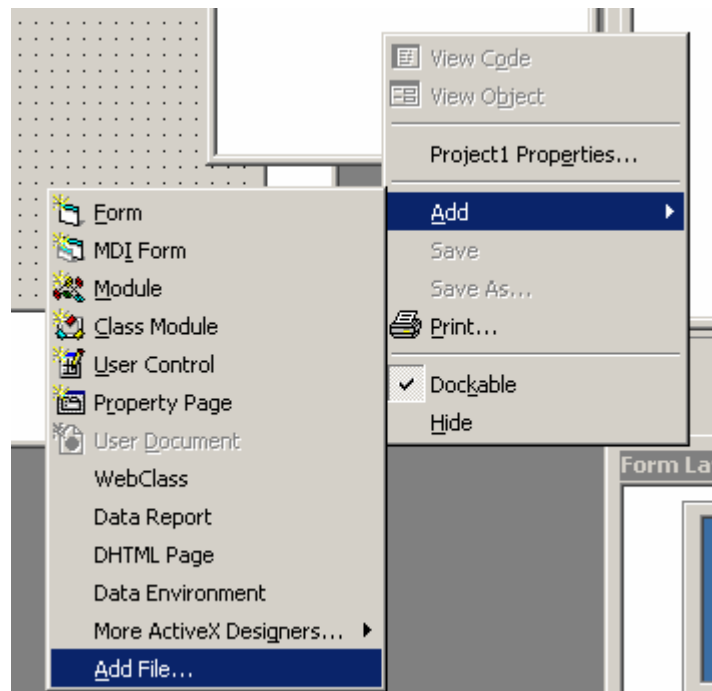


3、单击 Project Explorer 按钮之后，弹出 Project Explorer 窗口，在该窗口中单击右键，显示了一些可选的菜单

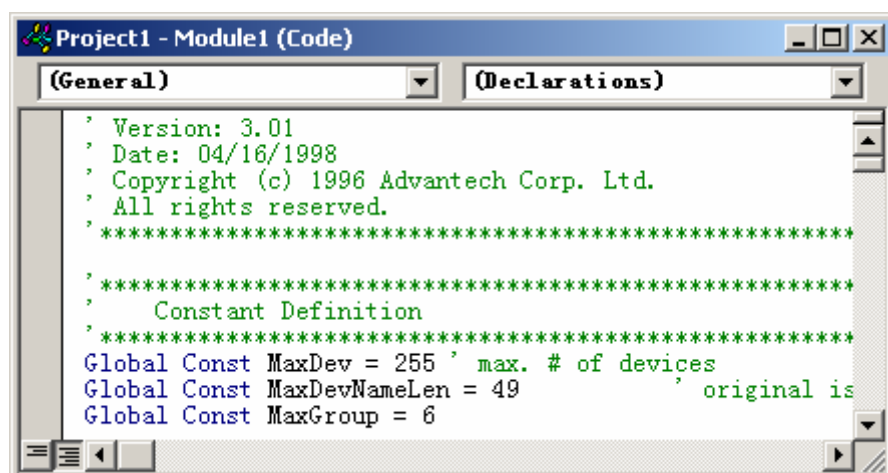
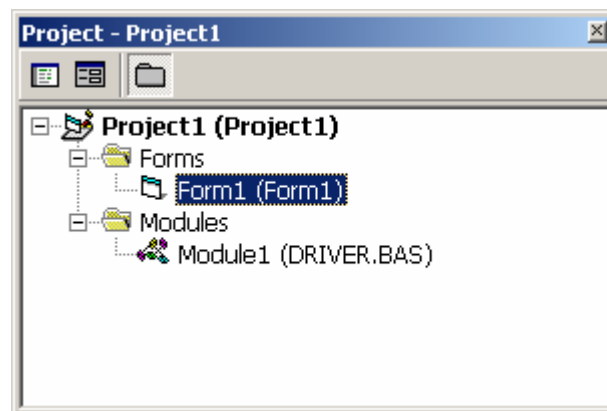


单击 AddFile 菜单 添加 Advantech 为 Visual Basic 的库文件 “Driver.bas “，随后会弹出文件

选择对话框。通常情况下着头文件是包含在驱动程序安装的默认路径下面。VB 下面如何调用动态连接库请参考 VB 使用手册。



加入库文件后就可以浏览工程中用到的常量，参数声明和动态连接库中的函数。

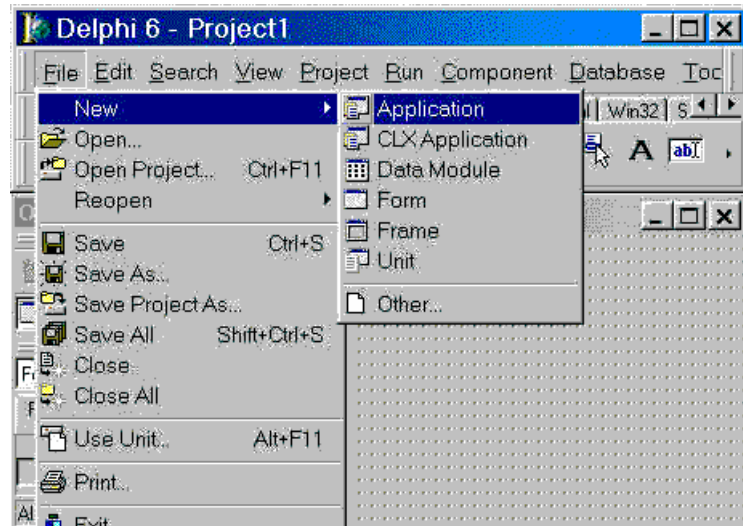


创建您自己的应用程序代码，并且使用动态连接库中的函数。

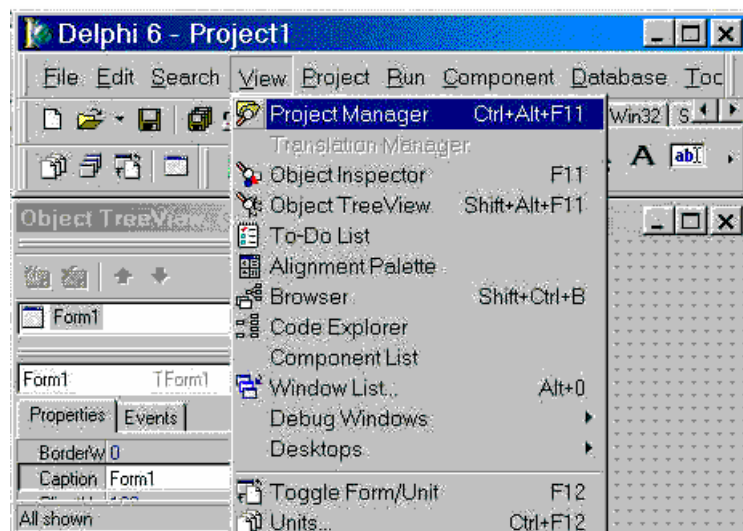
使用 Borland Dephi

在 Dephi 下面使研华提供的用动态连接库，需要按照下面的步骤操作；

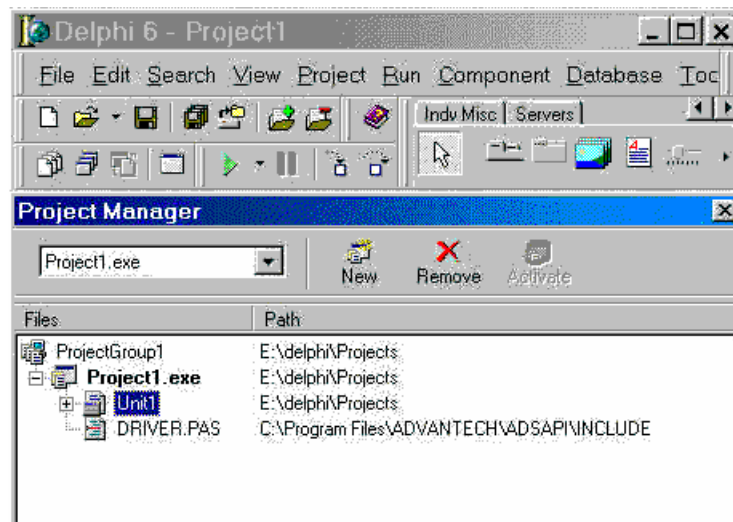
1) 选择File | New Application 创建一个新的工程



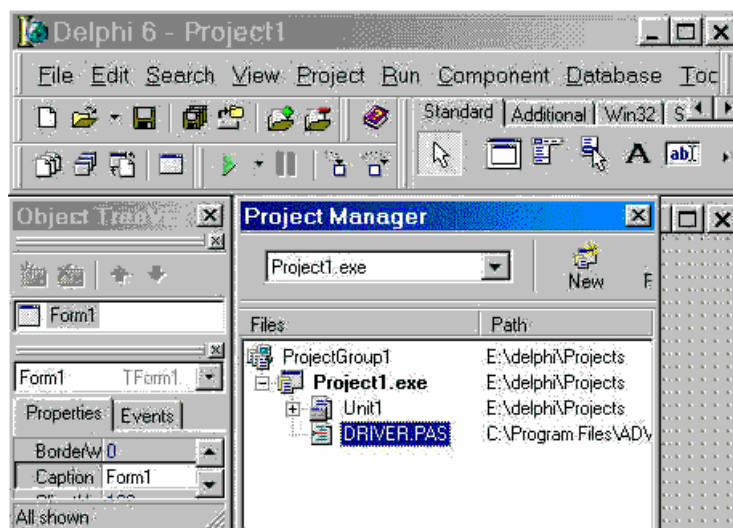
2) 选择 View | Project Manager 来设计应用程序窗体，编写代码。单击该菜单后会弹出一个 Project Manager 对话框。



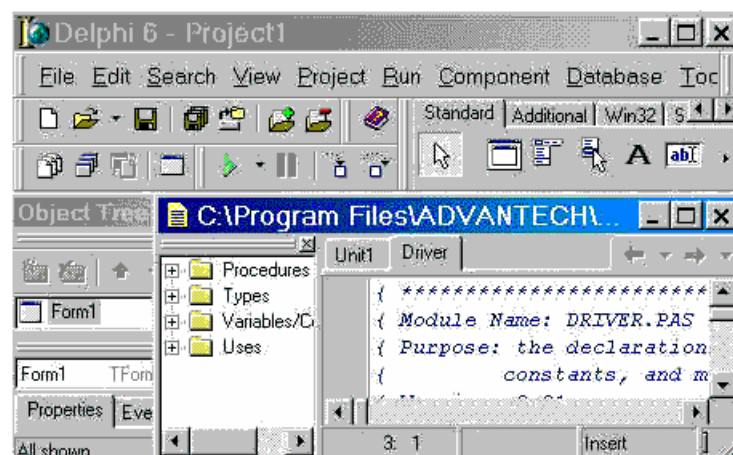
3. 首先通过 Project | Add to Project 在工程中添加 Advantech 32 位设备驱动程序的头文件，单击该菜单会弹出一个文件查找的对话框，可以从中选择所需的动态连接库文件。



4.选择动态连接库的库文件“Driver.pas”并确定，Project Manager 中将会显示该头文件及其位置。



5.双击头文件，就可以看到在程序中可以调用的常量的定义，参数声明和动态连接库函数。



6.创建你自己的 Delphi 程序界面代码。这时你可以参照驱动程序光盘上面的例子程序和相关代码，设计完毕之后单击 Run 菜单测试程序。

为 Windows 95/98/2000/NT 提供的驱动程序包括一个动态连接库和一个内核系统文件，主要的函数都包含在内核模式的设备驱动程序里面。动态连接库则提供了一个简单的编程接口。

驱动程序的动态连接库文件，ADSAPI32.DLL，在 95/98 系统的 SYSTEM 文件夹下面；2000/NT 系统在 SYSTEM32 文件夹下面；内核设备驱动程序文件，ADSIO.SYS 在 Windows\Drivers 目录下面。

除了上述的系统文件外，驱动程序还带有其他的一些有助于您开发 DA&C 应用程序的文件。

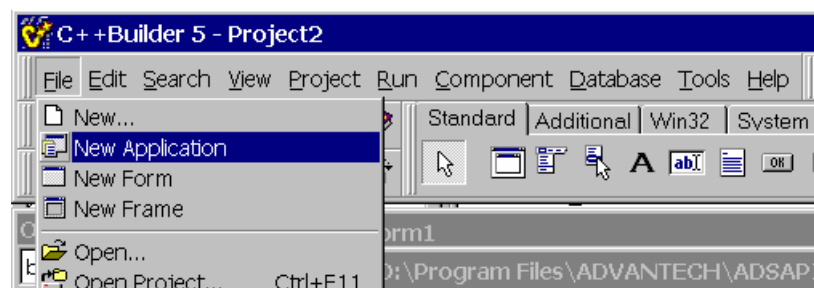
这些文件也都存在于您驱动程序安装的目录中。

注："Driver.bas"包含了所有的动态连接库函数的原型，在开发您的数据采集程序的时候您需要将这个文件包含到您的源文件中。

使用 Borlang C++或 C++ Builder

安装好设备驱动程序之后，在 C++ Builder 5.0 以上的版本中可以按照如下的步骤开始编程：

- 1) 使用 File|New Application 创建一个新的工程



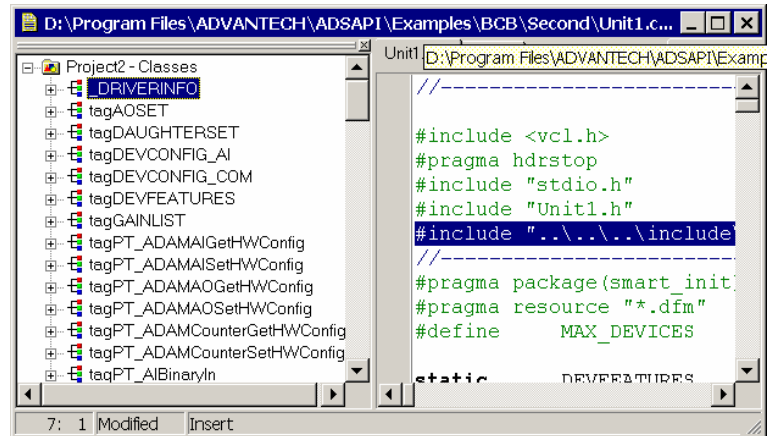
- 2) 添加变量：

.....

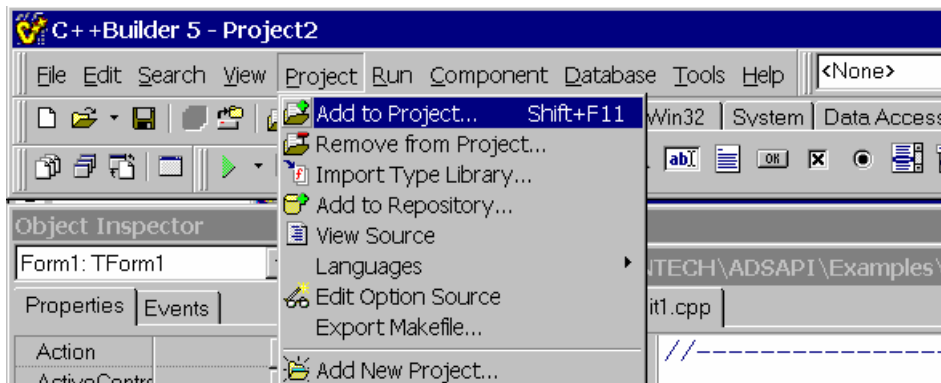
```
static      DEVFEATURES      DevFeatures;
```

.....

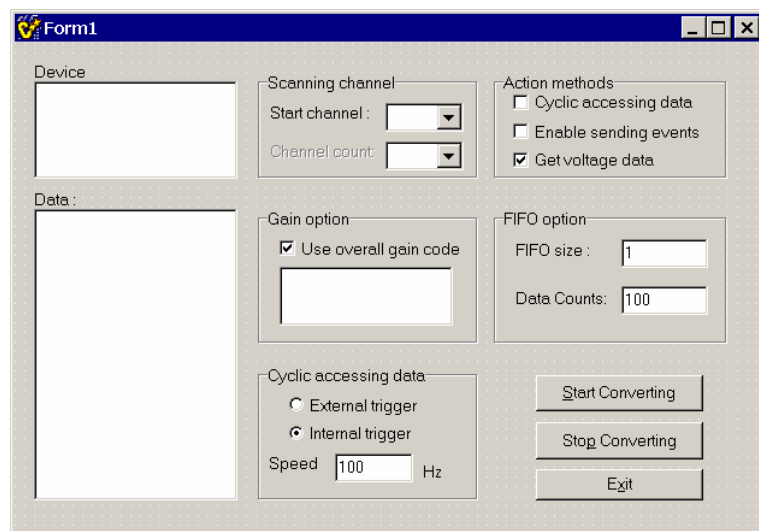
- 2) 添加这一句代码之后#include "..\..\..\include\driver.h"，编译程序就可以看到如下的 driver.h 中的代码。



2) 使用 Project/Add to Project...添加函数库（AdsApi32bcb.lib）到工程中：



3) 编辑程序界面如下图：



4) 在单击界面上面的按钮可以进入命令编辑界面，依次添加程序处理代码

```
void __fastcall TForm1::butStartClick(TObject *Sender)
```

{ }

具体代码参见例子程序

如果您使用的 C++Builder 比较低的版本那么就可能需要一些复杂的转换工作：

1.使用类库创建一个新工程，如果你希望直接使用我们提供的例子程序中的代码，那么在创建程序的时候，不要选择使用OWL类。

2.如果函数库（例如：**\ADSAPI\EXAMPLES\BC\LIB\ADSAPIBC.LIB**）与您的 Borland c++版本不兼容，您需要使用 implib 工具创建新的.LIB 文件。

Syntax: **IMPLIB** [options] **libname** [.lib] **dllname** [.dll]

语法: **IMPLIB** [可选] **库名** [.lib] 动态连接库名 [.dll]

例子:

IMPLIB C:\Program Files\Advantech\Adsapi\
Examples\BC\LIB\ADSAPIBC.LIB

C:\WinNT\ADSAPI32.DLL

注：提供给 Borland C++/Borland C++ Builder 的头函数同提供给 Visual C++的头文件是完全一样的。

1. 在工程中加入您自己的代码

2. 在工程的 Project Options 选项的 Processor 中将数据格式改成 Quad Word (8-Byte)。

因为 Visual C++ Visual Basic/Delphi 的默认数据格式就是四字节，你就不要在这些程序的环境中改变配置了。

思考：

除了了解如何使用研华的驱动程序之外，你还要需要考虑到在使用动态连接库时可能发生的一些问题。这一部分简单介绍了这些问题的本质，下面的一节给出了这些问题的解决办法（不同的编程语言，解决的办法也各不相同）

内存分配：

在 Windows 应用程序中分配内存要比在非 Windows 应用程序受到更多的限制，因为 Windows 有它自己的内存分配函数，这就要求你通过 Windows 内存管理器来分配内存。通常情况下你需要使用 Windows 提供的内存分配函数来分配内存，而不是使用个别语言自己提供的

字符串传递：

当 Windows 动态连接库调用一个字符串作为传递参数，它需要一个指向 null 结尾的字符串。不同的编程语言需要不同的字符串处理来支持这种类型。

参数传递：

你可以采用传值/传地址的方式来传递参数，而不同的编程语言有不同的默认设置。你必须使用传值/传址方式传递特定类型的参数给每个 Windows 函数。

注：DRV_GetAddress 函数只是在 Visual Basic 中才用到。因为在 VC++ 或者是 Delphi 中，用户可以使用指针变量或者取得一个变量的地址。然而，在 Visual Basic 中没有 Windows 标准函数来取得变量的内存地址。而 Advantech 的设备驱动程序多数函数需要一个地址参数作变量

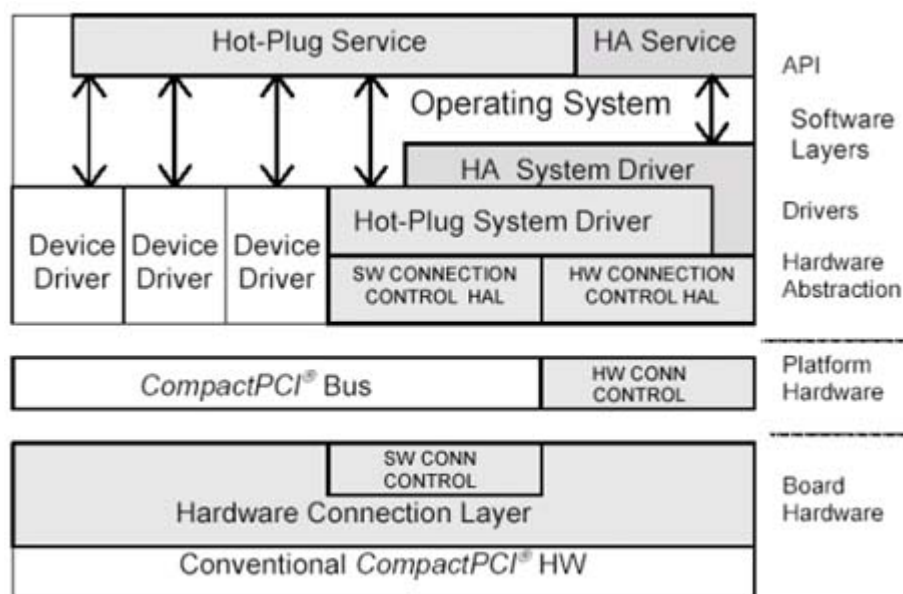
三、 Compact PCI Hot Swap

Hot Swap 介绍

PICMG 2.1 中详细地定义出 Hot Swap 的目的及应用，其最主要的目的在于提供一种平台在系统不断电条件下，周边卡甚至于系统卡能在不破坏整个系统的正常工作下，让操作者或管理者拔出或插入，以利于维修或系统重新组态。

Hot Swap 平台架构

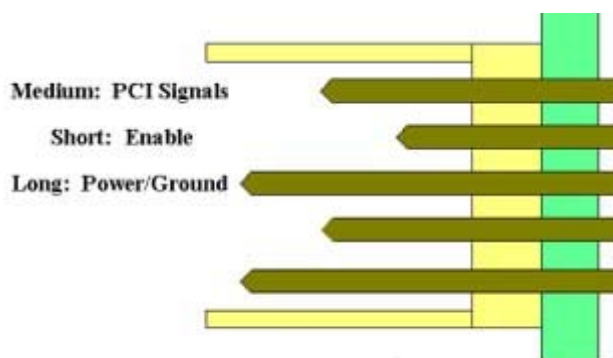
Hot Swap 系统分为 Basic Hot Swap、Full Hot Swap 和 High Availability 三类，此三类别的系统架构都含括在以下的架构图中。



图一：Hot Swap 架构图

在架构图中最底层的部份为可热插拔卡片所应支持的硬件设计，它必须有一个可以掌控卡片插拔状态及控制 Hot Swap LED 的芯片，在卡片热插拔时能够发出 ENUM#讯号告知系统 CPU 卡。

架构图中间的部份为 CompactPCI 的背板设计，因为 CompactPCI Bus 的独特硬件接触界面有长短针的分别(如图二)，所以在热插拔卡片时可以避免因为电源的不当输入而造成卡片的损害。另外，CompactPCI 背板上还提供了 ENUM#讯号的传递界面让可热插拔卡和系统 CPU 卡沟通。

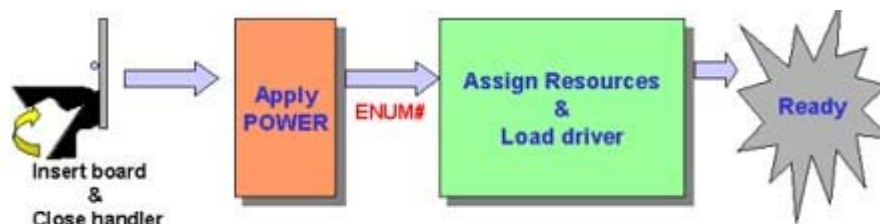


图二：CompactPCI Connector

最后，在架构图的最上层为系统 CPU 卡所必须支持的部份，它必须有一个可以知道卡片热插拔的界面(ENUM#或 PCI Bus)，并且必须要有一 Hot Swap driver 来负责卡片热插拔时的资源分配及卸载，否则即使 CPU 卡可以知道有任何的卡片插拔时，使用者也无法正常地使用。基于以上的架构，因为部份的组成组件的不支持便会形成不同的 Hot Swap 平台，如 Full Hot swap 平台即少了 HA Service、HA system driver 以及 Hardware Connection Control 等组成组件。本篇文章将以介绍 Full Hot Swap 平台为主。

Hot Swap 流程

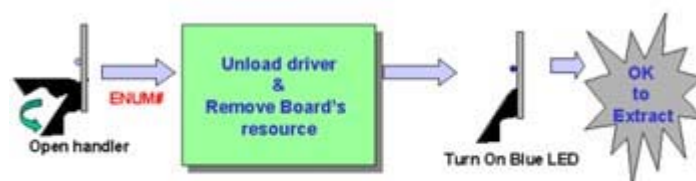
在 Full Hot Swap 的平台上，只有卡片的插拔是需要操作员来手动操作，其余的像讯号的连结，资源的分配及卸载，都是靠硬件及软件来达成。卡片插入的过程可由图三来表示。



图三：Insertion Process

当操作员插入卡片并将把手扣上时，可热插拔卡上的电源会开始供应并且发出 ENUM#的讯号告知系统 CPU 卡，CPU 卡在接收到 ENUM#讯号之后，会先确认 PCI 总线上哪张卡片是刚被插上，然后将此卡相关的资源需求安排好并且加载它的驱动程序，等驱动程序完整地加载之后便可以正常地使用这张卡。

外围卡的拔出正好是相反的动作，当操作员打开外围卡的把手时，此时卡片会送出 ENUM#讯号告知系统 CPU 卡，CPU 卡在接收到讯号之后，即会开始扫描总线上的卡以确认哪张卡要被移除，然后开始卸载此卡的驱动程序以及相关的资源要求，最后 Hot Swap driver 会再把卡片的 Hot Swap LED 点亮以告知操作员可以拔掉这张外围卡了。



图四：Extraction Process

由插入及拔出的过程，我们可以知道要支持热插拔是需要特殊的硬件及软件的设计。

Hot Swap 的特殊硬件设计

要支持热插拔，在硬件的设计上，首先要找到一颗支持热插拔的芯片，此芯片必须能够记录插拔的状态、能够控制 Hot Swap LED、能够反应从卡片把手所送来的状态改变，进而发送 ENUM#讯号。这部份的应用，在一般的 I/O 界面卡，通常会使用 PLX 的芯片，而 CPU 外围卡就必须依靠非透明桥(Non-Transparent Bridge)的支持。

另外，在外围卡的电源供应的设计上，必须考虑到卡片在插入或拔除时，板子上的所有组件不会被破坏，而且在 PCI 总线上的讯号能够同步，所以在设计上通常都会利用背板上的长短针来达到这些要求。

在热插拔的平台上，系统 CPU 卡所要设计的硬件最主要为 ENUM#讯号的接收。通常这类的设计是各家厂商有自己设计方式。如果在系统 CPU 卡上没有 ENUM#接收的硬件设计的话，通常也可以利 PCI 总线扫描的形式来达成热插拔的效果。

Hot Swap 的特殊软件需求

在软件的需求上，不外乎是能够反应外围卡所送来的讯号，进而分配或移除资源给外围卡以及加载或卸载外围卡的驱动程序。根据上节所描述的接收讯号的机制，GPI 或中断甚至于 PCI 总线扫描的形式，我们可将软件分为轮询(polling)以及中断服务程序(Interrupt service routine)两类。前者必须利用固定的时间定期去询问是否有卡片的插拔，然后再启动 Hot Swap 服务程序，而后者主要是利用中断来启动 Hot Swap 服务程序。

Hot Swap 服务程序包含了外围卡的资源分配及卸载、驱动程序的加载及移除、以及 Hot Swap 指示灯的致能。这一部份是最复杂的一部份，它必须考虑到所有可能的资源分配，包含 I/O 端口、内存以及中断。而且不同的操作系统有各自的处理模式，如果在这一方面没有处理好的话，并不能达到真正的 Hot Swap 的目的。

研华 Compact PCI Hot Swap 编程

研华公司提供以下函数实现 Compact PCI 的热插拔

- 1、 HSM_DeviceOpen: 打开设备，返回设备句柄
- 2、 HSM_DeviceClose: 关闭设备
- 3、 HSM_GetHsStatus: 取得 IO 槽的状态，0 为空
- 4、 HSM_RequestExtraction: 拔除指定槽的卡
- 5、 HSM_GetEnumStatus: 取得所有 IO 槽的状态，0 为空
- 6、 HSM_SetLED: Hot Swap LED 点亮/熄灭
- 7、 HSM_GetLED: Hot Swap LED 状态

例程如下，该程序实现

- 1 Set LED on
- 2 Set LED off;
- 3 拔除设备;

程序及说明如下

```
retValue = HSM_DeviceOpen(&hsHandle); //打开设备
scanf("%d",&usSlotNumber);
while(key != ESC)
{
.....
switch(key)
{
case '1':    retValue = HSM_SetLED(hsHandle,          //句柄
                                   usSlotNumber,      // IO 卡槽号
                                   TRUE);             // 置位 ON
break;
case '2':    retValue = HSM_SetLED(hsHandle, usSlotNumber, FALSE);
break;
```

```

case '3':    retValue = HSM_RequestExtraction( hsHandle,    // 设备句柄
usSlotNumber);// 槽号
break;
}
// 得到 IO 卡的状态
retValue = HSM_GetHsStatus(hsHandle, //设备句柄
    usSlotNumber,          // IO 卡槽号
    &usHsStatus            // 输出状态
);
// 得到所有 IO 卡的状态
retValue = HSM_GetEnumStatus(hsHandle, // 句柄
    &bEnum); // 输出状态
if(retValue)
    GetErrorStatus(retValue);
// LED 状态
retValue = HSM_GetLED(hsHandle,      //句柄
    usSlotNumber, // 槽号
    &bLED);

printf(" Hot Swap Status= %x    Enum = %d    LED = %d    \r",
    usHsStatus, bEnum, bLED);
Sleep(100);
}
HSM_DeviceClose(hsHandle);//关闭设备

```

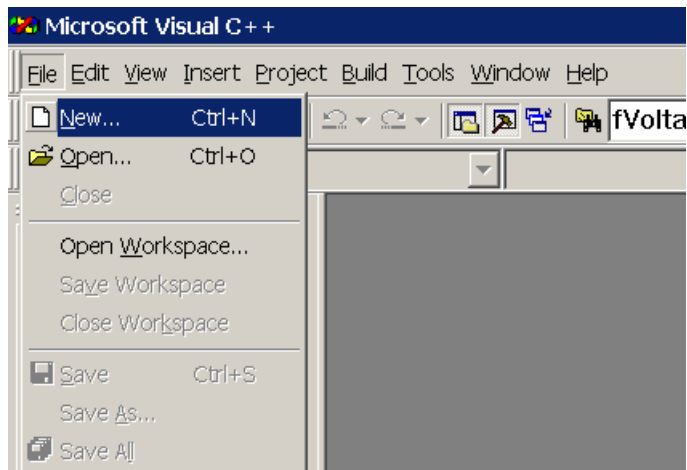
数据采集卡 VC 编程详解

一、 软件触发数据采集 Adsoft

安装设备并用 device manger 调试无误之后就可以开始编程工作。

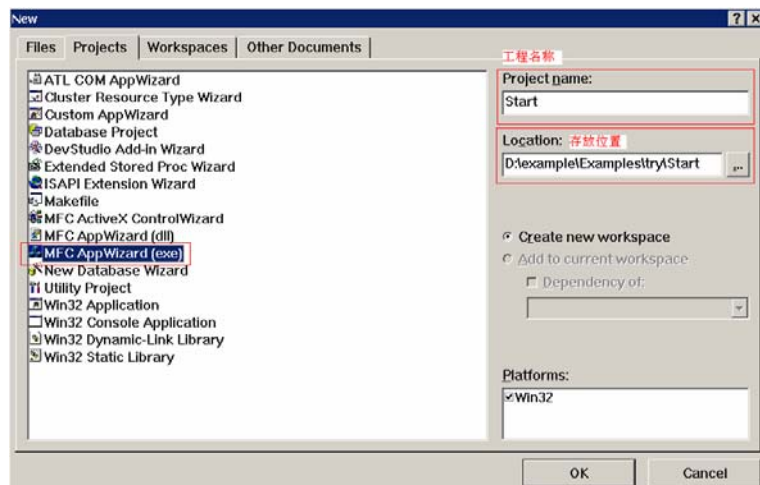
第一步：创建一个 Dialog Based 程序框架。

1) 我们需要新建一个工程：点击 File->New 菜单，图（1）所示，



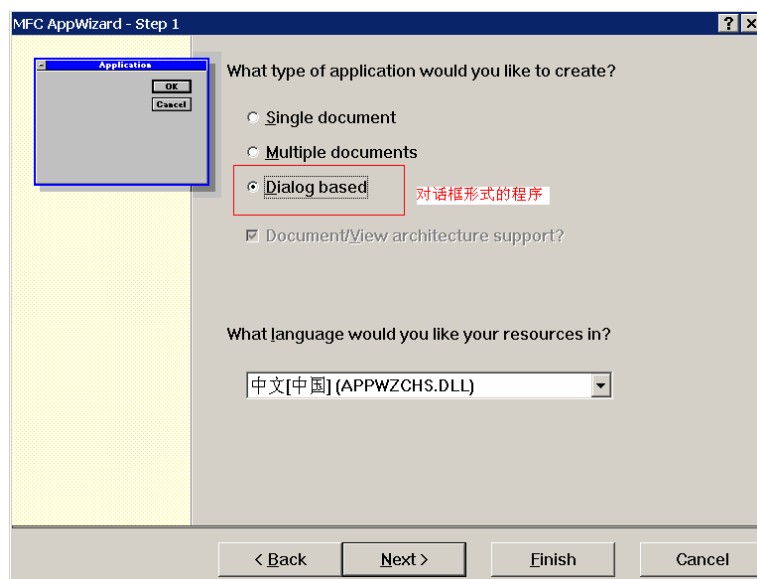
图（1）

2) 在弹出的 NEW 对话框（图（2））选中 MFC Application(exe)选项，在 project name 中输入要创建的工程名称，例如 Start；在下面的 Location 中可以选择要保存这个工程文件夹的路径。



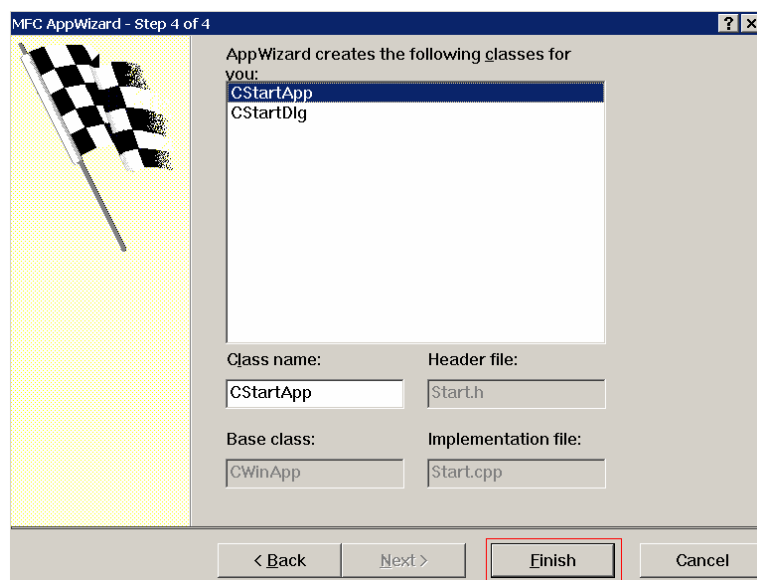
图（2）New 对话框

3) 点击 OK 按钮进入下一步 MFC Wizard Step1，如下图所示：选择程序类型为：Dialog based，即对话框形式的程序；选择之后点击 Next 按钮进入下一步。



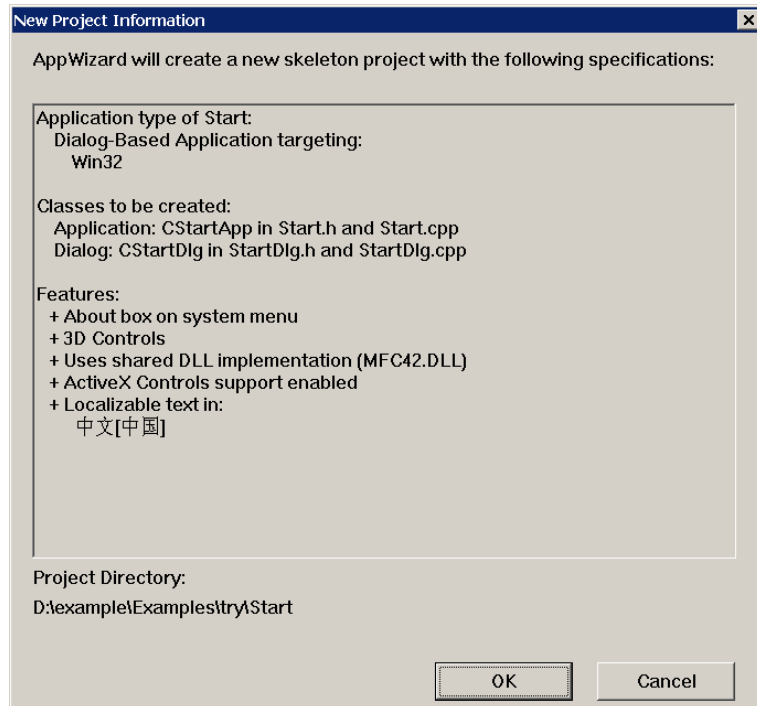
图（3）Step1

4) 下面的对话框中依次点击 Next 按钮，即：接受所有的默认选项；最后一项，如下所示：



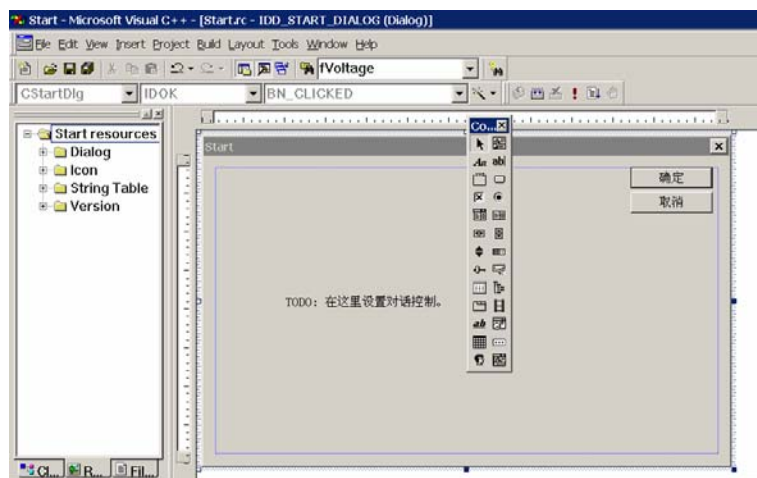
图（4）接受默认项到最后

5) 点击 Finish 按钮。弹出如下对话框，给出了这个新建工程的主要信息。点击 OK 按钮完成新工程的创建工作。通过对话框编辑器，就可以对这个对话框进行编辑；首先我们可以删掉上面的（Satic）静态文本框“TODO: 在这里设置对话控制”；



图（5）工程信息

6) 点击 OK 按钮之后的 Visual C++编译器的界面如下图:



图（6）

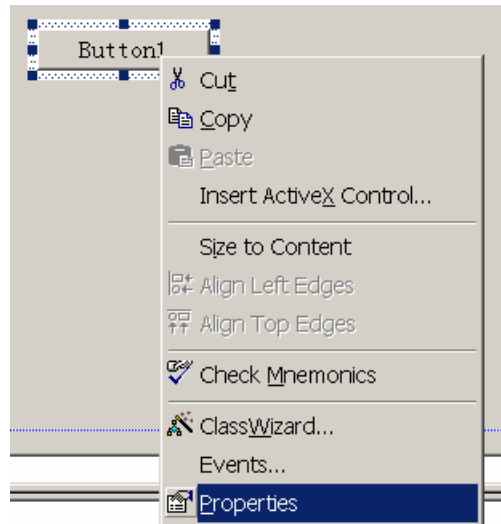
第二步：编辑对话框

1) 加入命令按钮和然后选择控件中的命令按钮，在对话框上单击或是按住并拖动鼠标左键就可以看到一个命令按钮。



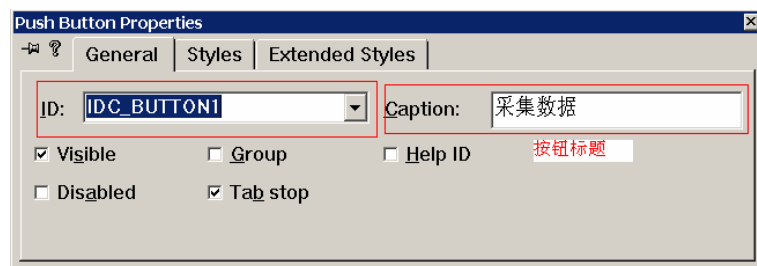
图（7）

2) 修改按钮的标题，在这个命令按钮上单击鼠标右键，如下图所示:



图（8）

选择其中的属性 Properties 项弹出该按钮属性对话框：

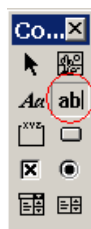


图（9）属性对话框

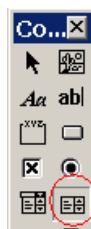
在这里为了简单明了，我们只修改这个“按钮标题”，其他的选项都保持系统默认的不

变。
3）在对话框上添加一个编辑框（Edit）和列表框（ListBox），用来显示我们采集到的数

据：



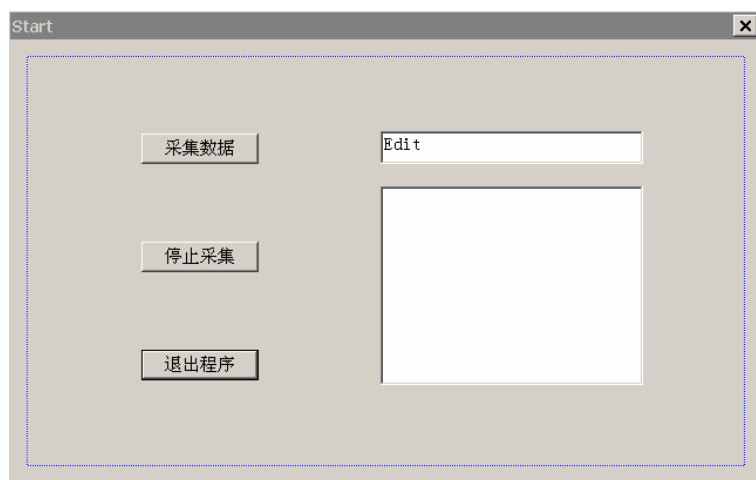
图（10）选择编辑框



图（11）选择列表框

这里编辑框和列表框的属性我们不作修改。

通过上述的方法，编辑对话框窗口布局如下图所示为止：



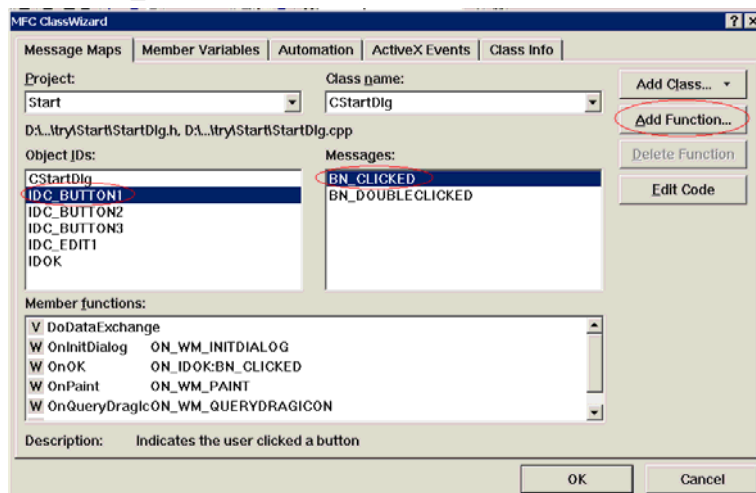
图（12）程序窗口布局

第三步：导入外部头文件和函数库。

导入方法见手册。另外一种比较简单的做法是把 adsapi32.lib,driver.h,os.h 直接拷贝到这个工程所在的文件夹中。

第四步：定义变量和函数。

1) 为命令按钮添加函数：选择 ClassWizard 中的 Message Map 属性页，来为按钮单击操作添加相对应的函数，方法如图中所示：选中 IDC_BUTTON1(标题为：采集数据，Data Acquisition)，同时选中 BN_CLICKED，单击 Add Function...按钮，



图（13）ClassWizard 对话框

弹出添加函数的对话框：

函数名称与该按钮的 ID 相对应，IDC_BUTTON1 ⇔ OnButton1，当然也可以有别的作法，我们在这里选择默认的函数名称，点击 OK 添加函数，同样的方法为其他的按钮添加函数。

“退出程序”按钮，我们只是修改原来的“OK”按钮的按钮标题，其他的不作改动，其 ID 为：ID_OK，故添加的函数为 OnOK。函数名称方面，用户也可以根据自己的喜好并参照 VC 方面的书籍作修改。



图（14）添加函数的对话框

2) 定义变量:

在 StartDlg.h 文件中添加变量: 以下为完整的 StartDlg.h 头文件, 其中加黑斜体部分代码为手动方式输入的变量, 其他部分为 Visual C++编译器自动生成的部分, 不要改动。

```
// StartDlg.h : header file
//
#if !defined(AFX_STARTDLG_H__89987CA9_3800_43ED_A6A0_929B0F6A6049__INC
LUDED_)
#define
AFX_STARTDLG_H__89987CA9_3800_43ED_A6A0_929B0F6A6049__INCLUDED_

    #include "driver.h"

    #if _MSC_VER > 1000
    #pragma once
    #endif // _MSC_VER > 1000

    //////////////////////////////////////
    // CStartDlg dialog

    static PT_AIVoltageIn ptAIVoltageIn;

    static PT_AIConfig ptAIConfig;

    static DEVFEATURES DevFeatures;

    static PT_DeviceGetFeatures ptDevFeatures;

class CStartDlg : public CDialog
{
// Construction
public:
    CListBox* pDataList;

    float fVoltage;

    USHORT gwChannel;                                // input
```

channel

USHORT gwGain; *// gain index*

LONG DriverHandle;

LRESULT ErrCde;

CStartDlg(CWnd* pParent = NULL); *// standard constructor*

// Dialog Data

```
//{{AFX_DATA(CStartDlg)  
enum { IDD = IDD_START_DIALOG };  
CString m_DataDisplay;  
//}}AFX_DATA
```

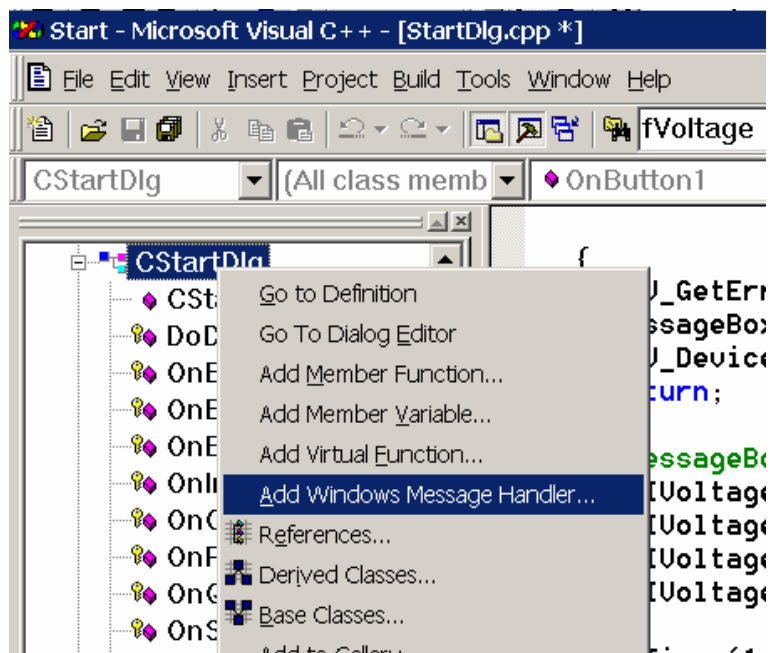
.....

3) 变量的初始化。在 `BOOL CStartDlg::OnInitDialog()` 函数中，对定义的变量进行初始化：

```
SetIcon(m_hIcon, TRUE); // Set big icon  
SetIcon(m_hIcon, FALSE); // Set small icon  
// TODO: Add extra initialization here  
DriverHandle = (LONG)NULL;  
gwChannel = 0; // input channel  
gwGain = 0; // gain index  
fVoltage=0;  
pDataList=(CListBox*)GetDlgItem(IDC_LIST1);
```

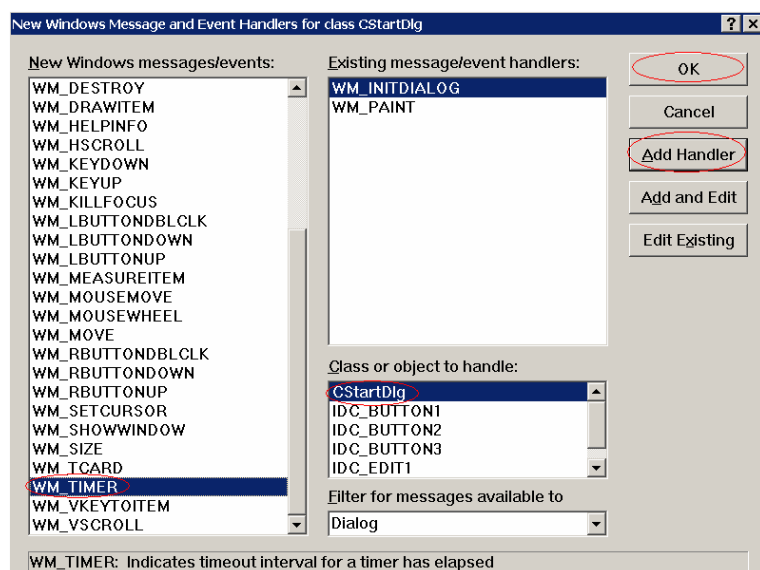
4) 添加处理计时器事件的句柄 (WM_TIMER)。

在 ClassView 中右键单击 CStratDlg 类, 在弹出菜单中选中 Add Windows Message Handler, 添加计时器事件处理程序



图(15)添加事件处理句柄

单击后出现如下窗口，选中 WM_TIMER->单击 Add Handler 按钮添加处理函数句柄，然后单击 OK 按钮退出。



图（16）添加处理句柄对话框

这时候我们可以看到在 StartDlg.cpp 文件的最后添加了这样的一些函数体，我们可以通过在这些函数体中添加代码实现我们所需要的一些功能。

第五步：编写程序代码

```
void CStartDlg::OnButton1()
{
    //采集数据
    // TODO: Add your control notification handler code here
    //open device
```

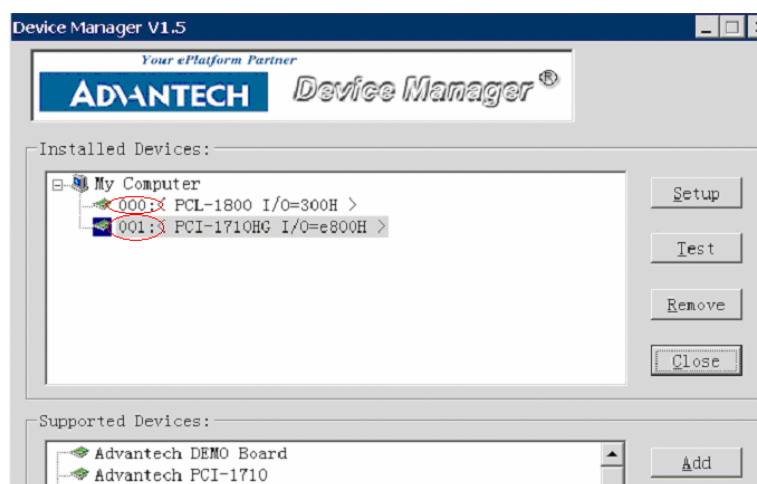
```

ErrCde = DRV_DeviceOpen(0,(LONG far *)&DriverHandle);//打开设备
char      szErrMsg[80];
if (ErrCde != SUCCESS)
{
    strcpy(szErrMsg,"Device open error !");
    MessageBox((LPCSTR)szErrMsg,"Device Open",MB_OK);
    return ;
}

```

编程流程 1) 打开设备的操作是在使用设备进行数据采集等编程操作之前必须进行的。

DRV_DeviceOpen(dwDeviceNum,(LONG far*)&DriverHandle)参数说明：dwDeviceNum 是设备号，该号码在研华提供的 Device Manger 中可以查看到，图中椭圆标记中所示数字，我们。



图（17）

DriverHandle: 后面编程过程中对板卡的输入输出进行操作都将以该变量中的参数为基础，不同的板卡的使用也可以通过这个参数来区分调用。

注意：每个 DRV_DeviceOpen 函数都必须对应一个 DRV_DeviceClose 函数来释放由其分配的内存。

该函数的返回值为一个设备打开成功/失败的标记：故可以通过 (ErrCde != SUCCESS) 判断设备是否成功打开。DRV_GetErrorMessage(ErrCde,(LPSTR)szErrMsg)函数通过 ErrCde 就可以取得打开设备过程中是那一部分出错，通过 MessageBox(hWnd,(LPCSTR)szErrMsg, "Device Open",MB_OK)显示出来。

编程流程 2) 取得设备设置。

首先初始化 ptDevFeatures 结构变量中的变量 buffer，size。然后调用 DRV_DeviceGetFeatures(DriverHandle,(LPT_DeviceGetFeatures)&ptDevFeatures)，通过 DriverHandle 参数取得该设备的设置。其他的语句进行出错判断，结构及含义同上面 (DRV_DeviceOpen) 类似，不再赘述。

注：如果在这里出错需要调用 DRV_DeviceClose((LONG far *)&DriverHandle)关闭设备。

```

ptDevFeatures.buffer = (LPDEVFEATURES)&DevFeatures;
ptDevFeatures.size = sizeof(DEVFEATURES);
if ((ErrCde = DRV_DeviceGetFeatures(DriverHandle,
                                     (LPT_DeviceGetFeatures)&ptDevFeatures)) != SUCCESS)
{
    DRV_GetErrorMessage(ErrCde,(LPSTR)szErrMsg);
    MessageBox((LPCSTR)szErrMsg,"Driver Message",MB_OK);
    DRV_DeviceClose((LONG far *)&DriverHandle);
    return;
}

```

编程流程 3) 配置设备增益。

```

ptAIConfig.DasGain = DevFeatures.glGainList[gwGain].usGainCde;
ptAIConfig.DasChan = gwChannel;
if ((ErrCde = DRV_AIConfig(DriverHandle,
                           (LPT_AIConfig)&ptAIConfig)) != 0)
{
    DRV_GetErrorMessage(ErrCde,(LPSTR)szErrMsg);
    MessageBox((LPCSTR)szErrMsg,"Driver Message",MB_OK);
    DRV_DeviceClose((LONG far *)&DriverHandle);
    return;
}
ptAIVoltageIn.chan = gwChannel;
ptAIVoltageIn.gain = ptAIConfig.DasGain;
ptAIVoltageIn.TrigMode = 0;           // internal trigger
ptAIVoltageIn.voltage = (FLOAT far *)&fVoltage;

```

编程流程 4) 启动计时器，进行软件触发数据采集，数据采集过程在 OnTimer 函数

中。

```

SetTimer(1, 500, 0);
}

```

```

void CStartDlg::OnTimer(UINT nIDEvent)
{

```

```

    // TODO: Add your message handler code here and/or call default

```

编程流程 5)调用 DRV_AIVoltageIn(DriverHandle, (LPT_AIVoltageIn)&ptAIVoltageIn) 函数，来采集数据。

```

CString szBuffer;
char szErrMsg[80];
strcpy(szErrMsg,"Device open error !");
szBuffer.Format("data = %10.6f", fVoltage);
if ((ErrCde = DRV_AIVoltageIn(DriverHandle,
                              (LPT_AIVoltageIn)&ptAIVoltageIn)) != 0)
{
    DRV_GetErrorMessage(ErrCde,(LPSTR)szErrMsg);

```

```

MessageBox((LPCSTR)szErrMsg,"Driver Message",MB_OK);
DRV_DeviceClose((LONG far *)&DriverHandle);
return;
}

```

编程流程 6) 在编辑框和列表框中显示数据。

```

CWnd* pWnd = GetDlgItem(IDC_EDIT1);
pWnd->SetWindowText(szBuffer);
pDataList->InsertString(0,szBuffer);
CDialog::OnTimer(nIDEvent);
}

```

编程流程 7) 编写停止采集数据和退出程序按钮。

```

void CStartDlg::OnButton2()
{
    //停止数据采集
    // TODO: Add your control notification handler code here
    KillTimer(1);
    DRV_DeviceClose((LONG far *)&DriverHandle);
}

void CStartDlg::OnOK()
{
    //退出程序
    if(DriverHandle !=(long)NULL)——判断句柄是否已经释放，如没有则在此释放
    DRV_DeviceClose((LONG far *)&DriverHandle);
    CDialog::OnOK();
}

```

第六步：运行程序

好了，就是这么简单，运行一下程序结果就是下面样子的，采集到的数据不断的加入到列表框中！



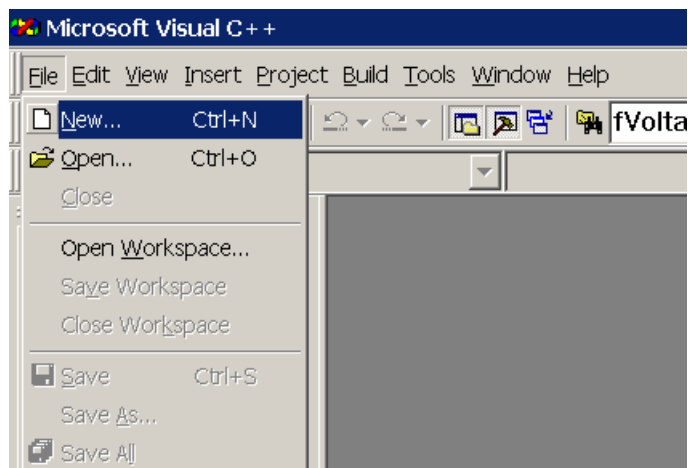
图（18）运行界面

二、应用中断进行数据采集 Adint

安装板卡并用 device manger 调试无误之后就可以开始编程工作。

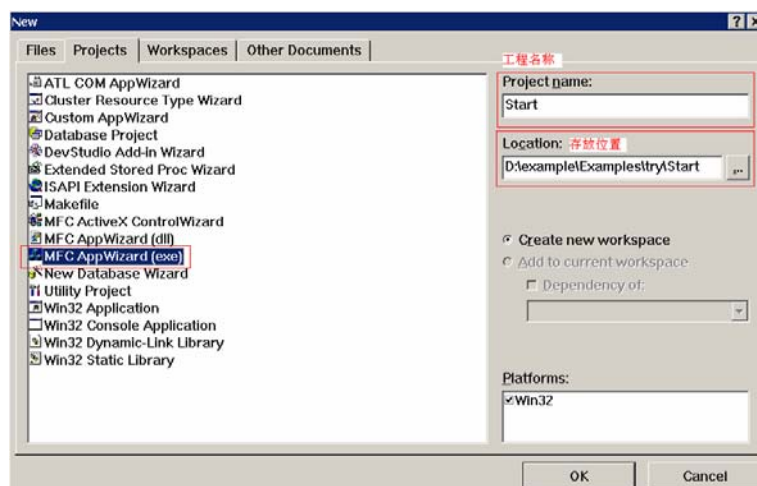
第一步：创建一个 Dialog Based 程序框架。

1) 我们需要新建一个工程：点击 File->New 菜单，图（1）所示，



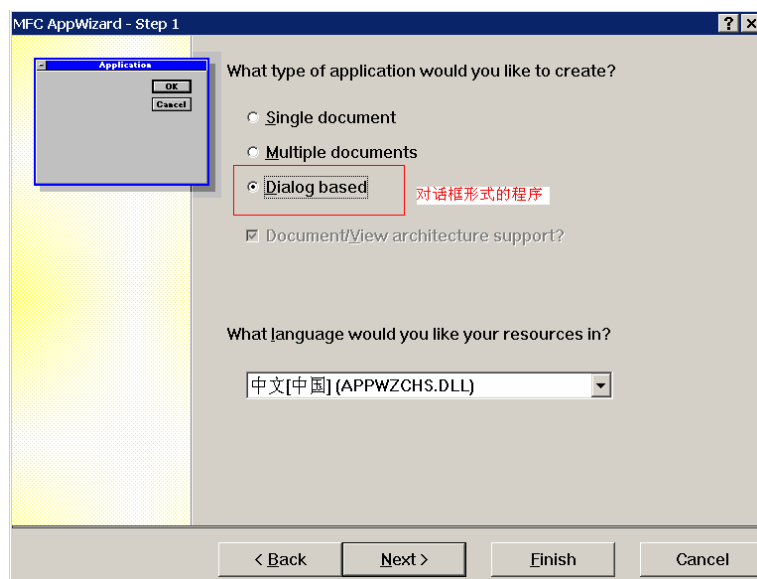
图（1）

2) 在弹出的 NEW 对话框（图（2））选中 MFC Application(exe)选项，在 project name 中输入要创建的工程名称，例如 Start；在下面的 Location 中可以选择要保存这个工程文件夹的路径。



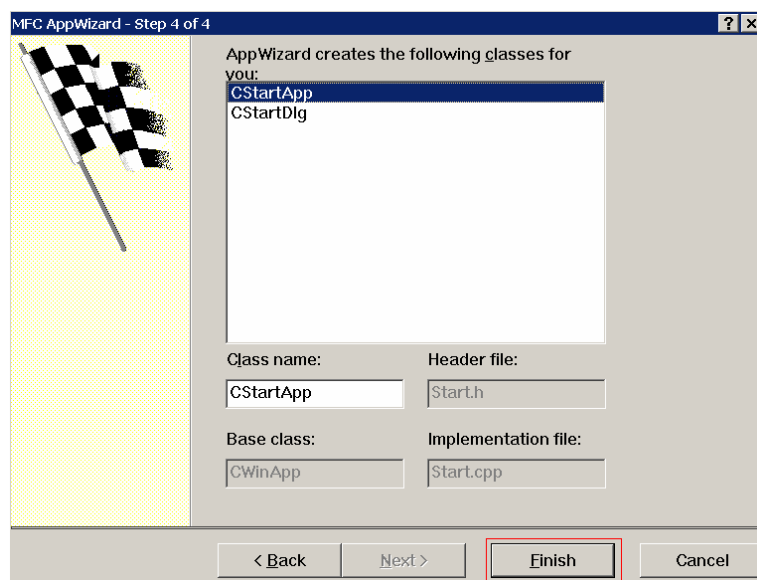
图（2）New 对话框

3) 点击 OK 按钮进入下一步 MFC Wizard Step1，如下图所示：选择程序类型为：Dialog based，即对话框形式的程序；选择之后点击 Next 按钮进入下一步。



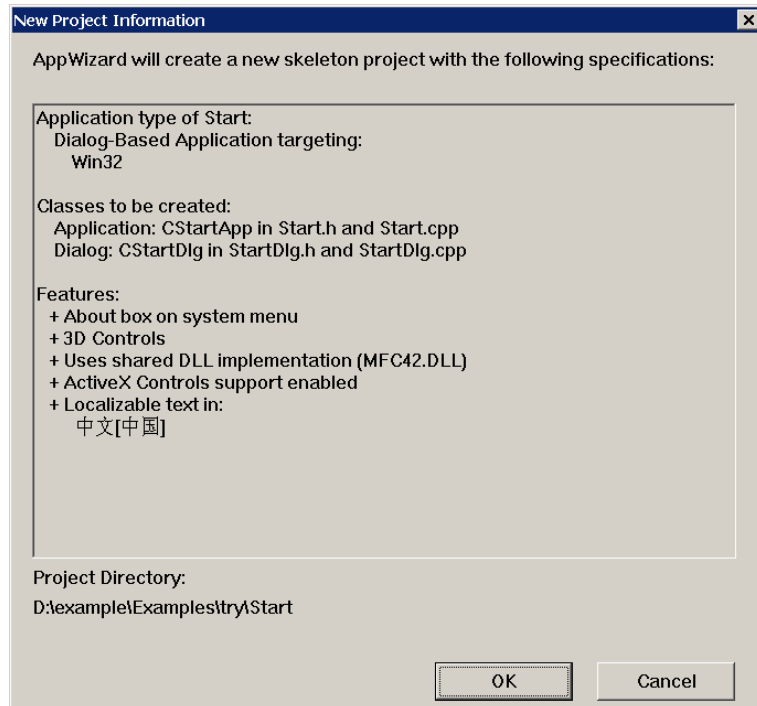
图（3）Step1

4) 下面的对话框中依次点击 Next 按钮，即：接受所有的默认选项；最后一项，如下所示：



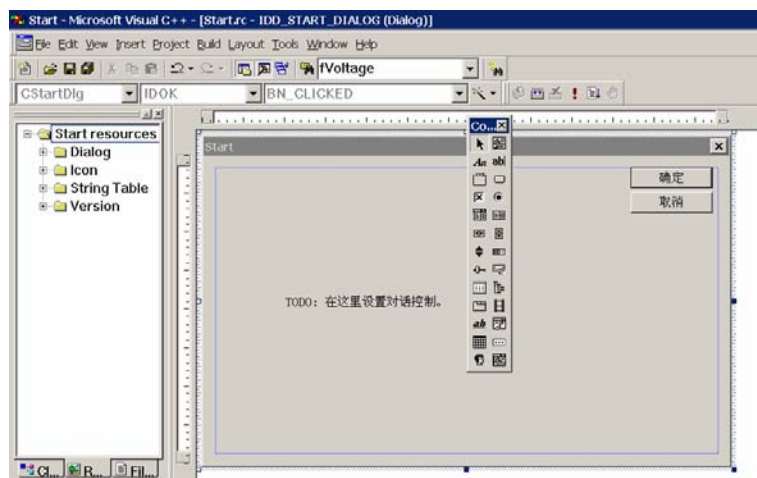
图（4）接受默认项到最后

5) 点击 Finish 按钮。弹出如下对话框，给出了这个新建工程的主要信息。点击 OK 按钮完成新工程的创建工作。通过对话框编辑期，就可以对这个对话框进行编辑；首先我们可以删掉上面的（Satic）静态文本框“TODO: 在这里设置对话控制”；



图（5）工程信息

6) 点击 OK 按钮之后的 Visual C++编译器的界面如下图:



图（6）

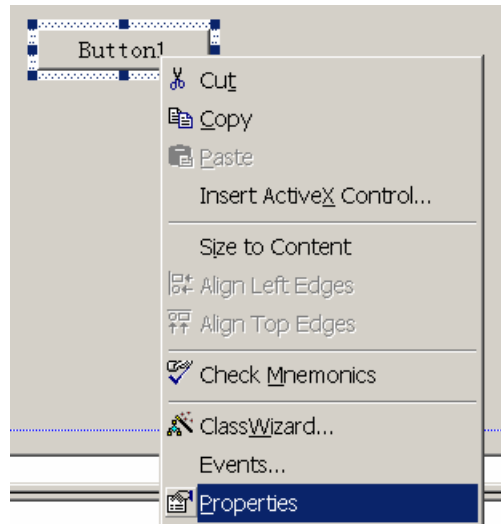
第二步：编辑对话框

2) 加入命令按钮和然后选择控件中的命令按钮，在对话框上单击或是按住并拖动鼠标左键就可以看到一个命令按钮。



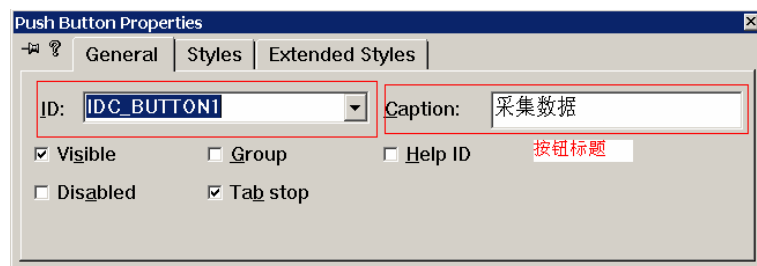
图（7）

2) 修改按钮的标题，在这个命令按钮上单击鼠标右键，如下图所示:



图（8）

选择其中的属性 Properties 项弹出该按钮属性对话框：



图（9）属性对话框

在这里为了简单明了，我们只修改这个“按钮标题”，其他的选项都保持系统默认的不

变。

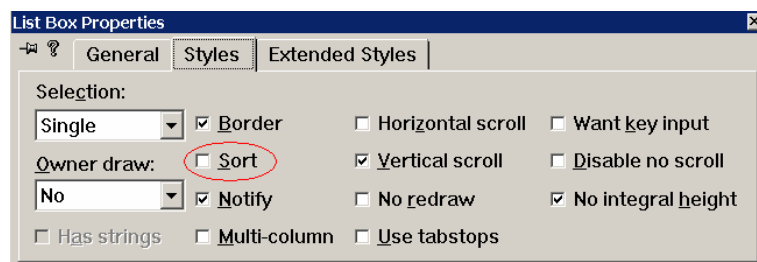
3) 添加列表框 (ListBox)，用来显示我们采集到的数据：



图（10）选择列表框

为了能够按照采集的顺序显示数据修改列表框的属性，在 ListBox 的属性框里面，选

Styles 页取消列表框的自动排序选项，Sort 项前面没有选中标志（即：去掉对钩）



图（11）ListBox 的属性页

通过上述的方法，编辑对话框窗口布局如下图所示为止：



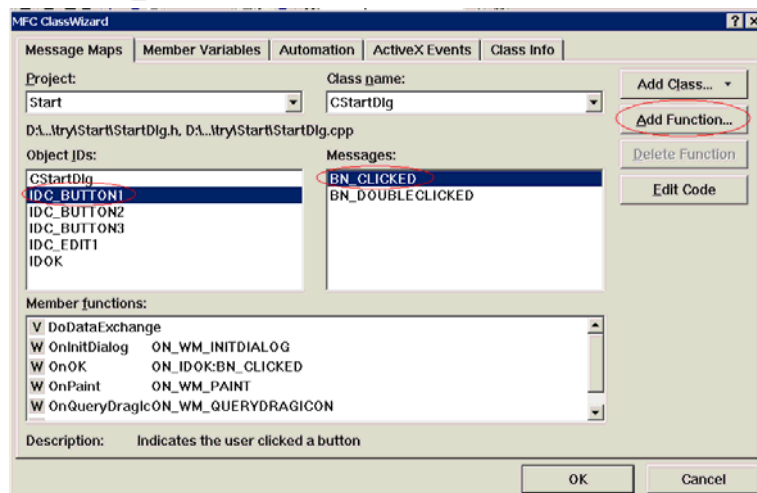
图（12）程序窗口布局

第三步：导入外部头文件和函数库。

导入方法见手册。另外一种比较简单的做法是把 adsapi32.lib,driver.h,os.h 直接拷贝到这个工程所在的文件夹中。

第四步：定义变量和函数。

1) 为命令按钮添加函数：选择 ClassWizard 中的 Message Map 属性页，来为按钮单击操作添加相对应的函数，方法如图中所示：选中 IDC_BUTTON1(标题为：采集数据，Data Acquisition)，同时选中 BN_CLICKED，单击 Add Function...按钮，



图（13）ClassWizard 对话框

弹出添加函数的对话框：

函数名称与该按钮的 ID 相对应，IDC_BUTTON1 ⇔ OnButton1，当然也可以有别的作法，我们在这里选择默认的函数名称，点击 OK 添加函数，同样的方法为其他的按钮添加函数。

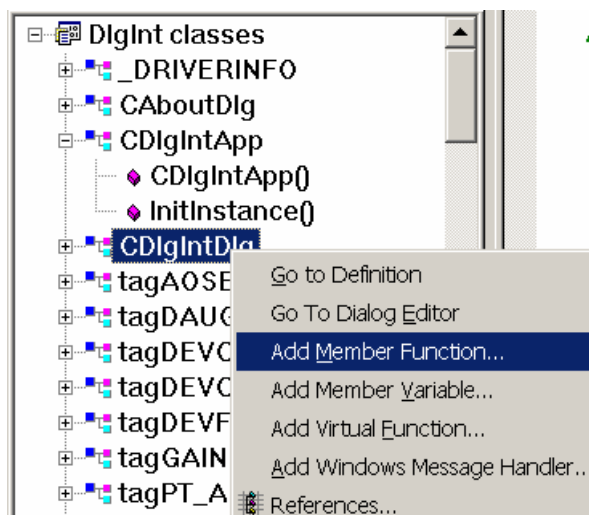
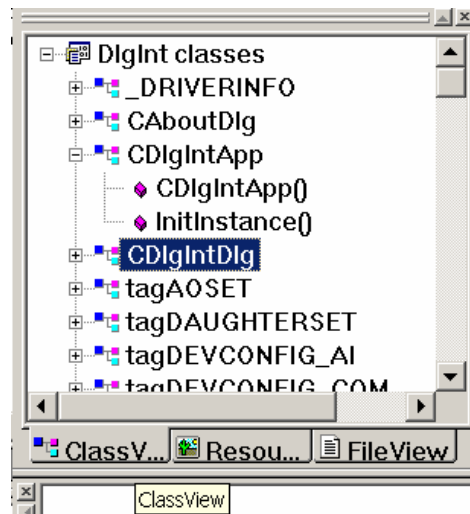
“退出程序”按钮，我们只是修改原来的“OK”按钮的按钮标题，其他的不作改动，其 ID 为：ID_OK，故添加的函数为 OnOK。函数名称方面，用户也可以根据自己的喜好并参照 VC 方面的书籍作修改。



图（14）添加函数的对话框

3) 定义变量:

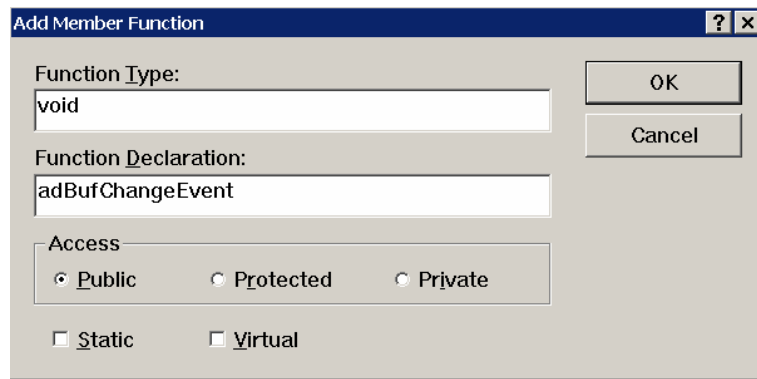
在工作区中的 ClassView 页中用鼠标右键点击 CDlgIntDlg, 在弹出的菜单中选择 Add Member Function...选项,



图（15）工作区 Workspace

图（16）添加成员函数

在 Add Member Function... 对话框中添加成员函数, `adInterruptEvent()`, `adBufChangeEvent()`, `adOverrunEvent()`, `adTerminateEvent()`, 接受对话框中的默认选项。



图（17）添加成员函数对话框

类似的方法添加一个成员变量：CListBox *pDataList。注：这变量需要在进行初始化：

BOOL CDlgIntDlg::OnInitDialog ()

```
{
.....

    // TODO: Add extra initialization here
    pDataList=(CListBox *)GetDlgItem(IDC_LIST1);
    return TRUE;  // return TRUE unless you set the focus to a control
}
```

3) 在 CDlgIntDlg.cpp 文件的开始部分添加下面的变量：

```
#include "stdafx.h"
#include "DlgInt.h"
#include "DlgIntDlg.h"

#ifdef _DEBUG
#define new DEBUG_NEW
#undef THIS_FILE
static char THIS_FILE[] = __FILE__;
#endif

//////////Add by programmer

static      DEVFEATURES      DevFeatures;          // structure for device features
static      PT_FAIntStart     ptFAIntStart;         // FAIntStart table
static      PT_FAITransfer     ptFAITransfer;        // FAITransfer table
static      PT_FAICheck        ptFAICheck;          // FAICheck table
static      PT_DeviceGetFeatures ptDevFeatures;     // Devfeatures table

static      PT_EnableEvent     ptEnableEvent;        // Enable event
static      PT_CheckEvent      ptCheckEvent;         // Check event
HANDLE      hEvent;
HANDLE      hThreadHandle;
DWORD       dwThreadID;

char        szErrMsg[80];          // Use for MESSAGEBOX function
char        szBuffer[40];          // Temporary buffer
```

```

LRESULT      ErrCde;                                // Return error code

USHORT      gwScanTime = 1000;                       // scan time
DWORD       gdwStartTime;                           // start time
DWORD       gdwElapseTime = 0;                      // elapse time
USHORT      gwOverrunCount = 0;                     // overrun count

static USHORT gwGain[16] = {0,0,0,0,0,0,0,0,
                          0,0,0,0,0,0,0,0};

static USHORT gwGainCode;
//static ULONG dwDeviceNum;                          // Device number

static LONG   DriverHandle = (LONG)NULL;             // driver handle
static USHORT gwDevice = 0, gwSubDevice = 0;         // device index
static SHORT  gnNumOfDevices, gnNumOfSubdevices;     // number of installed devices
static DWORD  gdwPacerRate = 100;                   // pacer rate
static ULONG  gulConvNum = 100;                     // conversion number
static USHORT gwStartChl = 0, gwStopChl = 0;         // scan channels
static USHORT gwDataType = 1;                       // display type : FLOAT(1)
static USHORT gwEvtFlag = 1;                        // event enable(0)
static USHORT gwFifoEnable = 0;                     // Fifo disable(0)

static USHORT gwGainList = 0;                       // gain list flag
static ULONG  gdwStartPt = 0, gdwStopPt = 99;        // display points
static USHORT gwExtTrig = 0;                        // external or internal trigger

static USHORT gwCyclicMode = 0;                     // cyclic or non-cyclic mode
static USHORT gwBufferMode = 0;                     // buffer: single or double
static USHORT gwIntrCount = 1;
static USHORT gwCyclicCount= 0;                     // number of cyclic count

static USHORT gwActiveBuf = 0;                      // return by FAICheck
static USHORT gwOverrun   = 0;                      // return by FAICheck, FAITransfer
static USHORT gwStopped   = 0;                      // return by FAICheck
static ULONG  gulRetrieved = 0;                      // return by FAICheck
static USHORT gwHalfReady = 0;                      // return by FAICheck

HGLOBAL hBuf, hUserBuf;
DWORD WINAPI UserThread();
CString strData[100];//用来存储采样数据的全局变量

.....

```

这时候我们可以看到在 CDlgIntDlg.cpp 文件的最后添加了这样的一些函数体, 我们可以

通过在这些函数体中添加代码实现我们需要的一些功能。

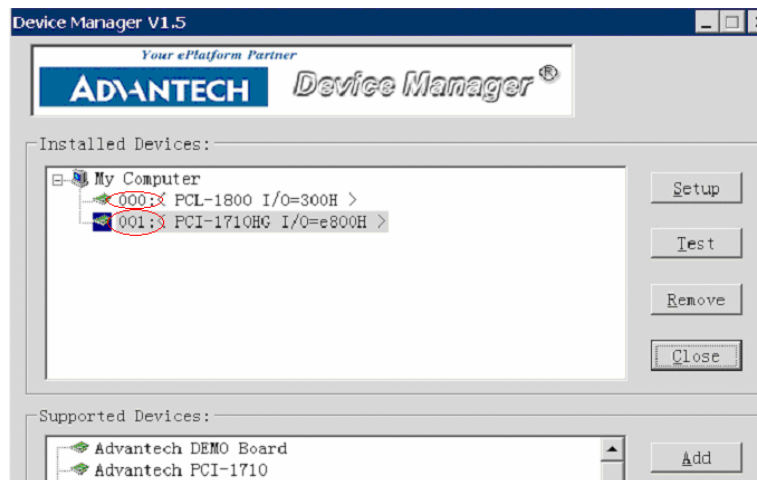
第五步：编写数据程序代码

```
void CDlgIntDlg::OnButton1()
{
    // TODO: Add your control notification handler code here
    // Step 1: Device open
    ErrCde = DRV_DeviceOpen(0,
        (LONG far *)&DriverHandle);
    if (ErrCde != SUCCESS)
    {
        DRV_GetErrorMessage(ErrCde,(LPSTR)szErrMsg);
        MessageBox((LPCSTR)szErrMsg,"Device Open",MB_OK);
        return ;
    }
}
```

设备编程第一步：打开设备。

打开设备的操作是在使用设备进行数据采集等编程操作之前必须进行的。

DRV_DeviceOpen(dwDeviceNum,(LONG far*)&DriverHandle)参数说明：dwDeviceNum是设备号，该号码在研华提供的 Device Manger 中可以查看到，图中椭圆标记中所示数字。



图（18）设备管理器

DriverHandle: 后面编程过程中对板卡的输入输出进行操作都将以该变量中的参数为基础，不同的板卡的使用也可以通过这个参数来区分调用。注意：每个 DRV_DeviceOpen 函数都必须对应一个 DRV_DeviceClose 函数来释放由其分配的内存。

该函数的返回值为一个设备打开成功/失败的标记：故可以通过 (ErrCde != SUCCESS) 判断设备是否成功打开。DRV_GetErrorMessage(ErrCde,(LPSTR)szErrMsg)函数通过 ErrCde 就可以取得打开设备过程中是那一部分出错，通过 MessageBox(hWnd,(LPCSTR)szErrMsg, "Device Open",MB_OK)显示出来。

```
// Step 2: Get device features
ptDevFeatures.buffer = (LPDEVFEATURES)&DevFeatures;
ptDevFeatures.size = sizeof(DEVFEATURES);
if ((ErrCde = DRV_DeviceGetFeatures(DriverHandle,
    (LPT_DeviceGetFeatures)&ptDevFeatures)) != SUCCESS)
{
    DRV_GetErrorMessage(ErrCde,(LPSTR)szErrMsg);
    MessageBox((LPCSTR)szErrMsg,"Driver Message",MB_OK);
    DRV_DeviceClose((LONG far *)&DriverHandle);
    return ;
}
```

设备编程第二步：取得设备设置。

首先初始化 ptDevFeatures 结构变量中的变量 buffer，size。然后调用 DRV_DeviceGetFeatures(DriverHandle,(LPT_DeviceGetFeatures)&ptDevFeatures)，通过 DriverHandle 参数取得该设备的设置。其他的语句进行出错判断，结构及含义同上面 (DRV_DeviceOpen) 类似，不再赘述。

注：如果在这里出错需要调用 DRV_DeviceClose((LONG far *)&DriverHandle)关闭设备。

```
// Step 3: Allocate memory for driver
if((hBuf=(USHORT far *)GlobalAlloc(GHND,
    sizeof(USHORT) * gulConvNum)) == 0)
{
    MessageBox("Not enough memory for buffer ",
        "High Speed",MB_OK);
    DRV_DeviceClose((LONG far *)&DriverHandle);
    return ;
}
```

设备编程第三步：驱动程序内存分配及锁定。

```
// Step 4: Allocate memory for Voltage data or Raw data
if((hUserBuf=(FLOAT far *)GlobalAlloc(GHND,
    sizeof(FLOAT) * gulConvNum)) == 0)
{
    MessageBox("Not enough memory for buffer ",
        "High Speed",MB_OK);
    GlobalFree(hBuf);
    DRV_DeviceClose((LONG far *)&DriverHandle);
    return ;
}
```

设备编程第四步：用户数据采集所需变量的内存分配及锁定。

```
// Prepare some informations to run
ptFAIIntStart.buffer = (USHORT far *)GlobalLock(hBuf);
ptFAITransfer.DataBuffer = (FLOAT far *)GlobalLock(hUserBuf);
```

这两条语句用设备编程三四步中准备好的变量来初始化 ptFAIIntStart 结构体中的变量。

// Step 5: Enable event feature

```
ptEnableEvent.EventType = ADS_EVT_INTERRUPT |
    ADS_EVT_BUFCHANGE |
    ADS_EVT_TERMINATED |
    ADS_EVT_OVERRUN;
ptEnableEvent.Enabled = gwEvtFlag;
ptEnableEvent.Count = 512;
if ((ErrCde = DRV_EnableEvent(DriverHandle,
    (LPT_EnableEvent)&ptEnableEvent)) != 0)
{
    DRV_GetErrorMessage(ErrCde, (LPSTR)szErrMsg);
    MessageBox((LPCSTR)szErrMsg, "Driver Message", MB_OK);
    GlobalUnlock(hBuf);
    GlobalUnlock(hUserBuf);
    GlobalFree(hBuf);
    GlobalFree(hUserBuf);
    DRV_DeviceClose((LONG far *)&DriverHandle);
    return ;
}
```

设备编程第五步：确定中断事件类型，并设置其可以进行中断。

注：如果程序在这里出错，则不仅要关闭设备，而且在关闭设备的操作之间必须解锁和释放前面定义和锁定的变量。

// Step 6: Start Interrupt transfer

```
ptFAIIntStart.TrigSrc = gwExtTrig;
ptFAIIntStart.SampleRate = gdwPacerRate;
ptFAIIntStart.chan = gwStartChl;
ptFAIIntStart.gain = DevFeatures.glGainList[gwGainCode].usGainCde;
ptFAIIntStart.count = gulConvNum;
ptFAIIntStart.cyclic = gwCyclicMode;
    if (gwFifoEnable)
        ptFAIIntStart.IntrCount = FIFO_SIZE;
    else
        ptFAIIntStart.IntrCount = 1;

if ((ErrCde = DRV_FAIIntStart(DriverHandle,
    (LPT_FAIIntStart)&ptFAIIntStart)) != 0)
{
    DRV_GetErrorMessage(ErrCde, (LPSTR)szErrMsg);
    MessageBox((LPCSTR)szErrMsg, "Driver Message", MB_OK);
    GlobalUnlock(hBuf);
    GlobalUnlock(hUserBuf);
    GlobalFree(hBuf);
    GlobalFree(hUserBuf);
}
```

```

        DRV_DeviceClose((LONG far *)&DriverHandle);
        return ;
    }
}

设备编程第六步：开始中断模式数据传输
    ptFAIntStart.TrigSrc    = gwExtTrig;—设置触发源（0，内部；1，外部）；
    ptFAIntStart.SampleRate = gdwPacerRate;—采样率(s/s)；
    ptFAIntStart.chan      = gwStartChl;—采样通道；
    ptFAIntStart.gain      = DevFeatures.glGainList[gwGainCode].usGainCde;—增益设
置；

    ptFAIntStart.count     = gulConvNum;—A/D 转换次数；
    ptFAIntStart.cyclic    = gwCyclicMode;—是否循环（0—非循环，1—循环）；
    if (gwFifoEnable)根据是否有 FIFO（由硬件决定）及其大小来确定数据传输时的中
断。

        ptFAIntStart.IntrCount = FIFO_SIZE;
    else
        ptFAIntStart.IntrCount = 1;

    gwCyclicCount = 0;
    上面的准备工作完成之后，如果时间类型允许中断，就可以创建用户线程进行数据采集。
    // if event feature is enabled , then create one thread.
    if (gwEvtFlag)
    {
        hThreadHandle = CreateThread(0, 0,
            (LPTHREAD_START_ROUTINE)UserThread,
            0, 0, (LPDWORD)&dwThreadID);
    }

    Sleep(0);
}

```

用户线程及其调用函数说明：

用户线程：

```

UINT UserThread(LPVOID pParam)
{
    USHORT usEventType;
    LONG   ErrCde;
    DWORD  TickValue;

    TickValue = GetTickCount();
    TickValue = TickValue + 500000;

    while(TickValue > GetTickCount())

```

```

{
    // Check message
    ptCheckEvent.EventType = &usEventType;
    //ptCheckEvent.Milliseconds = 5000;
    if ( gwCyclicMode )
    {
        ptCheckEvent.Milliseconds = INFINITE;
    }
    else
    {
        ptCheckEvent.Milliseconds = 1000 * (gulConvNum /
gdwPacerRate)+1000;
    }
    if ((ErrCde = DRV_CheckEvent(DriverHandle,
        (LPT_CheckEvent)&ptCheckEvent)) != 0)
    {
        MessageBox(hWnd, "Check Event Error !","Thread Message",MB_OK);
        return 0 ;
    }

    // Process interrupt event
    if (usEventType & ADS_EVT_INTERRUPT)
        adInterruptEvent();//

    // Process buffer change event
    if (usEventType & ADS_EVT_BUFCHANGE)
        adBufChangeEvent();

    // Process overrun event
    if (usEventType & ADS_EVT_OVERRUN)
        adOverrunEvent();

    // Process terminate event
    if (usEventType & ADS_EVT_TERMINATED)
    {
        adTerminateEvent();
        return 0 ;
    }
}

return 0;
}
调用函数:
void CDlgIntDlg::adInterruptEvent()

```

```

{
    return;
}

```

```

//-----

```

```

void adBufChangeEvent()

```

```

{
    ptFAICheck.ActiveBuf = &gwActiveBuf;
    ptFAICheck.stopped    = &gwStopped;
    ptFAICheck.retrieved  = &gulRetrieved;
    ptFAICheck.overrun    = &gwOverrun;
    ptFAICheck.HalfReady  = &gwHalfReady;

```

1)初始化 PT_FAIChek 结构并调用 DRV_FAICheck 函数检查和返回数据采集的当前状态:

```

    ptFAICheck.ActiveBuf = &gwActiveBuf;—已激活的 buffer
    ptFAICheck.stopped    = &gwStopped;—指示操作是否结束 (1—>结束, 0 —>未结束)
    ptFAICheck.retrieved  = &gulRetrieved;—内存中保存的 A/D 转换次数
                                ptFAICheck.overrun    = &gwOverrun;
                                —内存中的数据是否超过循环模式要求的量,即是否已经是第二次以后的循环。
    ptFAICheck.HalfReady  = &gwHalfReady;—一半满标志;

```

```

    if ((ErrCde = DRV_FAICheck(DriverHandle,
        (LPT_FAICheck)&ptFAICheck)) != 0)
    {
        DRV_GetErrorMessage(ErrCde,(LPSTR)szErrMsg);
        MessageBox(hWnd,(LPCSTR)szErrMsg,"Thread Message",MB_OK);
        return ;
    }

```

```

    if (gwHalfReady == 1) // first ready
    {
        ptFAITransfer.ActiveBuf = 0;    // single buffer
        ptFAITransfer.DataType = gwDataType;
        ptFAITransfer.start      = 0;
        ptFAITransfer.count      = gulConvNum/2;
        ptFAITransfer.overrun    = &gwOverrun;

        if ((ErrCde = DRV_FAITransfer(DriverHandle,
            (LPT_FAITransfer)&ptFAITransfer)) != 0)
        {
            DRV_GetErrorMessage(ErrCde,(LPSTR)szErrMsg);
            MessageBox(hWnd,(LPCSTR)szErrMsg,"Thread Message",MB_OK);

```

```

        return ;
    }
}
else if (gwHalfReady == 2) // second half ready
{
    ptFAITransfer.ActiveBuf = 0;    // single buffer
    ptFAITransfer.DataType = gwDataType;
    ptFAITransfer.start      = gulConvNum/2;
    ptFAITransfer.count      = gulConvNum/2;
    ptFAITransfer.overrun    = &gwOverrun;

    if ((ErrCde = DRV_FAITransfer(DriverHandle,
        (LPT_FAITransfer)&ptFAITransfer)) != 0)
    {
        DRV_GetErrorMessage(ErrCde,(LPSTR)szErrMsg);
        MessageBox(hWnd,(LPCSTR)szErrMsg,"Thread Message",MB_OK);
        return ;
    }
}

```

2) 调用 DRV_FAITransfer 函数进行数据采集。ActiveBuffer=0/1 表示是单 Buffer/双 Buffer; DataType=0/1 表示数据类型为无符号整型/浮点型; start/count 表示向数据 Buffer 拷贝数据的起始/终止位置。Overrun 表示是否为覆盖模式。初始化结束之后即可进行传输。

```

        gwCyclicCount ++;
    }

    return;
}
//-----
void adOverrunEvent()
{
    // clear overrun

    if ((ErrCde = DRV_ClearOverrun(DriverHandle)) != 0)
    {
        DRV_GetErrorMessage(ErrCde,(LPSTR)szErrMsg);
        MessageBox(hWnd,(LPCSTR)szErrMsg,"Thread Message",MB_OK);
        return ;
    }

    return;
}
//-----
void adTerminateEvent()
{
    A/D 转化结束事件处理程序
}

```

1) 停止 A/D 转换

```
// Stop A/D conversion
if ((ErrCde = DRV_FAIStop(DriverHandle)) != 0)
{
    DRV_GetErrorMessage(ErrCde,(LPSTR)szErrMsg);
    MessageBox(hWnd,(LPCSTR)szErrMsg,"Thread Message",MB_OK);
    return ;
}
```

2) 从内存中取得电压值。

```
// Get real voltage of buffer from driver
ptFAITransfer.ActiveBuf = 0;    // single buffer
ptFAITransfer.DataType = gwDataType;
ptFAITransfer.start      = 0;
ptFAITransfer.count      = gulConvNum;
ptFAITransfer.overrun    = &gwOverrun;
if ((ErrCde = DRV_FAITransfer(DriverHandle,
    (LPT_FAITransfer)&ptFAITransfer)) != 0)
{
    DRV_GetErrorMessage(ErrCde,(LPSTR)szErrMsg);
    MessageBox(hWnd,(LPCSTR)szErrMsg,"Thread Message",MB_OK);
    return ;
}
```

3) 通过数组取得数据

```
ULONG i;
LPVOID temp;
CString str;
for(i=0;i<100;i++)
{
    strData[i]="Initial string";
}
if (gwDataType == 0)
    temp = (USHORT far *)ptFAITransfer.DataBuffer;
else
    temp = (float far *)ptFAITransfer.DataBuffer;
for(i = gdwStartPt; i <= gdwStopPt; i++)
{
    if (gwDataType == 0)
        str.Format("Buf[%ld]=%4x",i,((USHORT far *)temp)[i]);
    else
        str.Format("Buf[%ld]=%10.6f",i,((float far *)temp)[i]);

    strData[i]=str;    //为显示字符串数组赋值
}
```

```

        //MessageBox(hWnd,str,MB_OK);//测试数据结果。
    }
    //////////////////////////////////////
    // Free buffer

    GlobalUnlock(hBuf);
    GlobalUnlock(hUserBuf);
    GlobalFree(hBuf);
    GlobalFree(hUserBuf);

    // Close driver
    DRV_DeviceClose((LONG far *)&DriverHandle);

    return;
}

```

第六步：编写停止采集数据和退出程序按钮。

```

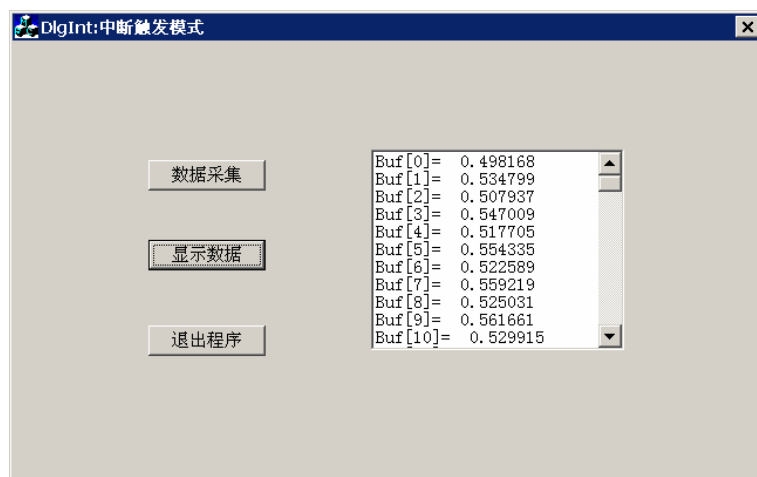
void CDlgIntDlg::OnButton2()
{
    // TODO: Add your control notification handler code here
    for(int i=0;i<100;i++)
    {
        pDataList->AddString(strData[i]); //在列表框中显示数据
    }
}

void CDlgIntDlg::OnOK()
{
    // TODO: Add extra validation here
    if(DriverHandle!=NULL)退出程序时，如果设备句柄没有释放则在此释放。
    DRV_DeviceClose((LONG far *)&DriverHandle);
    CDialog::OnOK();
}

```

第七步：运行程序

好了，就是这么简单，运行一下程序结果就是下面样子的，采集到的数据不断的加入到列表框中。!



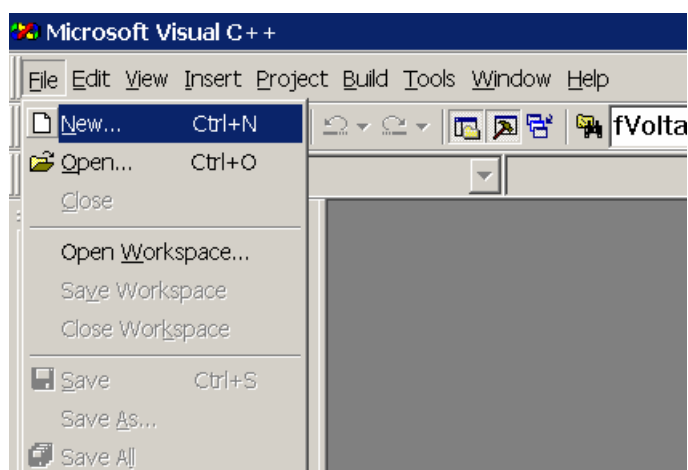
图（19）程序运行结果

三、应用 DMA 进行数据采集 addma

安装板卡并用 device manger 调试无误之后就可以开始编程工作。

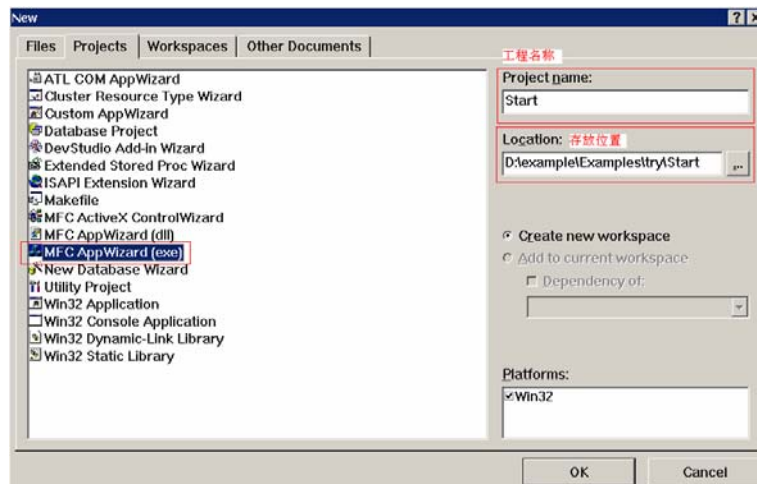
第一步：创建一个 Dialog Based 程序框架。

1) 我们需要新建一个工程：点击 File->New 菜单，图（1）所示，



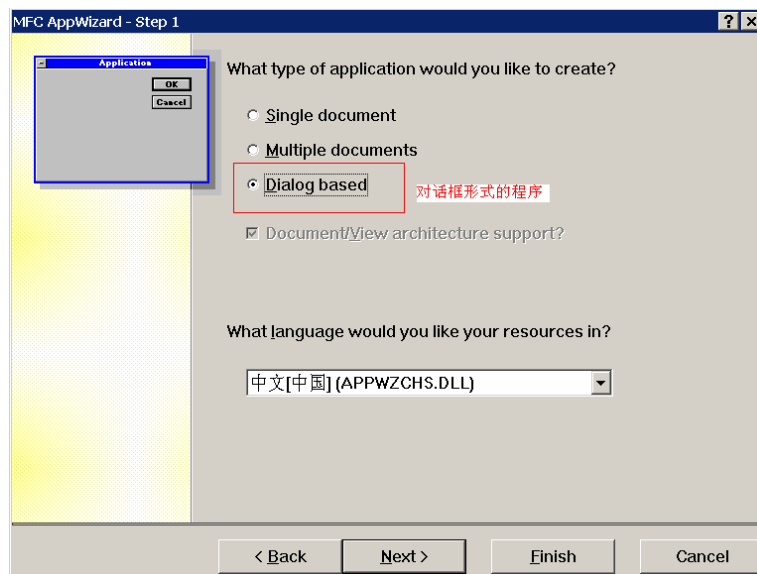
图（1）

2) 在弹出的 NEW 对话框（图（2））选中 MFC Application(exe)选项，在 project name 中输入要创建的工程名称，例如 Start；在下面的 Location 中可以选择要保存这个工程文件夹的路径。



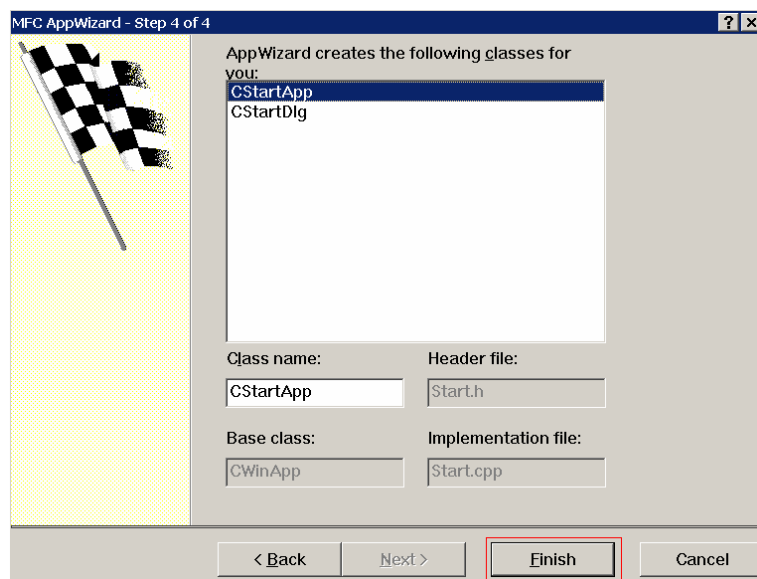
图（2）New 对话框

3) 点击 OK 按钮进入下一步 MFC Wizard Step1, 如下图所示: 选择程序类型为: Dialog based, 即对话框形式的程序; 选择之后点击 Next 按钮进入下一步。



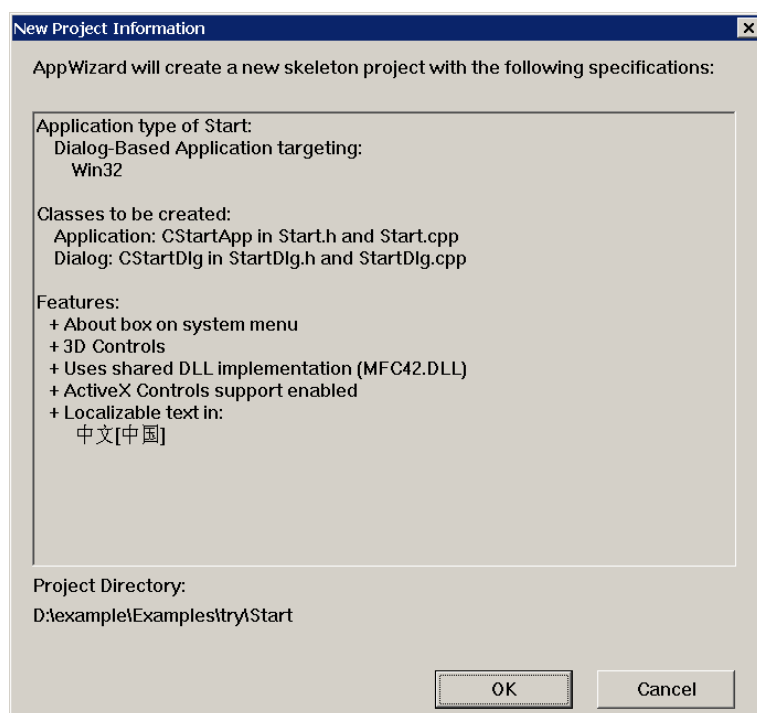
图（3）Step1

4) 下面的对话框中依次点击 Next 按钮, 即: 接受所有的默认选项; 最后一项, 如下所示:



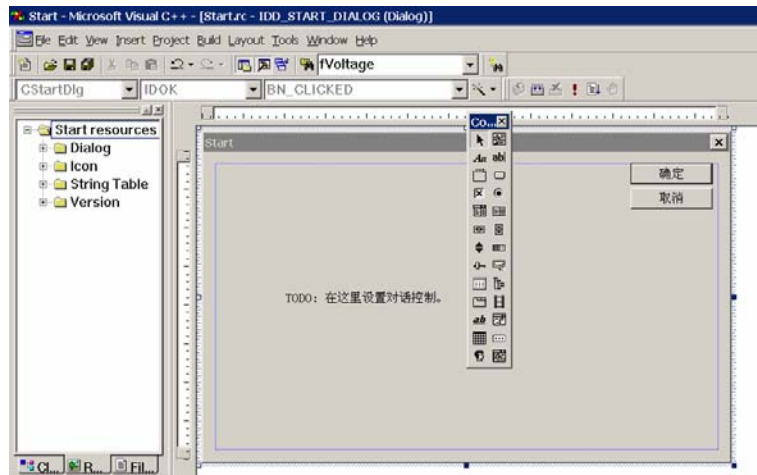
图（4）接受默认项到最后

5) 点击 Finish 按钮。弹出如下对话框，给出了这个新建工程的主要信息。点击 OK 按钮完成新工程的创建工作。通过对话框编辑期，就可以对这个对话框进行编辑；首先我们可以删掉上面的（Satic）静态文本框“TODO: 在这里设置对话控制”；



图（5）工程信息

6) 点击 OK 按钮之后的 Visual C++编译器的界面如下图：



图（6）

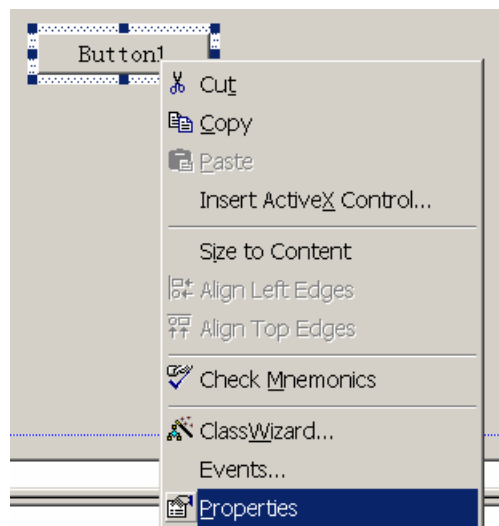
第二步：编辑对话框

4) 加入命令按钮和然后选择控件中的命令按钮，在对话框上单击或是按住并拖动鼠标左键就可以看到一个命令按钮。



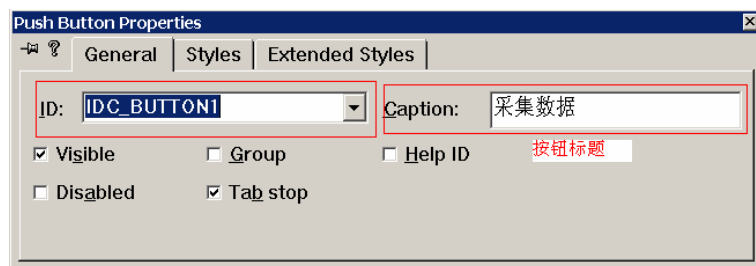
图（7）

2) 修改按钮的标题，在这个命令按钮上单击鼠标右键，如下图所示：



图（8）

选择其中的属性 Properties 项弹出该按钮属性对话框：

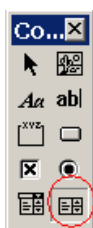


图（9）属性对话框

在这里为了简单明了，我们只修改这个“按钮标题”，其他的选项都保持系统默认的不

变。

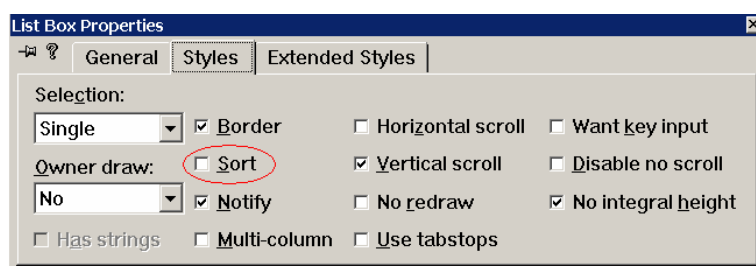
3) 在对话框上添加列表框 (ListBox)，用来显示我们采集到的数据：



图（10）选择列表框

为了能够按照采集的顺序显示数据修改列表框的属性，在 ListBox 的属性框里面，选

Styles 页取消列表框的自动排序选项，Sort 项前面没有选中标志（即：去掉对钩）



图（11）ListBox 的属性页

通过上述的方法，编辑对话框窗口布局如下图所示为止：



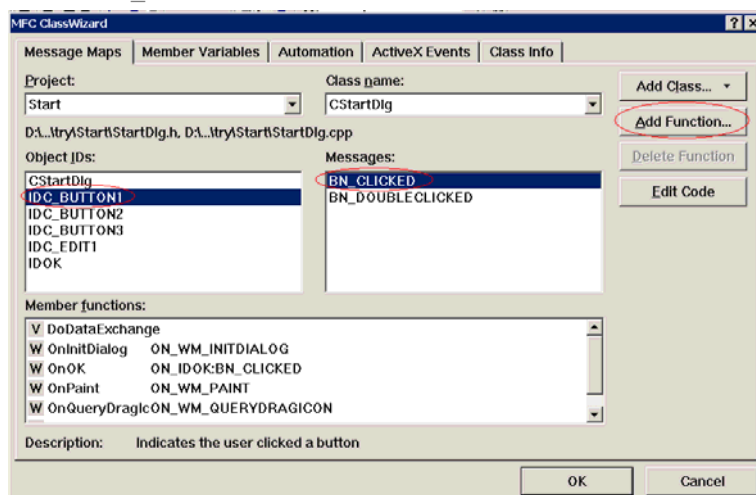
图（12）程序窗口布局

第三步：导入外部头文件和函数库。

导入方法见手册。另外一种比较简单的做法是把 adsapi32.lib,driver.h,os.h 直接拷贝到这个工程所在的文件夹中。

第四步：定义变量和函数。

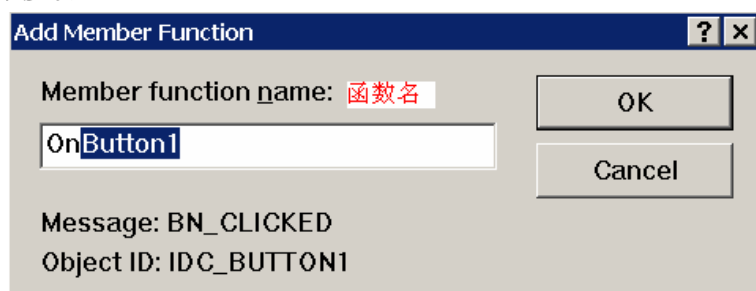
1) 为命令按钮添加函数：选择 ClassWizard 中的 Message Map 属性页，来为按钮单击操作添加相对应的函数，方法如图中所示：选中 IDC_BUTTON1(标题为：采集数据，Data Acquisition)，同时选中 BN_CLICKED，单击 Add Function...按钮，



图（13）ClassWizard 对话框

弹出添加函数的对话框：

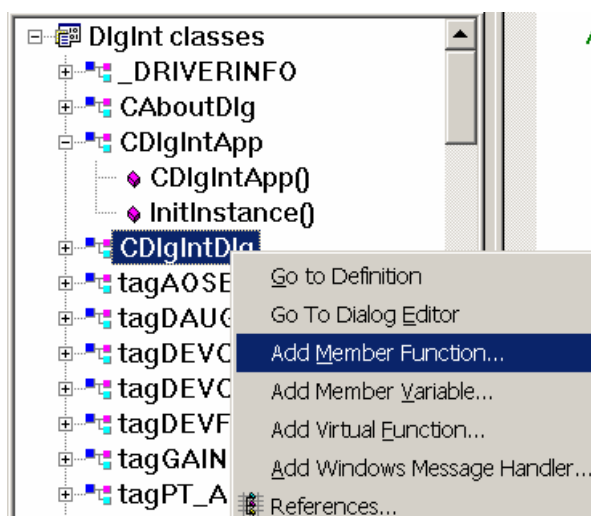
函数名称与该按钮的 ID 相对应，IDC_BUTTON1⇔OnButton1，当然也可以有别的作法，我们在这里选择默认的函数名称，点击 OK 添加函数，同样的方法为其他的按钮添加函数。“退出程序”按钮，我们只是修改原来的“OK”按钮的按钮标题，其他的不作改动，其 ID 为：ID_OK，故添加的函数为 OnOK。函数名称方面，用户也可以根据自己的喜好并参照 VC 方面的书籍作修改。



图（14）添加函数的对话框

5) 定义变量：

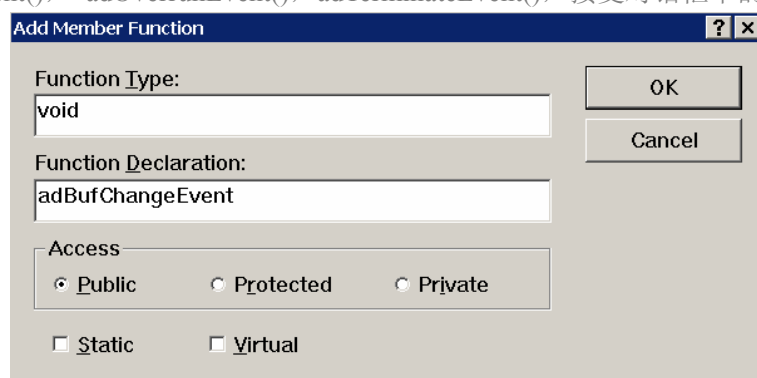
在工作区中的 ClassView 页中用鼠标右键点击 CDlgIntDlg，在弹出的菜单中选择 Add Member Function...选项，



图（15）工作区 Workspace

图（16）添加成员函数

在 Add Member Function... 对话框中添加成员函数，adInterruptEvent()，adBufChangeEvent()，adOverrunEvent()，adTerminateEvent()，接受对话框中的默认选项。



图（17）添加成员函数对话框

类似的方法添加一个成员变量：CListBox *pDataList。注：这变量需要在进行初始化：
 BOOL CDlgIntDlg::OnInitDialog ()

```
{
.....
// TODO: Add extra initialization here
```

```

        pDataList=(CListBox *)GetDlgItem(IDC_LIST1);
        return TRUE; // return TRUE unless you set the focus to a control
    }
}
3) 在 CDlgIntDlg.cpp 文件的开始部分添加下面的变量:
//////////Add by programmer

static      DEVFEATURES      DevFeatures;          // structure for device features
static      PT_FAIDmaStart    ptFAIDmaStart;        // FAIDmaStart table
static      PT_FAITransfer    ptFAITransfer;        // FAITransfer table
static      PT_FAICheck       ptFAICheck;          // FAICheck table
static      PT_DeviceGetFeatures ptDevFeatures;    // Devfeatures table
static      PT_AllocateDMABuffer ptAllocateDMABuffer; // buffer table

static      PT_EnableEvent     ptEnableEvent;       // Enable event
static      PT_CheckEvent      ptCheckEvent;        // Check event
HANDLE      hEvent;
HANDLE      hThreadHandle;
DWORD       dwThreadID;

char        szErrMsg[80];                          // Use for MESSAGEBOX function
char        szBuffer[40];                          // Temporary buffer
LRESULT     ErrCde;                                // Return error code

BOOL        bRun = FALSE;                          // flag for running
USHORT      gwScanTime = 1000;                      // scan time
DWORD       gdwStartTime;                          // start time
DWORD       gdwElapseTime = 0;                     // elapse time
USHORT      gwOverrunCount = 0;                    // overrun count

static      USHORT gwGain[16] = {0,0,0,0,0,0,0,0,
                                0,0,0,0,0,0,0,0};

static      USHORT gwGainCode;
static      ULONG   dwDeviceNum;                    // Device number

static      LONG    DriverHandle = (LONG)NULL;      // driver handle
static      USHORT  gwDevice = 0, gwSubDevice = 0;  // device index
static      SHORT   gnNumOfDevices, gnNumOfSubdevices; // number of installed devices
static      DWORD   gdwPacerRate = 4000;           // pacer rate
static      ULONG   gulConvNum = 4000;             // conversion number
static      USHORT  gwStartChl = 0, gwStopChl = 0;  // scan channels
static      USHORT  gwDataType = 1;                // display type : FLOAT(1)
static      USHORT  gwEvtFlag = 1;                 // event enable(0)
static      ULONG   gwActualBufSize = 0;           // actual allocate buffer size
static      USHORT  gwFifoEnable = 0;              // Fifo disable(0)

```

```

static USHORT gwGainList = 0;      // gain list flag
static ULONG  gdwStartPt = 0, gdwStopPt = 99;    // display points
static USHORT gwExtTrig = 0;      // external or internal trigger

static USHORT gwCyclicMode = 0;    // cyclic or non-cyclic mode
static USHORT gwBufferMode = 0;    // buffer: single or double
static USHORT gwIntrCount = 1;
static USHORT gwCyclicCount= 0;    // number of cyclic count

static USHORT gwActiveBuf = 0;     // return by FAICheck
static USHORT gwOverrun   = 0;     // return by FAICheck, FAITransfer
static USHORT gwStopped   = 0;     // return by FAICheck
static ULONG  gulRetrieved = 0;     // return by FAICheck
static USHORT gwHalfReady = 0;     // return by FAICheck
LONG pBuffer;

HGLOBAL hBuf, hUserBuf;
DWORD WINAPI UserThread();

CString strData[100];

//////////Above are added by user
.....

```

这时候我们可以看到在 CDlgIntDlg.cpp 文件的最后添加了这样的一些函数体, 我们可以通过在这些函数体中添加代码实现我们所需要的一些功能。

第五步：编写数据程序代码

```

void CDlgIntDlg::OnButton1()
{
    // TODO: Add your control notification handler code here
    // Step 1: Device open
    ErrCde = DRV_DeviceOpen(0,
        (LONG far *)&DriverHandle);
    if (ErrCde != SUCCESS)
    {
        DRV_GetErrorMessage(ErrCde,(LPSTR)szErrMsg);
        MessageBox((LPCSTR)szErrMsg,"Device Open",MB_OK);
        return ;
    }
}

```

设备编程第一步：打开设备。

打开设备的操作是在使用设备进行数据采集等编程操作之前必须进行的。

DRV_DeviceOpen(dwDeviceNum,(LONG far*)&DriverHandle)参数说明: dwDeviceNum 是设备号, 该号码在研华提供的 Device Manger 中可以查看到, 图中椭圆标记中所示数字。

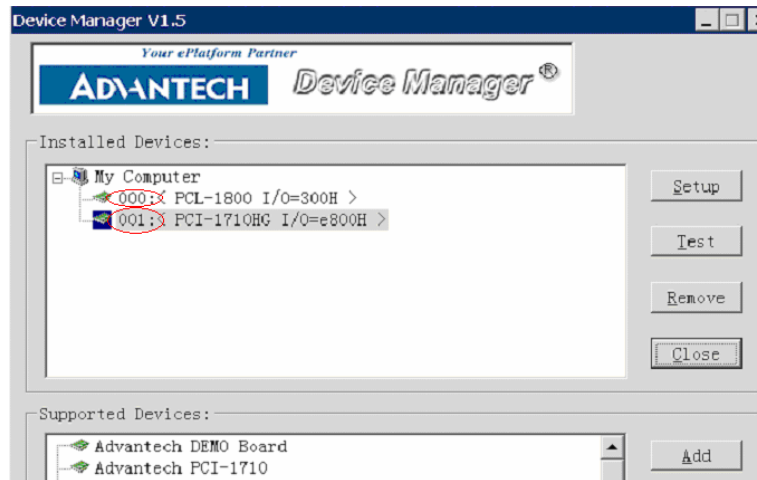


图 (18) 设备管理器

DriverHandle: 后面编程过程中对板卡的输入输出进行操作都将以该变量中的参数为基础, 不同的板卡的使用也可以通过这个参数来区分调用。注意: 每个 DRV_DeviceOpen 函数都必须对应一个 DRV_DeviceClose 函数来释放由其分配的内存。

该函数的返回值为一个设备打开成功/失败的标记: 故可以通过 (ErrCde != SUCCESS) 判断设备是否成功打开。DRV_GetErrorMessage(ErrCde,(LPSTR)szErrMsg)函数通过 ErrCde 就可以取得打开设备过程中是那一部分出错, 通过 MessageBox(hWnd,(LPCSTR)szErrMsg, "Device Open",MB_OK)显示出来。

```
// Step 2: Get device features
ptDevFeatures.buffer = (LPDEVFEATURES)&DevFeatures;
ptDevFeatures.size = sizeof(DEVFEATURES);
if ((ErrCde = DRV_DeviceGetFeatures(DriverHandle,
    (LPT_DeviceGetFeatures)&ptDevFeatures)) != SUCCESS)
{
    DRV_GetErrorMessage(ErrCde,(LPSTR)szErrMsg);
    MessageBox((LPCSTR)szErrMsg,"Driver Message",MB_OK);
    DRV_DeviceClose((LONG far *)&DriverHandle);
    return ;
}
```

设备编程第二步: 取得设备设置。

首先初始化 ptDevFeatures 结构变量中的变量 buffer, size。然后调用 DRV_DeviceGetFeatures(DriverHandle,(LPT_DeviceGetFeatures)&ptDevFeatures), 通过 DriverHandle 参数取得该设备的设置。其他的语句进行出错判断, 结构及含义同上面 (DRV_DeviceOpen) 类似, 不再赘述。

注: 如果在这里出错需要调用 DRV_DeviceClose((LONG far *)&DriverHandle)关闭设备。

```
// Step 3: Allocate DMA buffer for DMA transfer
```

```

ptAllocateDMABuffer.CyclicMode = gwCyclicMode;
ptAllocateDMABuffer.RequestBufSize = gulConvNum * 2;
ptAllocateDMABuffer.ActualBufSize = &gwActualBufSize;
ptAllocateDMABuffer.buffer = &pBuffer;
if ((ErrCde = DRV_AllocateDMABuffer(DriverHandle,
    (LPT_AllocateDMABuffer)&ptAllocateDMABuffer)) != SUCCESS)
{

    DRV_GetErrorMessage(ErrCde,(LPSTR)szErrMsg);
    MessageBox((LPCSTR)szErrMsg,"Driver Message",MB_OK);
    DRV_DeviceClose((LONG far *)&DriverHandle);
    return ;
}

```

设备编程第三步：为 DMA buffer 分配内存

初始化 ptAllocateDMABuffer 结构：

```

    ptAllocateDMABuffer.CyclicMode = gwCyclicMode;—是否采用循环模式（0，
    非循环；1，循环）
    ptAllocateDMABuffer.RequestBufSize = gulConvNum * 2;—需要的 buffer
大小；
    ptAllocateDMABuffer.ActualBufSize = &gwActualBufSize;—实际能够分配
的 buffer 大小。
    ptAllocateDMABuffer.buffer = &pBuffer;—内存地址。

```

// Step 4: Allocate memory for Voltage data or Raw data

```

if((hUserBuf=(FLOAT far *)GlobalAlloc(GHND,
    sizeof(FLOAT) * gulConvNum)) == 0)
{
    MessageBox("Not enough memory for buffer ",
        "High Speed",MB_OK);
    DRV_FreeDMABuffer(DriverHandle, (LPARAM)&pBuffer);
    DRV_DeviceClose((LONG far *)&DriverHandle);
    return ;
}
ptFAITransfer.DataBuffer = (FLOAT far *)GlobalLock(hUserBuf);

```

设备编程第四步：为电压数据或原始数据分配内存。

// Prepare some informations to run

```

ptFAITransfer.DataBuffer = (FLOAT far *)GlobalLock(hUserBuf);

```

这两条语句用设备编程三四步中准备好的变量来初始化 ptFAIIntStart 结构体中的变量。

// Step 5: Enable event feature

```

ptEnableEvent.EventType = ADS_EVT_INTERRUPT |
    ADS_EVT_BUFCHANGE |
    ADS_EVT_TERMINATED |
    ADS_EVT_OVERRUN;

```

```

ptEnableEvent.Enabled = gwEvtFlag;
ptEnableEvent.Count    = 512;

if ((ErrCde = DRV_EnableEvent(DriverHandle,
    (LPT_EnableEvent)&ptEnableEvent)) != 0)
{
    DRV_GetErrorMessage(ErrCde,(LPSTR)szErrMsg);
    MessageBox((LPCSTR)szErrMsg,"Driver Message",MB_OK);
    GlobalUnlock(hUserBuf);
    GlobalFree(hUserBuf);
    DRV_FreeDMABuffer(DriverHandle, (LPARAM)&pBuffer);
    DRV_DeviceClose((LONG far *)&DriverHandle);
    return ;
}

```

设备编程第五步：确定中断事件类型，并设置其可以进行中断。

注：如果程序在这里出错，则不仅要关闭设备，而且在关闭设备的操作之间必须解锁和释放前面定义和锁定的变量。

// Step 6: Start DMA transfer

```

ptFAIDmaStart.TrigSrc    = gwExtTrig;
ptFAIDmaStart.SampleRate = gdwPacerRate;
ptFAIDmaStart.chan      = gwStartChl;
ptFAIDmaStart.gain      = DevFeatures.glGainList[gwGainCode].usGainCde;
ptFAIDmaStart.count     = gulConvNum;
ptFAIDmaStart.buffer = (USHORT far *)pBuffer;

if ((ErrCde = DRV_FAIDmaStart(DriverHandle,
    (LPT_FAIDmaStart)&ptFAIDmaStart)) != 0)
{
    DRV_GetErrorMessage(ErrCde,(LPSTR)szErrMsg);
    MessageBox((LPCSTR)szErrMsg,"Driver Message",MB_OK);
    GlobalUnlock(hUserBuf);
    GlobalFree(hUserBuf);
    DRV_FreeDMABuffer(DriverHandle, (LPARAM)&pBuffer);
    DRV_DeviceClose((LONG far *)&DriverHandle);
    return ;
}

```

设备编程第六步：开始 DMA 模式数据传输

```

ptFAIDmaStart.TrigSrc    = gwExtTrig;—设置触发源（0，内部；1，外部）；
ptFAIDmaStart.SampleRate = gdwPacerRate;—采样率(s/s)；
ptFAIDmaStart.chan      = gwStartChl;—采样通道；
ptFAIDmaStart.gain      = DevFeatures.glGainList[gwGainCode].usGainCde;—增益

```

设置：

```

ptFAIDmaStart.count     = gulConvNum;—A/D 转换次数；
ptFAIDmaStart.buffer = (USHORT far *)pBuffer;—数据内存；

```

```

if ((ErrCde = DRV_FAIDmaStart(DriverHandle,
    (LPT_FAIDmaStart)&ptFAIDmaStart)) != 0)
{
    DRV_GetErrorMessage(ErrCde,(LPSTR)szErrMsg);
    MessageBox((LPCSTR)szErrMsg,"Driver Message",MB_OK);
    GlobalUnlock(hUserBuf);
    GlobalFree(hUserBuf);
    DRV_FreeDMABuffer(DriverHandle, (LPARAM)&pBuffer);
    DRV_DeviceClose((LONG far *)&DriverHandle);
    return ;
}
gwCyclicCount = 0;

```

上面的准备工作完成之后,如果时间类型允许中断,就可以创建用户线程进行数据采集。

```

// if event feature is enabled , then create one thread.
if (gwEvtFlag)
{
    hThreadHandle = CreateThread(0, 0,
        (LPTHREAD_START_ROUTINE)UserThread,
        0, 0, (LPDWORD)&dwThreadID);
}

Sleep(0);
}

```

用户线程及其调用函数说明:

用户线程:

```

UINT UserThread(LPVOID pParam)
{
    USHORT usEventType;
    LONG ErrCde;
    DWORD TickValue;

    TickValue = GetTickCount();
    TickValue = TickValue + 500000;

    while(TickValue > GetTickCount())
    {
        // Check message
        ptCheckEvent.EventType = &usEventType;
        //ptCheckEvent.Milliseconds = 5000;
        if ( gwCyclicMode )
        {

```

```

        ptCheckEvent.Milliseconds = INFINITE;
    }
else
    {
        ptCheckEvent.Milliseconds = 1000 * (gulConvNum /
gdwPacerRate)+1000;
    }
if ((ErrCde = DRV_CheckEvent(DriverHandle,
(LPT_CheckEvent)&ptCheckEvent)) != 0)
{
    MessageBox(hWnd,"Check Event Error !","Thread Message",MB_OK);
    return 0 ;
}

CDlgIntDlg dlg=new CDlgIntDlg;
// Process interrupt event
if (usEventType & ADS_EVT_INTERRUPT)
    dlg.adInterruptEvent();

// Process buffer change event
if (usEventType & ADS_EVT_BUFCHANGE)
    dlg.adBufChangeEvent();

// Process overrun event
if (usEventType & ADS_EVT_OVERRUN)
    dlg.adOverrunEvent();

// Process terminate event
if (usEventType & ADS_EVT_TERMINATED)
{
    dlg.adTerminateEvent();
    return 0 ;
}
delete(dlg);
}

return 0;
}
调用函数:
void CDlgIntDlg::adInterruptEvent()
{
    return;
}

```

```
//-----
```

```
void adBufChangeEvent()
```

```
{
    ptFAICheck.ActiveBuf = &gwActiveBuf;
    ptFAICheck.stopped    = &gwStopped;
    ptFAICheck.retrieved  = &gulRetrieved;
    ptFAICheck.overrun    = &gwOverrun;
    ptFAICheck.HalfReady  = &gwHalfReady;
```

1)初始化 PT_FAIChek 结构并调用 DRV_FAICheck 函数检查和返回数据采集的当前状态:

```
    ptFAICheck.ActiveBuf = &gwActiveBuf;—已激活的 buffer
    ptFAICheck.stopped    = &gwStopped;—指示操作是否结束（1—>结束，0 —>未结束）
    ptFAICheck.retrieved  = &gulRetrieved;—内存中保存的 A/D 转换次数
                                ptFAICheck.overrun    = &gwOverrun;
                                —内存中的数据是否超过循环模式要求的量，即
                                是否已经是第二次以后的循环。
    ptFAICheck.HalfReady  = &gwHalfReady;—一半满标志；
```

```
    if ((ErrCde = DRV_FAICheck(DriverHandle,
        (LPT_FAICheck)&ptFAICheck)) != 0)
    {
        DRV_GetErrorMessage(ErrCde,(LPSTR)szErrMsg);
        MessageBox(hWnd,(LPCSTR)szErrMsg,"Thread Message",MB_OK);
        return ;
    }
```

```
    if (gwHalfReady == 1) // first ready
    {
        ptFAITransfer.ActiveBuf = 0;    // single buffer
        ptFAITransfer.DataType = gwDataType;
        ptFAITransfer.start      = 0;
        ptFAITransfer.count      = gulConvNum/2;
        ptFAITransfer.overrun    = &gwOverrun;

        if ((ErrCde = DRV_FAITransfer(DriverHandle,
            (LPT_FAITransfer)&ptFAITransfer)) != 0)
        {
            DRV_GetErrorMessage(ErrCde,(LPSTR)szErrMsg);
            MessageBox(hWnd,(LPCSTR)szErrMsg,"Thread Message",MB_OK);
            return ;
        }
    }

    else if (gwHalfReady == 2) // second half ready
```

```

    {
        ptFAITransfer.ActiveBuf = 0;    // single buffer
        ptFAITransfer.DataType = gwDataType;
        ptFAITransfer.start      = gulConvNum/2;
        ptFAITransfer.count      = gulConvNum/2;
        ptFAITransfer.overrun    = &gwOverrun;

        if ((ErrCde = DRV_FAITransfer(DriverHandle,
            (LPT_FAITransfer)&ptFAITransfer)) != 0)
        {
            DRV_GetErrorMessage(ErrCde,(LPSTR)szErrMsg);
            MessageBox(hWnd,(LPCSTR)szErrMsg,"Thread Message",MB_OK);
            return ;
        }

        2) 调用 DRV_FAITransfer 函数进行数据采集。ActiveBuffer=0/1 表示是单 Buffer/双
        Buffer; DataType=0/1 表示数据类型为无符号整型/浮点型; start/count 表示向数据 Buffer 拷
        贝数据的起始/终止位置。Overrun 表示是否为覆盖模式。初始化结束之后即可进行传输。
        gwCyclicCount ++;
    }

    return;
}
//-----
void adOverrunEvent()
{
    // clear overrun

    if ((ErrCde = DRV_ClearOverrun(DriverHandle)) != 0)
    {
        DRV_GetErrorMessage(ErrCde,(LPSTR)szErrMsg);
        MessageBox(hWnd,(LPCSTR)szErrMsg,"Thread Message",MB_OK);
        return ;
    }

    return;
}
//-----

```

```

void adTerminateEvent()
{

```

A/D 转化结束事件处理程序

1) 停止 A/D 转换

```

    // Stop A/D conversion
    if ((ErrCde = DRV_FAIStop(DriverHandle)) != 0)
    {

```

```

        DRV_GetErrorMessage(ErrCde,(LPSTR)szErrMsg);
        MessageBox(hWnd,(LPCSTR)szErrMsg,"Thread Message",MB_OK);
        return ;
    }
2) 从内存中取得电压值。
    // Get real voltage of buffer from driver
    ptFAITransfer.ActiveBuf = 0;    // single buffer
    ptFAITransfer.DataType = gwDataType;
    ptFAITransfer.start      = 0;
    ptFAITransfer.count      = gulConvNum;
    ptFAITransfer.overrun    = &gwOverrun;
    if ((ErrCde = DRV_FAITransfer(DriverHandle,
        (LPT_FAITransfer)&ptFAITransfer)) != 0)
    {
        DRV_GetErrorMessage(ErrCde,(LPSTR)szErrMsg);
        MessageBox(hWnd,(LPCSTR)szErrMsg,"Thread Message",MB_OK);
        return ;
    }
3) 通过数组取得数据
    ULONG i;
    LPVOID temp;
    CString str;
    for(i=0;i<100;i++)
    {
        strData[i]="Initial string";
    }
    if (gwDataType == 0)
        temp = (USHORT far *)ptFAITransfer.DataBuffer;
    else
        temp = (float far *)ptFAITransfer.DataBuffer;
    for(i = gdwStartPt; i <= gdwStopPt; i++)
    {
        if (gwDataType == 0)
            str.Format("Buf[%ld]=%4x",i,((USHORT far *)temp)[i]);
        else
            str.Format("Buf[%ld]=%10.6f",i,((float far *)temp)[i]);

        strData[i]=str;    //为显示字符串数组赋值

        //MessageBox(hWnd,str,MB_OK);//测试数据结果。
    }
    //////////////////////////////////////
    // Free buffer

```

```

GlobalUnlock(hBuf);
GlobalUnlock(hUserBuf);
GlobalFree(hBuf);
GlobalFree(hUserBuf);

// Close driver
DRV_DeviceClose((LONG far *)&DriverHandle);

return;
}

```

第六步：编写停止采集数据和退出程序按钮。

```

void CDlgIntDlg::OnButton2()
{
    // TODO: Add your control notification handler code here
    for(int i=0;i<100;i++)
    {
        pDataList->AddString(strData[i]); //在列表框中显示数据
    }
}

void CDlgIntDlg::OnOK()
{
    // TODO: Add extra validation here
    if(DriverHandle!=NULL)退出程序时，如果设备句柄没有释放则在此释放。
    DRV_DeviceClose((LONG far *)&DriverHandle);
    CDialog::OnOK();
}

```

第七步：运行程序

好了，就是这么简单，运行一下程序结果就是下面样子的，采集到的数据显示到列表框中。!



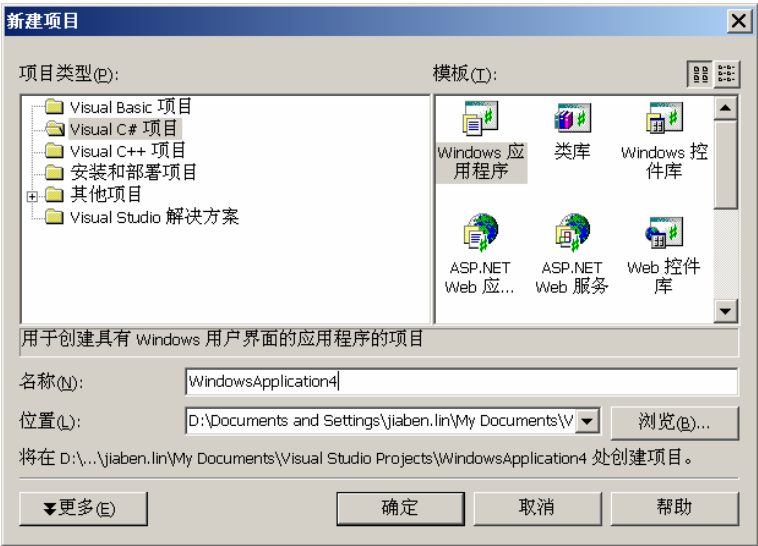
图（19）程序运行结果

第四章 数据采集控制卡 ActiveX 控件编程

Active 控件是研华公司提供的高效数据采集开发组件，可以方便地用于 VB、VC、BC、Delphi、C#以及各种支持 Active 控件的组态软件中。这里以 C#为例说明 Active 控件的使用方法。研华公司提供了 VB、Delphi 下的丰富的例程，这里不作详细描述，有兴趣的读者可以参考研华 Activedaq 的使用说明。

一、从最简单的开始

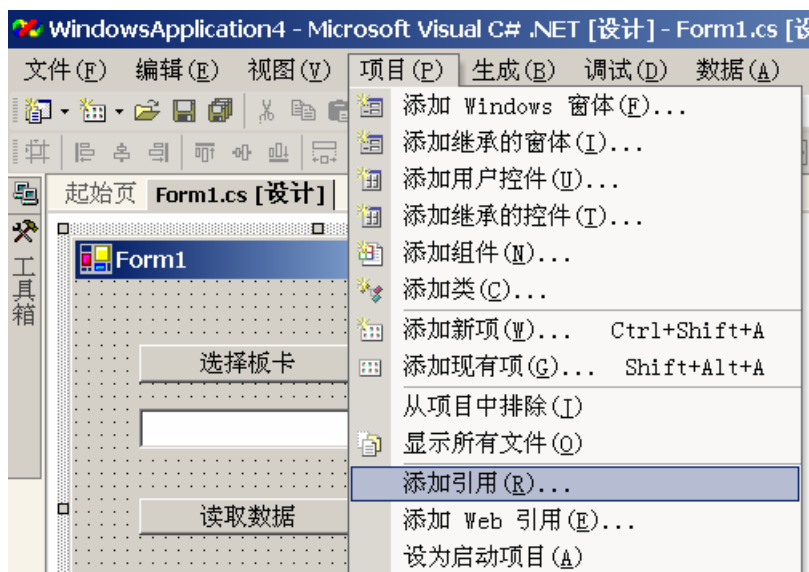
1) 打开一个 C#项目



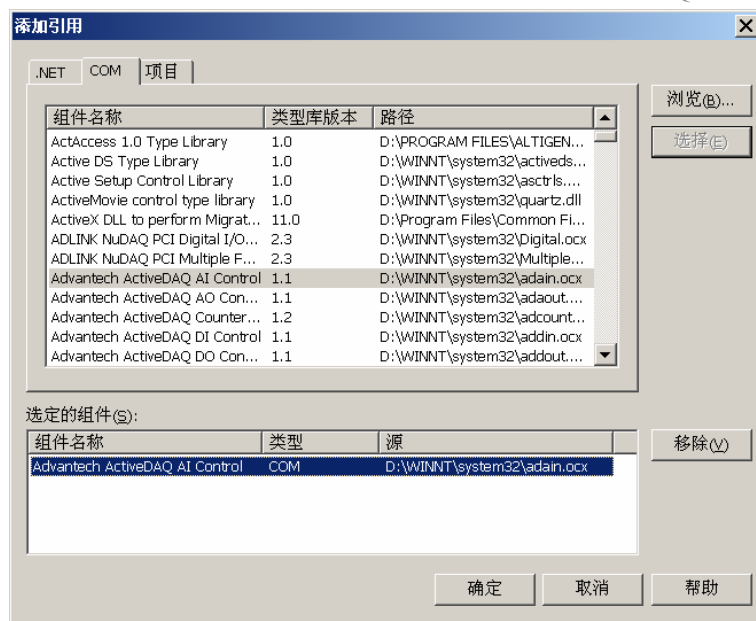
布置程序的界面如下所示：



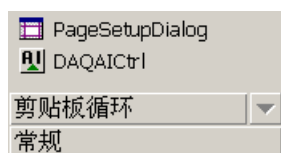
2) 将需要所使用的控件添加到工具箱中。方法：选择“项目”=>“添加引用”



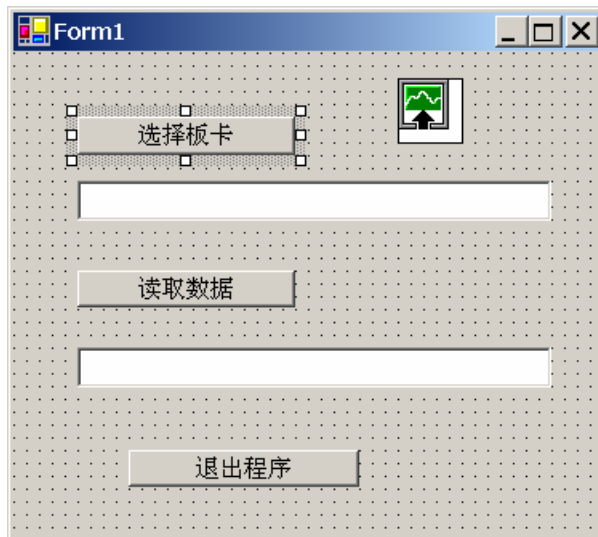
在“添加引用”对话框中选择“COM”中的“Advantech ActiveDAQ AI Control”……



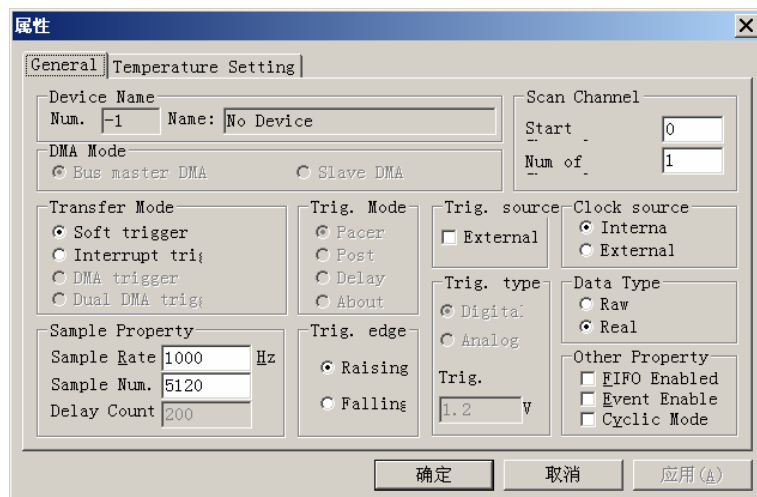
这时候在 C#的工具箱中就可以看到刚刚添加的控件  DAQAICtrl。



3) 将控件添加到 C#项目中，如下图所示：



查看控件的“属性”，可以看到控件的主要属性



4) 开始编程使用板卡

编辑按钮 **选择板卡** 的实现函数：

```
private void button1_Click(object sender, System.EventArgs e)
{
    axDAQAI1.SelectDevice();弹出板卡选择对话框
    textBox1.Text =axDAQAI1.DeviceName ;将所选板卡的名字显示在文本框中
    axDAQAI1.OpenDevice ();打开所选的板卡设备。
}
```

编辑按钮 **读取数据** 的实现函数：

```
private void button2_Click(object sender, System.EventArgs e)
{
    textBox2.Text =axDAQAI1.RealInput (0).ToString ();
}
```

编辑按钮 **退出程序** 的实现函数：

```
private void button3_Click(object sender, System.EventArgs e)
```

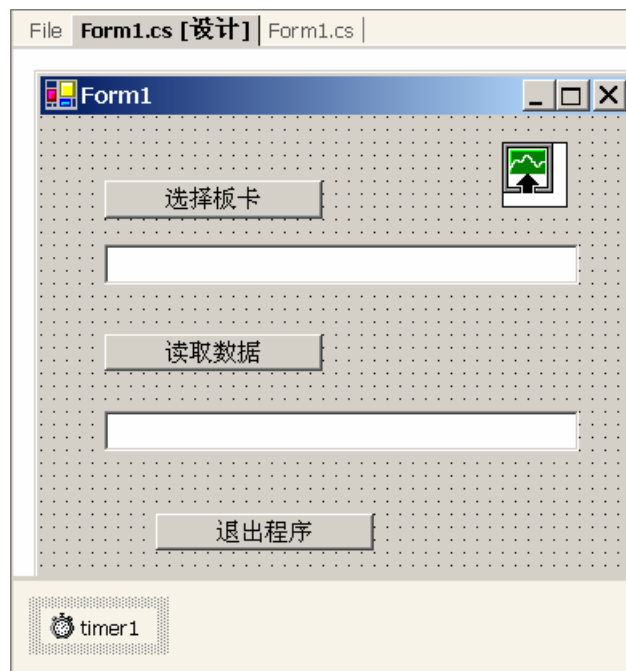
```

{
    axDAQAI1.CloseDevice ();
    Application.Exit ();
}

```

二、完善程序

在实际的应用系统中，我们往往需要定时连续地进行数据采集，而不是每一次都去手动的采集。



在项目中添加一个定时器 timer1，设定 Interval=100ms 并为其添加事件 timer1_Tick。

这时候我们需要在程序初始化的过程使 Timer1.Enable=false，防止程序一起动就开始数据采集，而在需要开始定时连续采集的时候才通过按钮使 Timer1.Enable=true，开始连续的数据采集。

重新编辑按钮 **读取数据** 的实现函数：

```

private void button2_Click(object sender, System.EventArgs e)
{
    timer1.Enabled =true;使能Timer1
}

```

同时我们也需要编辑timer1_Tick的实现函数：

```

private void timer1_Tick(object sender, System.EventArgs e)
{
    float i=axDAQAI1.RealInput (0);
    textBox2.Text =i.ToString ();
    //w.WriteLine(i.ToString());
}

private void timer1_Tick(object sender, System.EventArgs e)

```

```

{
    float i=axDAQ11.RealInput (0);
    textBox2.Text =i.ToString ();
    w.WriteLine(i.ToString());
}

```

三、打造自己的第一个综合工程应用程序

程序初始化的过程中打开待操作的文件，在后面的采集过程中随时可以将采集到的数据存入文件

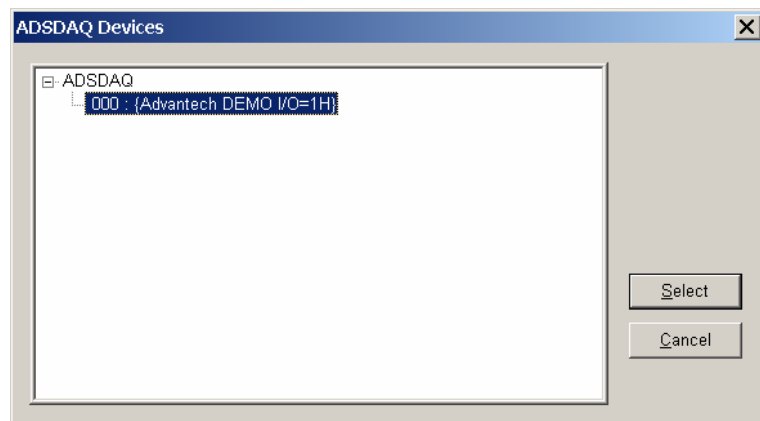
```
f = new FileInfo("Data.txt");
```

```
w = f.CreateText();
```

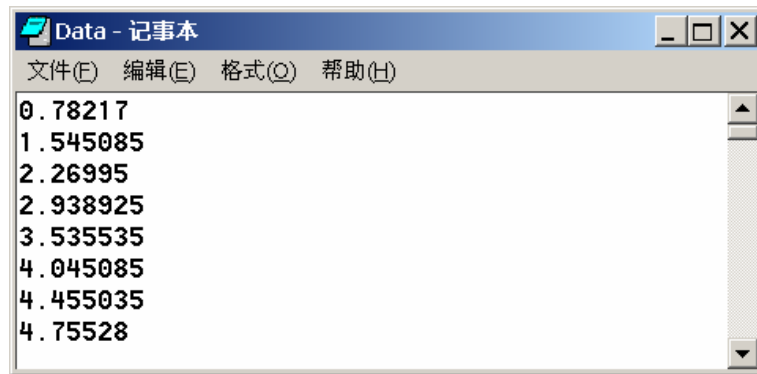
程序最终的测试结果如下：



选择板卡：



存档文件如下图所示：



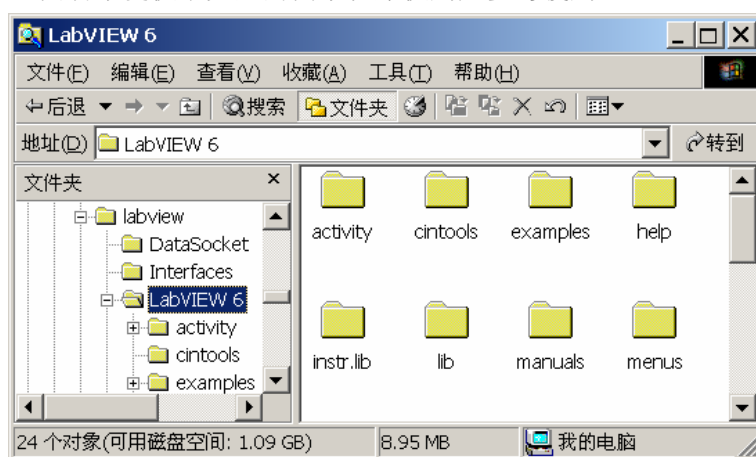
上面的系统中已经完成一个简单系统所需的大部分功能——数据采集，存储。大家可以在这个程序的基础上加入数据实时显示，数据读取、历史曲线显示等就可以形成一个完善的系统。

第五章 数据采集卡 LabVIEW 编程

一、研华 LabVIEW 驱动程序的安装

从数据采集卡附带的光盘或者公司网站可以下载驱动程序 LabVIEW.exe 文件，最新的驱动程序，直接运行 LabVIEW.exe 这个可执行文件即可。

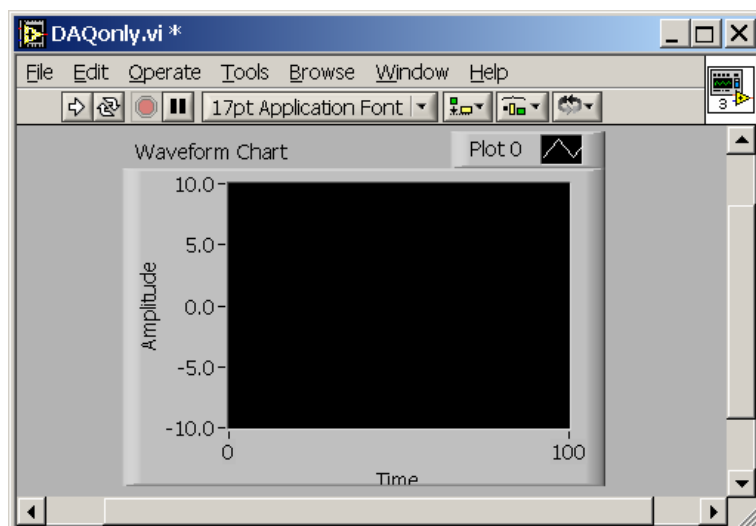
如果用户拿到的是比较早版本的驱动程序，那么安装过程中需要注意安装路径要如下图进行选择，即：用户安装 LabVIEW 的文件夹中的 LabVIEW6x/LabVIEW7 这个目录中，正确安装之后，在该目录下面的 examples 文件夹中就会有研华提供的例子程序文件夹 Advantech，在这里面研华提供了大量的例子程序供用户参考使用。



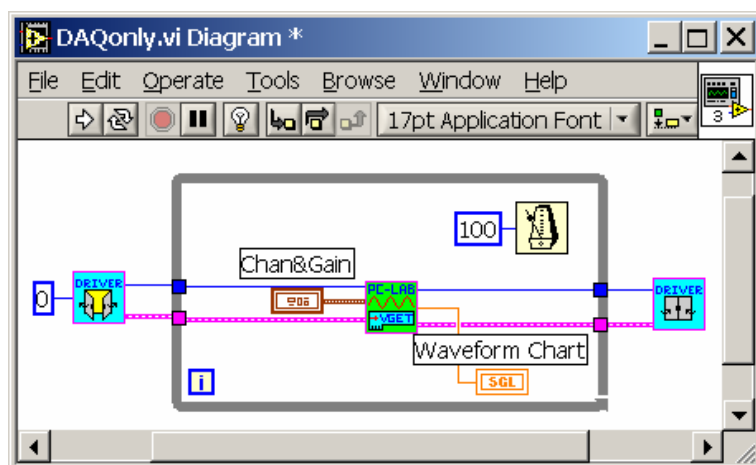
二、简单易用，轻松上手

我们从一个最简单的例子开始来看一下，在 LabVIEW 下面使用研华的数据采集卡是一件多么简单和惬意的事情。

(1) 首先我们在 LabVIEW 的“面板窗口”上面布置一个图形显示控件，用来显示从数据采集卡中取得的数据。



(2) 然后我们切换到程序框图窗口——面板窗口→Window→Show Diagram:
第一个简单的例子程序 的“程序框图”，我们将编辑成如下图所示：



从程序框图窗口中可以看到，我们只用三个研华公司提供的驱动函数就完成了了一个图形化的数据采集和显示系统，简单吧！

下面我们就一步步地来看如何实现这个程序。

我们先来看看，程序中用到的三个函数：



——打开设备函数



——采集电压函数

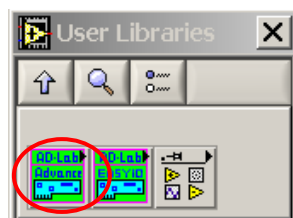


——关闭设备函数

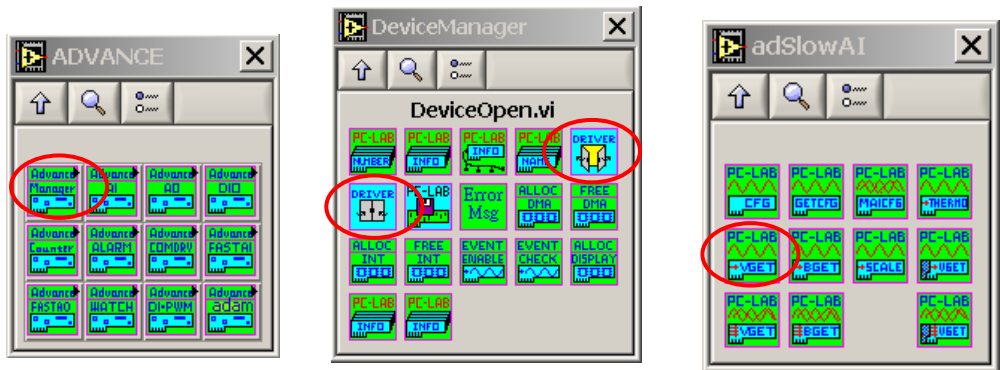
上面这三个控件在安装完研华公司提供的 LabVIEW 驱动程序之后就可以在 LabVIEW 的 Function 面板中看到研华公提供的对板卡操作的库函数，如右图所示。



点击  弹出 User Libraries
在 User Libraries 中选择 ADVANC

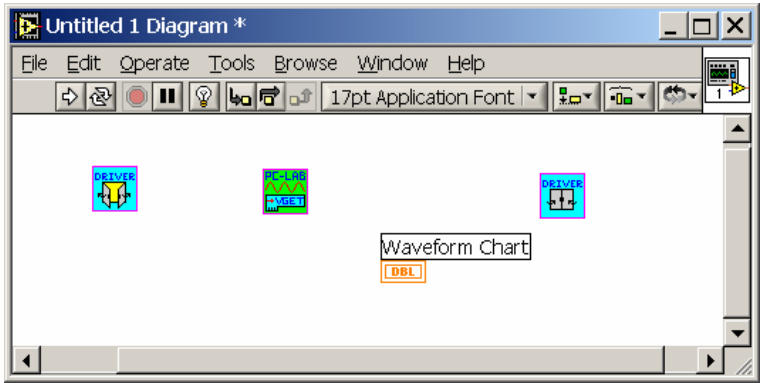


在 ADVANCE 函数库中包括以下函数：
Device Manager, AI, AO,



在 ADVANCE 选择 DeviceManager, 在 DeviceManager 中可以选择 Device Open, Device Close 函数拖动到程序框图窗口中。

在 ADVANCE 选择 adsSlowAI 函数库, 在 adsSlowAI 中选择 AIVoltageIn 函数拖放到程序框图窗口中, 函数布置如下图所示:



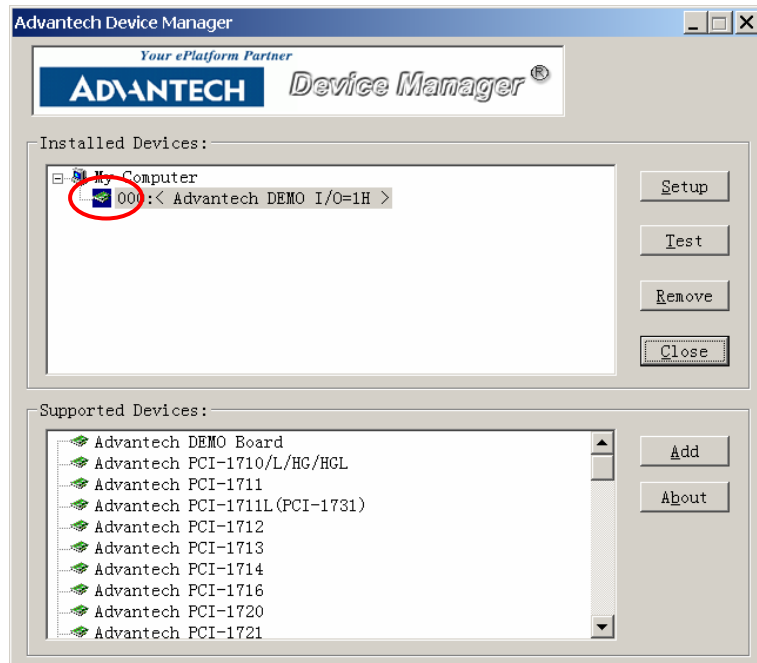
(3) 在程序框图界面中加入所需的操作板卡的函数之后, 我们就可以开始 “画程序”



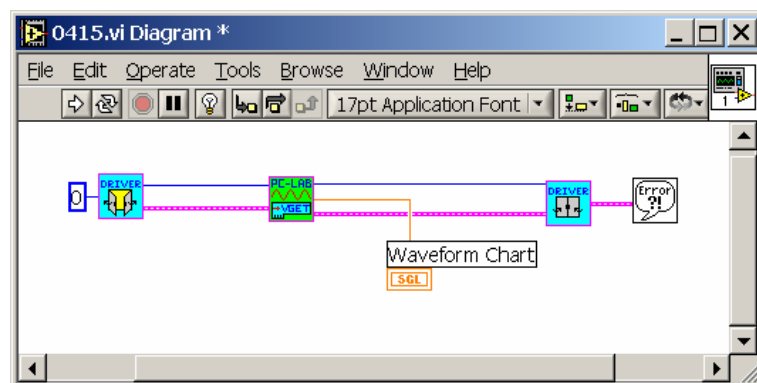
了。在 Tools 面板中，选择线轴工具，这时候鼠标就会变成线轴形状，当你把线轴挪动到这三个函数所定应的图标上面的时候，图标的对外接口就会出现。

我们首先给 Device Open 函数 提供一个 Device Number; 板卡的 Device Number, 可以在研华提供的板卡安装测试工具 Device Manger 中找到 (如图所示, 您的板卡可是要在这里测好噢, 这样在编程的时候就可以顺利进行了!). 我们在这里添加了一块 demo 板。

值得注意——用户不需要购买研华的板卡就可以开始学习研华板卡编程使用方法。



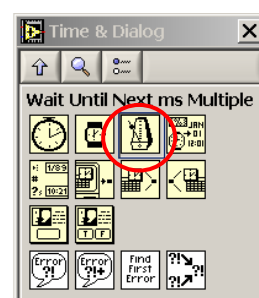
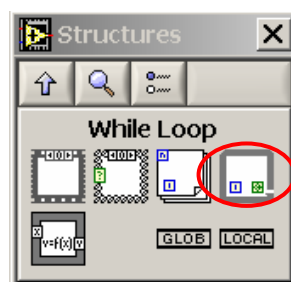
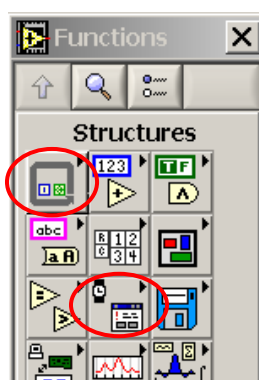
然后就可以把各个函数需要传递的参数连接起来：DevHandle 连接起来（蓝色线条），把出错信息连接起来（粉色线条）——前一个函数的 error out 连接到下一个函数的 error in 最后加一个出错提示的函数——这样一旦系统除向问题可以比较容易地判断问题出现在系统地那一个部分。最后把测量到的数据 voltage 送到显示控件 Waveform Chart。



这时候每点击一次  按钮，就可以采集一次数据，并显示在 Waveform Chart 数据显示控件中。感觉到了吧，就是这么简单。

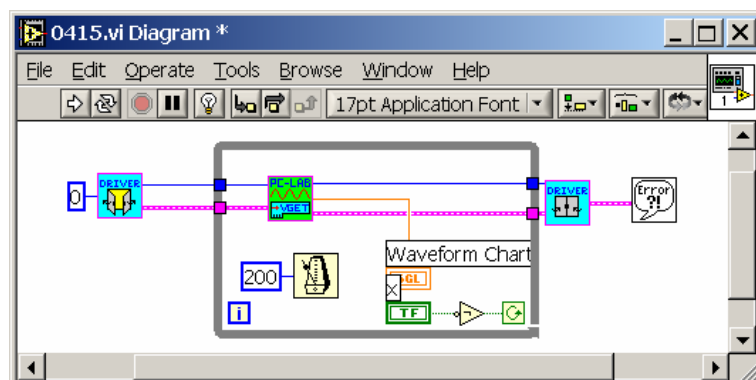
（4）完善我们的程序——定时连续采集

这时候我们需要在上面的程序中添加一个 While 循环，一个定时器。

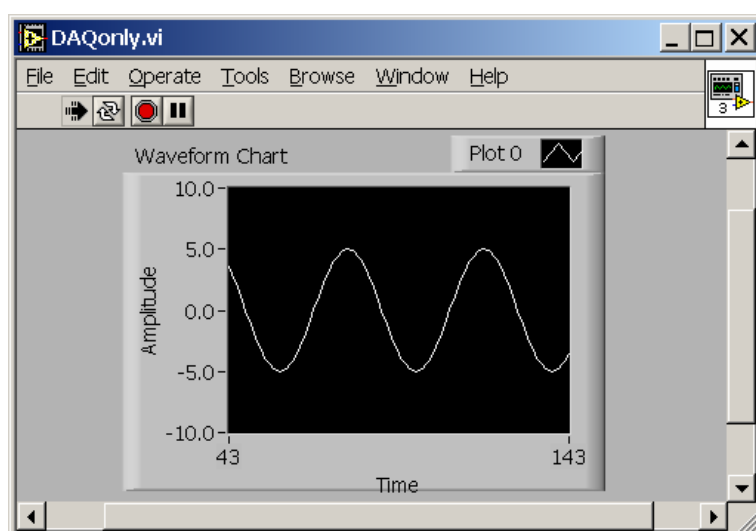


首先在 Function 面板中选择 Structure，在 Structure 面板中可以找到 While Loop 结构，在 Function 面板中选择 Time & Dialog，在 Time & Dialog 面板中可以找到 Timer 控件。

如下图所示：A、对 While Loop 加上一个为真控制条件，只要运行程序就可以进入 While 循环开始采集。B、加入时间控件，并为其添加时间间隔 200ms



(5) 运行程序，这时候  函数按照默认设置采集 demo 板的第一个通道上面的数据，并显示，程序的运行界面如下图所示：



三 更上一层楼，数据采集结果——显示且存储。

上面的程序当中我们仅仅使用了几个控件就实现了一个数据采集/显示的系统，但是在实际的工程系统当中我们采集的数据不仅仅要显示出来，而且要存储数据，作历史资料用。

这部分工作实际上使用 LabVIEW 工具本身所具有的文件读写函数就可以完成。与上面的过程类似我们实际上也是用到了几个函数来完成数据写入的功能。



——打开待写入的文件， ，打开的方式是创建或替换



——将采集到的数据写入文件，将来自  的字符串写入文件。



——关闭文件

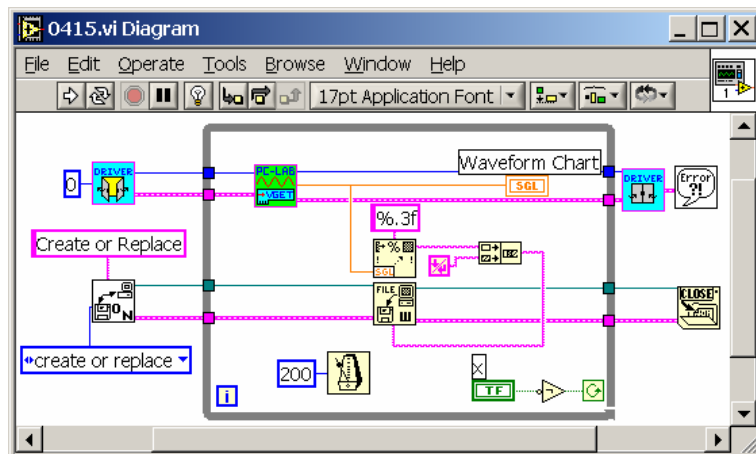


——字符串格式控制，这里将数据精度控制到小数点后 3 位，`%0.3f`。



——写入文件字符串格式调节，为了简便期间，这里在前面的字符串后加了一个换行符“`\n`”——即：一个数据一行——已后读出的时候很方便！

最后我们“画程序”的结果如下：如果在这些函数的使用方面有什么问题，请您参考 LabVIEW 方面的资料



四 结论

从上面的过程我们可以看到使用研华的数据采集卡，完成一个数据采集/显示/记录的程序是多么简单的事情。

当然上面的例子我们为了方便大家快速上手，省略了很多有关控件的接口函数的使用



方面的说明，例如控制电压采集函数 `AIVoltageIn`——采集的通道号/增益等等，这些细节用户可以在使用的时候参考研华提供的帮助文档。



多通道数据采集方面，您只需要选择相应的多通道采集函数 `MAIVoltageIn` 就可以了。

当然研华公司提供了丰富的例子程序，从模拟量到数字量；从单通道到多通道；从软件触发例程到 DMA 方式采集的例程应有尽有！用户只要稍作修改，甚至不需修改就可以应用到自己的系统当中，对最终应用客户来讲这应该还是比较有价值的资源。

第六章 研华数据采集卡例程使用

一、 系列产品介绍、硬件简介：AI,AO,DI,DO

研华数据采集与控制卡：

ISA 总线数据采集与控制卡

PCL-818H: 16 通道 100 kHz 多功能 DAS 卡

PCL-818L: 16 通道 40 kHz 多功能 DAS 卡

PCL-818HG: 16 通道 100 kHz 高增益多功能 DAS 卡

PCL-818HD: 16 通道 100 kHz 多功能 DAS 卡，带 1 K FIFO

PCL-1800: 16 通道 330 kHz 高速 DAS 卡

PCL-812PG: 16 通道 30 kHz 多功能 DAS 卡

PCL-711S: 8 通道 25 kHz 多功能 DAS 卡

PCL-813B: 32 通道 25 KHz A/D 卡，500 VDC 总线隔离

PCL-726: 6 通道 D/A 输出卡

PCL-727: 12 通道 D/A 输出卡

PCL-728 : 隔离 2 通道 D/A 输出卡

PCL-720+: 32 通道数字量输入/输出和计数卡

PCL-722: 144 位数字量输入/输出卡

PCL-725 : 8 通道继电器输出和 8 通道隔离数字量输入卡

PCL-730: 32 通道隔离数字量输入/输出卡

PCL-724: 24 位数字量输入/输出卡

PCL-731: 48 位数字量输入/输出卡

PCL-733: 32 通道隔离数字量输入卡

PCL-734: 32 通道隔离数字量输出卡

PCL-735: 12 通道继电器输出卡

PCL-836: 8 通道计数器/定时器卡

PCL-833: 3 轴正交编码和计数卡

PCL-839 : 3 轴步进电机控制卡

PCI 总线数据采集与控制卡

PCI-1710: PCI 总线 16 通道 100 KHz, 12 位多功能卡（高增益）

PCI-1711: 100K 采样速率、16 通道单端低价格多功能卡

PCI-1712: 1M 采样速率、12 位高速采集卡

PCI-1716: 250K 采样速率、16 位 16 通道高分辨率多功能卡

PCI-1713: 100K 采样速率、32 通道隔离模拟量输入卡

PCI-1721: 12 位 4 通道模拟量输出卡

PCI-1723: 16 位 8 通道模拟量输出卡

PCI-1753: 96/192 位数字量 I/O 卡

PCI-1730: 32 通道数字量 I/O 卡

PCI-1752: 64 通道隔离 I/O 卡

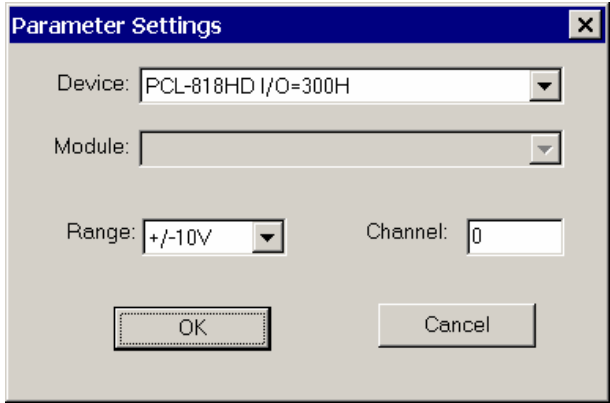
PCI-1761: 8 通道继电器/8 通道隔离 IO
PCI-1762: 16 通道隔离 DI/16 通道继电器
PCI-1720: 4 通道隔离 D/A 输出卡
PCI-1750 : PCI 总线 32 通道数字量输入/输出卡, 具有 2500 V DC 隔离保护
PCI-1751: PCI 总线 48 位数字量输入/输出卡
PCI-1760: 继电器输出和隔离数字量输入卡
PCI-1240: PCI 总线 4 通道步进/伺服电机卡
PCI-1784: PCI 总线 4 轴正交编码器及计数卡
PCL-833: 3 轴正交编码和计数卡
PCL-839 : 3 轴步进电机控制卡

二、 例程详解

ADSOFT/ADTRIG（软件触发方式例程）

单通道模拟量数据采集例程（软件触发模式）：该例程主要使用 DRV_AIConfig 配置模拟量输入通道等信息，使用模拟量输入函数（DRV_AIVoltageIn），通过软件触发方式（使用 Windows Timer）实现数据采集。

1) 单击 Setting 菜单弹出下面的对话框：

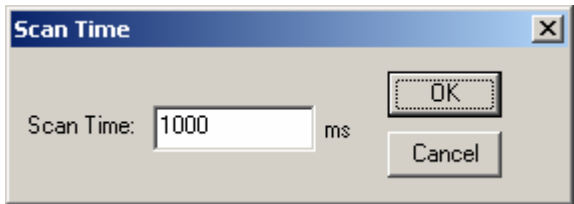


Device:显示所安装的设备，如果你安装了多块板卡可以在这里选择支持该例程的板卡；

Range:选择输入范围；

Channel:选择输入通道；

2) 单击 Scan,弹出下面的对话框：



可以设置计数的时间间隔，默认值为 1000 毫秒

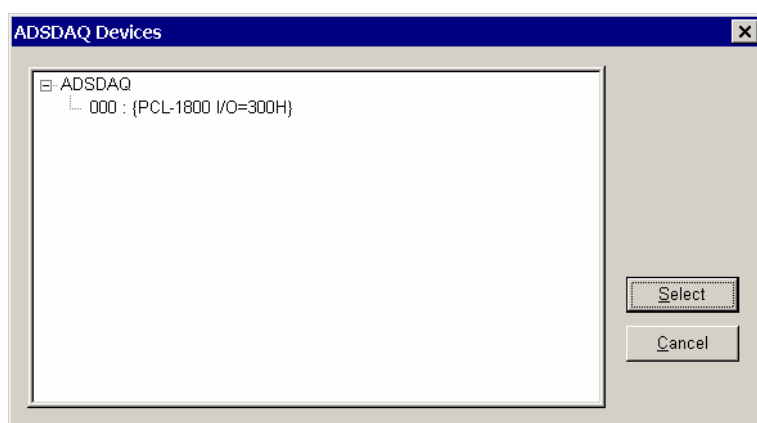
3) 单击 Run 菜单项中的 Start 菜单就可以开始软件触发模式数据采集，单击 Stop 项停

止。

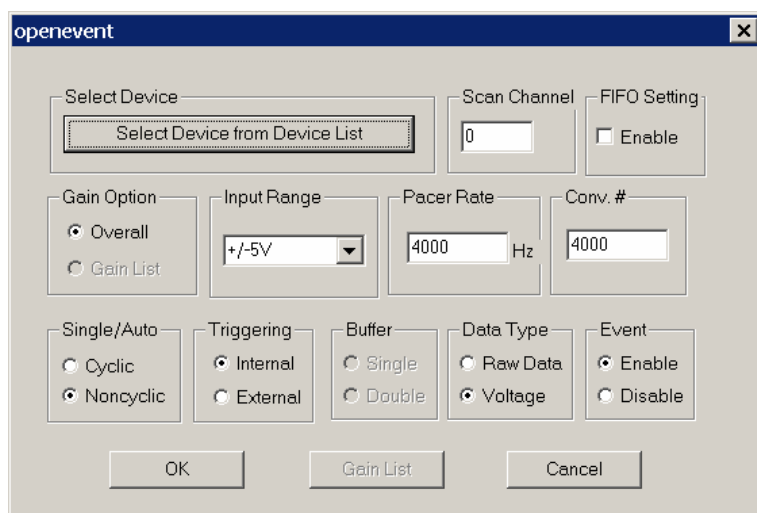
ADDMA（DMA 方式进行数据采集的例程）函数：

单通道模拟量数据采集例程（DMA 模式）：该例程通过 DRV_FAIDmaStart 函数启动了 DMA 功能，该功能运行于后台，可以使用 DRV_FAICheck 函数检查工作状态，同时可以使用 DRV_FAITransfer 函数传输数据，当工作结束，或者任何时刻，你都可以采用 DRV_FAIStop 来停止工作。

1) 单击 Setting，弹出设备选择窗口如下所示：



2) 选择设备点击 Select 按钮后的对话框如下图所示，



对话框重的参数含义如下：

Select Device from Device List 按钮可以弹出板卡选择的对话框。

Scan Channel: 可以输入要采集的通道数据号，其范围由板卡的通道数目确定。

FIFO Setting: 设置是否使用 FIFO。

Gain Option: 选择增益，这里只能选择 Overall 选项，表示这个板卡不支持各个通道独立设置输入量程，所有的通道都是采用相同的量程。

Input Range: 为所有的通道选择相同的量程范围。

Pacer Rate: 设置采样频率。

Conv.#: A/D 转化的数目。

Single/Auto:两个选项 Cyclic:循环模式; Noncyclic: 非循环模式;

Triggering: 触发方式, Internal 内部触发; External 外部触发;

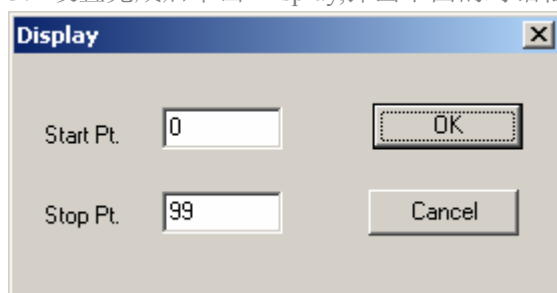
Buffer: 使用单个 buffer(single), 双 buffer(double)

Data Type: 数据类型; Raw Data 原始数据——二进制/十六进制——未经过 DA 转换的数据;

Voltage: 真实的电压数据。

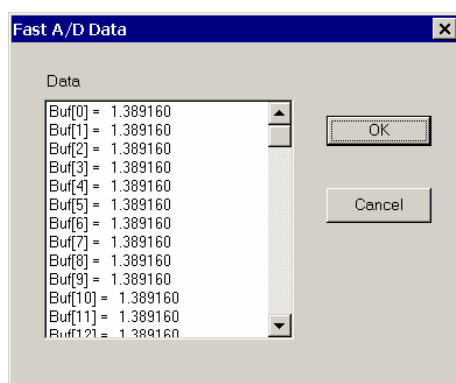
Event: 事件; 是否允许设置事件来编程 (只是对中断和 DMA 方式来讲), 如果选择 Enable, 则当程序完成 Conv.#设定的转换次数之后自动弹出数据显示对话框; 若选择的是 Disable, 则当用户单击 Stop 按钮的时候, 才会弹出数据显示对话框。

3) 设置完成后单击 Display,弹出下面的对话框:

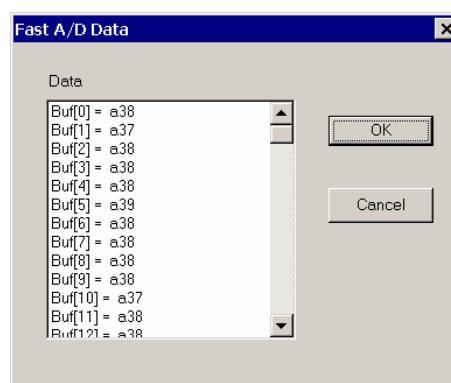


设置将要现实的数据的范围, 默认 (0~99) 注: Stop Pt 不能大于 Conv.#设置的数值。

4) 单击 Run 菜单项开始采集数据, 当采集完成 Conv.#设置的 A/D 转换次数之后, 就会显示 (假设没有修改 Display 菜单中的默认值) 0~99 点的数据。显示窗口如下所示:



(Real Voltage)

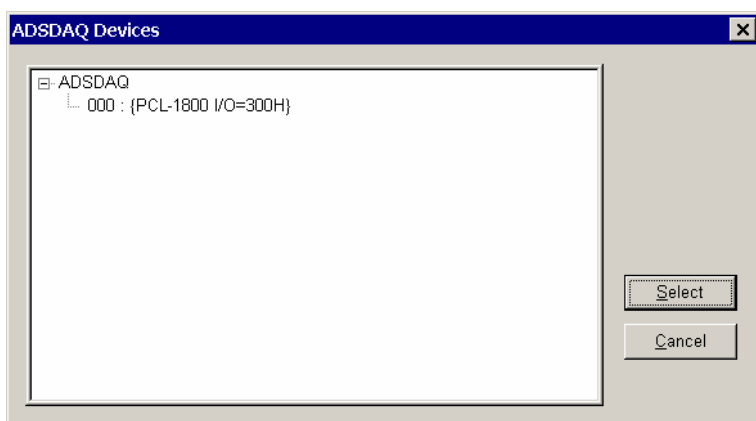


(Raw Data)

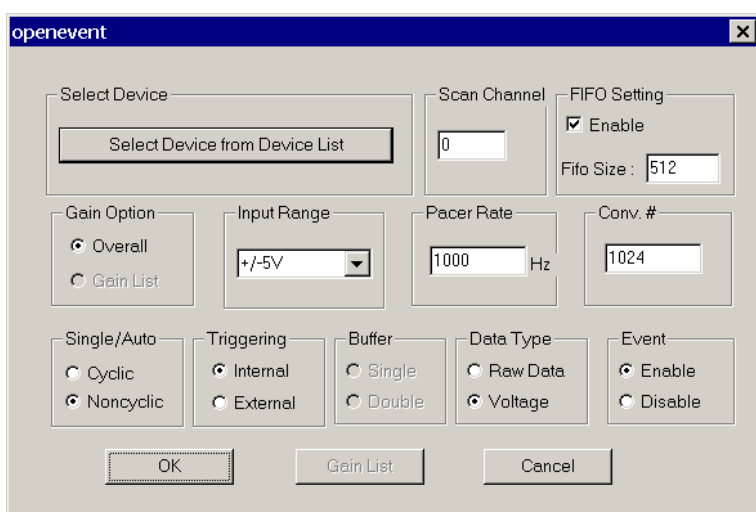
ADintf (中断方式进行数据采集的例程):

单通道模拟量数据采集例程 (中断模式): 该例程通过 DRV_FAIntStart 函数启动了中断功能, 该功能运行于后台, 可以使用 DRV_FAICheck 函数检查工作状态, 同时可以使用 DRV_FAITransfer 函数传输数据, 当工作结束, 或者任何时刻, 你都可以采用 DRV_FAIStop 来停止工作, 另外: 该例程支持用户设定 FIFO 大小。

1) 单击 Setting, 弹出设备选择窗口如下所示:



2) 选择设备点击 Select 按钮后的对话框如下图所示，



对话框中的参数含义如下：

Select Device from Device List 按钮可以弹出板卡选择的对话框。

Scan Channel: 可以输入要采集的通道数据号，其范围由板卡的通道数目确定。

FIFO Setting: 设置是否使用 FIFO 及其大小。

Gain Option: 选择增益，这里只能选择 Overall 选项，表示这个板卡不支持各个通道独立设置输入量程，所有的通道都是采用相同的量程。

Input Range: 为所有的通道选择相同的量程范围。

Pacer Rate: 设置采样频率

Conv.#: A/D 转化的数目，注意：这个数字必须是半 FIFO 大小的整数倍。

Single/Auto: 两个选项 Cyclic: 循环模式；Noncyclic: 非循环模式；

Triggering: 触发方式，Internal 内部触发；External 外部触发；

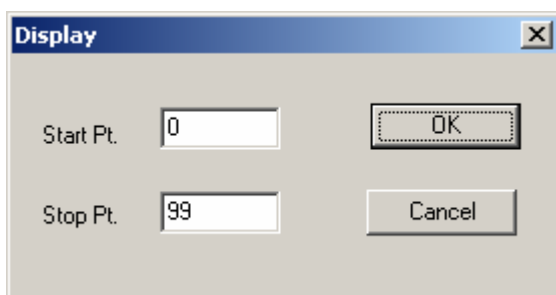
Buffer: 使用单个 buffer(single)，双 buffer(double)

Data Type: 数据类型；Raw Data 原始数据——二进制/十六进制——未经过 DA 转换的数据；

Voltage: 真实的电压数据。

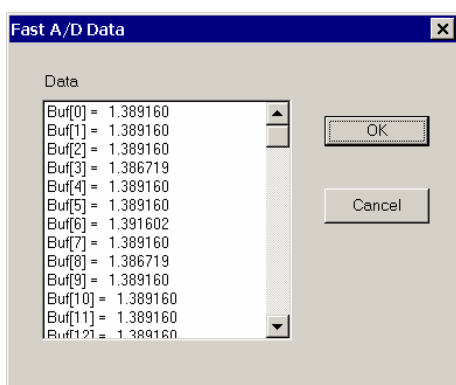
Event: 事件；是否允许设置事件来编程（只是对中断和 DMA 方式来讲），如果选择 Enable，则当程序完成 Conv.# 设定的转换次数之后自动弹出数据显示对话框；若选择的是 Disable，则当用户单击 Stop 按钮的时候，才会弹出数据显示对话框。

3) 设置完成后单击 Display, 弹出下面的对话框：



设置将要现实的数据的范围，默认（0~99）注：Stop Pt 不能大于 Conv.#设置的数值。

4)单击 Run 菜单项开始采集数据，当采集完成 Conv.#设置的那么多次的 A/D 转换之后，就会显示（假设没有修改 Display 菜单中的默认值）0~99 点的数据。显示窗口如下所示：

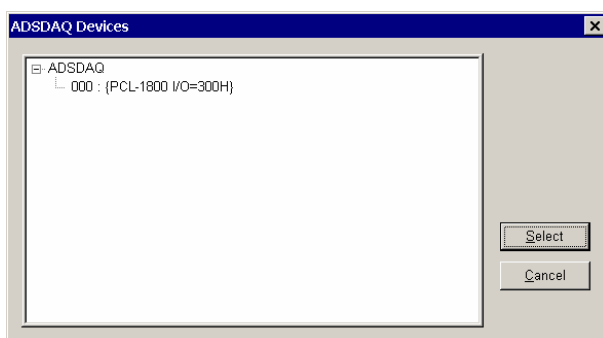


Adint(中断方式数据采集)与该程序界面类似，只是该例程使用固定大小的 FIFO，不再赘述。

DAINT(中断方式模拟量输出)

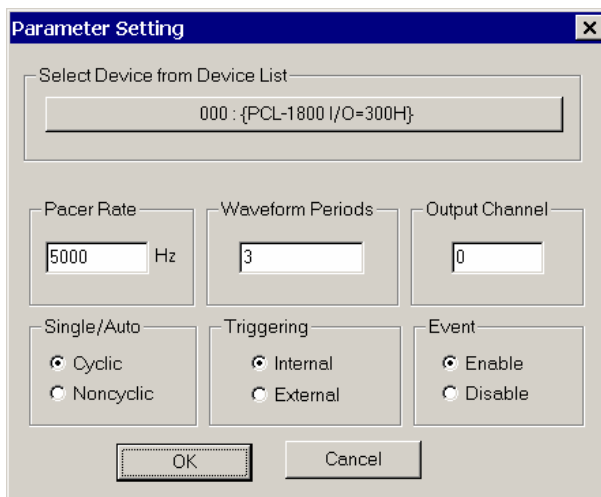
单通道模拟量数据输出例程（中断方式）：该例程主要使用 PT_FAOIntStart 配置模拟量输出通道等信息，使用模拟量输出函数（DRV_FAOIntStart），通过触发方式实现模拟量数据输出。

1) 单击 Setting 菜单弹出如下的设备选择对话框；



从中选择支持该例程的板卡。

2) 单击 Select 按钮，弹出如下的设置窗口：



Select Device from Device List:

单击下面的按钮就可以从已经安装的设备中选择支持该例程的板卡。

Pacer Rate: 脉冲频率, 默认值 1000Hz

Waveform Periods: 输出的波形数, 默认为 1。

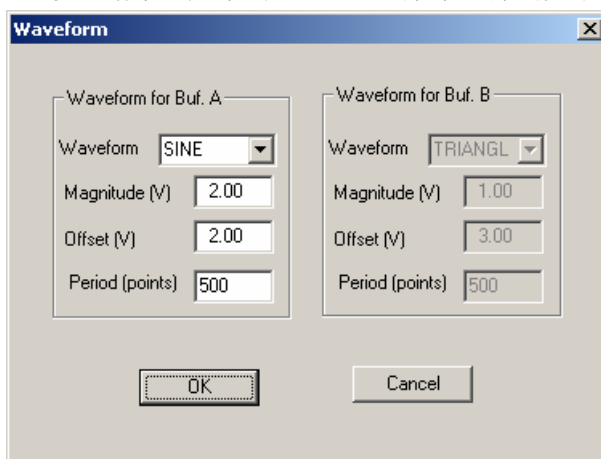
Output Channel: 输出通道, 默认值为 0

Single/Auto: 两个选项 Noncyclic: 非循环模式输出, 输出 Waveform Periods 设定的波形数的输出之后, 不再输出有效波形; Cyclic: 循环模式, 循环输出 Waveform Periods 数目的波形。

Triggering: 触发方式, Internal 内部时钟触发; External 外部时钟触发;

Event: 事件; 是否允许设置事件来编程 (只是对中断和 DMA 方式来讲), 如果选择 Enable, 则当程序完成 Waveform Periods 设定的波形数目的输出后, 程序自动停止输出; 若选择的是 Disable, 则当用户单击 Stop 按钮的时候, 才会停止输出。

3) 设置结束之后单击 Waveform 菜单, 弹出如下对话框:



Waveform for Buf.A: Buf.A 中的波形设定。

Waveform:: 输出波形选择 (SINE (正弦波), TRIANGLE (三角波), SQUARE (方波))

Magnitude(V): 幅值, 默认值 2.00 伏

Offset(V): 偏移量, 默认值 2.00 伏

Period(points): 每周期输出的点数, 默认值是 500 点。

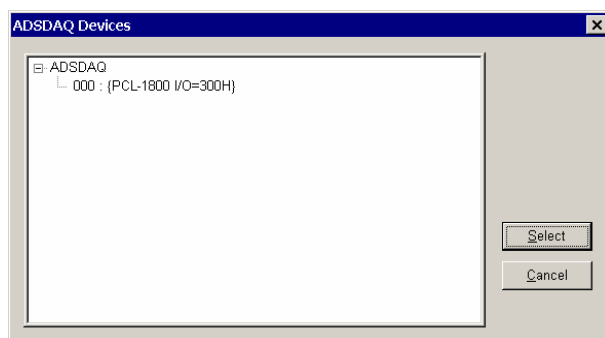
注: $\text{Pacer rate} = \text{Period point} \times \text{波形频率}$;

4) 波形选择完成之后, 单击 Run 菜单项即可开始输出模拟量, 通过模拟量输出通道可以监测输出波形。。

DADMA(DMA 方式模拟量输出)

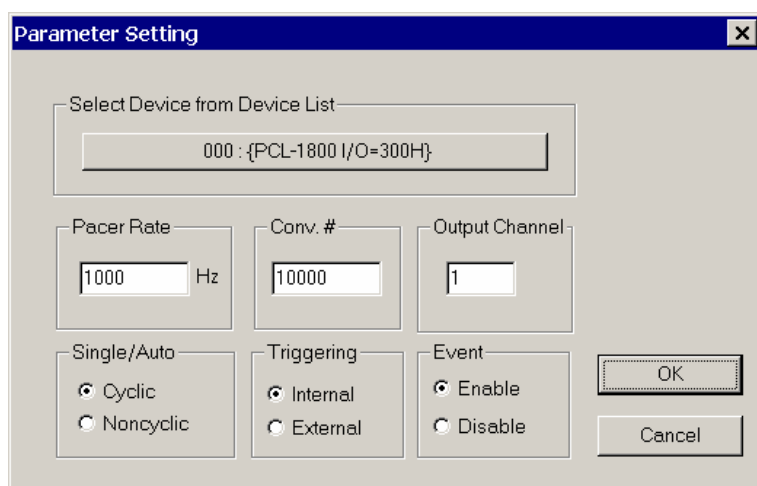
单通道模拟量数据输出例程（DMA 方式）：该例程主要使用 PT_FAODmaStart 配置模拟量输出通道等信息，使用模拟量输出函数（DRV_FAODmaStart），通过 DMA 方式实现模拟量数据输出。

1) 单击 Setting 菜单弹出如下的设备选择对话框；



从中选择支持该例程的板卡。

2) 单击 Select 按钮，弹出如下的设置窗口：



Select Device from Device List:

单击下面的按钮就可以从已经安装的设备中选择支持该例程的板卡。

Pacer Rate: 脉冲频率，默认值 10000Hz

Conv.#: D/A 转换的点数，默认值 10000。

Output Channel: 输出通道，默认值为 0。

Single/Auto: 两个选项 Cyclic: 循环模式；Noncyclic: 非循环模式；

Triggering: 触发方式，Internal 内部触发；External 外部触发；

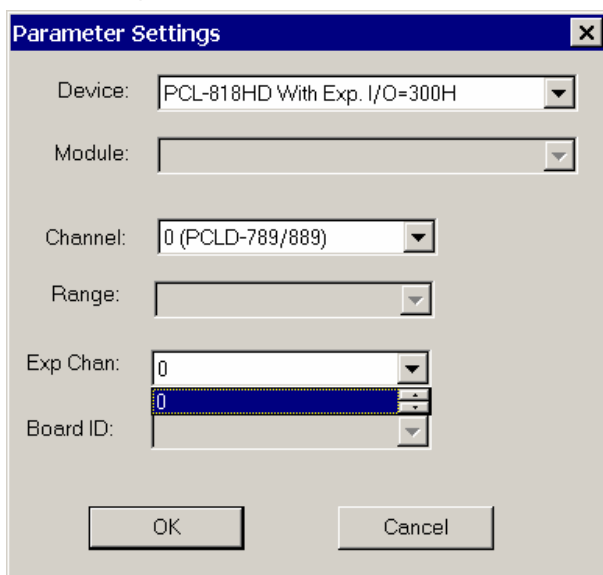
Event: 事件；是否允许设置事件来编程（只是对中断和 DMA 方式来讲），如果选择 Enable，则当程序完成 Waveform Periods 设定的波形数目的输出后，程序自动停止输出；若选择的是 Disable，则当用户单击 Stop 按钮的时候，才会停止输出。

后面的操作同 DAINTE 例程类似，不再赘述。

AIEXP（使用扩展板进行模拟量输入的例程）：

使用扩展板进行单通道模拟量输入采集例程：该例程主要使用 PT_AIVoltageInExp 配置模拟量输入通道等信息，使用模拟量输入函数（DRV_AIVoltageInExp），通过软件触发方式（使用 Windows Timer）实现数据采集。

单击 Setting 菜单弹出下面的对话框：



The 'Parameter Settings' dialog box contains the following fields and controls:

- Device: PCL-818HD With Exp. I/O=300H
- Module: (empty)
- Channel: 0 (PCLD-789/889)
- Range: (empty)
- Exp Chan: 0
- Board ID: 0
- Buttons: OK, Cancel

Device:显示所安装的设备，如果你安装了多块板卡可以在这里进行选择；

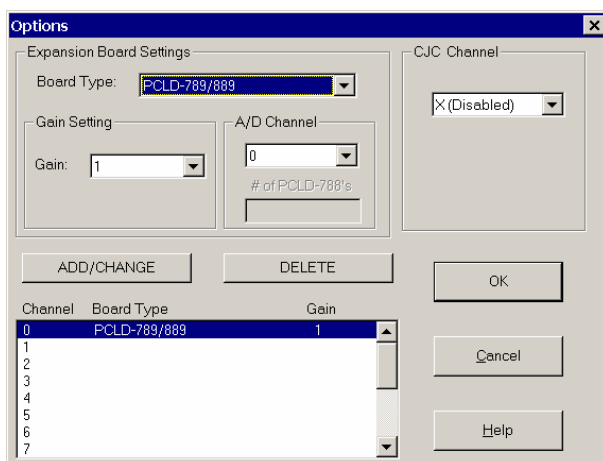
Range:选择输入输入范围；

Channel:选择通道号；

Exp Chan:选择外部通道

Board ID:板卡的 ID 号

注：在使用扩展板之前必须先通过 Device Manger 对数据采集板卡及其所带的扩展板进行设置，如本例在 Device Manger 的 Option 选项中就先进行了如下的设置：

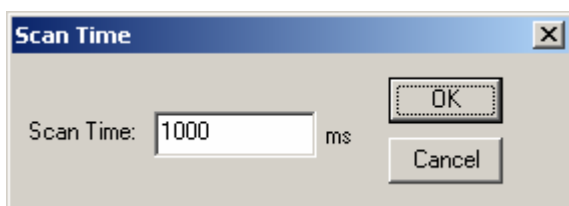


The 'Options' dialog box contains the following sections and controls:

- Expansion Board Settings:
 - Board Type: PCLD-789/889
 - Gain Setting: Gain: 1
 - A/D Channel: 0
 - # of PCLD-789's: (empty)
- CJC Channel: X (Disabled)
- Buttons: ADD/CHANGE, DELETE, OK, Cancel, Help
- Table:

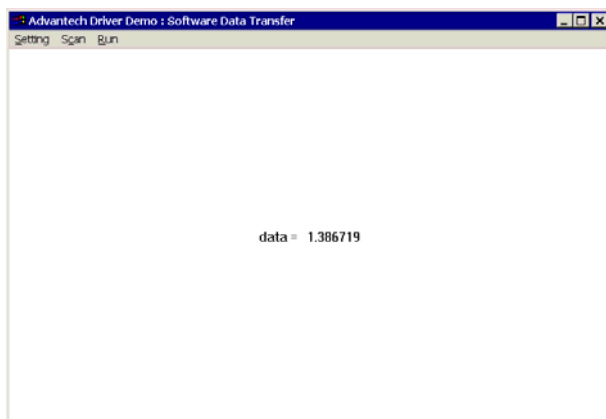
Channel	Board Type	Gain
0	PCLD-789/889	1
1		
2		
3		
4		
5		
6		
7		

2) 单击 Scan,弹出下面的对话框：



可以设置计数的时间间隔，默认值为 1000 毫秒

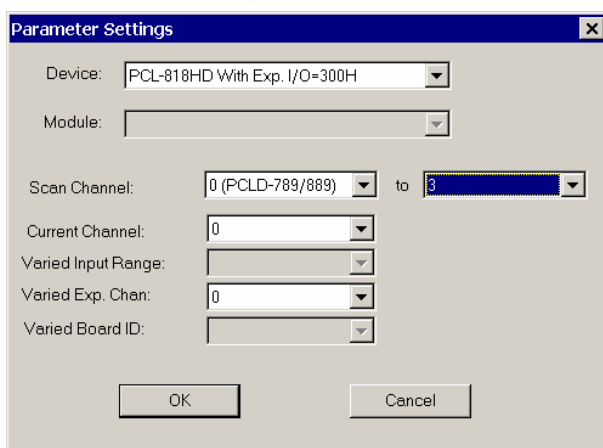
3)单击 Run 菜单项中的 Start 菜单就可以开始采集并显示模拟量(实验中在 PCLD-789D 的 CH0 的 HI 和 GND 之间跨接一电池，测的数据如下图所示)，单击 Stop 项停止。



MAIexp(扩展的多通道模拟量输入例程,软件触发)

使用扩展板进行多通道模拟量输入采集例程：该例程主要使用 PT_MAIVoltageInExp 配置模拟量输入通道等信息，使用模拟量输入函数（DRV_AIVoltageInExp），通过软件触发方式（使用 Windows Timer）实现数据采集。

1) 单击 Setting 菜单弹出下面的对话框：



Device:选择系统中已经安装的并且支持该函数的板卡

Scan Channel:扫描的通道从 0 到 N，N 的值可以在最右边的列表框中选择。

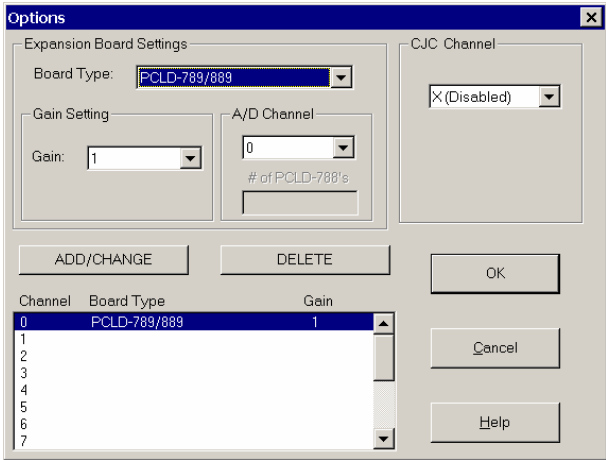
Current Channel:当前使用的通道。

Varied Input Range:输入范围选择

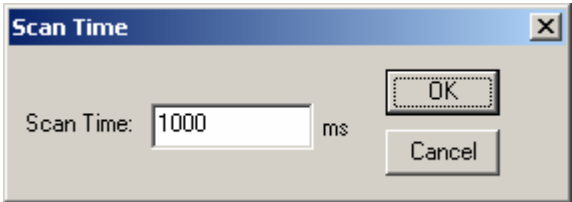
Varied Exp.Chan:扩展通道；显示的时候将显示连接的各个 PCLD-789D 的该通道输入的数据。

Varied Board ID:板卡 ID；

注：在使用扩展板之前必须先通过 Device Manger 对数据采集板卡及其所带的扩展板进行设置，如本例在 Device Manger 的 Option 选项中就先进行了如下的设置：



2) 单击 Scan 菜单弹出下面的对话框：



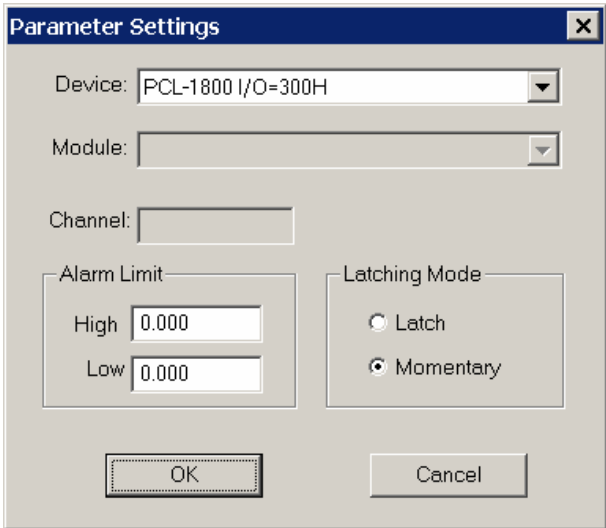
设置输出测量脉冲宽度的时间间隔，默认值为 1000 毫秒。

3) 单击 Run 菜单项中的 Start 菜单就可以开始采集，单击 Stop 项停止采集。

ALARM（报警例程）：

报警函数使用例程：该例程主要使用 PT_AlarmConfig 配置报警通道等信息，通过 PT_AlarmEnable 打开报警功能，使用报警检测函数（DRV_AlarmCheck），通过定时扫描方式（使用 Windows Timer）实现报警功能。

1) 单击 Setting 菜单弹出下面的对话框：

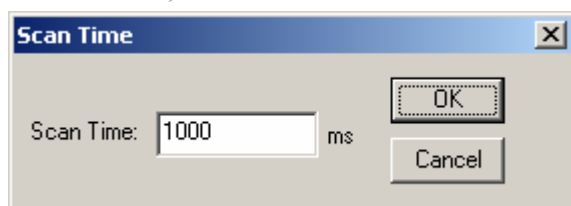


DEVICE:选择设备

Alarm Limit:设置报警的上下限，high:上限 low:下限；

Latching Mode: 设置报警的模式 Latch: 锁定模式;Mementary, 瞬时模式。

2) 单击 Scan,弹出下面的对话框:



可以设置计数的时间间隔，默认值为 1000 毫秒

3) 单击 Run 菜单项中的 Enable 菜单就可以开始检测报警，单击 Reset 项复位报警信号。

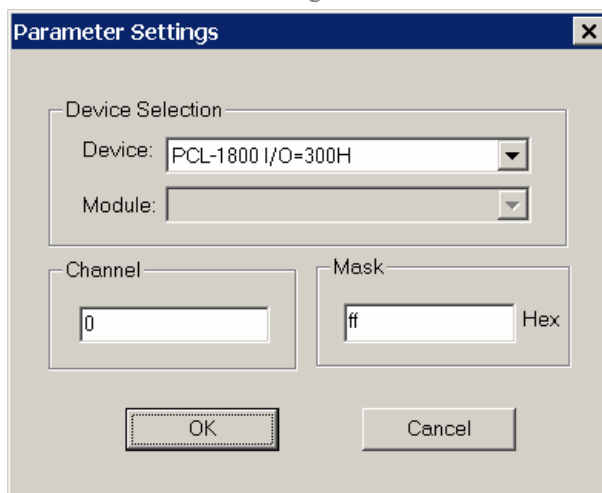
DIGOUT（数字量输出）:

数字量输出例程：该例程主要使用 PT_DioWriteBit/PT_DioWritePortByte 配置数字量输出通道等信息，使用数字量输出函数（DRV_DioWriteBit（）：按位输出；DRV_DioWritePortByte（）：按字节输出）；通过 PT_DioGetCurrentDOByte 配置回读通道等信息，使用 DRV_DioGetCurrentDOByte 读回当前的数字量输出状态。

1) 启动程序之后的界面如下图所示:



2) 单击 Setting 菜单后弹出 Parameter Setting 对话框:



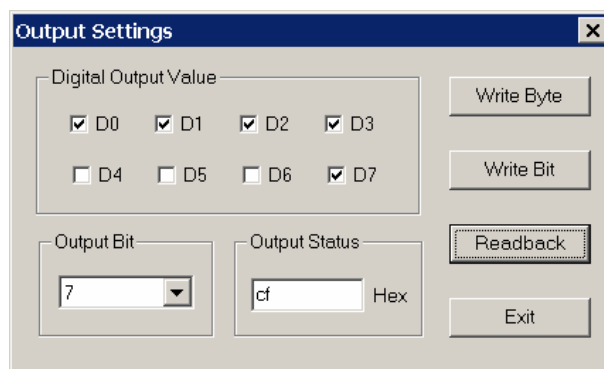
参数含义：Device 选择计算机中安装的板卡；

Module 选择计算机中安装的模块（因为本机未装模块，故不能用）；

Channel 输出通道的选择；这里要注意的是：因为后面的输出对话框中实际上只有 8 个 bit 的数据，所以板卡上面每个十六位的通道在这里实际上是对应两个通道的。

Mask: 输出形式数据类型为 16 进制数据

3) 设置结束之后点击 Run 菜单，即可弹出输出对话框，要使用这个对话框必须了解这个对话框中各个参数的含义



Write Byte: 按字节输出；

Write Bit: 按位输出；

ReadBack: 回读输出值并显示在 Output Status 编辑框中；

D0~D7: 选中与否标着这个位是否输出；

Output Bit: 用来选择输出的 bit 位是哪一位（0~7 对应 D0~D7），

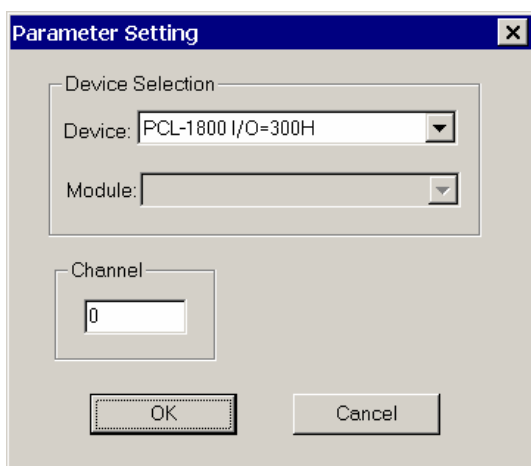
在使用 Write Bit 的时候，只有 Output Bit （0~7)对应的（D0~D7）那一位改变的时候 ReadBack 的返回值（Output Status）才会改变。

Exit: 退出当前窗口。

COUNTER（计数程序）

计数例程: 该例程通过 PT_CounterConfig/ PT_CounterStart/PT_CounterEventRead 来配置计数通道等设置，通过 DRV_CounterEventStart 函数启动了计数功能，使用 DRV_CounterEventRead 函数读取计数结果。

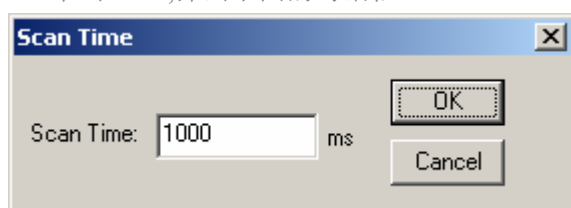
1) 单击 Setting 菜单弹出下面的对话框：



Device:显示所安装的设备，如果你安装了多块板卡可以在这里进行选择；

Channel: 选择计数通道（1800 选择计数器 0，将待计数信号从 Counter0 CLK，GND 接入）；

2) 单击 Scan,弹出下面的对话框：



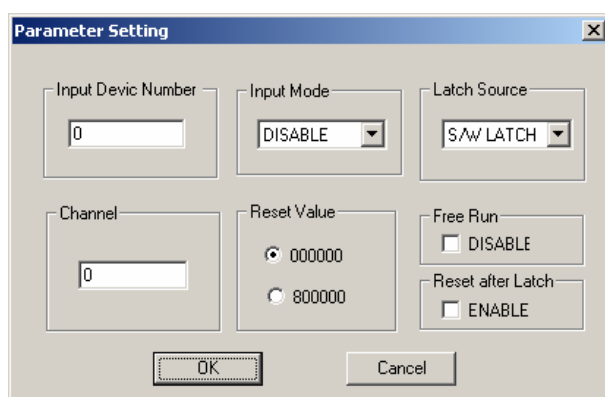
可以设置计数的时间间隔，默认值为 1000 毫秒

1) 单击 Run 菜单项中的 Start 菜单就可以开始计数，显示在窗口中间，单击 Stop 项停止计数

QCOUNTER(计数程序)

计数例程：该例程通过 PT_QCounterConfig/ PT_QCounterStart/PT_QCounterRead 来配置计数通道等设置，通过 DRV_QCounterStart 函数启动了计数功能，使用 DRV_QCounterRead 函数读取计数结果。

1) 单击 Setting 菜单弹出下面的对话框：



Input Device Number:输入设备号，默认值为 0。

Input Mode: 输入模式

Latch Source: 锁定源

Channel: 选择计数通道，默认值为 0；

Reset Value: 重置值

Free Run: 自由运行

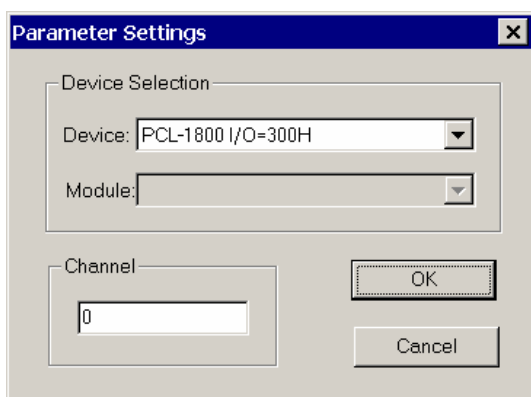
Reset after Latch: 锁定后重置

3) 单击 Run 菜单开始计数

Digin (数字量输入例程)

数字量输入例程（软件触发模式）：该例程主要使用 PT_DioReadPortByte 配置数字量输入通道等信息，使用数字量输入函数（DRV_DioReadPortByte，读字节函数），通过软件触发方式（使用 Windows Timer）实现数据采集。

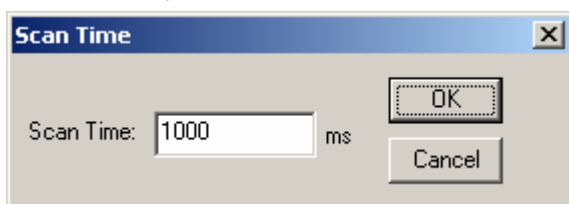
1) 单击 Setting 菜单弹出下面的对话框：



Device:显示所安装的设备，如果你安装了多块板卡可以在这里进行选择；

Channel: 选择数字量输入通道；

2) 单击 Scan,弹出下面的对话框：



可以设置计数的时间间隔，默认值为 1000 毫秒

3) 单击 Run 菜单项中的 Start 菜单就可以开始察看数字量输入值，单击 Stop 项停止输入。

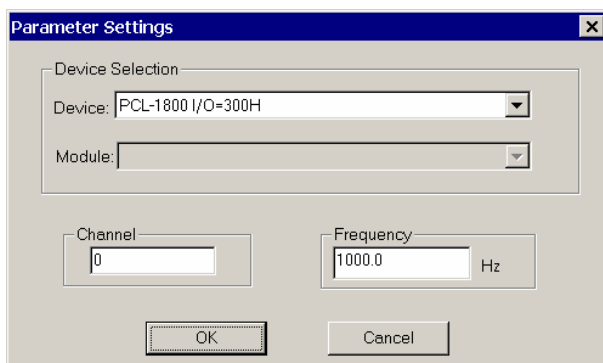
注：这里在屏幕中央看到的是读字节函数返回的结果。

FREQ/Daout/ (计频例程/模拟量/电流输出例程界面类似)

PULSE(脉冲输出例程)

脉冲输出例程：该例程通过 PT_CounterPulseStart 配置计数器输入通道等信息，使用 DRV_CounterPulseStart () 函数完成脉冲输出。

1) 单击 Setting 菜单弹出下面的对话框：



Device:显示所安装的设备，如果你安装了多块板卡可以在这里进行选择；

Channel: 选择脉冲输出通道，默认值 0 通道；

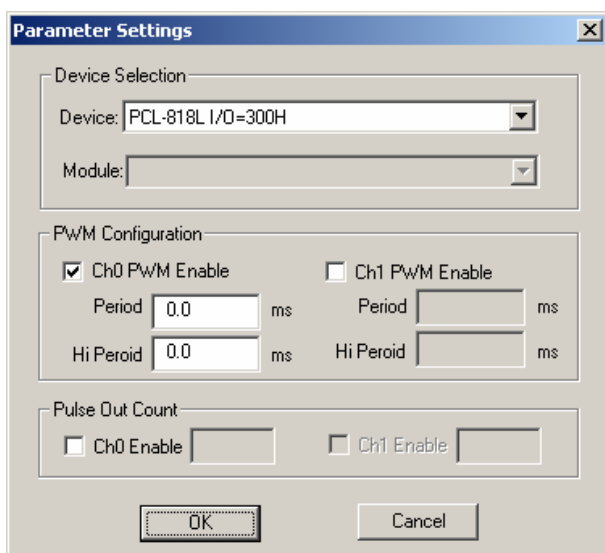
Frequency: 输出脉冲的频率，默认值 1000Hz;

4) 单击 Run 菜单项中的 Start 菜单就可以开始脉冲输出（用示波器连接 Counter0 Out 和 AGND，可以察看波形），单击 Stop 项停止输出。

PWM(宽度可调的脉冲输出)

宽度可调脉冲输出例程：该例程通过 PT_CounterPWMSetting 配置计数器输入通道等信息，使用 DRV_CounterPWMSetting（）和 DRV_CounterPWMAEnable（）函数完成脉冲输出。

1) 单击 Setting 菜单弹出下面的对话框：



The 'Parameter Settings' dialog box is used for configuring PWM output. It contains three main sections: 'Device Selection', 'PWM Configuration', and 'Pulse Out Count'. In 'Device Selection', 'Device' is set to 'PCL-818L I/O=300H' and 'Module' is empty. In 'PWM Configuration', 'Ch0 PWM Enable' is checked with a period of 0.0 ms, while 'Ch1 PWM Enable' is unchecked. In 'Pulse Out Count', both 'Ch0 Enable' and 'Ch1 Enable' are unchecked. 'OK' and 'Cancel' buttons are at the bottom.

Device:显示所安装的设备，如果你安装了多块板卡可以在这里进行选择；

PWM Configuration:选择 PWM 的输出通道；Period/ Hi Period:设置输出脉冲一个周期的时间/高电平的宽度，单位毫秒。

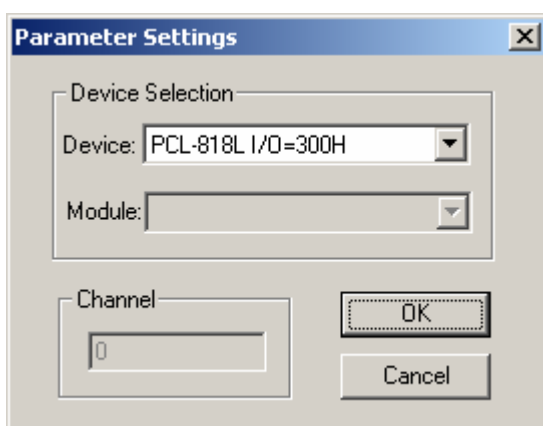
Pulse Out Count:输出脉冲计数

5) 单击 Run 菜单项中的 Start 菜单就可以开始脉冲输出，单击 Stop 项停止输出。

Pwmin(输入脉冲宽度测量)

输入脉冲宽度测量例程：该例程通过 PT_PWMStartRead 配置计数器输入通道等信息，使用 DRV_PWMStartRead（）函数完成脉冲宽度测量。

1) 单击 Setting 菜单弹出下图的对话框：

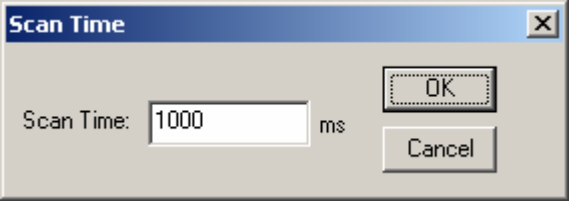


This 'Parameter Settings' dialog box is for pulse width measurement. It includes 'Device Selection' with 'Device' set to 'PCL-818L I/O=300H' and an empty 'Module' field. It also has a 'Channel' input field with the value '0'. 'OK' and 'Cancel' buttons are at the bottom.

Device:显示所安装的设备，如果你安装了多块板卡可以在这里进行选择；

Channel: 选择脉冲输出通道，默认值 0 通道；

6) 单击 Scan 菜单弹出下面的对话框：



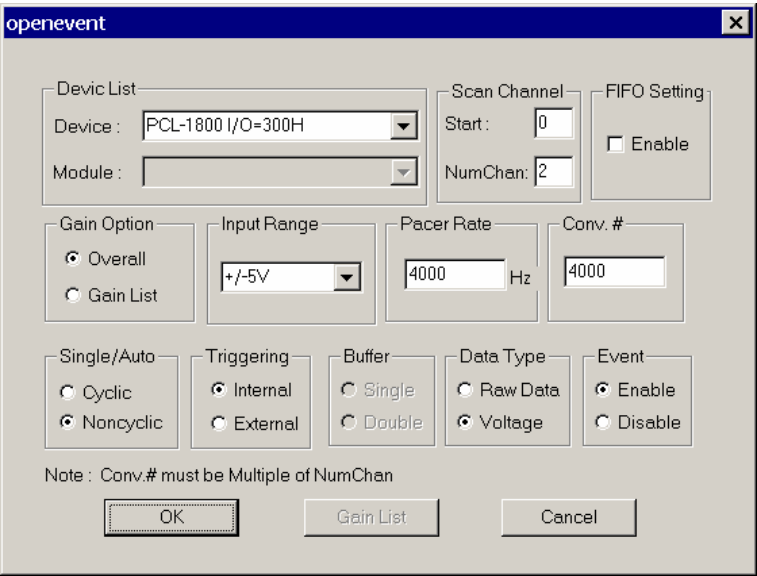
设置输出测量脉冲宽度的时间间隔，默认值为 1000 毫秒。

3)单击 Run 菜单项中的 Start 菜单就可以开始测量脉冲宽度并在窗口的中央显示测量结果，单击 Stop 项停止测量。

MAIntf(多通道中断采集例程)

多通道模拟量数据采集例程（中断模式）：该例程通过 PT_FAIntScanStart 函数启动了中断功能，该功能运行于后台，可以使用 DRV_FAICheck 函数检查工作状态，同时可以使用 DRV_FAITransfer 函数传输数据.另外：该例程支持用户设定 FIFO 大小。

1) 单击 Setting 菜单弹出如下对话框：



对话框重的参数含义如下：

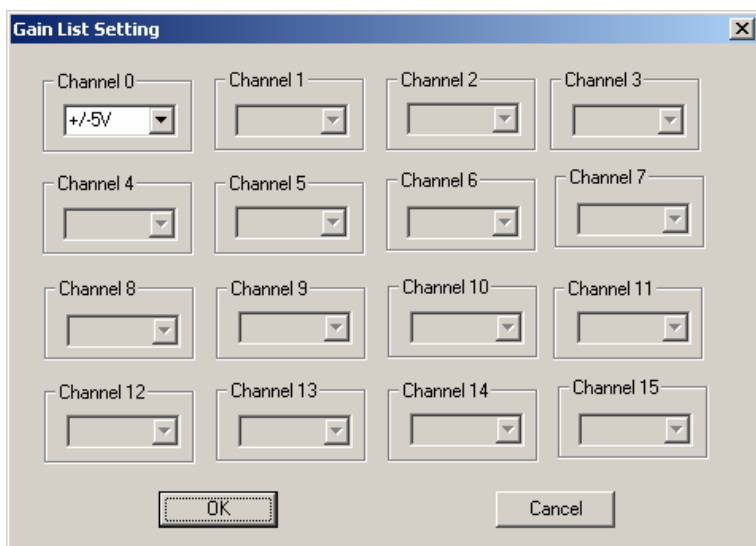
Device List 列表框，可以选择已安装的设备。

Scan Channel 中：Start::设置要扫描的起始通道号，NumChan: 设置从起始开始往后要扫描的通道的数目。

FIFO Setting:设置是否使用 FIFO, 选中 Enable 后可以设置在程序重要使用的 FIFO 的大小，注意设置值不能超过板卡上面 FIFO 的大小。

Gain Option:选择增益，这里选择 Overall 选项，Input Range: 为所有的通道选择相同的量程。

若选择 GainList 选项，则可以看到下面的 Gain List 按钮被激活，单击该按钮就可以对各个通道分别进行设置，设置的对话框如下所示：



Pacer Rate: 设置采样频率

Conv.#: A/D 转化的数目, 注意: Conv.#设置的数值必须是 FIFO 大小一半的整数倍, 同时还必须是设定的通道数 (NumChan) 的整数倍。

Single/Auto:两个选项 Cyclic:循环模式; Noncyclic: 非循环模式;

Triggering: 触发方式, Internal 内部触发; External 外部触发;

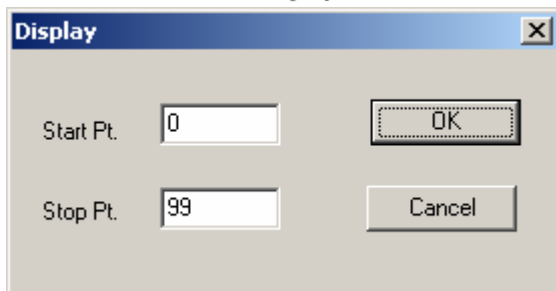
Buffer: 使用单个 buffer(single), 双 buffer(double)

Data Type: 数据类型; Raw Data 原始数据——二进制/十六进制——未经过 DA 转换的数据;

Voltage: 真实的电压数据。

Event: 事件; 是否允许设置事件来编程 (只是对中断和 DMA 方式来讲)。

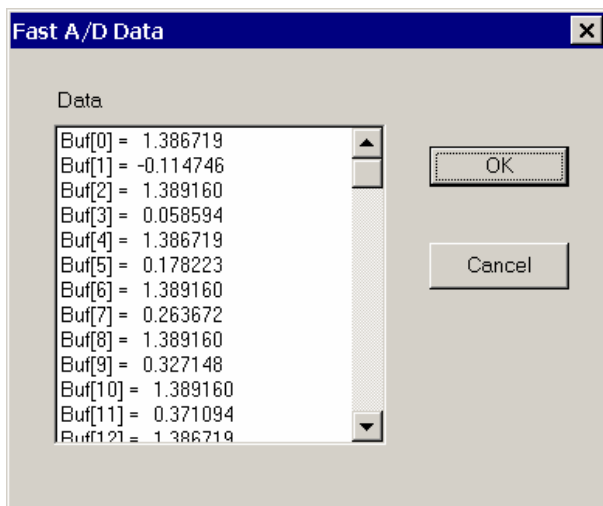
3) 设置完成后单击 Display,弹出下面的对话框:



设置将要现实的数据的范围, 默认 (0~99) 注: Stop Pt 不能大于 Conv.#设置的数值。

2) 单击 Run 菜单项开始采集数据, 当采集完成 Conv.#设置的 A/D 转换次数之后, 就会显示 (假设没有修改 Display 菜单中的默认值) 0~99 点的数据。数据显示窗口如下所示:

注: 采集数据的时候我们在通道 0 上接了一个干电池, 通道 1 上面没有接任何信号, 采集到的数据为随机的量。

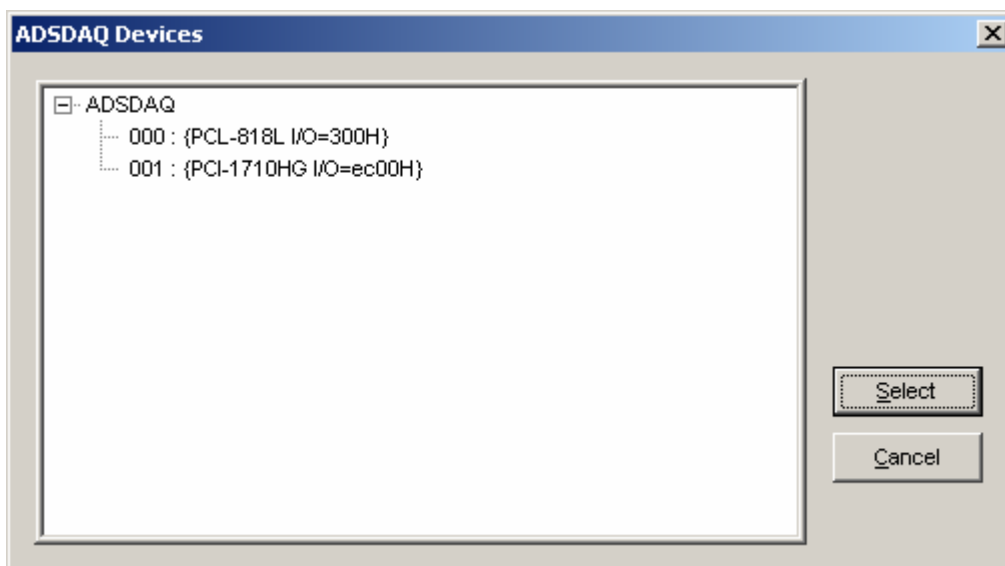


Madint/Maddma/DualDMA 例程与上面的例程界面类似，不过有的例程不能修改 FIFO 的大小。

CDADINT(看门狗中断数据传输例程)

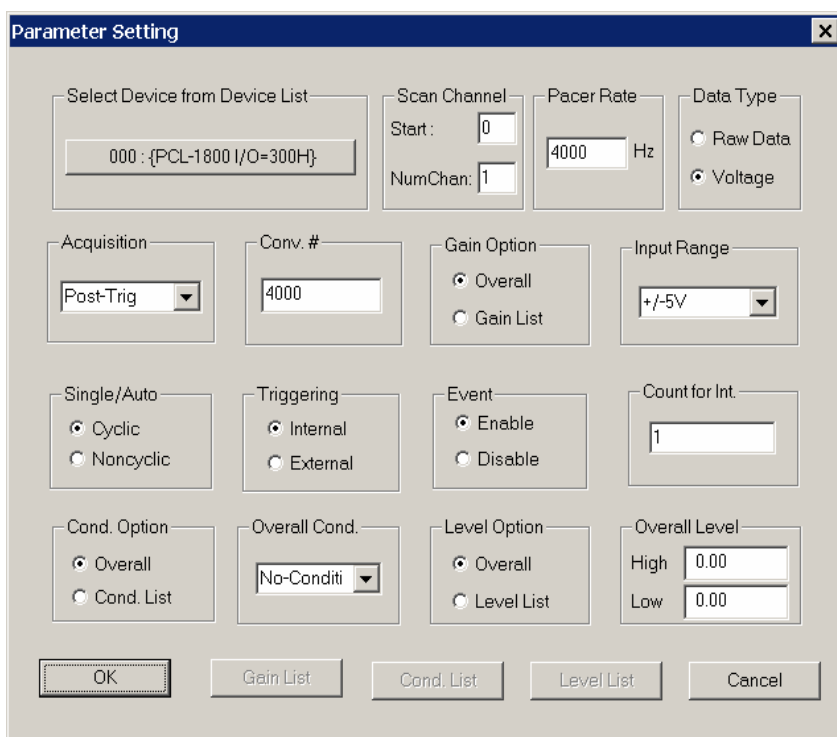
看门狗中断数据传输例程：该例程主要使用 DRV_FAIWatchdogConfig 配置模拟量输出通道等信息，使用模拟量输出函数（DRV_FAIDmaWatchdogStart），通过中断方式实现模拟量数据采集。

1) 单击 Setting 菜单弹出如下的设备选择对话框；



从中选择支持该例程的设备。

2) 单击 Select 按钮，弹出如下的设置窗口：



The **Parameter Setting** dialog box contains the following controls:

- Select Device from Device List:** A text box showing "000 : {PCL-1800 I/O=300H}".
- Scan Channel:** "Start:" is a text box with "0"; "NumChan:" is a text box with "1".
- Pacer Rate:** A text box with "4000" and a "Hz" label.
- Data Type:** Radio buttons for "Raw Data" and "Voltage" (selected).
- Acquisition:** A dropdown menu showing "Post-Trig".
- Conv. #:** A text box with "4000".
- Gain Option:** Radio buttons for "Overall" (selected) and "Gain List".
- Input Range:** A dropdown menu showing "+/-5V".
- Single/Auto:** Radio buttons for "Cyclic" (selected) and "Noncyclic".
- Triggering:** Radio buttons for "Internal" (selected) and "External".
- Event:** Radio buttons for "Enable" (selected) and "Disable".
- Count for Int.:** A text box with "1".
- Cond. Option:** Radio buttons for "Overall" (selected) and "Cond. List".
- Overall Cond.:** A dropdown menu showing "No-Conditi".
- Level Option:** Radio buttons for "Overall" (selected) and "Level List".
- Overall Level:** "High" and "Low" text boxes, both containing "0.00".
- Buttons:** "OK", "Gain List", "Cond. List", "Level List", and "Cancel".

Device List 按钮可以重新选择已安装的设备。

Scan Channel 中: Start::设置要扫描的起始通道号, NumChan: 设置从起始通道开始往后要扫描的通道的数目。

Pacer Rate: 设置采样频率

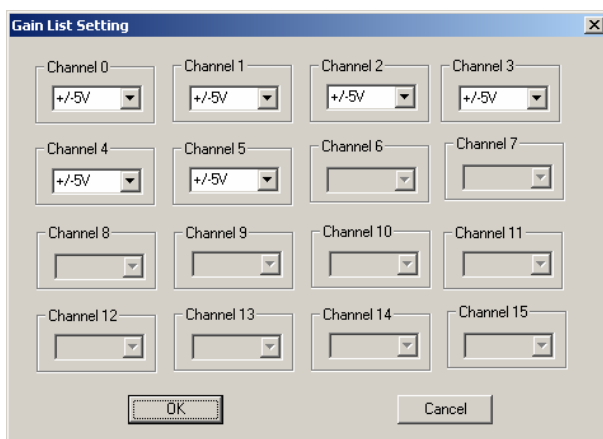
Data Type: 数据类型; Raw Data 原始数据——二进制/十六进制——未经过 DA 转换的数据;

Voltage: 真实的电压数据。

Acquisition:采集的类型, Post-Trig:Pre-Trig

Conv.#: A/D 转化的次数, 必须为通道数的整数倍

Gain Option:选择增益, 这里选择 Overall 选项, 则 Input Range: 为所有的通道选择相同的量程。若选择 GainList 选项, 则可以看到下面的 Gain List 按钮被激活, 单击该按钮就可以对各个通道分别进行设置, 设置的对话框如下所示:



The **Gain List Setting** dialog box displays 16 channels (0 to 15) in a 4x4 grid. Each channel has a dropdown menu for its input range. Channels 0 through 5 show "+/-5V", while channels 6 through 15 show an empty dropdown. At the bottom are "OK" and "Cancel" buttons.

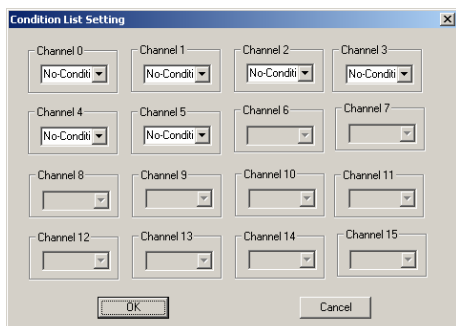
Single/Auto:两个选项 Cyclic:循环模式; Noncyclic: 非循环模式;

Triggering: 触发方式, Internal 内部触发; External 外部触发;

Event: 事件; 是否允许设置事件来编程 (只是对中断和 DMA 方式来讲)。

Count for Int: 产生中断的事件数

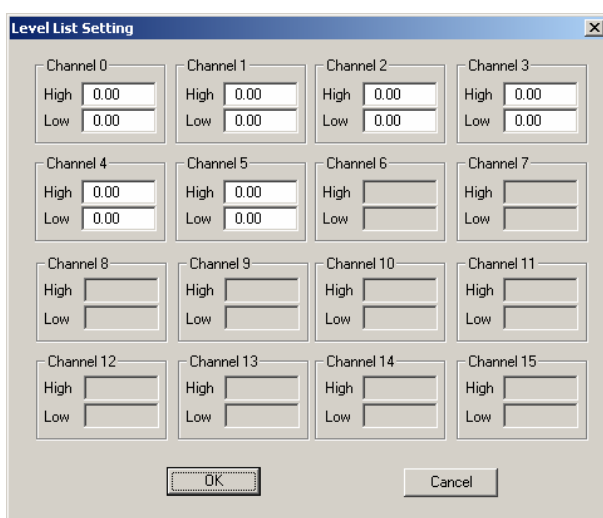
Cond Option:选择增益, 这里选择 Overall 选项, 则 Overall Cond: 为所有的通道选择相同的量程。若选择 Cond.List 选项, 则可以看到下面的 Cond.List 按钮被激活, 单击该按钮就可以对各个通道分别进行设置, 设置的对话框如下所示:



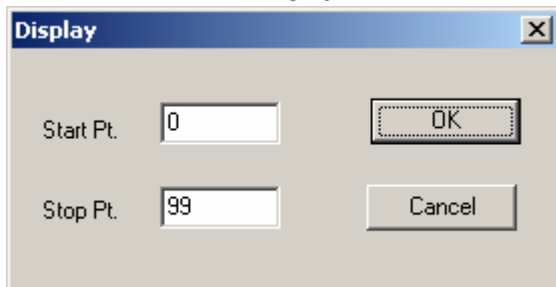
单击其中的一个列表框可以看到可选的信号调理的选项:

less:小于; between :介于中间; greater:大于; outside:在范围之外。

Level Option:选择增益, 这里选择 Overall 选项, 则 Overall Level 为所有的通道选择相同的量程。若选择 Level List 选项, 则可以看到下面的 Level List 按钮被激活, 单击该按钮就可以对各个通道分别进行设置要采集的数据的上下界, 设置的对话框如下所示:



3) 设置完成后单击 Display,弹出下面的对话框:



设置将要现实的数据的范围, 默认 (0~99) 注: Stop Pt 不能大于 Conv.#设置的数值。

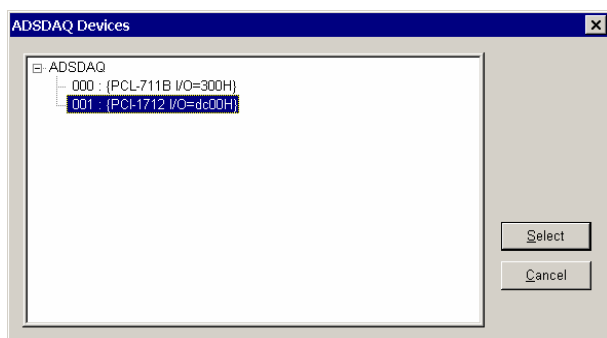
4) 单击 Run 菜单项开始采集数据, 当采集完成 Conv.#设置的 A/D 转换之后, 就会显示 (假设没有修改 Display 菜单中的默认值) 0~99 点的数据。数据显示窗口如下所示:

CDADDMA(看门狗 DMA 数据传输例程)界面及设置与该例类似, 不再赘述。

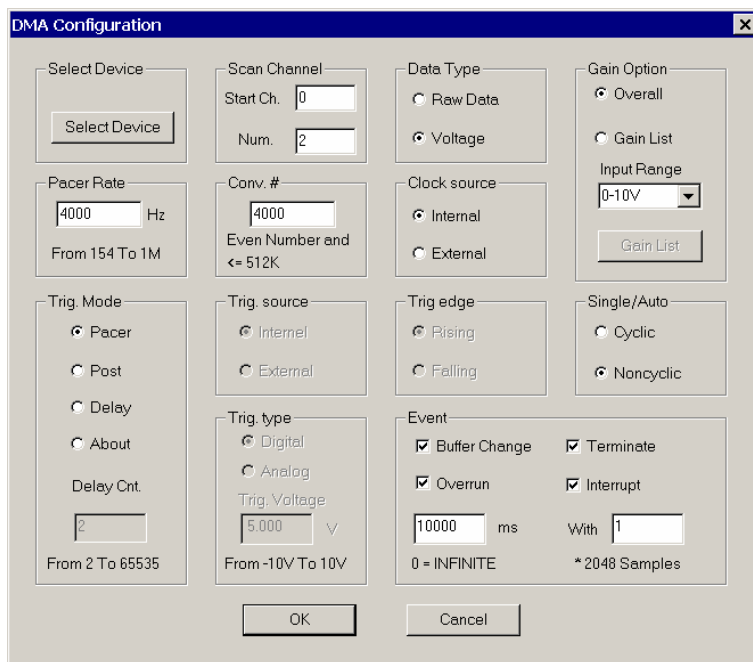
BDADDMA（总线控制 DMA 传输方式）

总线控制 DMA 数据采集例程：该例程通过 DRV_FAODmaStart 函数启动了 DMA 功能，该功能运行于后台，可以使用 DRV_FAOCheck 函数检查工作状态，当工作结束，或者任何时刻，你都可以采用 DRV_FAOStop 来停止工作。

1) 单击 Setting，弹出设备选择窗口如下所示：



2) 选择设备点击 Select 按钮后的对话框如下图所示，



对话框中的参数含义如下：

Select Device from Device List 按钮可以弹出板卡选择的对话框。

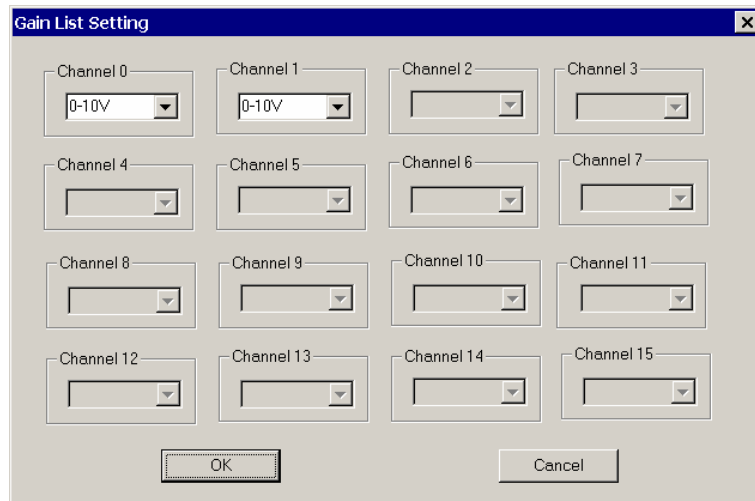
Scan Channel: Start Chan，起始通道号；Num，要扫描的通道数目。

Data Type: 数据类型；Raw Data 原始数据——二进制/十六进制——未经过 DA 转换的数据；

Voltage: 真实的电压数据。

Gain Option: 选择增益，选择 Overall 选项，表示对这个板卡的各个通道的输入量程进行统一设置，下面的 Input Range: 为所有的通道选择相同的量程范围。

选择 GainList 选项，则弹出如下对话框对各个通道的输入增益进行独立设置。



Pacer Rate: 设置采样频率, 范围在 154~1M, 默认值 4000。

Conv.#: A/D 转化的数目, 默认值 4000, 偶数且小于 512k, 且应为通道数的整数倍。

Clock Source: 时钟源, Internal, 内部时钟; External: 外部时钟;

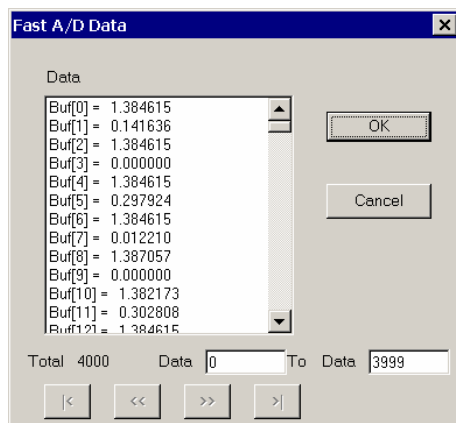
Trigger Mode: Pacer, 脉冲触发; Post: 后触发; Delay: 延迟触发; About: 中间触发;

Post: 后触发选择之后, Trig Edge/Trig.Type 成为可用, 其中: Rising 上升沿触发; Falling: 下降沿触发; Digital: 数字信号触发; Analog: 模拟量触发, 下面的选择框可以选择触发的电平。

Single/Auto: 两个选项 Cyclic: 循环模式; Noncyclic: 非循环模式;

Event: 允许的事件类型: Buffer Change, Buffer 改变事件; Terminate: 结束事件; Overrun: 溢出事件; Interrupt: 中断事件;

3) Run 菜单项开始采集数据, 当采集完成 Conv.# 设置的 A/D 转换次数之后, 就会显示 0~3999 点的数据。显示窗口如下所示:



Interrupt Counter = 2
Buffer change Counter = 1
Overrun Counter = 0
Terminate Counter = 1

Total: A/D 转换的总数, 显示的数据从第 0 个到第 3999 个。

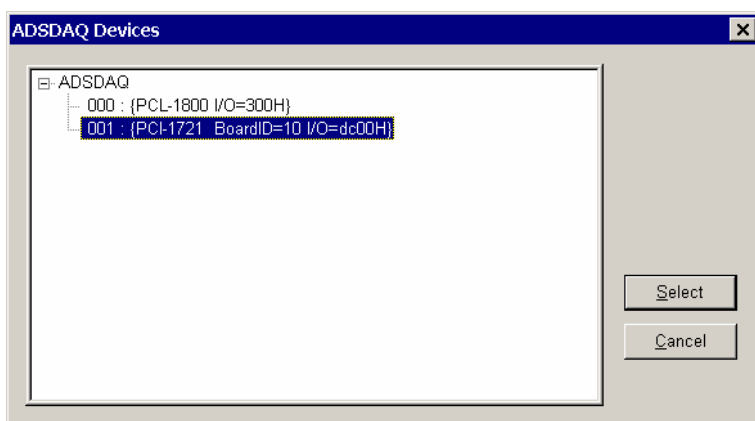
采集数据: 在通道 0 上加一节干电池, 通道 1 上加一个小的正弦信号。

右边的一幅图显示了在采集过程中产生的 (Interrupt Counter) 中断事件, (Buffer change) Buffer 改变事件, (Overrun Counter) 溢出事件, (Terminate Counter) A/D 转换结束事件数目。

WDADMA(总线控制 DMA 方式模拟波形输出)

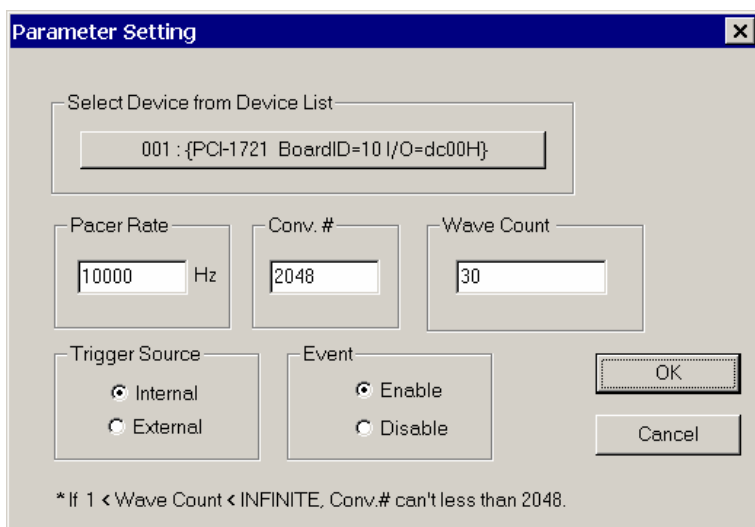
总线控制 DMA 方式数据输出例程：该例程主要使用 PT_FAOWaveFormStart 配置模拟量输出通道等信息，使用模拟量输出函数（DRV_FAOWaveFormStar），通过 DMA 方式实现模拟量波形输出。

1) 单击 Setting 菜单弹出如下的设备选择对话框：



从中选择支持该例程的板卡。

2) 单击 Select 按钮，弹出如下的设置窗口：



Select Device from Device List:

单击下面的按钮就可以从已经安装的设备中选择支持该例程的板卡。

Pacer Rate: 脉冲频率，默认值 10000Hz

Conv.#: D/A 转换的点数，默认值 2048。

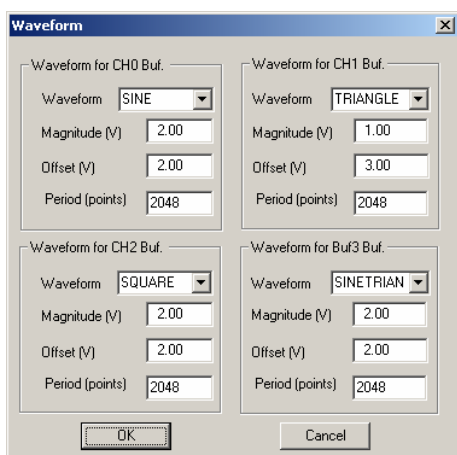
Wave Count: 波形数。注：Pacer Rate=Wave Count *(Period)Point 的整数

Triggering: 触发方式，Internal 内部触发；External 外部触发；

Event: 事件；是否允许设置事件来编程（只是对中断和 DMA 方式来讲）。

注：如果波形数大于 1 小于无穷，则转换点数（Conv.#）,不能小于 2048。

2) 设置完毕后，单击 Waveform 菜单弹出如下对话框：



Waveform for Ch0 Buf, Ch0 Buf 中的波形设定。

Waveform:: 输出波形选择 (SINE (正弦波), TRIANGLE (三角波), SQUARE (方波))

Magnitude(V): 幅值, 默认值 2.00 伏

Offset(V): 偏移量, 默认值 2.00 伏

Period(points): 每周期输出的点数, 默认值是 2048 点。

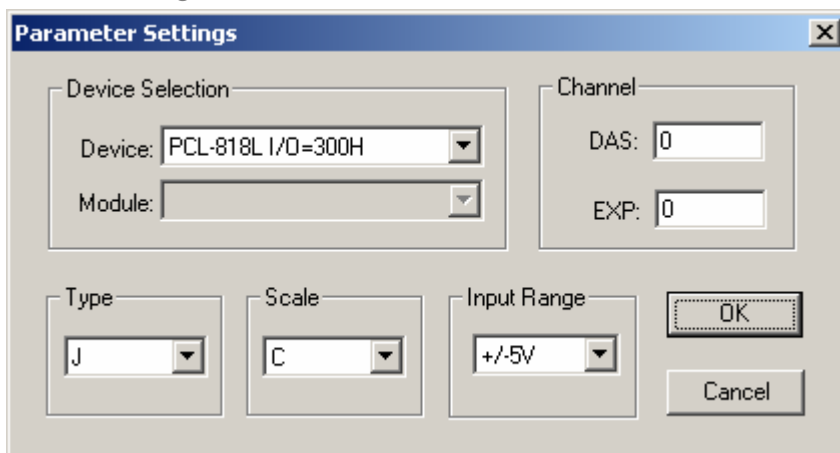
其它通道的设置类似, 不再赘述。

3) 波形设置完成之后, 可以在单击 Run 菜单即可进行数据输出。

THERMO(热电偶数据采集)

热电偶数据输入采集例程 (软件触发模式): 该例程主要使用 PT_TCMuxRead 配置模拟量输入通道等信息, 使用热电偶输入函数 (DRV_TCMuxRead), 通过软件触发方式 (使用 Windows Timer) 实现数据采集。

1) 单击 Setting 菜单弹出下面的对话框:



Device:显示所安装的设备, 如果你安装了多块板卡可以在这里进行选择;

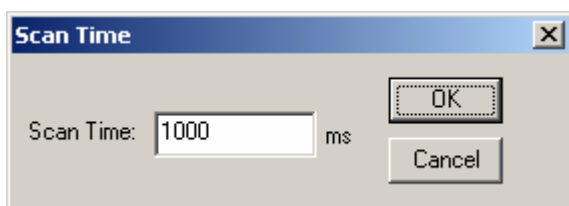
Channel: 选择通道 DAS: 数据采集卡的通道 EXP: 扩展通道;

Type: 热电偶类型选择

Scale: 比例放大器?。

Input Range: 输入范围选择。

2) 单击 Scan,弹出下面的对话框:



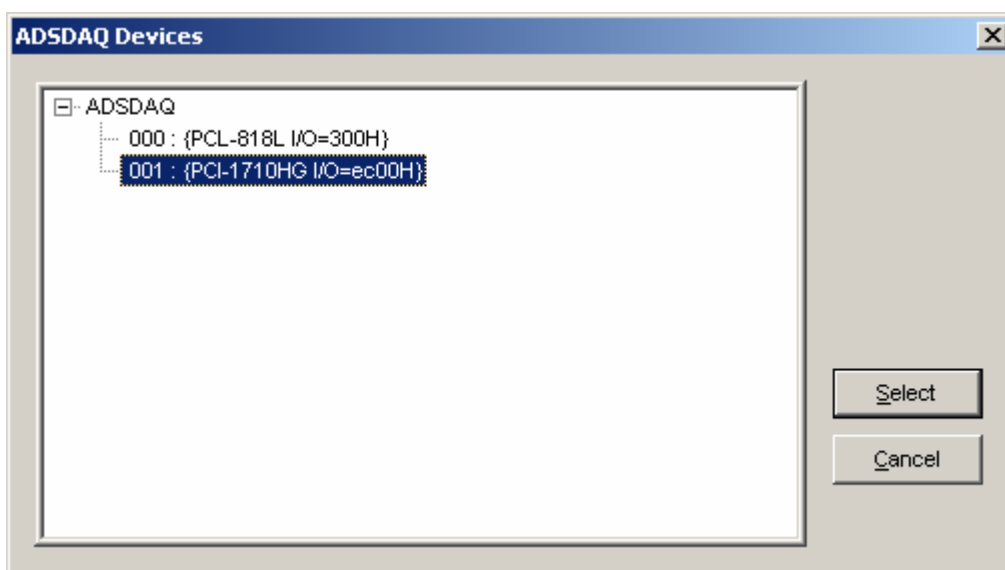
可以设置计数的时间间隔，默认值为 1000 毫秒

3) 单击 Run 菜单项中的 Start 菜单就可以开始采集输入值，单击 Stop 项停止。

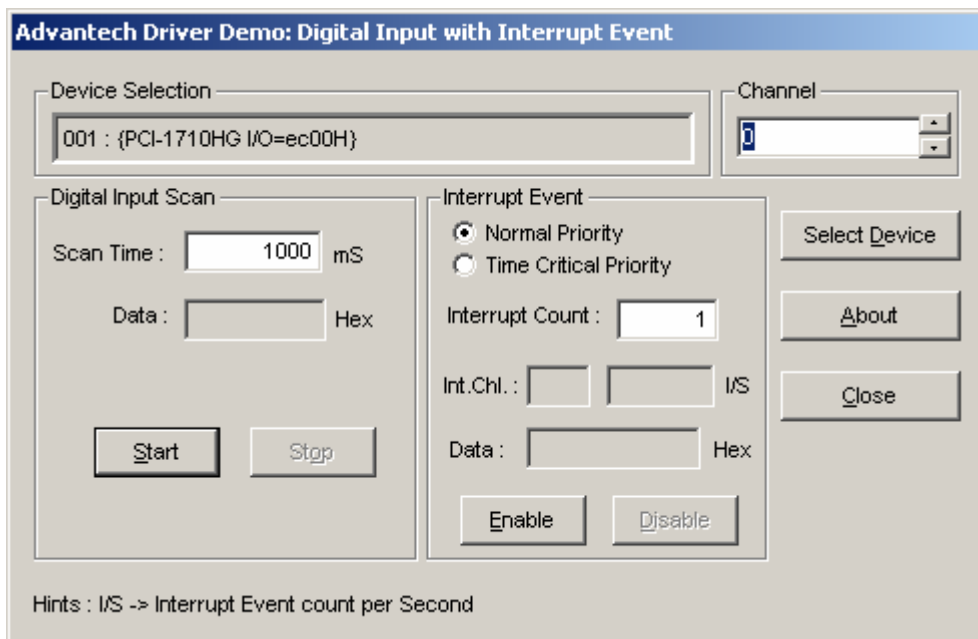
FDIGIN (中断事件数字量输入例程):

中断事件数字量输入例程：该例程主要说明了 PT_CheckEvent 及数字量输入函数 (DRV_FDITransfer) 的使用，实现了中断计数和数字量采集。

1) 运行程序，弹出如下对话框



2)选择设备后弹出如下对话框:



Device Selection:选择的设备;

Channel:输入通道号;

Scan time:设置端口扫描时间

Data:读取的端口数据（16 进制显示）

Interrupt Event:

设置中断的优先级别：Normal Priority，普通；Time Critical Priority,

Interrupt Count：中断数目；

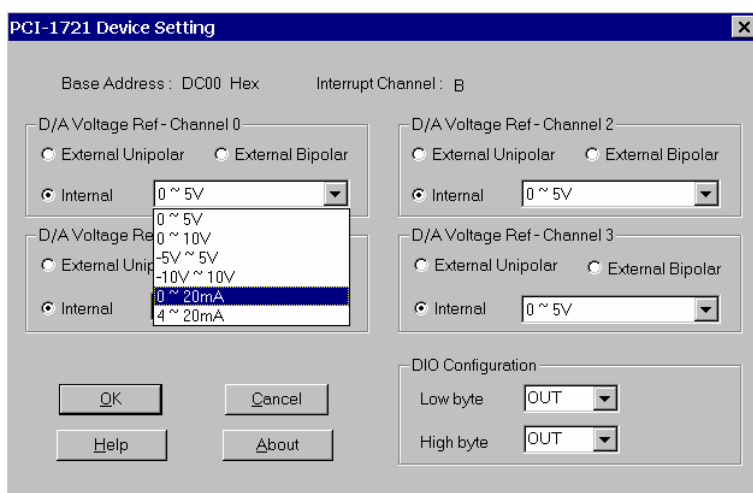
Int.chl:

Data:

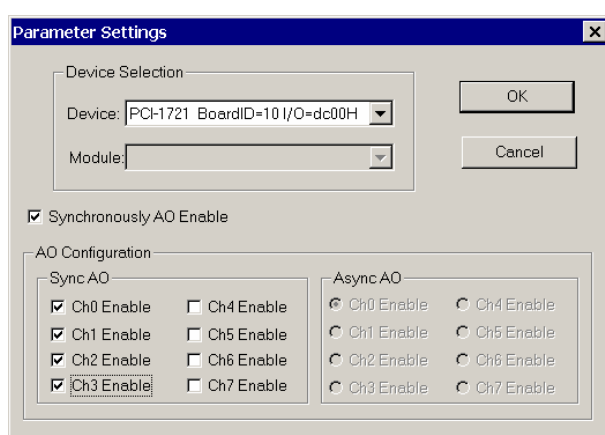
CSYNCAO（同步/异步电流输出）

电压输出例程（软件触发模式）：该例程主要使用 PT_AOVoltageOut 配置电压输出通道等信息，使用电压输出函数（DRV_AOVoltageOut），通过软件触发方式（使用 Windows Timer）实现电压输出。

注意：在使用电流输出的例程的时候，要先使用 Device Manger 通过 SetUp 将板卡（以 PCI-1721 为例）的输出调整到电流输出：



1) 点击 Setting 菜单，弹出如下的对话框：

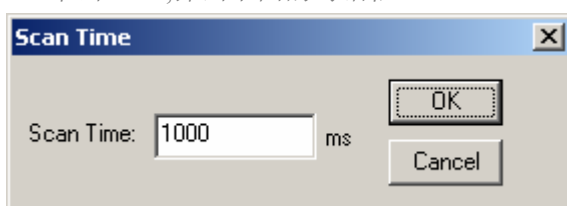


Device Selection:选择已安装的支持该例程的板卡

Synchronously AO Enable:选择是同步输出还是异步输出

当未选中该项的时候，右边的 AsyncAO，但选按钮可用
选中该项的时候，左边的复选框可用，这时候可以选择同步输出的通道。

2) 单击 Scan,弹出下面的对话框：



可以设置计数的时间间隔，默认值为 1000 毫秒

2) 单击 Run 菜单项中的 Start 菜单就可以开始电流输出。

注意：测量电流输出连接的管脚是：IOUTx(电流输入管脚)和+12V 脚；简单的办法是在这两个管脚之间跨一个电阻测电压。

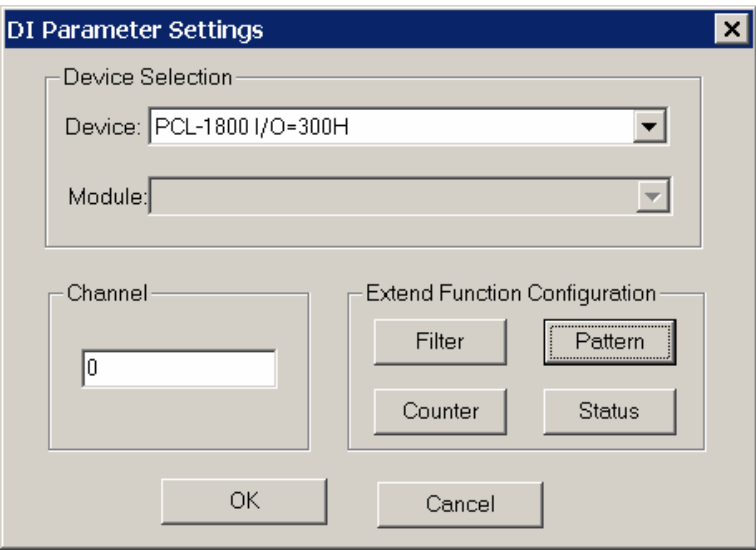
SYNCAO 为电压输出例程（使用 DRV_AOCurrentOut 函数），界面类似，不再赘述。

DiginEx（使用扩展板的数字量输入例程）

使用扩展板的数字量输入例程(软件触发模式):该例程主要使用 PT_DioReadPortByte 配置数字量输入通道等信息,使用 PT_EnableEventEx 启动扩展板输入功能，使用数字量输入

函数（DRV_DioReadPortByte，读字节函数），通过软件触发方式（使用 Windows Timer）实现数据采集

1) 单击 Setting 菜单弹出如下对话框：

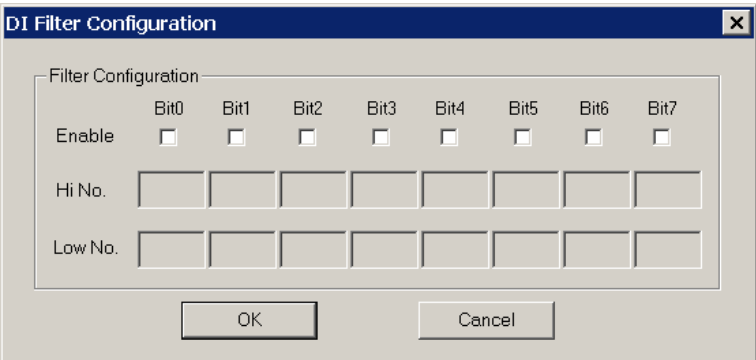


Device Selection:选择已安装的支持该例程的板卡

Channel： 选择通道；

Extended Function Configuration： 扩展函数配置

Filter： 数字滤波方式选择



Enable： 滤波使能端；

Hi No.： 高数字； ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ?

Low No.： 低数字； ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ?

Pattern： 数字模式配置

DI Pattern Configuration

Pattern Match Configuration

	Bit0	Bit1	Bit2	Bit3	Bit4	Bit5	Bit6	Bit7
Enable	☐	☐	☐	☐	☐	☐	☐	☐
High	☐	☐	☐	☐	☐	☐	☐	☐
Low	☐	☐	☐	☐	☐	☐	☐	☐

OK Cancel

Enable: 模式匹配使能断;

High: 高信号使能;

Low: 低信号使能;

Counter: 计数器配置

DI Counter Configuration

Counter Configuration

	Bit0	Bit1	Bit2	Bit3	Bit4	Bit5	Bit6	Bit7
Counter Enable	☐	☐	☐	☐	☐	☐	☐	☐
Preset Value								
Match Check	☐	☐	☐	☐	☐	☐	☐	☐
Match Value								
Overflow Check	☐	☐	☐	☐	☐	☐	☐	☐
Rising edge Trig	☐	☐	☐	☐	☐	☐	☐	☐
Falling edge Trig	☐	☐	☐	☐	☐	☐	☐	☐

OK Cancel

Counter Enable: 计数器使能复选框;

Preset Value: 预设值;

Match Check: 匹配检查;

Match Value: 匹配值;

Over Flow Check: 溢出检查;

Rising Edge Trig: 上升沿触发;

Falling Edge Trig: 下降沿触发;

Status: 状态配置

DI Status Configuration

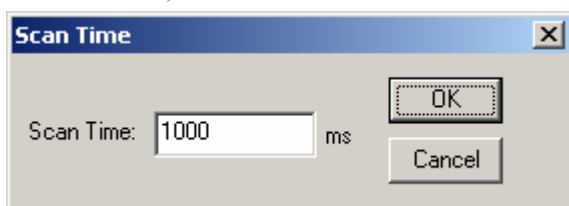
Status Change Configuration

	Bit0	Bit1	Bit2	Bit3	Bit4	Bit5	Bit6	Bit7
Enable	☐	☐	☐	☐	☐	☐	☐	☐
Rising Edge Trig	☐	☐	☐	☐	☐	☐	☐	☐
Falling Edge Trig	☐	☐	☐	☐	☐	☐	☐	☐

OK Cancel

Enable: 状态配置使能复选矿;
Rising Edge Trig: 上升沿触发;
Falling Edge Trig: 下降沿触发;

2) 单击 Scan,弹出下面的对话框:



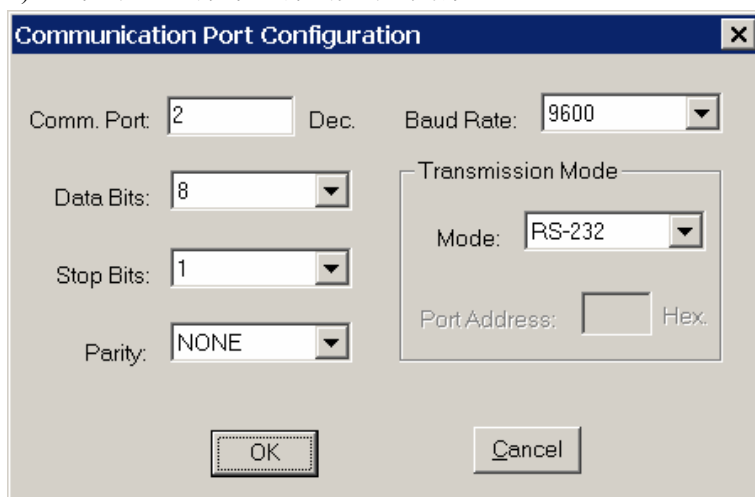
可以设置计数的时间间隔,默认值为 1000 毫秒

3) 单击 Run 菜单项中的 Start 菜单就可以开始输

CommPort (串口通信测试例程)

串口通信测试程序: 该例程完成串口通信测试功能。

1) 单击 Conf 菜单, 弹出如下对话框



Comm.Port:通讯端口选择 (十进制数)

Baud Rate:波特率;

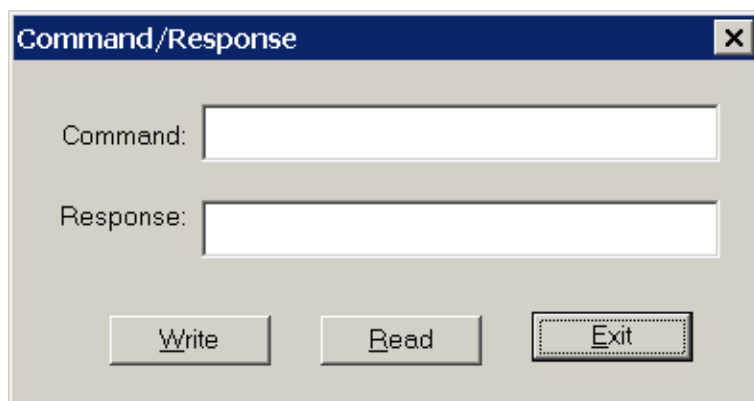
Data Bits:数据位;

Stop Bits:停止位;

Parity:奇偶校验选择

Transmission Mode:传输方式选择;

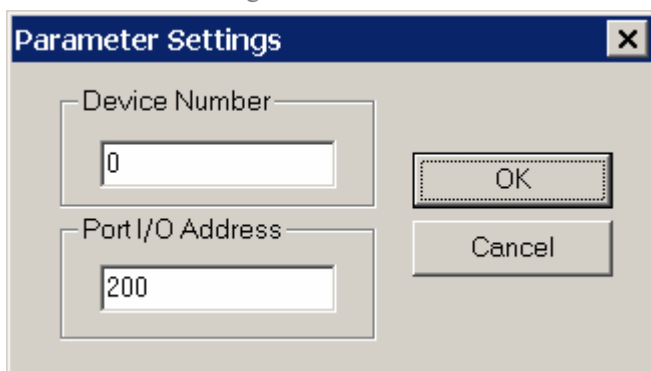
2) 单击 Read/Write 菜单, 弹出如下对话框



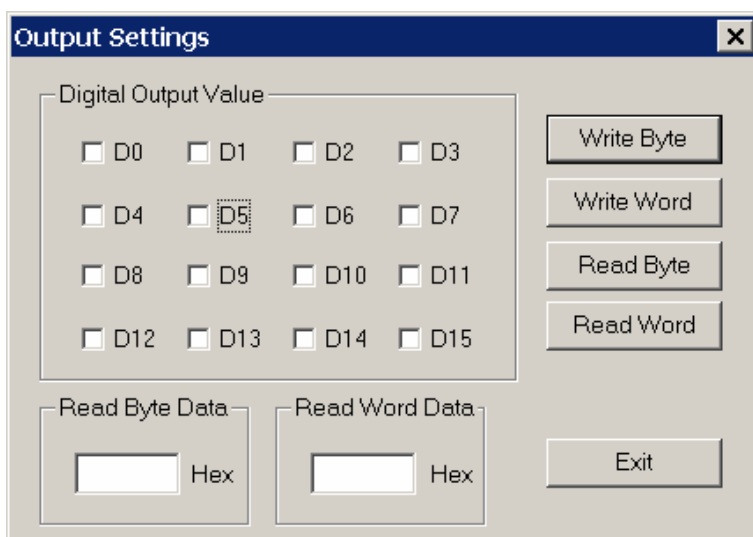
PortIO(端口输入输出例程)

直接对端口进行读入写出例程:通过 DRV_outp/DRV_outpw/DRV_inpw/DRV_inpw 直接对板卡的输入输出端口进行读写 (按字/字节)。

1) 单击 Setting 弹出如下对话框:



2) 单击 Run 弹出如下对话框:



板卡测试

CADDMA 程序用 PCL1800 进行测试的时候总是出现 Invalid DMA Channel 的出错信息。各个 DMA 通道都试过了, 5、6、7, 5&6 都不行。

第七章 驱动开发基础

10.1 虚拟设备初步

Windows95 是一个运行在最高级特权，第 0 级别的多线程操作系统。所有的应用程序都运行在最低级特权，第 3 级别上。这样就限制了应用程序对系统的操作。它们不能使用 cpu 特权指令，不能直接访问 I/O 端口，等等。你对 gdi32, kernel32 和 user32 这三个大的系统组件一

定很熟悉。你肯定会认为这样重要的代码段一定是在第 0 级别下运行的。但是实际上，它们和其他的应用程序一样，是在第三级别下运行的。这就是说它们并不比 Windows 计算器，或者扫雷游戏有更多的权限。系统的控制实权掌握在**虚拟级管理器(VMM)**和**虚拟设备驱动程序(VxD)**手中。

这一切都是由 dos 引起的。在 Window 3.x 的时代，在市场上有很多成功的 dos 软件。Windows 3.x 必须同时运行普通的 Windows 程序和 dos 程序，否则，它就会失去市场。这个局面是很难处理的，因为 dos 程序和 Windows 程序有本质的不同。dos 程序认为它们拥有系统的一切：键盘，cpu，内存，硬盘等等。dos 程序不知道怎样和其他程序合作，而 Windows 程序(从那时候起)是可靠的多任务合作系统。也就是每个 Windows 程序都必须通过 GetMessage 或 PeekMessage 来和其他程序进行交流。

解决办法就是，在一个 8086 虚拟机上运行所有的 dos 程序，而在另一个叫做**系统虚拟机**的虚拟机上运行其他所有的 Windows 程序。Windows 负责把 cpu 运算时间轮流的分给每个虚拟机。这样，在 Windows 3.x 里。Windows 程序之间用的是合作多任务，而虚拟机之间用的是优先级多任务。

什么是一个虚拟机？一个虚拟机是被软件创建的一个假象。一个虚拟机和在它上面运行的程序交互，就像这个程序是在真正的机器上运行一样。这样，一个程序不知道也不关心自己是否是在虚拟机上运行。只要虚拟机准确的像一个真的机器一样响应程序，我们就可以把它当成一个真正的机器。

你可以把虚拟机这种实机器和软件之间的接口看作一种 API。这种不寻常的 API 由中断，BIOS 调用和 I/O 端口组成。如果 Windows 能够以某种方法完美的模拟这个 API，那么在虚拟机上运行的程序就会表现的和它们在实际器上运行时完全一样。

这就是为什么会出现 VMM 和 VxD 的原因。为了协调和监视虚拟机(VMs)，Windows 需要一个程序来分配任务。这个程序就是虚拟机管理器(VMM)。

10.2 虚拟机管理器

VMM 是一个 32 位的保护模式程序。它的主要任务是建立和维护一个支持虚拟机的框架。例如，它要创建，运行和结束一个虚拟机。VMM 是众多的系统 VxD 程序之一，它被放在你的系统目录下的 VMM32.VxD 文件中。VMM 本身是一个 VxD 程序，但它被当作一个监视其他 VxD 程序的监视器。让我们来看一下 Windows95 的启动次序：

1. 加载 io.sys。
2. 执行 config.sys 和 autoexec.bat。

3. 调用 win.com。
4. win.com 运行 VMM32.VxD, VMM32.VxD 实际上是个简单的 dos 的 exe 文件。
5. VMM32.VxD 用 xms 驱动程序把 VMM 加载到扩展内存。
6. VMM 初始化自身及其它的默认 VxD。
7. VMM 把机器转入到保护模式并创建系统虚拟机。
8. 最后被加载的虚拟外壳设备在系统虚拟机上通过运行 krnl386.exe 来启动 Windows。
9. krnl386.exe 加载所有的文件, 最后是 Windows95 外壳。

正如你所看到的, VMM 是第一个被加载到内存的 VxD 程序。它创建系统虚拟机并初始化其他的 VxD 程序。它也为这些 VxD 程序提供许多服务。VMM 和 VxD 的操作模式和真正的程序不同。在大多数时候, 它们是潜伏的。当应用程序在系统中运行时, 这些 VxD 程序没有被激活。当某些需要它们处理的中断/错误/事件发生时, 它们才被唤醒。

VMM 是不可重入的。这意味着 VxD 程序必须使它们的访问和 VMM 服务同步。在有些情况下调用 VMM 服务是不安全的, 比如 VMM 正在处理一个硬件中断。在这段时间内, VMM 是不允许重进入的。作为一个 VxD 编写者, 你必须对你的所作所为极度的小心。记住, 你是在最高特权级别, 第 0 层级别, 如果你代码有错的话, 谁也管不到。

9.3 虚拟设备驱动程序

虚拟设备驱动程序被简称为 VxD。x 代表各种设备的名字, 如虚拟键盘驱动程序(vkd), 虚拟鼠标驱动程序(vmd)等等。VxD 程序是硬件成功初始化的途径。记得 dos 程序认为它们拥有系统的一切, 当它们在虚拟机中运行时, Windows 需要给它们一个实机器的替身。VxD 程序就是这些替身。VxD 程序通常虚拟一些硬件设备, 所以, 例如当一个 dos 程序认为它在同键盘通讯时, 实际是虚拟键盘驱动程序在和 dos 程序通讯。一个 VxD 程序通常控制真正的硬件设备并对该设备在各个虚拟机之间的共享进行管理。

尽管如此, 并不是说每个 VxD 程序必须和一个硬件设备相连。虽然 VxD 程序是用来虚拟硬件设备的, 但是我们也可以把 VxD 程序看作是在第 0 级别的 dll。例如, 如果你需要做一些只有在第 0 级别才能做的工作, 你就可以编一个 VxD 程序来为你完成这个工作。这样, 由于此 VxD 程序并没有虚拟任何设备, 你就可以把它仅仅看作是程序的扩展。

在我们更深入的讨论 VxD 和创建我们的 VxD 程序之前, 让我先说一些有关于 VxD 的事情。

- VxD 程序是 Windows 9x 特有的, 它在 Windows NT 下不能运行。所以如果你的程序是依靠 VxD 的, 它就不能被移植到 Windows NT 平台上去。
- VxD 是系统中权力最大的实体。由于它们可以对系统作任何事情, 所以它们是极度危险的。一个恶意的/错误的 VxD 程序可以毁掉整个系统。对于恶意的/错误的 VxD 程序没有任何的保护措施。
- 通常的, 不用 VxD 也有很多办法能达到你的目的。在采用 VxD 的解决办法之前一定要三思。如果用其他的可以在第三层级别实施的办法, 就使用这个办法。

Windows 95 下有两种 VxD:

- 静态 VxD
- 动态 VxD

静态 VxD 是那些从系统启动就被加载，在系统关闭之前一直存在于内存中的 VxD 程序。这种 VxD 可以追溯至 Windows 3.x 的时代。动态 VxD 是只有 Windows 9x 下才有的。动态 VxD 程序可以在需要的时候被加载/卸载。这些程序大多数都是用来控制设置管理器和输入输出监视器加载的即插即用设备的。你可以在你的 win32 应用程序里加载或卸载动态 VxD 程序。

9.4 VxD 程序之间的通讯

VxD 程序，包括 VMM，通过以下三种途径在相互之间进行通讯：

- 控制消息
- 服务 API
- 回调

控制消息：当有 VMM 感兴趣的事件发生时，它就向系统中所有载入的 VxD 程序发送控制消息。控制消息就像是第三层级别的 Windows 应用程序的消息。每个 VxD 程序都有一个接受和处理控制消息的函数，叫做**设备控制函数**。系统控制消息总共有 50 多个。控制消息不多的原因是系统中通常加载了很多 VxD 程序，而每个 VxD 程序在收到一个控制消息时都要进行处理。如果控制消息太多，就会导致系统停滞。所以控制消息只包括那些与虚拟机有关的重要消息，如：一个虚拟机被创建，被销毁等等。作为对系统控制消息的附加，一个 VxD 程序可以定义自己的控制消息，这些消息可以用来和那些能响应这些消息的 VxD 程序通讯。

服务函数：一个 VxD 程序，包括 VMM 在内，通常要导出一系列的被别的 VxD 程序调用的公共函数，这些函数被称为 VxD 服务。调用这些服务的机制和在第三层级别运行的应用程序有很大的不同：每个导出 VxD 服务的 VxD 程序**必须**有一个唯一的 ID，你可以从 Microsoft 得到一个这样的 ID。这个 ID 是一个包含了一个 VxD 唯一的身份验证的 16 位的数字，例如：

```

        UNDEFINED_DEVICE_ID    EQU 00000H
VMM_DEVICE_ID    EQU 00001H
DEBUG_DEVICE_ID  EQU 00002H
VPICD_DEVICE_ID  EQU 00003H
VDMAD_DEVICE_ID  EQU 00004H
VTD_DEVICE_ID    EQU 00005H

```

你可以看到 VMM 的 ID 是 1，VPICD 的 ID 是 3，等等。VMM 用这些 ID 来找到导出所需 VxD 服务的 VxD 程序。当一个 VxD 程序导出 VxD 服务时，它把所有服务的地址存在一个表里面。所以，你还需要通过服务分支表里面服务的索引来找到你所要的服务。例如，如果你要调用第一个服务，GetVersion 服务，你就要指定 0(这个索引是从 0 开始的)。调用 VxD 服务的实机机制包括中断 20h，你的代码产生一个中断 20h，并带有一个双字的值，这个值包含了设备 ID 和服务索引。例如，如果你要调用一个 VxD 程序导出的 VxD 服务，假设 VxD 程序设备 ID 是 000DH，服务号码是 1，那么代码应该是：

```

        int 20h
        dd 000D0001h

```

跟在中断 20H 后的双字的高字包含设备 ID。低字是在服务列表中的索引。当 20H 中断执行时，VMM 得到了控制权，并马上检测跟着的双字。然后它提出设备 ID 用来找到 VxD 程序，用服务索引来定位在那个 VxD 程序中的所要求的服务的地址。你可以看到这个操作时很费时的。VMM 必须浪费很多时间来定位 VxD 程序和所要服务的地址，所以 VMM 作了个小小的弊。当中断 20H 操作成功后，VMM **抓取链接**。这就是说，VMM 用直接的服务调用来替代 20H 中断和它后面的双字。所以上面的 20H 中断代码片断就被改变成：

```
call dword ptr [VxD_Service_Address]
```

这个把戏是成功的，因为 **int 20h+dword** 加一个双字用 6 个字节，正好和 **call dword ptr** 结构相等。所以接下来的服务调用是快速而有效的。这个方法具有直接性，简洁性。在好的一方面，它减轻了 VMM 和 VxD 载入器的工作量，因为它们不用定位 VxD 中**所有**的服务，那些没有执行过的服务将会保持原样。再不那么好的一方面，一旦一个静态 VxD 程序导出的服务被调用，那么就不可能把这个静态的 VxD 程序卸载了。由于 VMM 把调用锁定到 VxD 服务的实际地址上，如果提供这个服务的 VxD 程序从内存中被卸载了，其他 VxD 程序调用这个服务时就会很快的因为调用无效的内存地址而导致系统崩溃。没有办法来**消除**抓取的链接。这个问题的结论是动态 VxD 不适合作为服务提供者。

回调：回调或者回调函数是在 VxD 程序中给其他的 VxD 程序调用的函数，不要把回调函数和 VxD 服务搞混淆了。回调函数不像服务那样是公共的，它们是私有函数，VxD 在特定的情况下把它们的地址送给其他的 VxD 程序。例如，当一个 VxD 程序在处理一个硬件中断时，由于 VMM 是不可重入的，这个 VxD 程序不能使用 VxD 服务，否则会引起页面错误(重入 VMM)。这个 VxD 程序可以把它自己的一个回调函数的地址给 VMM，这样 VMM 就可以在能忍受页面错误时调用这个函数。回调函数的想法不是 VxD 独有的。许多 Windows API 都在用。最好的例子也许是窗口函数，你把窗口函数的地址填在 WINDCLASS 或 WINDCLASSEX 结构里并把它当作函数来调用 RegisterClass 或者 RegisterClassEx。当有这个窗口的消息传来时，Windows 就会调用你的窗口函数。另一个例子是窗口接管函数。你的程序把接管函数的地址送给 Windows，这样当你感兴趣的事件发生时，Windows 就会调用你的接管函数。

上述三种方法是 VxD 之间通讯的。

第八章 数据采集应用案例

为加深对于 PC_Based 数据采集系统的理解，我们精选部分应用案例供参考。

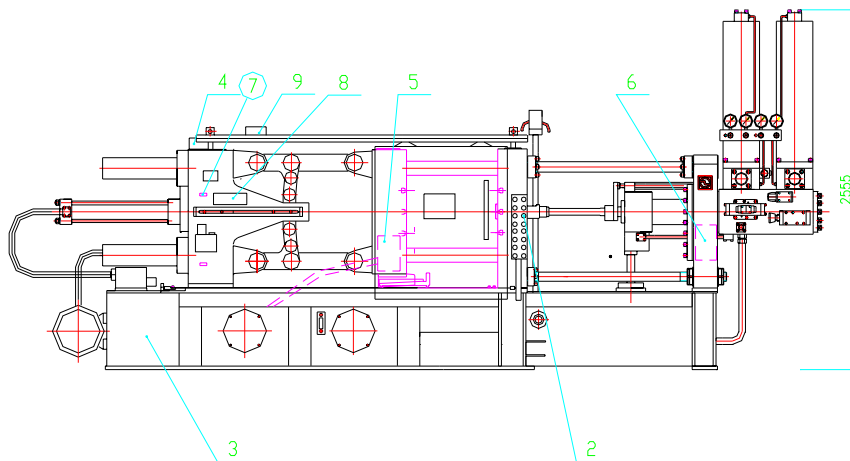
一、 压铸机计算机测控系统

引 言

随着压铸工业的迅速发展，尤其是对压铸产品质量越来越高的需求，给我们压铸工艺提出新的要求。那种只凭经验和用定性，而无定量工艺分析方法，将不再作为压铸工艺员的手段。然而我们目前用国外先进的测验系统，也十分昂贵。该测试系统可用于压铸生产时，对压铸机的各种工艺参数进行监控。由于采用了工业计算机、数据采集系统，使整个系统的精度较高，先进性也有很大提高。该系统上层软件采用组态软件，所以界面比较应用灵活，操作简便，画面美观，管理维护方便。压铸机合型力、压射力、压射速度、增压时间等主要参数，可随机得到，以帮助工艺分析，一目了然，并汇总成表，为系统分析作为基础，压铸机工艺参数测试仪，对广大压铸工艺人员、模具设计人员，压铸机设计人员将有较大的帮助。

传统的铸造多属于重力铸造，由翻砂造型加工得到，在这基础上发展了蜡模等等精密铸造。以后又有了离心铸造，低压铸造等等，压铸是浇铸到型腔后加压力成型的一种方法，它属于高压铸造。这种成形铸件比其它方法得到的铸件具有以下特点：1 表面光洁度好(可称无切削加工)2 强度高（强重比高）3 铸造效率高 4 有很好的经济性。

压铸机是专门用于满足以上工艺要求的机床，它主要有以下几个方面组成：1 合型机构：合型实现模具的造形，锁模力是主要参数，开型则可取加工另件。2 压射机构：通过压射冲头传递，使铸件在成型前得到要求比压，这一过程就决定铸件的品质 3 液压泵站：实现主机的各运动 4 电气控制：对主机的各种运动进行有效控制，满足工艺要求。参见如下简图：



自第一台压铸机问世以来，人类已经走过了二个多世纪，也逐步形成和完善了这门学科，压铸技术的确是一门综合性科学技术.我国自六十年代初第一台压铸机研制成功问世以来，至今已有四十多年的历史，我国的压铸工业也有了突飞猛进的发展，尤其是八十年代以后，乡镇企业和民营企业的出现，使传统的铸造业有了翻天覆地的变化，在加入 WTO 世贸组织后的今天，我们又有了一次很好的机遇，我们也面临着更多的挑战。世界上许多发达国家的加工铸件转入我国，加工铸件多了，但加工铸件的质量要求高.竞争价格低.交货期短等等,一系列的问题又特出了,我们的汽车工业要求铸件加工,必须有铸件加工过程记录报告,这也是铸件品质保证依据，现今许多汽车配件厂家只能依靠进口设备来完成上述要求，价格十分昂贵。根据我们掌握的情况，越来越多的压铸机将装备压铸检测装置，所以对压铸机检测仪器的研制和研究十分迫切，压铸检测仪的广泛使用必将对这一行业产生巨大作用，这一产品必有很好的发展前景。

1. 压铸机测控系统的技术要求

1.1 压铸性能的决定因素

压铸件的品质直接有压射性能决定,严格地掌握压射中参数变化的规律使其始终处于恒定状态，一直是压铸工作者努力的方向。图 1 是压铸机理想的冲头速度。从图中可以发现冲头的速度既是冲头所处位置的函数，又与冲头行程有关。位置 P1 为慢速起始速度，位置 P2 为向前运行的速度，在位置 P3 为充型阶段的速度以及在位置 P4 为瞬时实现制动到零的阶段。而在实际生产中不可能出现如图 1 所示的冲头速度曲线。实际的冲头速度曲线如图 2 所示。

利用现代化的测试手段，再加上测得各种最佳控制条件，使每次压射过程中反映在曲线上偏差最小，有其再现性，使铸件的前后质量都能有所保证。从铸件的实际需要出发，设定一种环绕着以速度曲线为底线的公差带,只要使每次压射时的速度处于公差带范围之内就能生产出合格的铸件。在每次压射过程中，如果超出了公差带的范围，则必须通过质量复检才能确定铸件是否合格。

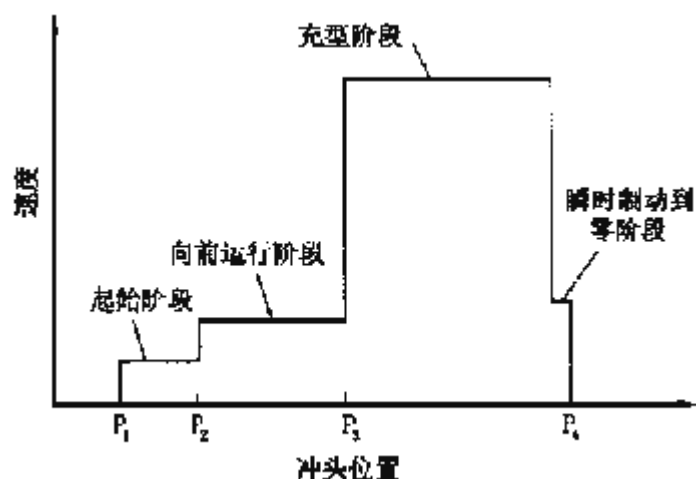


图 1 理想的冲头速度曲线

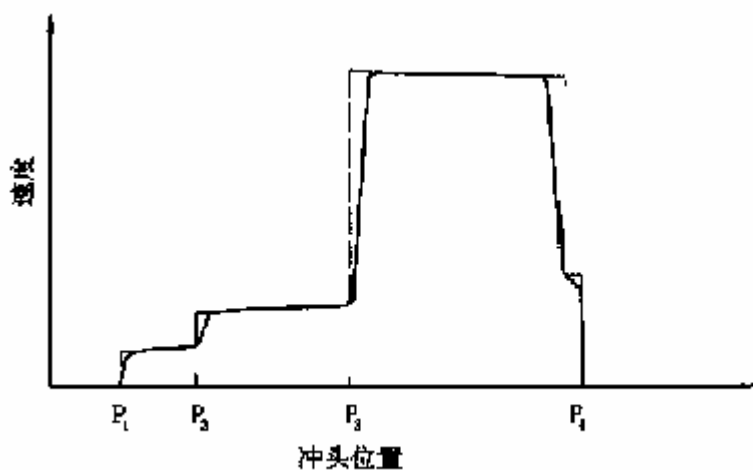


图 2 实际的冲头速度曲线

实际控制中，如果生产实际检测得到所出现的曲线如果与理想曲线偏差太大，必须通过压射控制调节来得到补偿。如充型时间平均为 20—80 ms，得良好的效果速度曲线的控制必须以测得参数要求来进行调整。此处总的反应时间的定义为：“通过传感器所确定的偏差值直至在瞬间冲头对此作出反应的这段时间间隔”。就是说液压系统中包括阀和驱动油缸的响应时间上然包括在总的反应时间内。这就要求系统阻尼小，运动部件质量轻。以上这些与我们讨论有关，与本次项目要求无关，但这个系统的建立为以后压铸机伺服系统的研制奠定基础。

1.2 测控系统的技术性能参数

- | | |
|--------------|------------------|
| 1、合型大杆直线变形位移 | 0~1mm, 误差±5% |
| 2、压射位移测试精度 | ±2mm, (高速 10M/S) |
| 3、压射压力测试精度 | ±5% |
| 4、压射过程采集时间范围 | 0~1、2、3、4、5 秒 |
| 5、压射行程测量 | 200~1000mm |

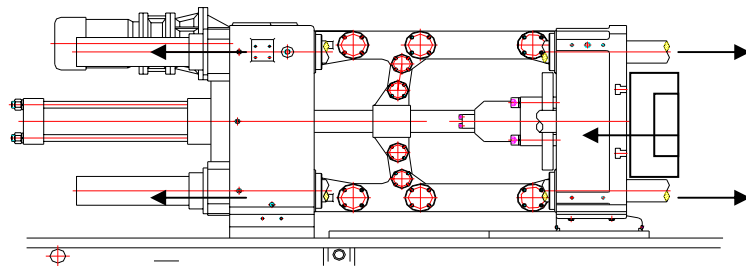
- | | |
|---------------|---------------|
| 6、使用环境温度 | 0~40℃ |
| 7、使用相对湿度 | ≤85% |
| 8、电源 | AC220V, 500VA |
| 9、系统压力为 14Mpa | |

本项目采用工控机控制,通过输入和输出实现各种运动的逻辑控制,对压铸机压射过程的锁模力和压射全过程进行参数记录,并进行系统数据管理,帮助压铸工艺员、模具设计员分析,指导生产。

2. 检测对象信号的分析及传感器的选用

主要检测对象: 压铸机锁模力

压铸机压射压力、压射行程、压射速度、压射建压时间



2.1 锁模力的检测

动模板受压,四根大杠如上图受力方向,所以四根大杠的受力之和等于锁模力。因此我们可以测定大杠受力等于锁模力。大杠受拉力,因此我们可借用虎定理来测定。

现行我们采用贴应变片,这种方法简便和价格低,但每次贴片到使用有一段时间,贴时有位置要求,而且使用寿命短,不易生产现场使用。参考国外使用情况,在大杠一端打深孔,而后有测得 ΔL ,有力学计算公式求得每根大杠受力 $F = \Delta L \cdot E \cdot S / L$,而等效求得大杠受力。这种方法结构复杂,但这一方法十分有效,

2.2 压射压力检测

压射力由油缸作用得到,所以由测定油缸压力而得到压射力 $F = P \cdot S$,压力传感器可安装在油缸活塞后端,管式安装。根据系统使用工况和采样定理要求,传感器必须选用 3KHZ 以上响应频率,量程选用在 0---50Mpa 内。

比较几种传感器的原理、结构和价格,我们选用的压力传感器是以单晶体,按照特定晶面,根据受力形式分别加工成杯、梁、膜等形状,采用集成电路工艺技术扩散成四个等效电阻,组成一个惠斯登电桥,当传感器受力后,电桥失去平衡,输出一个与压力成比例的电信号,它的主要特点是响应快、精度高。

2.3 压射行程的检测有多种方案,可用磁栅、光栅、电阻式等等形式测量位移,

根据安装位置、使用场合、抗冲击能力等等，我们选用德国巴鲁夫生产的直线位移传感器，它是有 5MM 间隔距离，一块 20×50×5 感应体作为运动体，运动体的移动改变传感器发射波的返回时间,即时输出一个与位移成比例信号,它的特点是精度高、响应快、抗冲击强、环境适应能力强等等特点。

2.4 压射速度、建压时间的检测：

压射速度可以通过对记录位移的一阶微分处理，得到相应速度曲线。

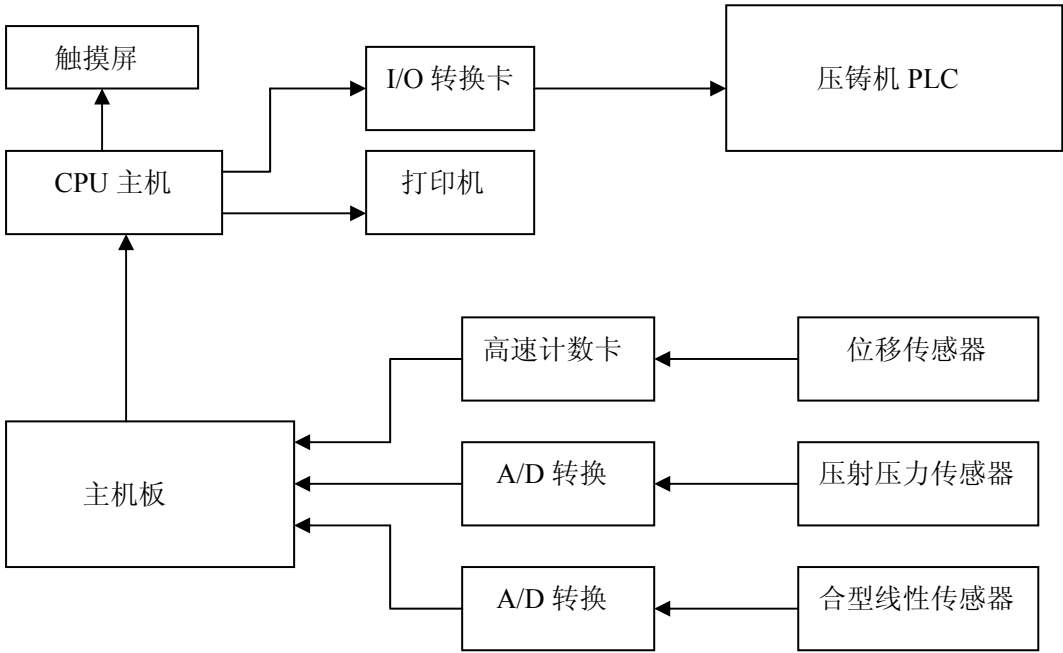
建压时间的定义：当压射结束，压射注料活塞运动速度为令，增压油缸开始运动而使压射力达到要求值（第一冲击峰值下降后的稳态值），这一过程所需时间，一般是 15—50ms。

以上是合型力、压射压力、压射行程、压射速度、建压时间等参数检测方案。压力和位移都是非电量，通过传感器信号变送后都转为标准电信号（0—10V）或（4---20mA），这些信号送入工控机数据采集卡。

2.5 其它信号的处理 压铸机的位置通过行程开关、动作由按钮和转换开关发信输入，经过逻辑运算输出动作控制电磁阀，所以可通过 I/O 控制板进行处理

4. 拟采用的设计方案和途径

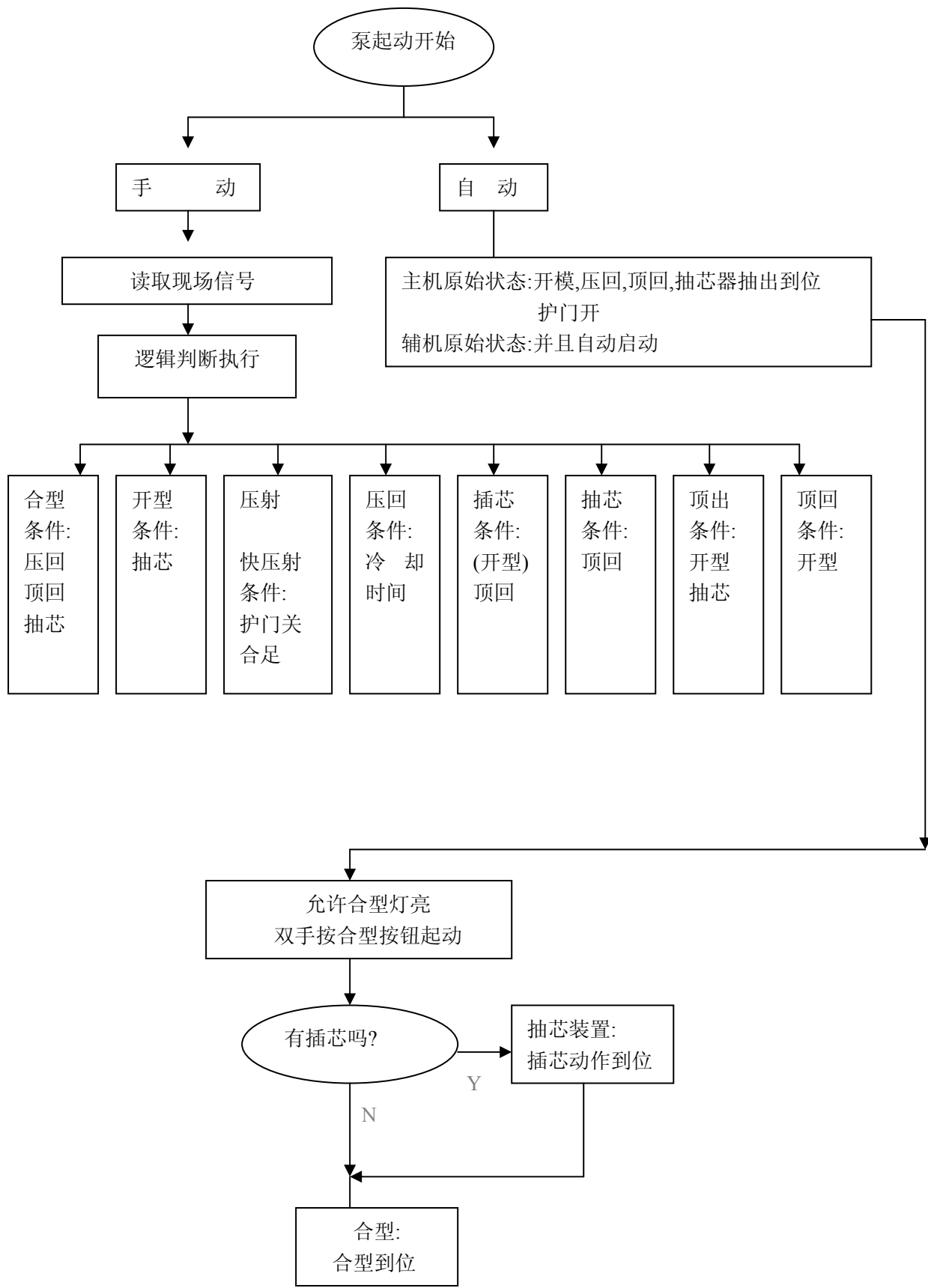
4.1 系统结构框图 参考国内外同行资料,拟本系统为开环控制,框图如下：

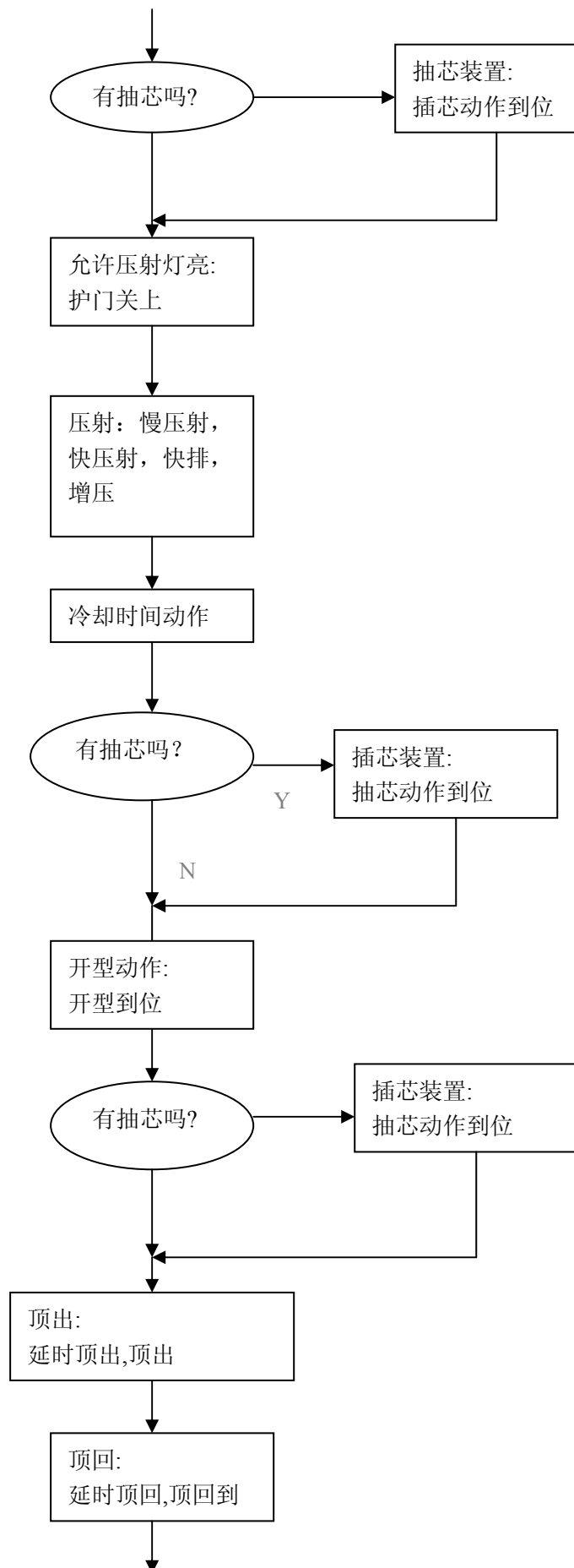


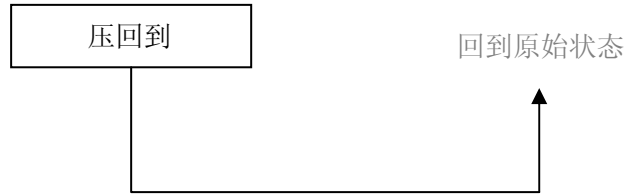
4.2 拟压铸机系统程序框图：

根据压铸机工艺流程要求，我们拟定框图如下：
工作方式分为：手动和自动二种

压铸机工作程序框图



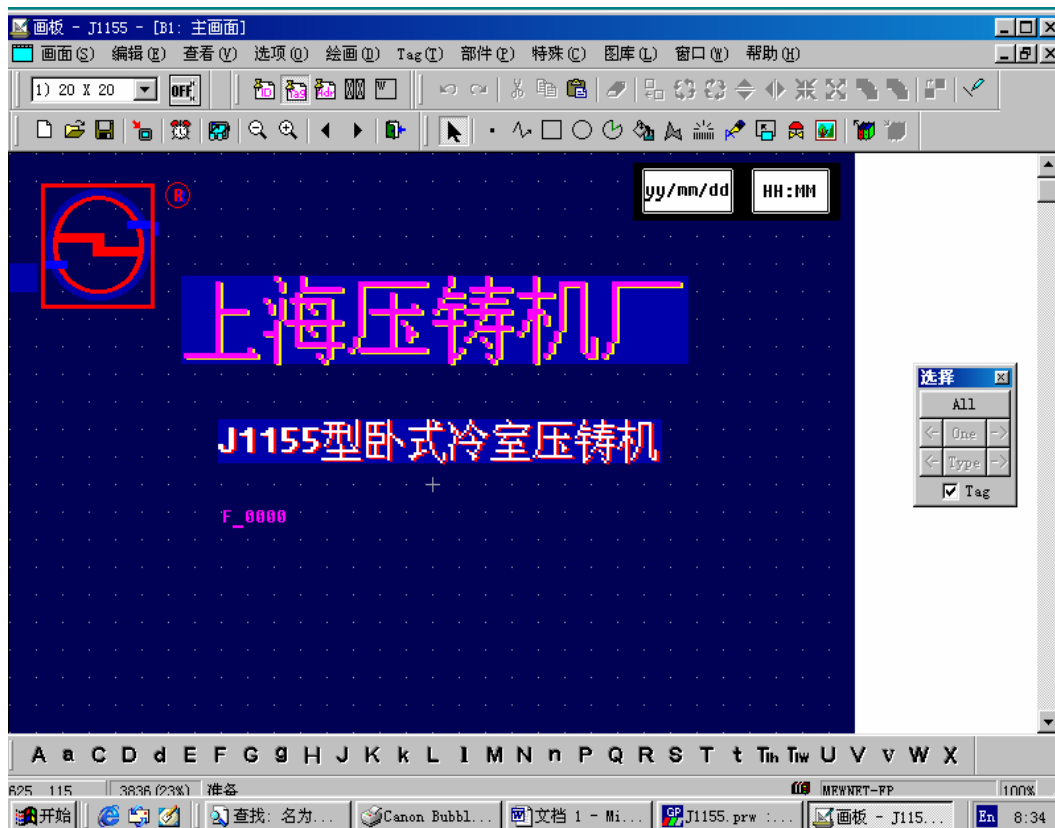




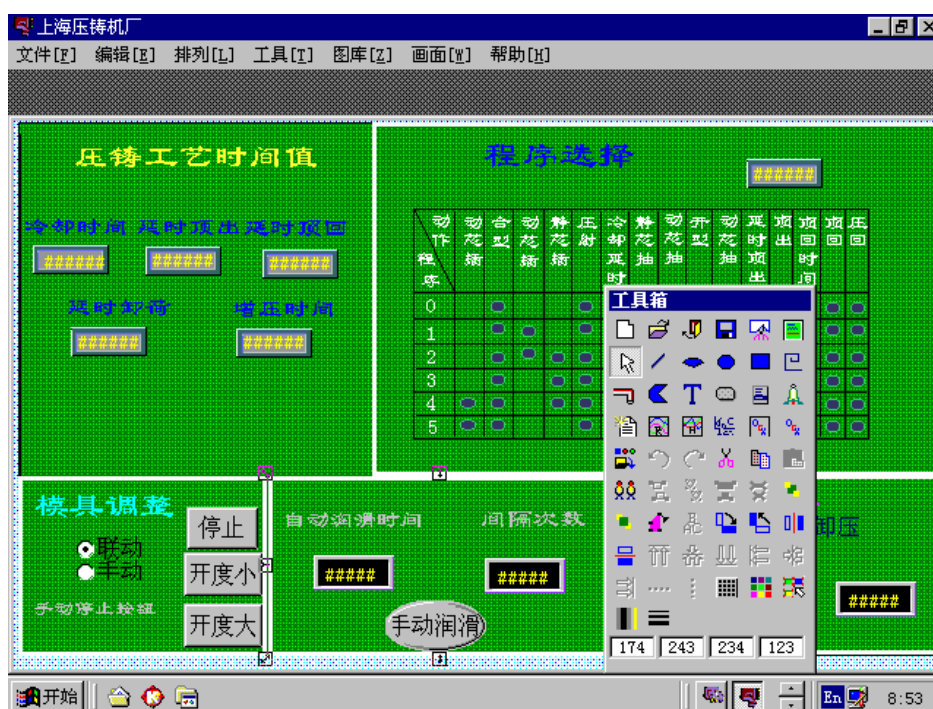
4.3 拟定压铸机测控中硬件：工控机：IPC-6806PJ 机箱
 CPU 卡：PCA-6159L
 数据采集卡：PCL-818H, PCL-1710H
 硬盘：8G 显示器：14LCD+触摸屏

4.4 拟定压铸机测控中软件：本系统的软件采用组态软件编制，以下几个典型画面：

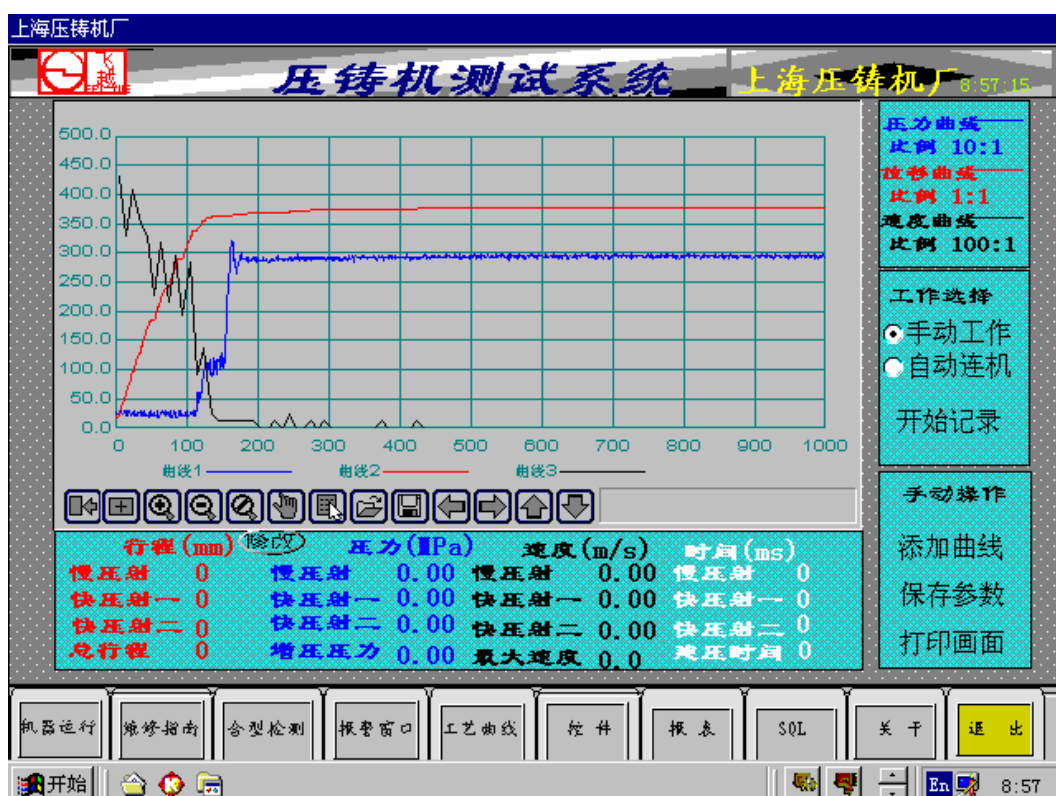
主画面：



设定画面：



压铸工艺曲线:



其它画面：记录汇总表，维修指南，报警信息

5. 结束语

使用研华工控机在压铸机测控系统上应用，完成测控。与世界相同产品相媲美，在这一领域国内首创，为我国压铸机制造和压铸工业的发展有一个很大的推动作用，为压铸机产品的升级换代奠定了基础。

二、 内燃机车辅助电机综合试验台测控系统

摘 要 针对在铁路各机务段和内燃机车大修工厂广泛使用内燃机车辅助电机综合试验台存在的问题，采用研华的工控技术，研制出一套替代现有的内燃机车辅助电机综合试验台的多功能、智能化的测控系统。本文对该系统的试验原理、硬件系统和软件系统进行了论述。

关键词 内燃机车 电机 测控系统

应用背景：内燃机车辅助电机（主要是启动发电机、励磁发电机、空压机电机、制动电阻器风机电机）在大修以后，需要进行试验。在铁路各机务段和内燃机车大修工厂广泛使用内燃机车辅助电机试验台。我们采用研华的工控技术开发了一套测控系统。该系统能完成东风 4B、东风 4C、东风 8B、东风 11 内燃机车的启动发电机、励磁发电机、空压机电机、制动电阻器风机电机等四大类十余种电机的试验。试验项目完全满足铁道部颁发的《内燃机车大修规程》。

面临的问题：

- 1、一套系统能完成多类电机（启动发电机为直流发电机，励磁发电机为交流发电机，空压机电机和制动电阻器风机电机为直流电动机）的多种试验的多功能试验台；
- 2、试验台的全自动控制。目前该类试验台使用计算机都用于测量，而不进行试验过程的自动控制。

也就是说我们需要应用研华的工控技术开发一套多功能、试验过程自动控制的高精度的智能化内燃机车辅助电机综合试验台测控系统。

解决方案：

1 试验原理：

1. 1 启动发电机试验原理：

启动发电机的试验项目主要有空转试验、空载特性试验、调整特性试验、小时温升试验、换向试验、超速试验和匝间耐压试验。试验基本电路如图 1 所示。被试电机、陪试电机和拖动电机由机械装置连接成试验机组。拖动电机由直流调速装置和励磁电源 3 驱动，为机组提供稳定的转速。被试电机和陪试电机分别以发电机和电动机状态运行，构成能量回馈系

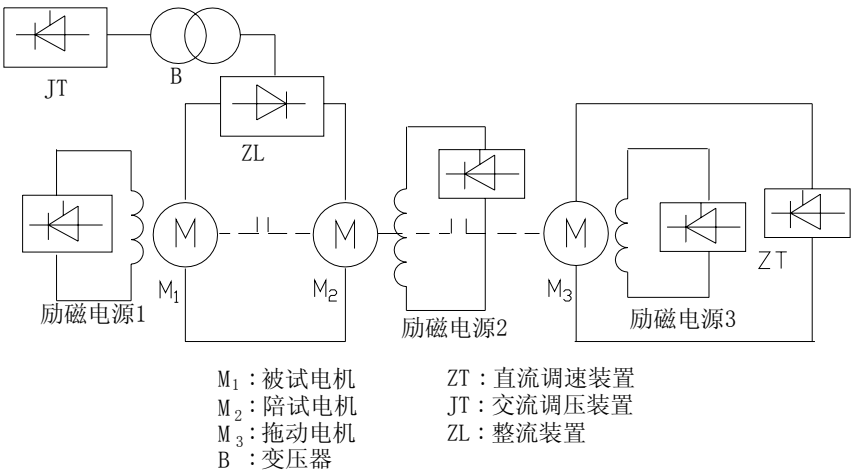


图 1

统。由交流调压装置、变压器和整流装置组成铜耗补偿装置。调节励磁电源 1 和励磁电源 2

达到改变被试电机电压和电流的目的。通过接触器的转换进行不同的试验。

1. 2 励磁发电机试验原理

励磁发电机的试验项目主要有空转试验、空载特性试验、三相短路试验、超速试验、匝间耐压试验。试验基本电路如图 2 所示。被试电机和拖动电机由机械装置连接成试验机组。拖动电机由直流调速装置和励磁电源 3 驱动，为机组提供稳定的转速。调节励磁电源 1 达到改变被试电机电压和电流的目的。通过接触器的转换进行不同的试验。

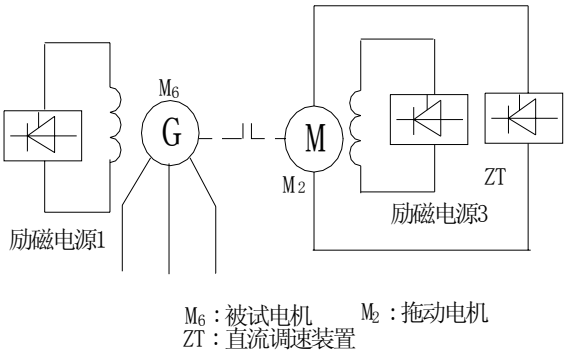


图 2

1. 3 空压机电机和制动电阻器风机电机

空压机电机和制动电阻器风机电机的试验项目主要有空转试验、小时温升试验、换向试验、超速试验和匝间耐压试验。试验基本电路如图 3 所示。被试电机和陪试电机由机械装置连接成试验机组。被试电机和陪试电机分别以电动机和发电机状态运行，构成能量回馈系统。由交流调压装置、变压器和整流装置组成铜耗补偿装置。调节直流调速装置和交流调压装置达到改变被试电机电压、电流和转速的目的。通过接触器的转换进行不同的试验。

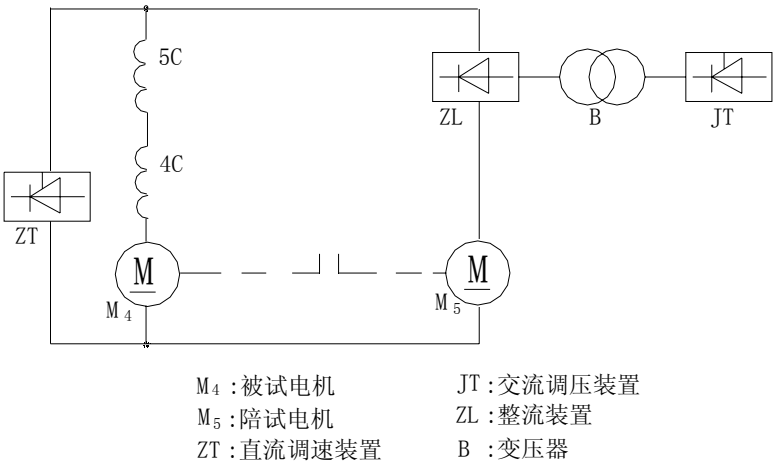


图 3

2 硬件系统：

工控系统需要完成各种电机的自动测试工作。包括试验电机的选择、试验参数的数据采集处理与实时显示、试验参数（电压、电流、转速）的自动调节。工控系统由工控机、各类传感器、直流调速装置、励磁电源、控制电路等部分组成。系统如图 4 所示。

2. 1 工控机：

工控机采用研华 IPC-610 工业控制计算机。使用了 PCL-733、PCL-734、PCL-813B、PCL-728、PCI-1720 四块板卡。

PCL-733 是 32 路隔离数字量输入卡。本套试验系统要进行四大类十余种电机的试验。

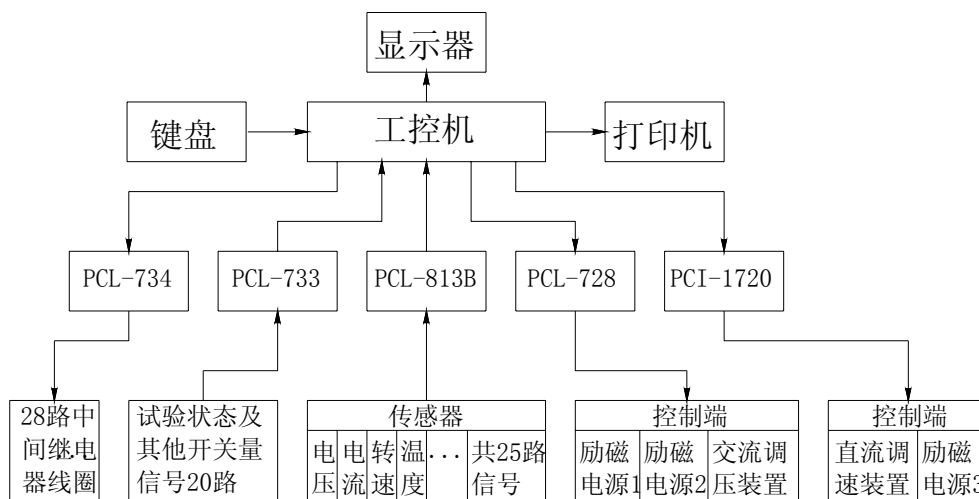


图 4

我们在控制台设有手动与自动、车型、电机类型、试验项目等选择开关。使用 PCL-733 板卡来输入这些选项。工控机用程序来判断与软件中的选项是否相同，通过两次选择来避免误操作。

PCL-734 是 32 路隔离数字量输出卡。根据所选的试验项目，由 PCL-734 输出控制信号，通过中间继电器来驱动接触器，进行所选试验项目的试验。

PCL-813B 是 32 路单端隔离模拟量输入卡。用于采集各个电机的电压、电流、转速、温度等 25 个参数。

PCL-728 和 PCI-1720 分别为 2 路和 4 路模拟量输出卡。用于直流调速装置、三个励磁电源和交流调压装置的控制。保证试验设备自动运行。（现在可选用 PCL-726 板卡来代替这两块板卡）

2.2 传感器和其他设备：

电压、电流传感器选用霍尔元件的隔离传感器；由磁电转速传感器和频率-电压转换器组成转速检测系统；温度用远红外温度传感器测温。直流调速装置采用双闭环直流调速系统；交流调压装置和 3 个励磁电源采用智能稳压模块。

3 软件系统

工控系统软件的主要功能有：参数的设置、数据采集处理、电机试验的自动测试、试验报表和特性曲线的自动生成及历史数据的查询和打印。软件的总体结构见图 5。

系统采用模块化、结构化、层次化程序设计技术。系统中模块化程序主要有：数字量输入板卡子程序、数字量输出板卡子程序、模拟量输入板卡子程序、模拟输出板卡子程序、启动发电机调速子程序、励磁发电机调速子程序、空压机电机和制动电阻器风机电机调速子程序、电压调整子程序、电流调整子程序、滤波子程序和保护子程序等。

滤波子程序是进行数字滤波的子程序。数字滤波是为了提高检测精度而采取的软件抗干扰措施。与由硬件组成的模拟滤波器相比有以下优点：

- (1) 多通道共享一个滤波器（公用一个滤波器）；
- (2) 由于不用硬件设备，所以不存在各回路间的阻抗匹配的问题，故可靠性高，稳定性好；
- (3) 可以对频率很低的信号进行滤波，克服了模拟滤波器的缺陷；
- (4) 可以方便地修改滤波器的参数，保证滤波效果。

数字滤波通常采用的方法有中值滤波、算术平均值滤波等多种滤波方法。我们采用算术平均值滤波法。算术平均值滤波是寻找一个 y 值，使该值与各采样值之间误差的平方和为最

小。

$$\text{即 } E = \min \left[\sum_{i=1}^n e_i^2 \right] = \min \left[\sum_{i=1}^n (y - x_i)^2 \right]$$

由一元函数求极值的原理可求得：
$$y = \frac{1}{n} \left[\sum_{i=1}^n x_i \right]$$

式中 x_i 为第 i 次采样值， n 为采样次数， y 为 n 次的采样的算术平均值。

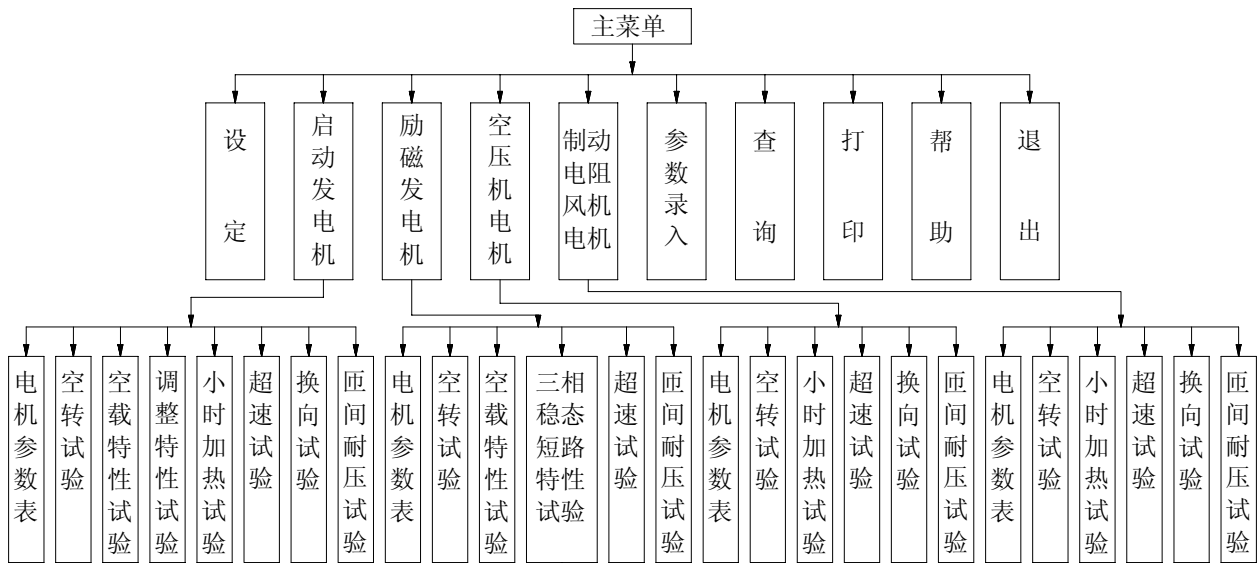


图 5

应用体会：

1、把研华的工控技术用于开发一套能完成几类电机多种试验的多功能电机试验台测控系统，可以大大简化控制电路，节省设备投资。

在这里我们以保护设置为例进行说明。在考虑过电流、过电压和超速保护时，由于试验台是多功能的。一方面调速、调压和励磁电源要共用。另一方面各种电机的过电流、过电压和超速保护值都不相同。如果使用传统的保护方式可以能用两种方式：一是用一套保护装置。做不同的电机时，根据电机参数进行整定。这显然是不符合实际的想法，是无法满足现场工作需要的。这样就要每种电机设一套保护装置，这就不仅使控制电路很复杂，同时也加大了设备投资。我们采取的办法是在软件中设计了保护子程序，只要在参数录入中将电机的有关参数输入，程序就可以根据这些参数在这种电机进行试验时进行保护。

2、实现智能化的全自动控制。

本系统实现了在操作人员确定接线无误，要求试验开始后，从电机启动、参数的采集处理、自动采点到停机和试验结果的判断、特性曲线和报表的生成等全过程的自动化。

3、可以方便的增加试验电机。

如有新的电机需要做同类试验。只要电机类型和技术参数在本实验台设计范围内。由于本系统的是智能化的，只要在参数录入将电机的有关参数输入，就可以进行试验。

总之。我们应用研华的工控技术研制成功了一套多功能、智能化的内燃机车辅助电机综合试验台。现已投入使用。并得到用户的好评。

三、 PCI-1714 及其在船舶状态监测中的具体应用

1、 问题的提出

从某种意义上讲,船舶的辐射噪声的强弱对其自身的安全有着极其重要的影响,因此船舶辐射噪声的测量一直倍受关注。由于发电机旋转时的声音和振动信号的采集与发动机非正常失速过程的记录要求进行多通道、大容量数据的实时测量、采集和存储、高速多通道的 FFT 分析和可视化的数据显示,数据的采集速率至少要 $8\sim 10\text{MS/s}$,因此对于一套能实施可行的可用于对发电机状态监测系统而言,除了需要具有高速处理能力的计算机系统外,更为重要的是需要具有**板载高速存储、同步锁存、高速且多路同步数据采集**、同时具有较高的信号抗混叠特性的 **DAQ 硬件板卡**。目前具有高达 10MS/s 以上**高速且同步数据采集板卡**并不多见,只有少数几家公司才拥有此项关键技术。在过去的几年内,由于受工业控制计算机 (IPC) 的运算速度,可靠性以及 A/D 采集卡的采集速率的限制以及采集设备本身的抗信号混叠特性的制约,使得船舶辐射噪声测量是一件极其复杂的工作。即使能从欧美等先进的国家能进口到高速采集卡,但由于其配套的应用软件极其昂贵,同时高速采集卡也是价格不菲,往往使使用单位和船舶噪声监测设备的研发单位望尘莫及。随着我国工业控制技术的飞速发展,我国的 IPC 研发能力以及高速采集卡的研制设计能力也得到了长足的发展。Advantech(研华)是目前工业控制领域中 eAutomation 的引领者,其 PCI-1714 专为高速数据采集与处理应用场合而设计,由于数据采集速度快,可达 30MS/s ,同时具有较高的信号抗混叠特性,可满足现代船舶辐射噪声测量的要求,正好满足此项要求。

2 设计实施

- 整个系统硬件采用研华 ACP-4001 工业工作站、 30MS/s 的 PCI-1714 高速数据采集卡,以及 5MS/s 的模拟数据输出卡 PCI-1721。系统架构图见图 1 所示,其中:

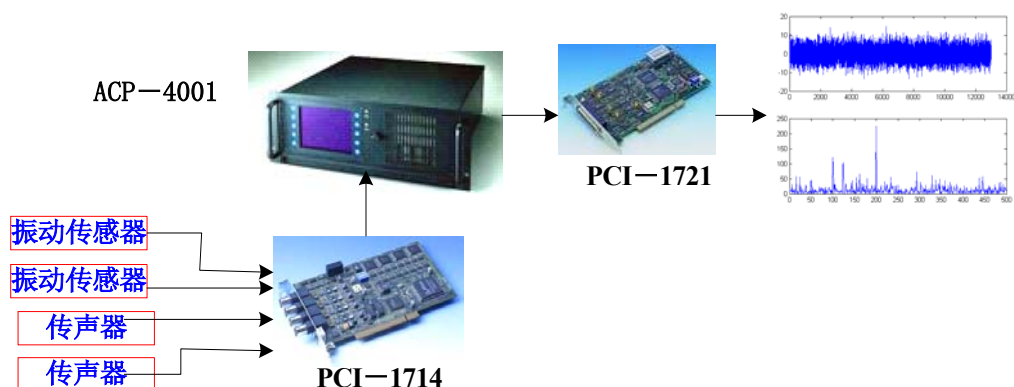


图 1: 系统架构图

- ACP-4001 是一款 4U 高、19 寸架装机箱,可以上标准 19"工业机柜、带 6.4 寸 TFT LED 显示、分辨率为 640×480 ; 内置超薄键盘和触摸板、前置 USB 接口整合型 IPC 系统,非常适合低成本,节省显示器空间工业自动化监控系统。
- PCI-1721 是具有 12 位分辨率 D/A 卡,具有 5M 数字更新率、每个模拟量输出通道具有独立的 D/A 转换,因此可以实时波形输出,因此正好利用这以特性,可以将实时采集的发电机的声音信号根据高速采集时的频率通过其输出,然后经放大器将信号放大后通过电声器输出,供专家“监听”,判别故障。
- PCI-1714: 采样速率为 30MS/s 、内含四个独立模拟数字转换器 (ADC),可使四个通道同步取样、 32K 的 FIFO,这是系统设计的关键部件。

系统的工作原理见图 2 所示。

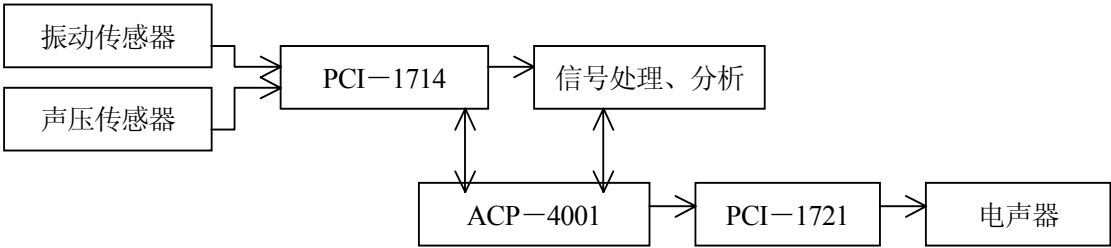


图 2: 系统的工作原理

发动机在正常时，其振动的声音及其自身的振动频谱是有规律的，但是一旦发动机出现异常时，其频谱便会出现变异和失真。因此，可以通过监测发动机转动时的声音及其自身振动信号来判别发动机的健康状态。在发电机状态监测中，发动机转动时自身发出的声音的监测是一个关键，其次，为整个电机机身的振动监测。在旋转机械声音处理方面，采用滤波器的输出用 A/D 转换器以每秒 8~9MS/s 采样，处理器每连续存储 10240 或 20480 个取样数据为一帧信息，声音处理的框架图见图 3 所示：

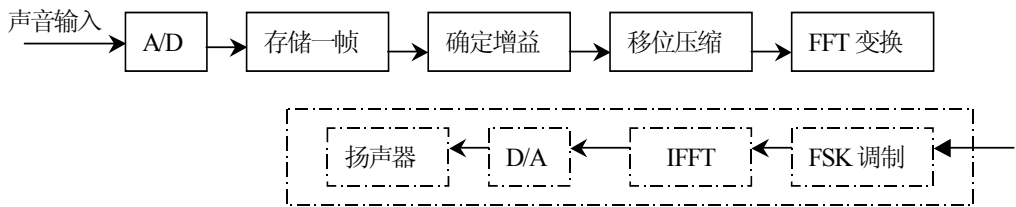


图 3: 声音信号处理的流程图

3 PCI-1714 应用的几项关键技术

振动和噪声测试技术的实时性很强，要求其数据的存储和传输必须通畅迅速。为了达到性能上的要求，在使用 PCI-1714 时应特别注意以下几方面：

3.1 FIFO 的运用

为了实时处理振动信号，避免数据覆盖，软件设置 FIFO 作为数据缓冲区，大小设置为振动信息帧长度的 4 倍左右。在 PCI-1714 板卡中由于每路均内嵌 32K FIFO 高速同步缓存，足以满足声音、振动对存储的需要。将采集到的数据存入 FIFO，只有当其中的数据达到一个语音帧的长度后，才将数据由 FIFO 取出，送入编码数据缓冲区。同样地，经过解码的数据也是存入发送 FIFO，在中断到来时，从发送 FIFO 中取得数据，送到发送寄存器中输出。程序中利用 PCI-1714 的结构特性，将 FIFO 设置为循环缓冲区(Circ-Buf)，这样在指针到达缓冲区的底部时，系统会自动修改指针指向缓冲区的头部，而不需人为地修改。

3.2 数据精度的处理

PCI-1714 每路均有独立的 S/H（采样保持）以及 12bit 的 A/D，可以满足系统对发电机状态监测的精度要求。但为确保运算精度和防止数据溢出，在实现过程中主要采用了两种方法：一是对某些精度要求较高的运算，将计算的中间变量采用 32 位来表示；二是对于幅度范围变化比较大的变量或数组，如激励增益、自适应码本的自相关数组等，采用标量浮点或块浮点表示，用一个字存储使该数或数组规格化所需的左移位数(EXP)，其余字存储规格化后的尾数，不仅能够提高运算速度，还可以确保数据的精度。

3.3 数据传输

在高速数据采集过程中，数据的位传输数率一直是制约高速数据采集实现的一个关键因

素, PCI-1714 采用 Bus Master DMA 把 A/D 转换数据直接写入系统内存, 有效解决了这一问题。为了防止 PCI 总线延时(latency)造成数据丢失, 采用了板上提供的 32k 采样点的 FIFO 作为缓存, 保证了信息帧的完整性和可读性。

4 软件实施

整个系统采用了 Windows2000 操作系统平台, Visual C++ 为程序开发平台, 在本系统中, 数字信号处理是这个程序设计的关键, 包含有大量的数据计算、频谱分析和三维图形显示。Visual C++ 具有极强的编译能力, 但数据处理与图形的编辑能力较弱, Matlab 是一种基于解释语言的编程环境, 对数字信号处理以及图形的处理能力较强, 因此, 在本系统中, 采用 Visual C++ 进行数据采集、线程以及界面的设计与管理。数据采集和存储在前台运行, 而让 Matlab 于 Visual C++ 的后台运行, 用于对采集的数据进行 FFT、频谱分析以及三维图形显示。这一方法充分利用了 Visual C++ 以及 Matlab 的优势。

软件实施过程中, 还有一个关键, 便是如何防止数据的丢失, 数据丢失一般是由数据显示与数据采集之间的矛盾引起的, 因为 Windows2000 是一个多任务、多线程的非实时系统, 数据显示、图形显示必然会对要求实时高速数据采集产生一定的时延作用, 造成数据丢失、声音的变异和失真, 极大影响系统对发动机诊断的准确性。为克服这一缺憾, 我们采用间隙高速数据采集的方式, 即在某个时间段内让计算机集中高速采集一组实时数据, 并将其暂存于数据缓存池中, 然后停止数据采集, 集中时间来处理数据、分析、数据输出和屏幕刷新显示等过程。这样即保证了信号的完整性, 又满足了系统对信号处理的要求。实际上, 在我们的试验中, 发动机一旦有异常, 其声音信号和振动信号会周期性出现较长时间, 因此, 间隙式的高速采集一般不会影响系统诊断的功能。图 4 为实时监测界面。

从图 4 可以发现, 发动机的振动频率主要为 100Hz 和 200Hz, 主要为 200Hz, 同时还参杂一些类似白噪声的高频信号, 这些信号可能是数据采集通道、振动传感器等本身的噪声所引起的。在振动和噪声测试中, 我们发现:

- 在发动机振动剧烈时, 如发现还伴有声响, 信号的频谱会发生较大的偏移, 甚至高频信号会周期性的出现, 这种故障一般是其转轮等旋转部件与其他静止部件发生了碰撞, 此时, 应立即停止检修;
- 当船舶航行速度加快, 发动机的噪声加剧, 同时机组的振动频率以及幅值也在加快, 说明此时发动机自身存在缺陷, 发动机的输出功率在其额定功率附近。另外可能是喷油嘴处的喷油量不足引起的;
- 当船舶航行速度加快, 机组的横倾信号的幅值加大, 船体航行不平稳, 说明可能是螺旋桨的叶片大小不均匀、也有可能是其中的某一叶片破损, 断裂等故障。

船舶不同的故障往往与其声音、振动信号出现奇异相关联, 因此, 只要信号出现奇异, 便可判定船舶有故障, 同时, 进一步对其频谱进行分析, 便可了解故障的特征。

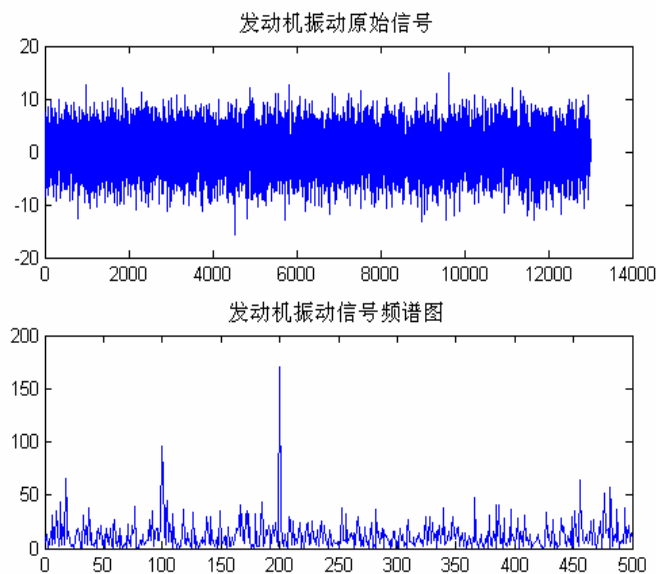


图 4：声音与振动实时监测分析图

4 结束语

基于高速数据采集的船舶健康状态的监测目前是一们新兴学科，关键是基于 PCI 总线的高性能高速数据采集板卡的研制在基于 PC 系统的 SCADA 系统中一直是一个难点，因为高速数据采集板卡的研制不仅要求数据采集具有急速的信号采样频率，更为重要的是要求有较大的板载高速存储空间、信号的同步技术、高速数据锁存技术以及数据传输技术。作为 eAutomation 的倡导者的 Advantech 公司，推出的 PCI-1714 板卡，采样速率高达 30MS/s、具有四路同步锁存、同步采集、同步存储，对采集的数据采用 DMA 传输方式等特性，满足了现代工业对一些语音、振动信号处理的要求，本系统的成功应用便是一个证实。

四、 基于 PCI-1712 的移动式电网谐波分析仪的研制

摘要: 本文主要详细介绍了采用 Advantech 公司的 PCI-1712 高速数据采集卡以及计数/定时板卡 PCL-836、IPC 集成的移动式电网谐波分析仪的研制原理、测试方法以及在设计中的几项关键技巧等。

关键词: PCI-1712 谐波分析 测试

1 开发背景与需求

由于电力机车以及大功率整流器件的广泛使用,使电网的谐波含量逐渐增加,因此国家电网公司提出了“绿色电网、绿色电能”的概念。目的就是为了采取一切可能的措施净化电网。在有关部门的配合下,我们采用 IPC 平台、并采用高速数据采集器 PCI-1712 与定时/计数卡 PCL-836 在 Windows 操作系统下开发了一套移动式的电网谐波分析仪。

2 测试原理

电力系统中电参数的定义,一般是在正弦条件下得到的,可以在时域或频域定义;由于电网中存在谐波致使电网为非正弦信号,而在非正弦的电力系统中电参数的定义,一般在频域定义。要得到有用的功率形式,必须有相应的测量算法,避免处理时间过长而不具备实时性。

用微机测量交流信号时,要求在一个工频周期内使采样次数 N 与采样周期 T_s 的乘积等于被测信号的周期 T (即 $N \times T_s = T$)。电网中一般含有奇次谐波,要真正利用 Shannon 采样定理采集谐波信号,需要根据具体的信号特征以及分析方法来确定采用频率。在实际谐波测量中,要求对电压和电流信号进行交流同步采样,目前交流采样的同步方法主要有硬件同步、软件同步、定时采样 3 种。

- 硬件同步的精度最高,但它要求采样装置具备专用同步电路;
- 软件同步精度次之,它也需要电网频率跟踪测量环节;
- 定时采样不需要任何附加同步电路,但同步误差大,一般不用于高精度测量场合。

考虑到实际问题,本系统采用定时采样。定时采样实际上是假定电网频率为某一默认值,根据这一默认值和每周期内的采样点数确定定时器的定时值,以此实现同步。但是当电网频率与默认值不符或发生变化时,定时采样的同步误差就会增大。为此,系统有必要对电网频率实行软件跟踪,即利用采样值估算电网周期,然后根据估算周期调整采样周期的值。

目前,谐波测量最常采用的方法为离散傅里叶变换(DFT)和快速傅里叶变换(FFT)。由于快速傅里叶变换是离散傅里叶变换的快速算法,它能使 DFT 的运算效率提高 1~2 个数量级,因此在谐波分析中被广泛采用,且采用其中最为常见的基 2 的 FFT 算法。

对电压和电流信号进行同步采样,在一个工频周期内分别采样 N 点,得到离散数字序列 $\{u_k\}$ 和 $\{i_k\}$ 。显然两序列均为实序列,将它们按下式构成一复序列 $\{f_k\}$

$$f_k = u_k + ji_k \quad (k=0, 1, 2, \dots, N-1) \quad \{f_k\} \text{ 频谱为:}$$

$$F_n = \frac{2}{N} \sum_{k=0}^{N-1} f_k \cdot e^{-j\frac{2\pi}{N}kn} \quad (n=0, 1, 2, \dots, N-1)$$

对 $\{f_k\}$ 按 FFT 算法求得 F_n 后,就可以得出 $\{u_k\}$ 、 $\{i_k\}$ 的频谱。

$$U_n = \frac{1}{2}(F_n + F_{N-n}^*) \quad I_n = \frac{1}{2}(F_n - F_{N-n}^*)$$

其中 U_n 、 I_n 分别为第 n 次谐波电压和谐波电流向量表达式。

3 测试系统硬件的选用

在利用 FFT 算法测量谐波时，采样点数一般要求为 2 的幂次方，最好为 1024 点。也即要在 4ms 内采集 1024 点采样值。因此采样频率应为：1024/4ms=256KS/s。假如需要“三相同步采样”采集三相电压信号，采样频率至少为：256 KS/s×3=768KS/s。

根据这个要求，可供选用的板卡不是很多，Advantech 公司的 PCI-1712 可以满足要求。PCI-1712 的特性为：

- 1MS/S 采样频率
- 12 位 A/D 分辨率，每秒 1M 采样速率；
- 16 路单端或 8 路差分或组合模拟量输入，
- 数据传输可使用 PCI 总线 master 功能，
- 对模拟量输入可使用预触发，后触发，模式匹配以及延迟触发等多种触发模式
- 2 路 12 位模拟量输出通道，可产生连续波形输出，
- 用于模拟量输入的 1K 字 FIFO，和用于模拟量输出的 4K 字 FIFO，
- 3 个 16 位时基为 10MHZ 的可编程计数器/定时器。

由于电网的频率并不稳定，要在一个周期内准确采样 1024 点，往往存在同步误差 ΔT ($\Delta T = NT_s - T$)，会影响测量精度。为了减少电网频率波动对测量结果的影响，必要时要实时测量其频率。因此，选用 PCL-836 计数/定时卡定时检测电网的频率，以动态确定采用周期。同时选用研华 ACP-4001 工业工作站，电网测试分析仪的架构图见图 1 所示。

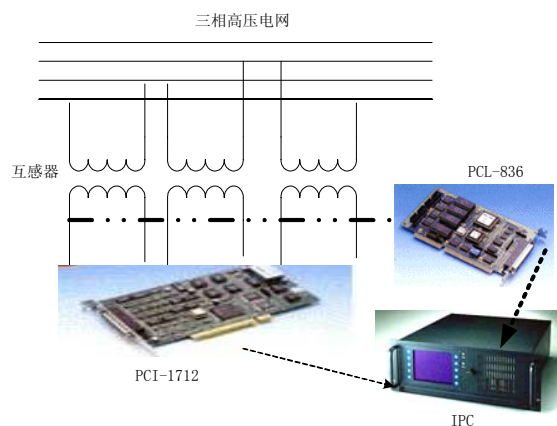


图 1：电网测试分析仪的架构图

4 实施过程中的几项关键技巧

4.1 数据采集方式选用

由于需要采集的数据量很大，同时还要进行数据的实时处理等工作，这样就要求高速数据采集时不能过多占用 CPU 资源，因此采用 DMA 数据传输方式及相应的中断控制。系统的工作原理为，三相高压电网经电压互感器 PT 将高压电网信号转变成适合 PC 采样的一 10V~10V 交变信号，再经 PCI-1712 采集、比例变换等简单的处理后送入 FIFO 进行缓冲，PCI 总线控制器工作在主控模式下，借助 PCI 总线高带宽、低延迟的特性，经 DMA 数据传输，将采集的数据从 FIFO 中读出后直接送至计算机内存，配合设备驱动程序及上层应用程序

序对结果进行分析、处理、显示和存储，整个传输过程无需 CPU 干预，完全由硬件来实现，有效地提高了数据的传输速率。这样，采用 DMA 方式进行大段数据传输，采样周期由板卡严格控制，避免了由于软件系统时间控制的不精确而产生的信号失真。系统可以在样本采集的同时进行其他工作，增加了系统的灵活性。

4.2 数据存储方式

在本系统应用中，必须快速且可靠的把大量数据记录到储存介质中去，以便事后的分析或处理。然而对应用程序开发者来说，大量且实时地纪录数据一直是一项极大的挑战。一般做法为：让数据由数据采集卡通过 DMA 方式，经过 PCI 总线直接储存在硬盘，但是这种方法极易造成数据传输的瓶颈。一般而言，PCI 总线以 32bit/33MHz 的总线时钟传输数据，大约可以达到 40Mb/s 的速度，基本能满足数据传输要求。其实，数据记录的瓶颈是从内存写到硬盘这个过程，一般这个过程是由操作系统的文件系统 (File System) 来处理。但是，单纯地将数据记录在文件系统内，则必然受限于原来操作系统与文件系统的限制，这些先进的文件系统反而会对高速存盘造成累赘。若硬盘为 IDE 硬盘，通常储存速度低于 2Mbytes/sec，即使使用 SCSI 硬盘大概也只能达到 10Mbytes/sec 左右的储存速度，假如跳过文件系统直接存取 SCSI 装置，储存的速度将会有非常大的提高。在此，我们借用 Adaptec 公司的 DAQCreator 软件的解决方法实现 40Mbytes/sec 的实时数据记录功能，这个方法的基本观念就是跳过文件系统，直接控制 SCSI 适配卡，而 ACP-4001 主板上板载 SCSI2 接口，可以直接挂接 SCSI 硬盘。

程序设计时，采用了由 Adaptec 公司所提供的 ASPI (Advanced SCSI Programming Interface)，我们可以直接使用 ASPI 高阶的函数及指令，通过 ASPI driver 传送 SCSI2 指令。利用 ASPI 函数能控制各种 SCSI 的外围装置，而且程序可以兼容各种 SCSI 适配卡。而在 Win32 的环境下，ASPI Manager 是以动态连结函数库 DLL 的型式安装在系统内，提供并管理 ASPI 函数的功能。在 Win32 系统下，ASPI Manager (wnaspi32.dll) 提供下列五个函数：

- DWORD GetASPI32SupportInfo (void);
- DWORD SendASPI32Command(LPSRB);
- BOOL GetASPI32Buffer(PASPI32BUFF);
- BOOL FreeASPI32Buffer(PASPI32BUFF);
- BOOL TranslateASPI32Address(PDWORD, PDWORD);

在我们的应用程序中，我们使用 GetASPI32SupportInfo() 与 SendASPI32Command() 这两个函数。同时通过存取 GetASPI32SupportInfo()，检查 ASPI driver 与 SCSI 卡是否安装正确；而 SendASPI32Command() 则是用来传递需要的 ASPI 指令。

5 测试系统软件的设计

利用了 Windows 多任务的特性，控制界面和实时显示设计为前台窗口，数据的采集和分析设计为后台运行，以便于试验人员操作和对测试流程的掌握。由于 Windows 并不是作为实时操作系统设计的。Windows 是抢先式、多任务、基于消息传递机制的操作系统，但仅凭消息调度机制，显然不能满足实时系统的要求，难以保证准确实时地完成前后台控制任务。因此在 Windows 环境中，采用多线程技术，可以有效地利用 Windows 等待时间，加快程序的反应速度，提高执行效率。用一个线程管理采集卡和计算机数据通信，另一个线程进行数据处理，这样，在满足数据连续采集的同时，增强了信号分析的实时性。

在本系统中，主要有数据采集、标志位辨别、数据传输、信号处理等线程，使用线程 Semaphore 方法已足以满足要求，但是在设计线程时，要合理规划多个线程对同一资源的使用顺序，这可以通过设定每个线程的优先级别来实现，在本系统中数据采集具有最高的优先级，其次为数据传输。线程优先级常量设定如下：

THREAD_PRIORITY_ABOVE_NORMAL	设定一个比普通级高一级的优先级
THREAD_PRIORITY_BELOW_NORMAL	设定一个比普通级低一级的优先级
THREAD_PRIORITY_HIGHEST	设定一个比普通级高二级的优先级
THREAD_PRIORITY_LOWEST	设定一个比普通级低二级的优先级
THREAD_PRIORITY_NORMAL	设定为正常优先级

应用程序为：

```

UNIT THREADSAMPLE(LPVOID param)
{
    Csemaphore Semaphore(2,6);
    someResource.UseResource();
    ::MessageBox(HWND) param;
    _outp(BaseAdr+2,0x34); //选择 Pacer 触发 A D
    _outp(BaseAdr+4,0x01); //启动 Pacer 。。。。。。 }

UNIT THREADTRANSDATA(LPVOID param)
{
    float data1,data2;
    someResource.UseResource();
    ::MessageBox(HWND) param;
    _fmemmove(data1,data2,Samplen); 。。。。。。 }

```

这样，系统开始工作时，数据采集卡开始向内存缓冲区中添加采集到的样本数据，经过采样时间后，此时已经采集到一个完整样本，信号处理线程开始工作，从内存缓冲区中读取样本数据，进行特征分析、显示和存储，然后等待下一个样本，数据采集线程继续采集信号数据。数据采集和数据分析处理同时进行，提高了系统资源利用率的同时，也实现了数据的连续采集和实时性。

整个系统的软件模块见图 2 所示。

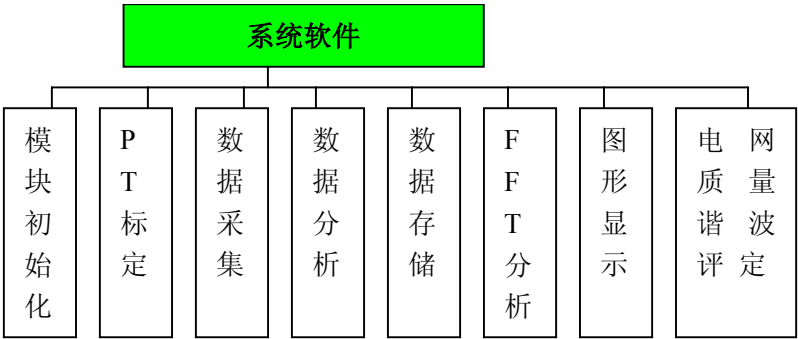


图 2：整个系统的软件模块

6 应用体会

PCI—1712 是 Advantech 公司新近推出的基于 PCI 总线的高速数据采集板卡，配合 PCL—836 定时计数板卡的使用，并借以提供的 ADSAPI.dll 库函数，使我们以最快的速度开发了本系统，由于板卡质量稳定，自投入运行以来，测试系统一直运行良好，深受用户好评。

五、 用 VC++实现 PCL-812PG 数据采集卡的应用

摘要: 介绍了研华 PCL-812PG 多功能数据采集卡的主要功能特征、硬件设置和软件编程操作方法,同时通过一个具体的实例阐述了在 VC++6.0 环境下如何对 PCL-812PG 进行 A/D 和 D/A 操作。

关键词: PCL-812PG;VC++6.0;数据采集

Using the VC++ to Realize The Application of PCL-812PG Data Acquisition Card

TANG Yi, HUANG Hong

(Automatic Control Department, Beijing Institute of Technology ,Beijing 100081, China)

Abstract: The paper introduces the key function and feature of Advantech PCL-812PG Enhanced Multi-Lab Card ,discusses how to set up the hardware and develop using the software. At the same time, the paper elaborated how to use the PCL-812PG Card to process A/D and D/A convert under the VC++6.0 development environment.

Key words: PCL-812PG;VC++ 6.0;Data Acquisition

1. 引言

PCL-812PG 是台湾研华公司推出的一款应用于 IBM PC/XT/AT 及其兼容机的高性能、高速度的多功能数据采集卡。此卡不仅功能强大,性能优越,而且相应的支持软件多,使得它成为工业控制和实验室应用的理想选择,可广泛应用于数据采集、过程控制、自动检测、工厂自动化。

该卡具有 16 路单端模拟输入通道,可用来作为 A/D 转换器使用;软件的可编程模拟输入范围为双极,范围为: $\pm 10V$, $\pm 5V$, $\pm 2.5V$, $\pm 1.25V$, $\pm 0.625V$, $\pm 0.3125V$; A/D 转换有 3 种触发方式:软件触发、可编程序步测触发、外部脉冲触发;2 个 12 位 D/A 转换输出通道,可以通过数据卡上的 $-5V$ 或 $-10V$ 参考电压来实现 0 到 $+5V$ 或者 0 到 $+10V$ 的输出范围选择。

2. PCL-812PG 硬件的参数设置

PCL-812PG 是一种通用型数据采集卡,可以满足不同用户的不同要求。它有一个 DIP 转换开关和 9 个跳线器,可由用户自行设置,以满足不同的需求。PCL_812PG 的控制面板示意图见图 1:

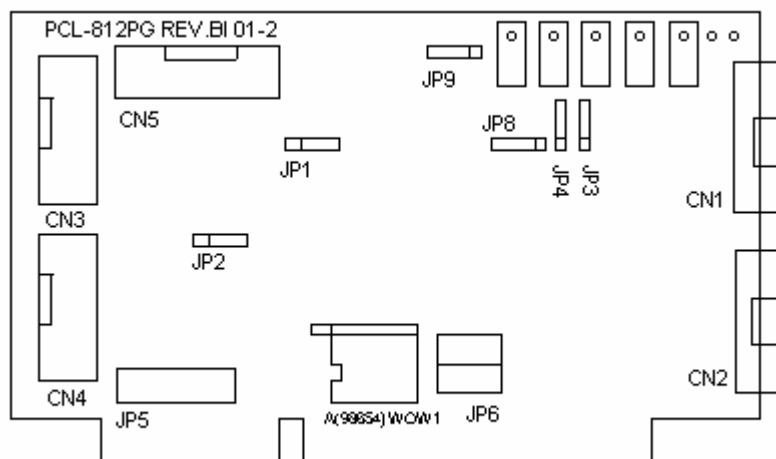


图 1 PCL-812PG 面板示意图

2. 1 基准地址的设置

开关名：SW1。大多数 PC 的外围设备和接口板都是由输入/输出（I/O）口控制的。这些端口是由 I/O 口的地址空间来寻址的。通过 8 位 DIP 开关，PCL-812PG 数据采集卡 I/O 口的基准地址是可选择的。PCL-812PG 需要有 16 位连续地址位置，有效地址来自 16 进制的 200H 到 300H。出厂时，PCL-812PG 的基准地址设置成 220H。如果默认的基准地址与计算机的其他设备地址冲突，你就需要把它调节成其他的地址范围，适用于不同基准地址的开关装置列表可以参考板卡所附带的 A/O 地址开关位置表来设置。

2. 2 等待状态的选择

一些高速的 PC 要求在总线 I/O 中设置等待状态以得到稳定的数据转换。PCL-812PG 可为各个信号传输设置 0，2，4，6s 的等待延迟，等待时间的长短可由 SW1 上的 7、8 引脚选择（可参考板卡所附带的时间延迟设置表来设置）。

2. 3 主要的跳线设置

JP1：触发源的选择。A/D 转换的触发来源可以是内部板上的可编程的定速器或外部脉冲信号（默认方式为内部触发）。

JP8：D/A 内部参考电源的选择。内部参考电压可以是-5V 或-10V。参考电压是-10V 时，D/A 输出范围是 0 到+10V；参考电压是-5V 时，D/A 输出范围是 0 到+5V。注意：这一方式仅当 JP3 和 JP4 设置为 INT 方式时才有效

JP9：选择 A/D 最大输入电压的选择。A/D 转换范围可以是±5V 或者±10V。当 JP9 被设置成±5V 范围，PCL-812PG 的模拟输入范围是±5V，±2.5V，±1.25V，±0.625V，±0.3125V；如果 JP9 被设置成±10V 范围，那么 PCL-812PG 的模拟输入范围是±10V，±5V，±2.5V，±1.25V，±0.625V。JP9 的缺省设置为±5V 范围。

3. VC++6.0 环境下 PCL-812PG 的软件开发

PCL-812PG 有着完善的软件支持功能，既可应用于 DOS 平台，也可以应用于 WINDOWS 平台。研华公司为该卡提供了一种基于 WINDOWS 的标准动态链接库，可以在 Visual C++，Visual Basic，Delphi 等编程环境中，通过调用动态链接库中的库函数，方便地实现对数据采集卡的底层进行操作。下面就一些关键的库函数及结构变量作一简单介绍。

3. 1 关键函数的说明

DRV_DeviceOpen()：该函数是数据采集卡能否正常工作的前提，所以它必须在调用其他库函数之前使用。它的功能是从注册文件或者是配置文件中得到合适的使设备工作的参数，同时分配内存来存储这些参数，使设备做好 I/O 操作的准备工作；同时返回一个函数值，以

确定设备驱动程序是否成功装载，若成功装载则返回一个非 0 值，否则返回 0。

DRV_AOVoltageOut(): 该函数用来接收浮点数, 根据比例将其转化为合适的二进制数, 同时将这个二进制数写到模拟输出通道并转换成电压值送出。

DRV_AIConfig(): 该函数用来为指定的模拟通道配置增益设置。同时返回一个函数值, 若成功配置则返回一个非 0 值, 否则返回 0。

DRV_AIVoltageIn(): 该函数用来从输入通道中读取采集的模拟量, 同时根据设置的比例返回一个电压值存储在 lpAIVoltageIn 参数中。

DRV_DeviceClose(): 该函数用来释放由 DRV_DeviceOpen() 函数分配的存储参数值以关闭装载的数据采集卡。同时返回一个函数值, 以确定设备驱动程序是否成功卸载, 若成功卸载则返回一个非 0 值, 否则返回 0。

3. 2 关键结构体变量的定义

PT_AOVoltageOut: 该结构体变量是在 DRV_AOVoltageOut 函数中使用, 当 DRV_AOVoltageOut 函数成功配置之后, 将由该结构体记录得到的输出通道和输出电压值等变量。

PT_AIVoltageIn: 该结构体变量是在 DRV_AIVoltageIn 函数中使用, 当 DRV_AIVoltageIn 函数成功配置之后, 将由该结构体记录得的模拟输入通道, 增益代码, 模拟触发模式和模拟转化后的电压值等变量。

PT_AIConfig: 该结构体变量是在 DRV_AIConfig 函数中使用, 当 DRV_AIConfig 函数成功配置之后, 将由该结构体记录得到的模拟输入通道和增益代码值等变量。

4. 应用实例

本节通过举例说明在 VC++6.0 编程环境下如何对 PCL-812PG 数据采集卡进行 A/D 和 D/A 转换。设计要求: 通过 PCL-812PG 数据采集卡 D/A 部分产生正弦信号, 同时启动 PCL-812PG 数据采集卡 A/D 部分采集信号。正弦函数的格式: $A \cdot \sin(2 \cdot \pi \cdot f \cdot t)$, 采用在一个周期内采集 100 个点的方式实现, 正弦信号周期大于 10 秒 (即采集时间间隔大于 100ms, 如果采集时间间隔小于 55ms 就涉及到高精度定时器, 请参考有关高精度定时器的文献, 现不在此赘述)。详细步骤如下所示:

(1) 首先, 参照使用说明书, 根据实际需要对 PCL-812PG 数据采集卡进行硬件的跳线设置。在本例中, 基地址设为默认方式 (即 220H), 等待状态设为 0 延迟, A/D 转换输入范围设为 $\pm 5V$, D/A 转换输出范围设为 0-5 伏, 最后将 PCL-812PG 数据采集卡插入到 PC 机中的 ISA 扩展槽中, 将 20 针的带状电缆与连接器相连, 本例中连接 CN2 连接器。

(2) 根据自己所用机器的操作系统, 安装好相应的 32 位 DLL 驱动程序, 然后在开始程序中运行 Device Intallation Utility 程序, 装载上 PCL-812PG 数据采集卡, 最后使用 Device Test Utility 程序来测试数据采集卡是否工作正常。至此, 硬件部分准备工作完毕。

(3) 在 VC++6.0 中建立一个单文档应用工程, 将其命名为 Demo。

(4) 在资源编辑器中, 给菜单添加数据采集卡测试, id 号为 ID_TEST。

(5) 在 VC++ 中单击菜单项 Inser 中的 Resource 选项, 插入一个如图 2 所示的对话框资源, ID 号为 IDD_DEMO。

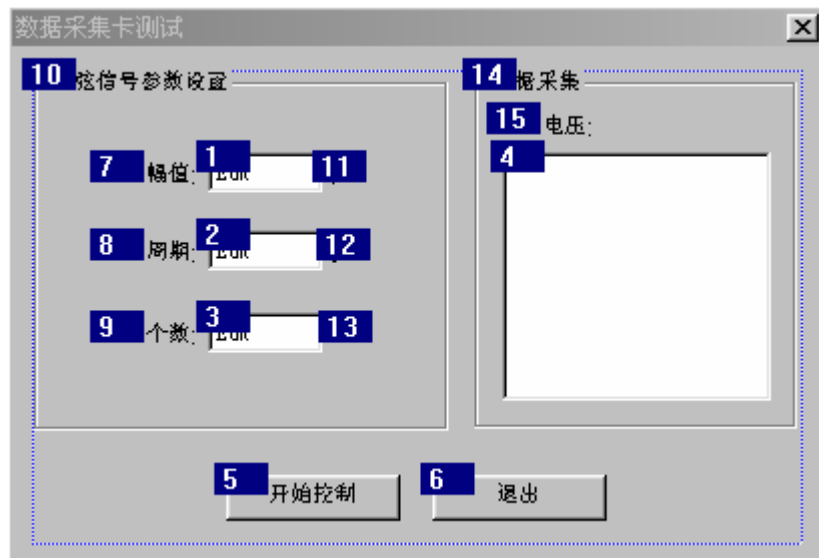


图 2 编辑完毕后的对话框资源

创建对话框类，命名为 Ctest, 加入控制的对应变量。对话框资源中的主要控制和风格及加入的变量与之对应的控制 ID 号见表 1。

控制类型	变量名称	类型	对应的 ID 号	Property 窗口选项
1. 编辑框	m_fSignalRange	float	ID_SINNARANGE	缺省
2. 编辑框	m_nSignalPeriod	int	ID_SINNALPERIOD	缺省
3. 编辑框	m_nSignalNumber	int	ID_SINNALNUMBER	缺省
4. 列表框	m_cSampleVotage	CListBox	ID_SAMPLEVOLTAGE	不选择 Sort, 其余缺省
5. 按钮			ID_START	缺省
6. 按钮			ID_STOP	缺省

表 1 对话框资源中的主要控制和风格及加入的变量与之对应的控制 ID 号

加入与控制有关的消息处理函数，ID_START 对应的函数名称为 OnStart，ID_STOP 对应的函数名称为 OnStop。

(6) 在 Test.h 文件中定义一些必要的变量，代码如下：

```
#define sinMaxPoint    101           //对正弦信号进行 100 等分。
#define PI             3.1415926     //定义 PI
protected:
    CString strADConvertResult; //送列表框的字符串,用于显示 AD 转换结果
    char    szErrMsg[80];       //存储数据采集卡错误信息的字符串数组
    float    fOutValue[sinMaxPoint*10]; //用于存储 D/A 转换的正弦值的数组
    float    fInValue[sinMaxPoint*10]; //用于存储 A/D 转换采集的数据
    int      sampleFlagPoint; //正弦值输出时的标记指针
    int      sampleFlagNumber; //正弦波输出个数的标记指针
    float    fData; //正弦值送数据采集卡时的标记指针
    float    fVoltage; //采集数据的标记指针结束语
```

同时，将安装 DLL 动态链接库时生成的 Advantech\ADSAPI\Include 目录下的 Driver.h 和 Os.h 文件拷贝到 Demo 目录下。在 Test.Cpp 文件中加入必要的头文件，代码如下：

```
#include "driver.h"
```

```
#include "os.h"
```

```
#include "math.h"
```

在构造函数中初始化定义的变量，代码如下

```
CTest::CTest(CWnd* pParent /*=NULL*/)
```

```
: CDialog(CTest::IDD, pParent)
```

```
{
```

```
    sampleFlagPoint=0;
```

```
    sampleFlagNumber=0;
```

```
    fVoltage=0;
```

```
}
```

(7) 利用 Class Wizard 生成一个定时器函数 OnTimer，用于进行 D/A 转换输出正弦信号同时进行 A/D 转换采集数据及数据的显示。代码如下：

```
void CTest::OnTimer(UINT nIDEvent)
```

```
{
```

```
    UpdateData(TRUE);
```

```
    static PT_AOVoltageOut ptAOVoltageOut;
```

```
    static PT_AIVoltageIn ptAIVoltageIn;
```

```
    static PT_AIConfig ptAIConfig;
```

```
    LRESULT ErrCde;//返回错误代号
```

```
    LONG DriverHandle=(LONG)NULL;//指向所选设备编号的句柄指针
```

```
    // 打开设备
```

```
    ErrCde = DRV_DeviceOpen(0, (LONG far *)&DriverHandle);
```

```
    if (ErrCde != SUCCESS)
```

```
    {
```

```
        strcpy(szErrMsg, "Device open error !");
```

```
        MessageBox((LPCSTR)szErrMsg, "Device Open", MB_OK);
```

```
    }
```

```
    if(sampleFlagPoint<=(m_nSignalNumber*sinMaxPoint-m_nSignalNumber))
```

```
    {
```

```
        fData = fOutValue[sampleFlagPoint];
```

```
        //进行 da 转换
```

```
        ptAOVoltageOut.chan = 0;//DA 通道选为 0 通道
```

```
        ptAOVoltageOut.OutputValue = fData;//fData 指针指向待 D/A 转化的正弦值
```

```
        if((ErrCde = DRV_AOVoltageOut(DriverHandle,
```

```
            (LPT_AOVoltageOut)&ptAOVoltageOut)) != 0)
```

```
        {
```

```
            DRV_GetErrorMessage(ErrCde, (LPSTR)szErrMsg);
```

```
            MessageBox((LPCSTR)szErrMsg, "Driver Message", MB_OK);
```

```
            DRV_DeviceClose((LONG far*)&DriverHandle);
```

```
        }
```

```
        ptAIConfig.DasGain = 0;////为指定的模拟量输入通道配置增益为 0
```

```
        ptAIConfig.DasChan = 13;//设置 A/D 转换的通道为 13 通道
```

```

if ((ErrCde = DRV_AIConfig(DriverHandle,
    (LPT_AIConfig)&ptAIConfig)) != 0)
{
    DRV_GetErrorMessage(ErrCde, (LPSTR) szErrMsg);
    MessageBox((LPCSTR) szErrMsg, "Driver Message", MB_OK);
    DRV_DeviceClose((LONG far *)&DriverHandle);
}
//进行 AD 转换
ptAIVoltageIn.chan = 13; //模拟量输入通道为
ptAIVoltageIn.gain = 0; //模拟量输入通道增益
ptAIVoltageIn.TrigMode = 0; //触发方式为内部触发
ptAIVoltageIn.voltage=(FLOAT far *)&fVoltage; //采集的数据由 fVoltage 标记
if ((ErrCde = DRV_AIVoltageIn(DriverHandle,
    (LPT_AIVoltageIn)&ptAIVoltageIn)) != 0)
{
    DRV_GetErrorMessage(ErrCde, (LPSTR) szErrMsg);
    MessageBox((LPCSTR) szErrMsg, "Driver Message", MB_OK);
    DRV_DeviceClose((LONG far *)&DriverHandle);
}
//往列表框中送数来显示
strADConvertResult.Format("%.8f", fVoltage);
m_cSampleVoltage.AddString(strADConvertResult);
m_cSampleVoltage.SetTopIndex(m_cSampleVoltage.GetCount()-1);
fInValue[sampleFlagPoint]=fVoltage; //采集的数据存入数组
sampleFlagPoint++; //准备送下一个正弦信号的值
}
CDialog::OnTimer(nIDEvent);
}

```

(9) 拷贝安装的 Advantech\ADSAPI\目录下的 Adsapi32.lib 库文件到 Demo 目录下。在 Project 菜单中打开 Project Setting 对话框, 并在 Link 项中加入应用程序必须的库文件 Adsapi32.lib。

(10) 利用 Class Wizard 给菜单项 ID 为 ID_TEST 的菜单栏添加视图类命令 OnTest, 编辑代码如下。

```

void CDemoView::OnTest()
{
    CTest test;
    test.DoModal();
}

```

同时在 DemoView.cpp 文件中添加#include "test.h"头文件。

(11) 最后, 给 OnStop 添加代码 OnOK()。编译并运行程序, 输入正弦信号的参数, 点击开始控制按钮, 如果 13 通道与 0 通道直接相连接的话, 可以在 Clist 中看到转化的数据正是输入的正弦信号值, 通过示波器也可以看到这是一个正弦信号。

5. 结束语

本文介绍了研华公司的 PCL-812PG 数据卡的基本功能和使用方法, 及其在 VC++6.0 开发

环境下的 MFC 实现，并且给出了一个具体实例加以说明。可见，采用 PCL-812PG 数据采集卡，以 VC++6.0 为软件开发平台，充分利用 VC++6.0 生成友好界面的优势和 PCL-812PG 的功能和资源，快速有效的开发出功能强大的实时数据采集控制系统，这样不但提高了整个控制系统的可靠性，而且缩短了工程应用开发的研究周期，提高了工作效率。

.

附录一 PCL-813 快速安装使用入门

一、 产品介绍

PCL-813 是一款 32 路单端带隔离的模拟量输入卡，广泛应用于工业现场和实验室，例如，变送器、传感器输出量检测，波形获取和检测，过程监控以及振动瞬时分析。

1.1 特点：

1. 32 路单端带隔离的模拟量输入；
2. 12 位逐次比较式 A/D 转换；
3. 多种模拟量输入范围，可以通过软件设定： ± 5 ， ± 2.5 ， ± 1.25 ， ± 0.625 ， $0\sim 10V$ ， $0\sim 5V$ ， $0\sim 2.5V$ ， $0\sim 1.25V$ ；
4. 软件触发模式；
5. 支持多种语言开发，如 BASIC, Delphi, C 和 C++等

1.2 规格：

1.2.1 模拟输入(A/D 转换)

1. 通道：32 路单端带隔离输入
2. 分辨率：12 位
3. 输入范围： 双极性： $\pm 5V$ ， $\pm 2.5V$ ， $\pm 1.25V$ ， $\pm 0.625V$ (软件可编程)
单极性： $0\sim 10V$ ， $0\sim 5V$ ， $0\sim 2.5V$ ， $0\sim 1.25V$ (软件可编程)
(用 JP100 来设置单或双极性)
4. 转换器：采用 AD574 或与之兼容的芯片, 25 微妙转换时间
5. 数据传输速率：最高可达 25KHz, 软件控制
6. 隔离电压： $>500VDC$ (从模拟量输入到输出)
7. 精度：读取数值的 $0.001\% \pm 1 \text{ LSB}$
8. 非线性度：最大 $\pm 1 \text{ BIT}$
9. 放大倍数： $X1, X2, X4, X8$ 可编程
10. 触发模式：软件触发
11. 温度系数： $\pm 25 \text{ PPM}/^{\circ}C$
12. 过载电压：最大可承受连续 $\pm 30V$
13. 输入阻抗： $>10M\Omega$

1.3 一般特性:

1. 功耗: +5V: 660mA(典型值) +12V: 140mA(典型值)
2. I/O 接口: 用于输入端口的 DB-37 针型接口(模拟输入端口)
3. 工作温度: 0°~50°C(32°~ 122°F)
4. 储存温度: -20°~50°C(-4°~149°F)
5. 面板尺寸: 99mm X 219mm
6. 重量: 210gm(7.04202)

二、 安装与测试

2.1 初始检查

研华 PCL-813, 包含如下三部分: 一个 32 通道单端带隔离的模拟量输入数据采集卡 PCL-813, 一本使用手册和一个内含板卡驱动的光盘。打开包装后, 请您查看这三件是否齐全, 请仔细检查有没有在运送过程中对板卡造成的损坏, 如果有损坏或者规格不符, 请立即告知我们的服务部门或是本地经销代理商, 我们将会负责维修或者更换。取出板卡后, 请保留它的防震包装, 以便在您不使用时将采集卡保护存放。在您用手持板卡之前, 请先释放手上的静电(例如, 通过触摸您电脑机箱的金属底盘释放静电), 不要接触易带静电的材料, 比如塑料材料等。手持板卡时只能握它的边沿, 以免您手上的静电损坏面板上的集成电路或组件。

2.2 开关和跳线的设置

面板上有一个 DIP 开关, 这个开关是用来选择设置 PCL-813 在您计算机上的基地址; JP100 用来设置单极性 or 双极性输入模式。

2.2.1 I/O 地址的选择

大多数的外设和接口卡都是通过计算机的 I/O 口来控制的, 它们各自都有一个独立的 I/O 存储空间以免相互之间发生地址冲突, PCL-813 使用 16 个连续的 I/O 地址空间, 下图给出了它的 I/O 地址选择, 地址的选择可通过面板上的 6 位 DIP 开关 SW1 的设置来设定。PCL-813 的有效地址范围是 000 到 3F0 (十六进制), 初始默认地址为 220, 您可以根据系统的资源占用情况, 给 PCL-813 分配正确的地址, 按照下图来设置它的地址。

CARD I/O Address (SW1)						
I/O ADDRESS RANGE	SWITCH POSITION					
	1	2	3	4	5	6
(HEXADECIMAL)	A9	A8	A7	A6	A5	A4
000 - 100	●	●	●	●	●	●
100 - 10F	●	○	●	●	●	●
——	-	-	-	-	-	-
——	-	-	-	-	-	-
200 - 20F	○	●	●	●	●	●
* 210 - 21F	○	●	●	●	●	○
220 - 22F	○	●	●	●	○	●
——	-	-	-	-	-	-
300 - 30F	○	○	●	●	●	●
——	-	-	-	-	-	-
3F0 - 3FF	○	○	○	○	○	○

○ = Off ● = On * = Factory default setting

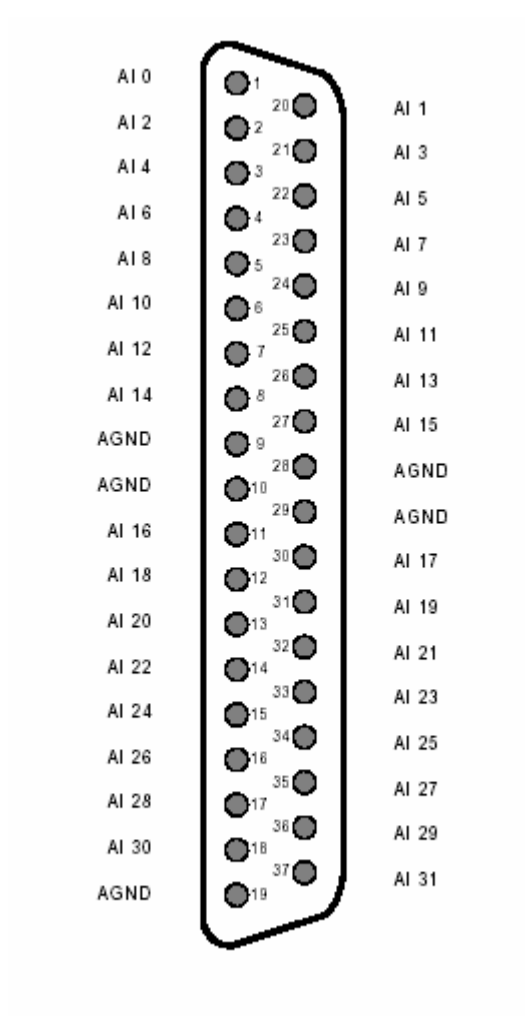
注意：A4到A9是和计算机上的地址线相对应

输入电压的选择

单极性模拟输入电压：将JP100放在“U”的位置，输入电压的范围可设为0~10V，0~5V，0~2.5V，0~1.25V。

引脚图

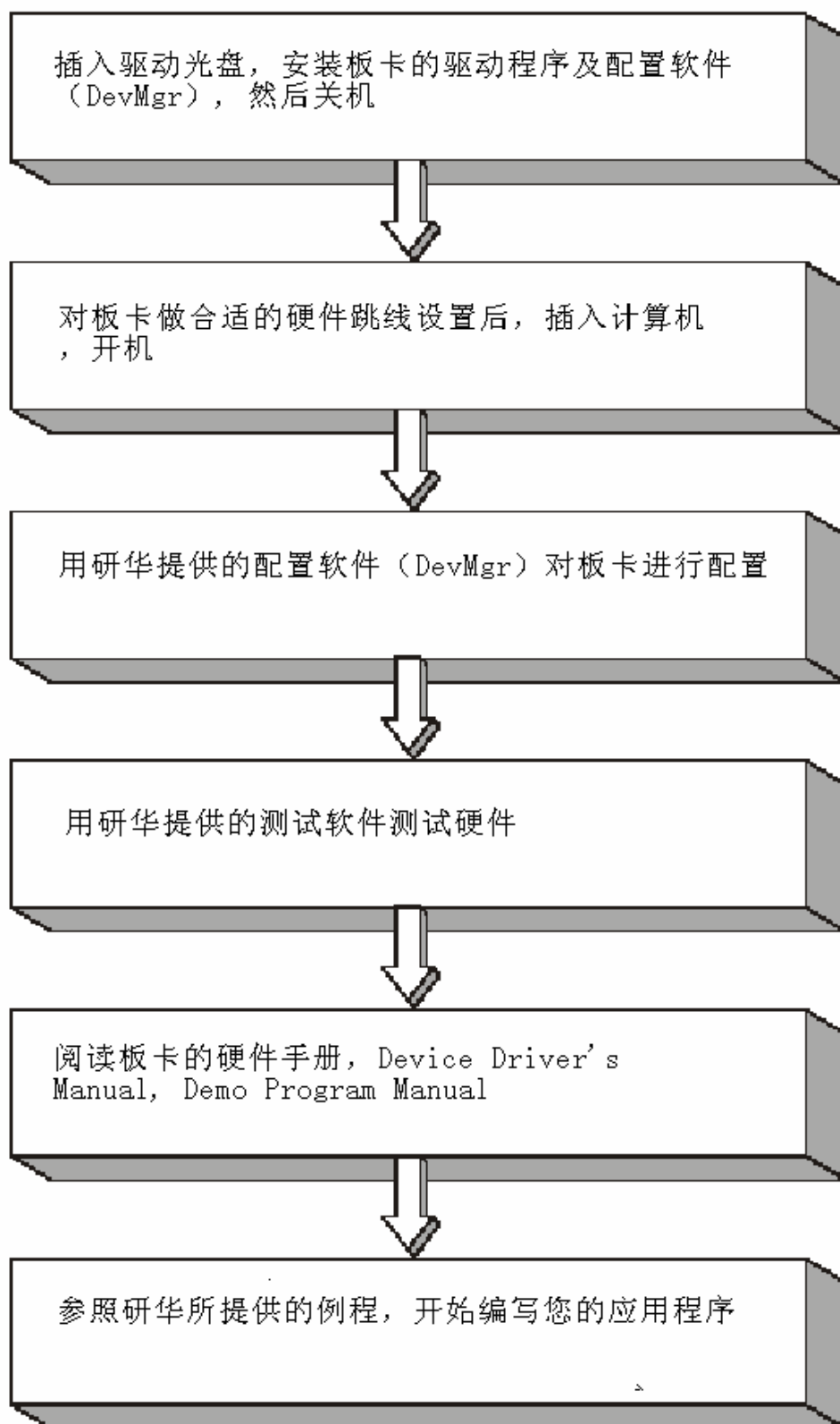
PCL-813 的管脚分配如下图所示：



关键词： AI：模拟输入 AGND：模拟地

2.3 Windows2K/XP/9X 下板卡的安装

安装流程图，如下：



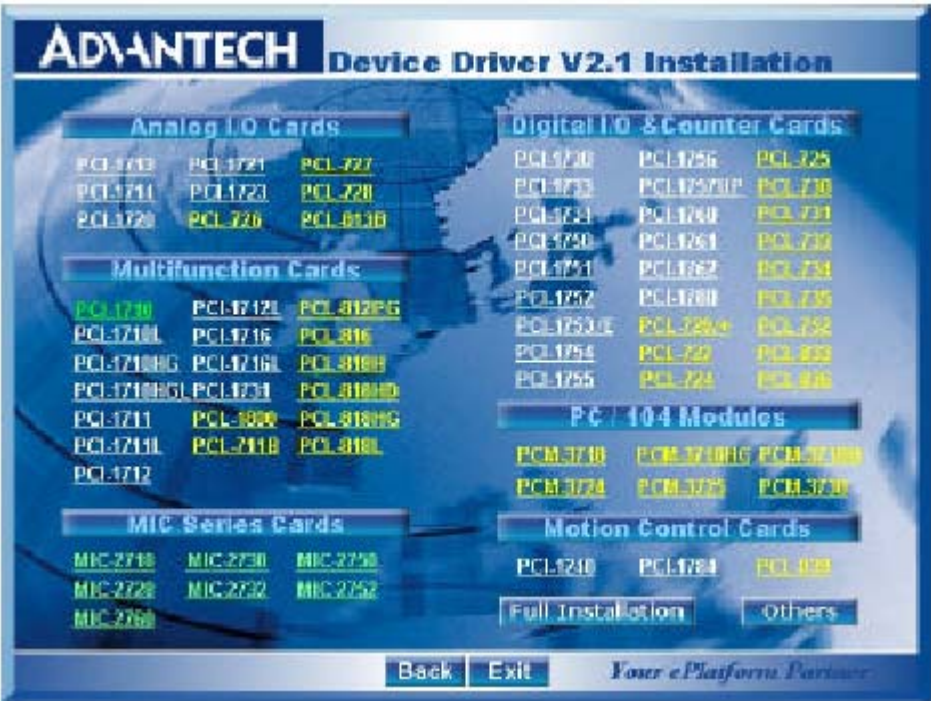
2.3.1 软件的安装:

- 第一步: 将启动光盘插入光驱;
- 第二步: 安装执行程序将会自动启动安装 , 这时您会看到下面的安装界面:



注意: 如果您的计算机没有启用自动安装, 可在 H: 盘文件中点击 SETUP.EXE 文件启动安装程

第三步: 点击 CONTINUE, 出现下图界面:



第四步: 选择点击您所安装的板卡型号, 然后按照提示就可一步一步完成驱动程序的安装。

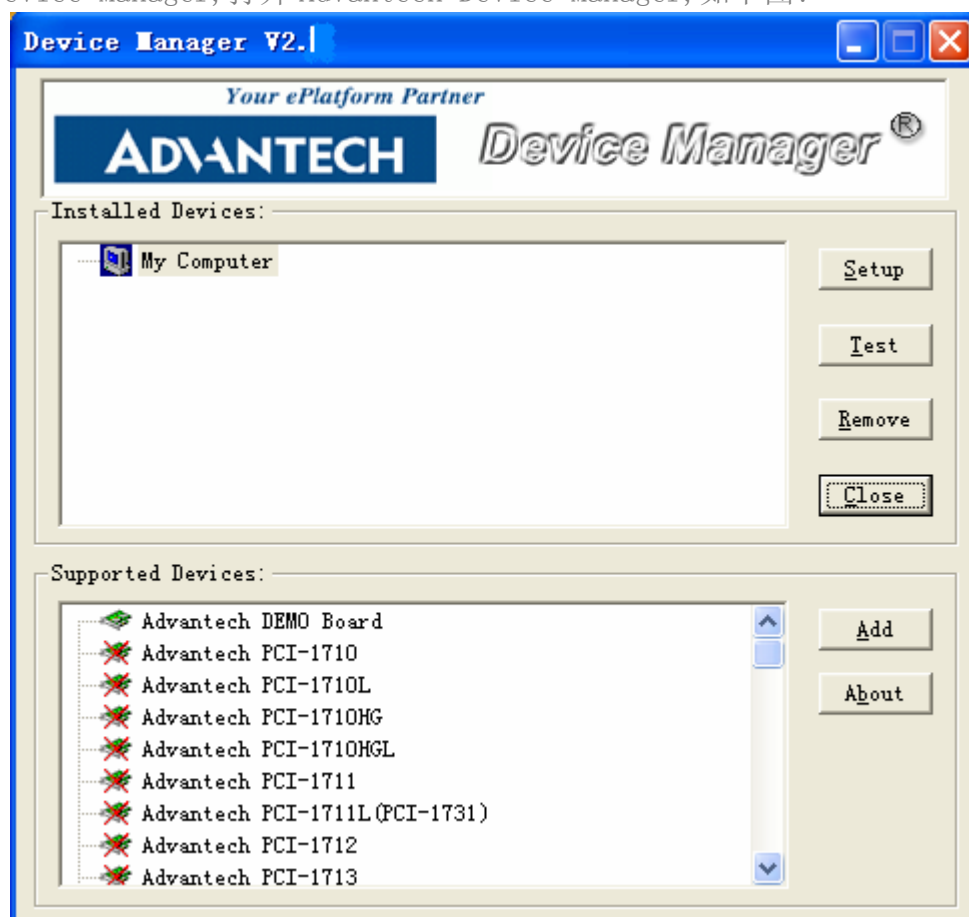
2.3.2 硬件的安装:

第一步: 参照 2.2 节, 完成板卡开关和跳线的设置

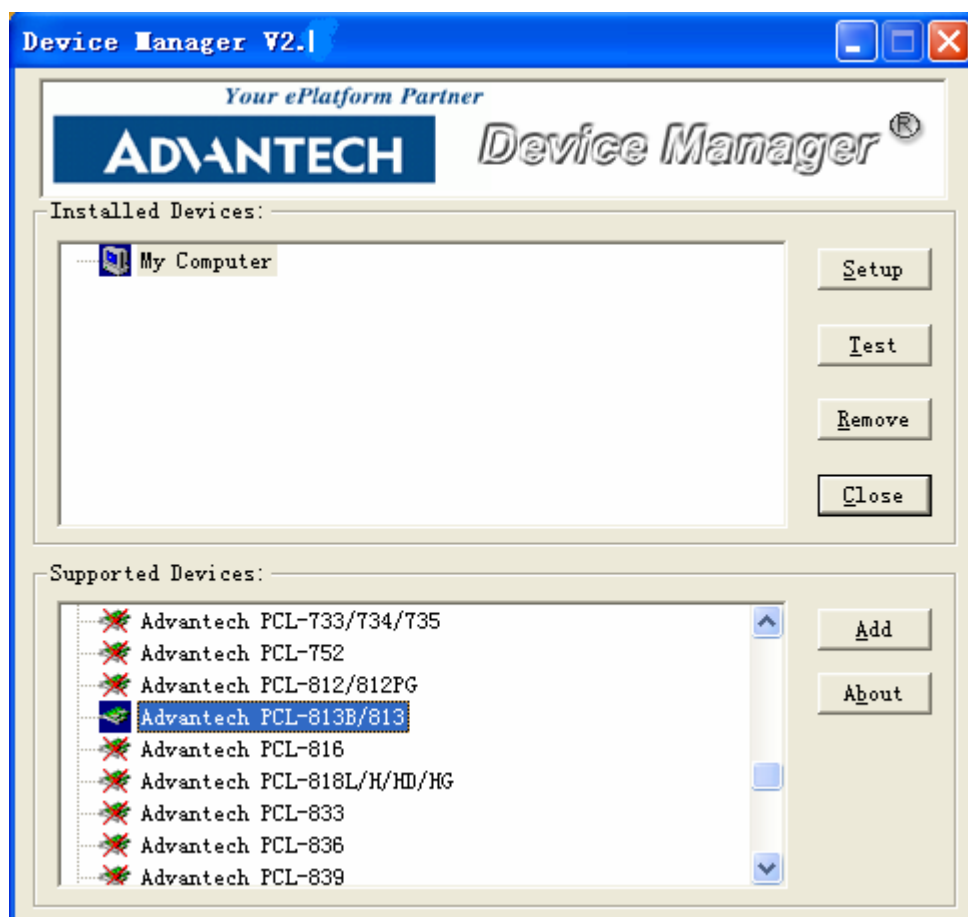
第二步: 关掉计算机, 将您的板卡插入到计算机后面空闲的 ISA 插槽中

(注意: 在您手持板卡之前触摸一下计算机的金属机箱壳以免手上的静电损坏板卡。)

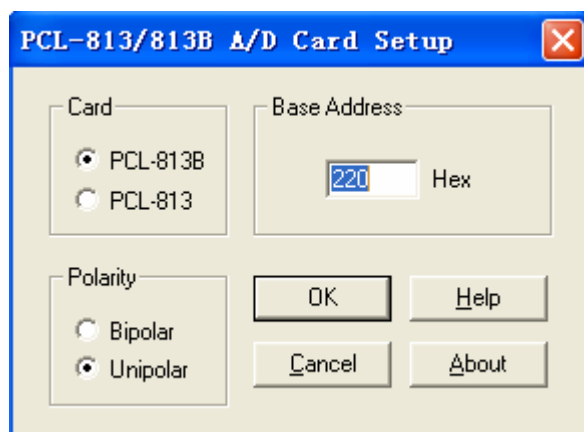
第三步: 从开始菜单/程序/Advantech Device Driver V2.1/ Advantech Device Manager, 打开 Advantech Device Manager, 如下图:



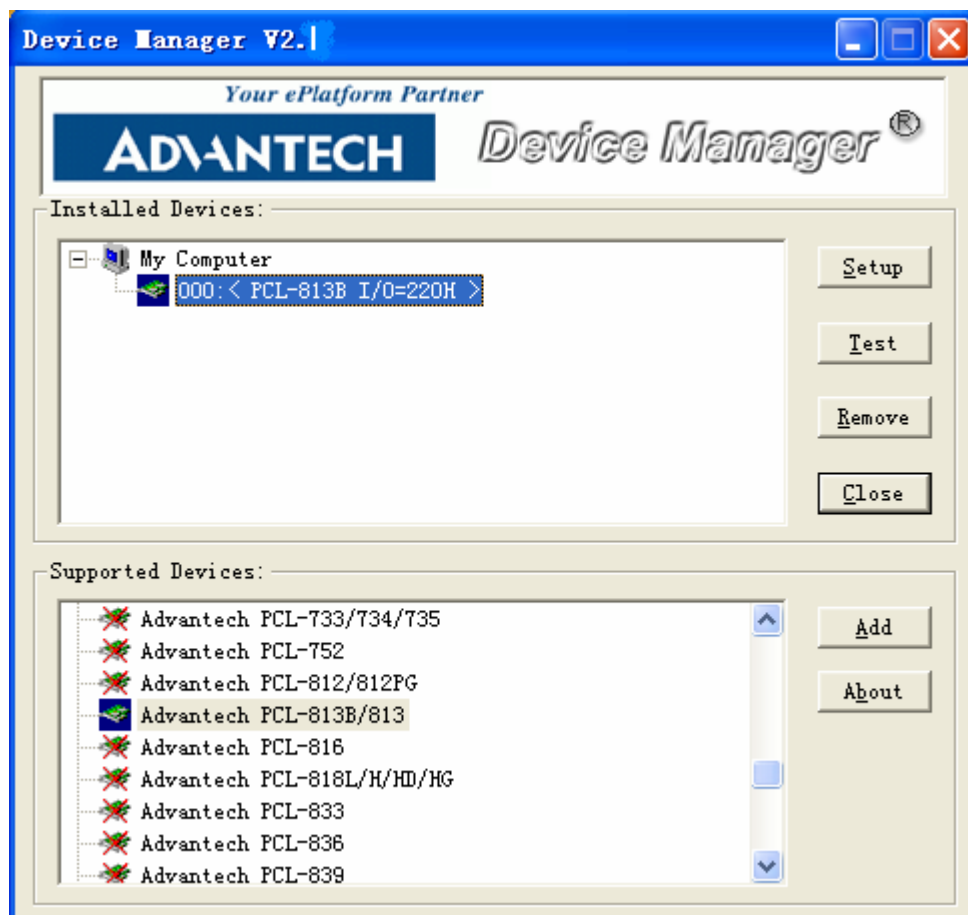
在 Supported Devices 列表中选中您所要安装的器件, 比如 PCL-813(注意: 当您的计算机上已经安装好某个产品的驱动程序后, 它前面将没有红色叉号, 说明驱动程序已经安装成功。比如下图中的 PCL-813 前面就没有红色叉号)



点击“Add”，弹出下图，进行地址和单双极性的选择（注意：地址的选择和单双极性的选择要和您的硬件设置相符）



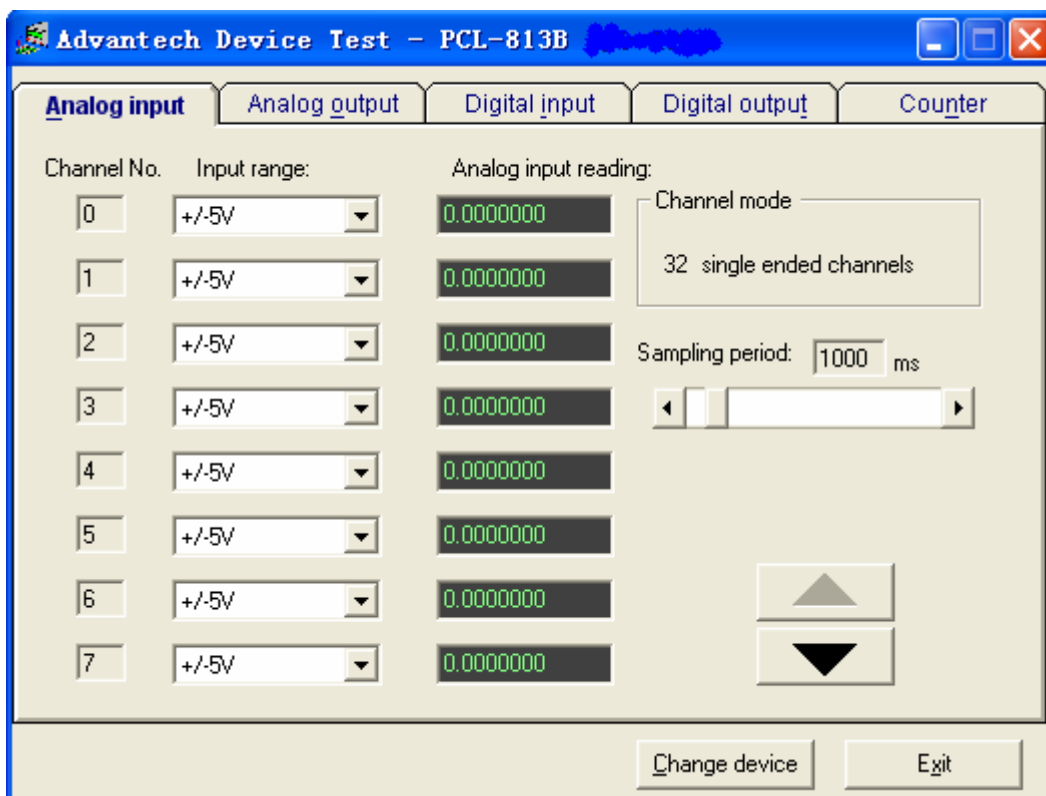
完成后点击“O K”就会在 Installed Devices 栏中 My Computer 下显示出所加的器件，如下图所示：



到此，PCL-813 数据采集卡的软件和硬件已经安装完毕，可进行板卡测试。

2.4 测试

在上图的界面中点击“Test”，弹出下图：



模拟输入功能测试 （因为 PLC-813 是模拟量输入数据采集卡，所以只有模拟输入功能测试）

测试界面说明：

Channel No: 模拟量输入通道号(0-31);

Input range: 输入范围选择;

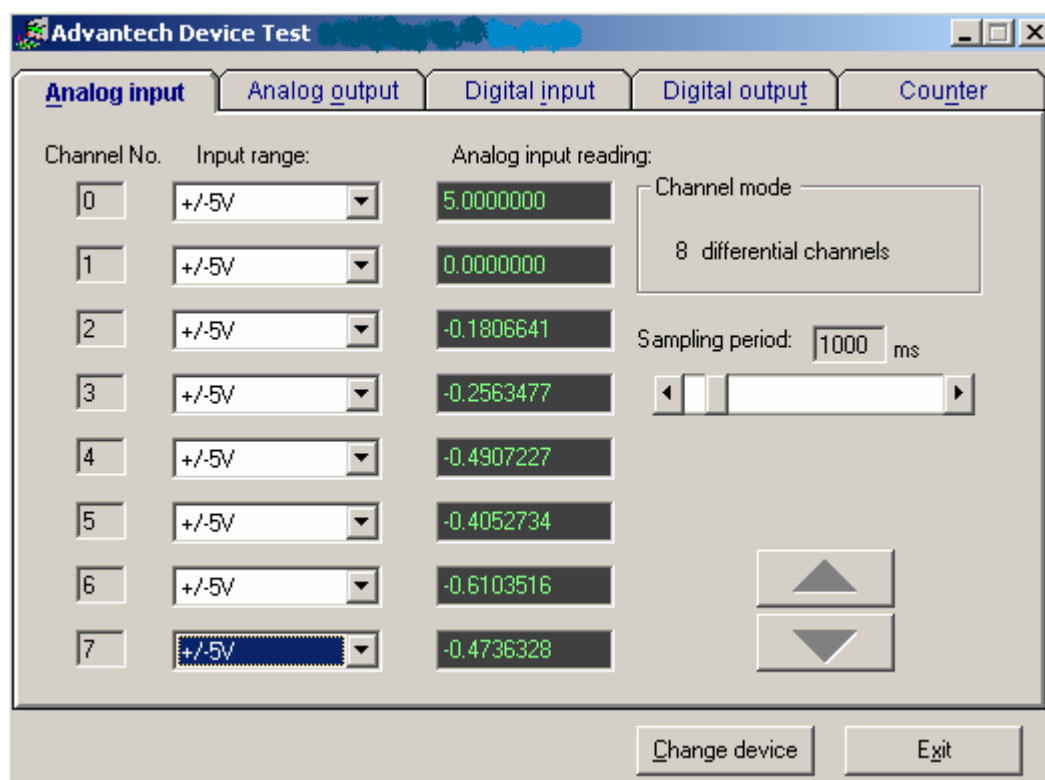
Analog input reading : 模拟量输入通道读取的数值;

Channel mode: 通道设定模式;

sampling period : 采样时间间隔;

测试时可用 PCL-10137-1/2/3 (37 芯 D 型电缆, 1, 2 或 3 米) 将 PCL-813 与 ADAM-3937 (可 DIN 导轨安装的 DB-37 接线端子板) 连接, 这样 PCL-813 的 32 个模拟输入通道就和 ADAM-3739 的 32 个接线端子一一对应, 可通过将输入信号连接到接线端子来测试 PCL-813 的 32 路模拟输入。

例如: 将 Channel 1 对应的接线端子与接线端子上对应的模拟地短接, 在 Channel 1 的 Analog input reading 中将读到值为 0.000000; 您也可以在模拟输入端外加一个电压信号进行测试, 但注意不要超出您设定的测量范围, 例如下图, 是在第一通道 (Channel 0) 输入+5V 电压, 第二个通道输入 0V 电压。



注意：在不接任何信号的时候，程序也会读到一个随机数值，这个数值是没有意义的，当接入实际电压时就会显示实际电压值。

三、 信号输入连接

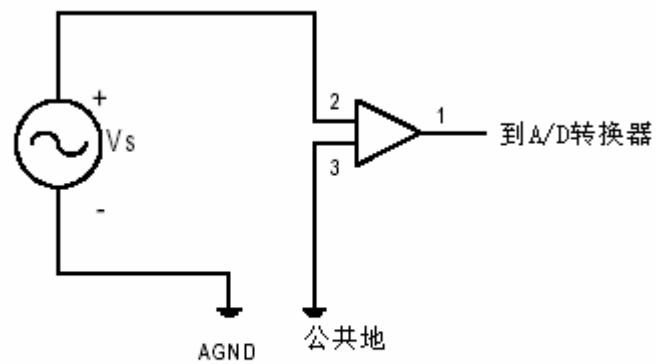
在数据采集应用中，模拟量输入基本上都是以电压信号输入。为了达到准确测量并防止损坏您的应用系统，正确的信号连接是非常重要的。这一章我们将向您介绍如何来正确连接模拟信号的输入。

模拟信号输入连接：

PCL-813 是 32 路单端带隔离模拟量输入数据采集卡，当测量一个电压信号时，将信号源一端接入到一个输入端子上，信号源另一端接到地端子上。（五个模拟地中的任何一个）。例如：测量一个电池的电压，只需将电池的正极接到一个单端输入接线端子上，将电池的负极接到模拟地端子上即可。

（注意： PCL-813 不支持差分信号输入。）

下图为单端信号输入示意图：



单端信号输入连接

面板上单端输入电路一端接输入信号（2），一端是公共地（3）。输入信号一端接到 2，另一端接到模拟地。

附录二 工业通讯卡使用入门

介绍:

本文将向您介绍研华通讯卡在不同操作系统下的安装方法。另外，对于串口测试工具 TTY 也有相应的介绍，利用这个工具，你可以非常方便的测试通讯卡是否安装成功

一、 如何在 Win 95/98/ME 下安装 PCL 系列的通讯卡

Window 95/98 提供了标准串口驱动并且支持多串口共享中断模式，安装步骤如下：

1. 请按如下步骤先添加新的串口：
[开始] => [设置] => [控制面板] => [添加新硬件]
2. 点击[下一步] 并选择[否] 取消自动搜索新硬件
3. 从硬件列表中选择 [端口 (COM & LPT)] 选项.
4. 从左边的列表框中选[标准端口类型]
从右边的列表框中选[通讯端口]
5. 接下来弹出的窗口中会显示出默认的 IRQ 和 I/O 地址。点击[下一步] 和 [完成].
到现在为止，我们已经添加了一个串口，但它的 IRQ 和 I/O 地址是不正确的。您可以按照以上五个步骤把所有的串口都添加上(Win 95/98 最多支持 256 个串口).
6. 接下来，我们需要将串口的资源修改正确
[开始] => [设置] => [控制面板] => [系统] => [设备管理器]
7. 选择你想要修改的[通讯端口] 并双击
8. 选择 [资源] 选项卡并将 “使用自动设置” 这一项取消，然后 [更改设置] 按钮就可以使用了
9. 注意 “设置基于” 这一项改为 “默认设置 8”.
10. 双击 “中断请求” 选择正确的 IRQ，同样的方法选择正确的 “输入/输出范围” 按照步骤 6-10 配置其余的串口。
11. 重新启动 Window 95/98 使新的设置生效

注意：

有一些 PCL 通讯卡可以采用标准工作模式或增强模式：

在标准模式下，每个端口可采用不同的基地址和中断（PCL-746+ ver A1）

Port1	CH1	Base Address	AAA,	CH1	IRQ	M
Port2	CH2	Base Address	BBB,	CH2	IRQ	N
Port3	CH3	Base Address	CCC,	CH3	IRQ	O
Port4	CH4	Base Address	DDD,	CH4	IRQ	P

在增强模式下，四个串口可使用不同的基地址但共享中断。

二、如何在 Win NT 下安装 PCL 系列的通讯卡

Win NT 提供了标准串口驱动并且支持多串口共享中断模式。但是在默认情况下，共享中断是不被允许的。如果您想使用独立的中断配置串口，可按以下步骤进行：

1. 请按以下步骤添加一个新的串口：
[开始] => [设置] => [控制面板] => [端口]
2. 点击 [添加] 并且设置 I/O 地址和 IRQ.
3. 如果还有其他串口要添加，就先不要重启。
4. 重复第二步添加其他串口。

例如：741, 743, 745

Port 1 => COM3, I/O 3E8h, IRQ 5

Port 2 => COM4, I/O 2E8h, IRQ 7

例如：746+, 749, 840, 849

Port 1 => COM3, I/O 300h, IRQ 5

Port 2 => COM4, I/O 308h, IRQ 7

Port 3 => COM5, I/O 310h, IRQ 9

Port 4 => COM6, I/O 318h, IRQ 10

5. 重启 WinNT 使新的设置生效.

共享中断方式：

如果您需要在 WinNT 中使用共享中断方式，必须先修改注册表。操作方法请参考以下步骤：

1. 请按以下步骤添加一个新的串口：
[开始] => [设置] => [控制面板] => [端口]
2. 点击 [添加] 并且设置 I/O 地址和 IRQ.
3. 因为还有其他串口要添加，先不要重启。
4. 重复第二步添加其他串口：

例如：746+, 749, 840, 849

Port 1 => COM3, I/O 300h, IRQ 12

Port 2 => COM4, I/O 308h, IRQ 12

Port 3 => COM5, I/O 310h, IRQ 12

Port 4 => COM6, I/O 318h, IRQ 12

5. 运行 regedit.exe 修改注册表
[开始] => [运行] => 键入 "regedit"
6. 按照下面的路径找到串口的注册表（如 Fig.1 所示）

HKEY_LOCAL_MACHINE => SYSTEM => CurrentControlSet => Services => Serial

7. 从右边的列表框中选择 “ShareInterrupt” 并且将键值从 0 改为 1 (Fig.2)
8. 重启 WinNT 使新的设置生效.

注意：

1. 系统要求 SP3 或以上版本。否则将不会出现 ShareInterrupt 这一项。
2. 对于 IRQ 和基地址, 硬件跳线必须与软件设置一致。并且必须保证所使用的中断和基地址与其他设备没有冲突，否则将无法正确安装。

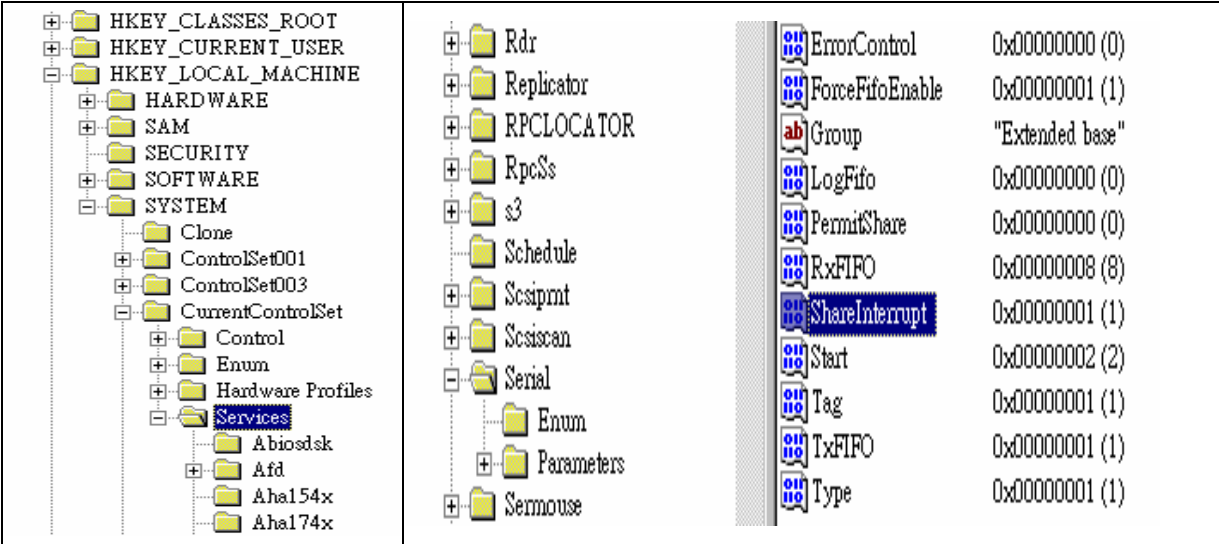


Figure 1

Figure 2

三、 如何在 Win 2000 下安装 PCL 系列的通讯卡

Win 2000 同样提供了标准串口驱动并在默认情况下支持共享中断模式。因此，用户可在 Win 2000 中使用标准和增强两种模式而不必修改注册表。下面是添加串口的步骤：

1. 请按以下步骤添加一个新的串口：

[开始] => [设置] => [控制面板] => [添加/删除新硬件]

选择[添加/排除设备故障] 并点击 [下一步]，操作系统会自动搜索新硬件

选择 [添加新设备]，然后点击 [下一步]。选择[否，我想从列表中选择]，点击 [下一步]。

选择 [端口]，然后从左边选中[标准端口类型] 从右边选中[通讯端口] 点击 [下一步]。

5. 选择 [资源] 选项卡并将 “使用自动设置” 这一项取消，然后 [更改设置] 按钮就可以使用了

6. 注意 “设置基于” 这一项改为 “默认设置 8”。

7. 双击 “中断请求” 选择正确的 IRQ，同样的方法选择正确的 “输入/输出范围”。注意不要使用与已有设备冲突的基地址和中断。

8. 如果还有其他串口要添加，先不要重启。

9. 重复以上步骤添加其他串口

例如：741, 743, 745

Port 1 => COM3, I/O is 3E8h, IRQ is 5

Port 2 => COM4, I/O is 2E8h, IRQ is 7

例如：746+, 749, 840, 849

Port 1 => COM3, I/O is 300h, IRQ is 5

Port 2 => COM4, I/O is 308h, IRQ is 7

Port 3 => COM5, I/O is 310h, IRQ is 9

Port 4 => COM6, I/O is 318h, IRQ is 10

四、 如何在 Win 95/98/ME 下安装 PCI 系列的通讯卡

研华 PCI ICOM driver 提供的串口编号从 COM 5 开始。

1. 首先安装研华提供的 PCI ICOM driver ,装完后再开始菜单中可以看到他。

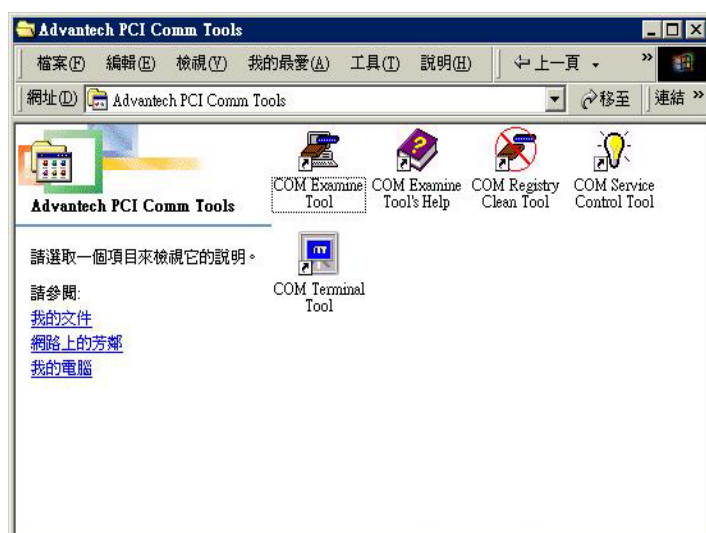


Figure 3 Advantech PCI Comm Tools

完成安装 Win95/98/ME 驱动后请重启计算机。这样，Windows 95/98/ME 会自动找到 UARTs 设备。

当 PCI UARTs 设备安装完成后，Windows 95/98/ME 接下来会安装 PCI Bridge device. 这两项安装都是自动完成的。

当 PCI Bridge device driver 安装完毕后，会有一个如下图所示的窗口弹出，点击[完成]。接下来，串口也会被自动安装。



Figure 3 Advantech PCI Bridge

为了检查 Win95/98/ME 下驱动的安装情况，系统完成安装之后，建议到控制面板/系统/设备管理器 中察看“设备名称”。看是否工作正常

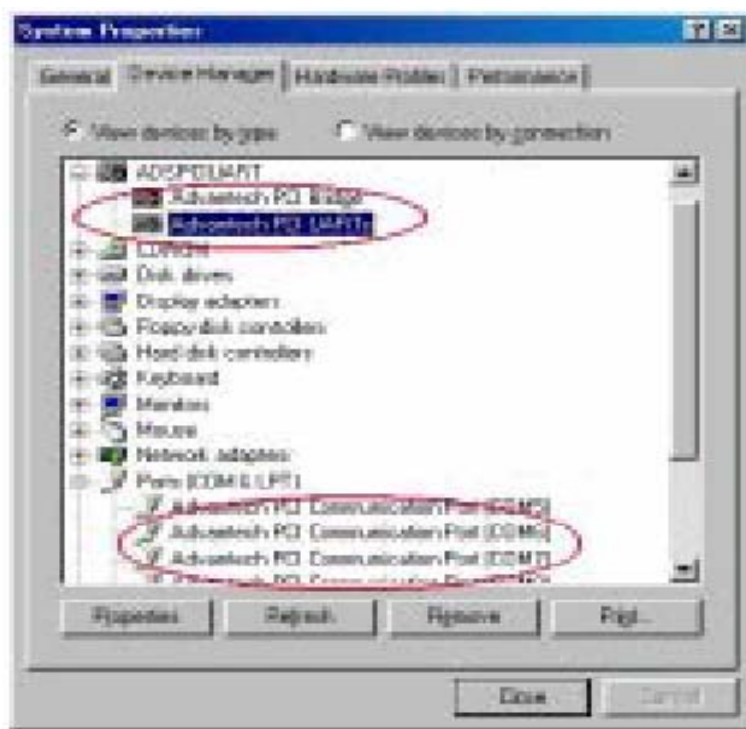


Figure 4 Advantech PCI Communication card

五、 如何在 Win NT 下安装 PCI 系列的通讯卡

Windows NT 最多支持 256 个串口，从 COM1 到 COM256。

请按照以下步骤完成安装：

首先安装 Advantech PCI ICOM Series Windows NT Driver，安装程序会在开始菜单中创建 Advantech ICOM Tools 目录。



Figure 5 Advantech PCI Com Tools

2. 装完驱动之后，在 开始/程序/Advantech PCI Comm Tools/COM Service Start up Tool 中运行硬件驱动。
3. 接下来，Advantech PCI Serial Service Manager 对话框就会显示出来。选择 Install and Start Service 这一项，然后点击 ok。（如 Figure6 所示）



Figure 6 Service Manager

4. 完成通讯卡的安装之后，进入 控制面板/端口 就可以看到已经添加的串口，如 Figure 7. 所示

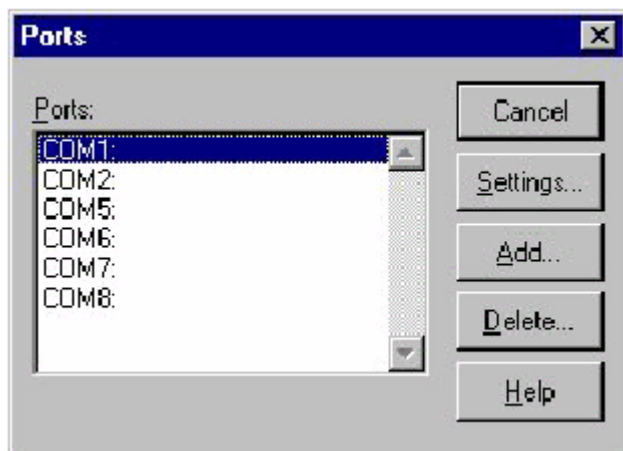


Figure 7 端口 in NT

5. 您可以通过 Windows NT 诊断工具 看到驱动的运行状况. 在 Services 选项卡上, 可以看到 Advantech PCI serial driver 正在运行. 如 Figure 8 所示.

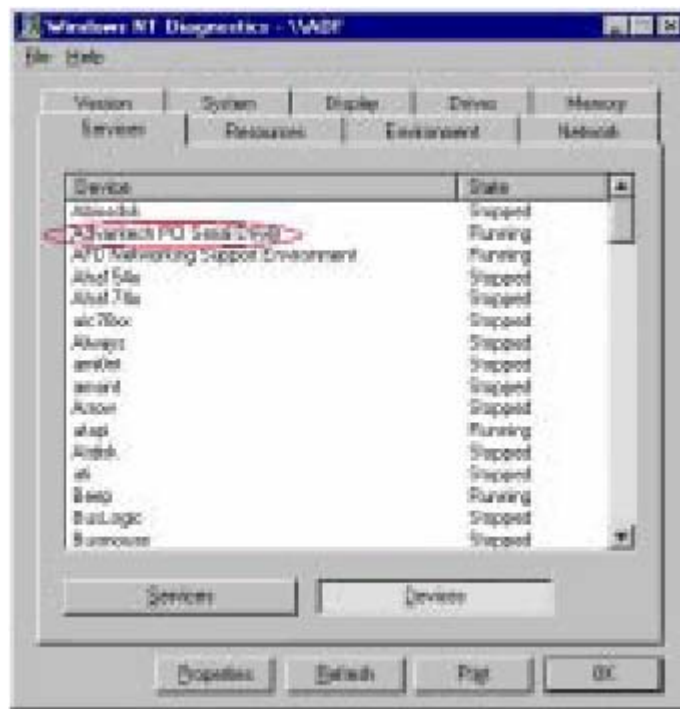


Figure 8 Windows NT Diagnostics utility

六、如何设置通讯卡上串口的序号

Win95/98/NT:

首先需要安装“Advantech Icom Tools ver 1.1 in Win9x/NT”，然后运行程序“Adsstart”如下图所示，可以在“Options”中设置开始的串口序号。这样，就可以按照您的要求来设置串口的序号了。



Figure 9 Remapping Dialog

Win2000:

用户可以直接在“设备管理器”中更改串口序号。先选中“端口 (COM & LPT)”并且点开右边的加号，选中您想要修改的“Advantech PCI communication port”，然后单击“Properties”，在“Setting”中更改串口序号。

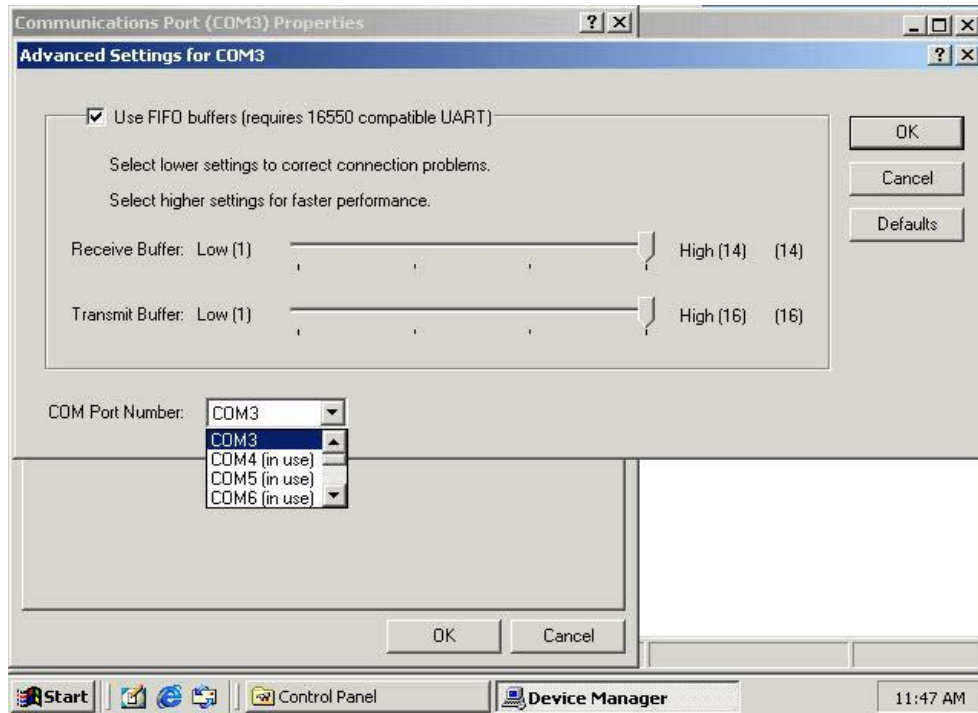


Figure 10 Advanced 设置 for Com Port

七、 如何测试已安装的通讯卡

为了检查通讯卡是否已被成功的安装，您可以使用“超级终端”这样的工具来检测串口。除此之外，研华还提供了工具“TTY”用来进行串口通讯的测试如 Figure 11 所示。

例如，您在系统中添加了 COM1~COM4，并且想要用 TTY 来测试串口的收发是否正常，就可以按照下面的步骤来测试：

用交叉串口线连接 COM1 和 COM2 然后打开两个 TTY（或超级终端）程序，一个给 COM1 用，另一个给 COM2 用。

在两个程序的“Settings”菜单中分别选中 COM1 和 COM2，然后在“Action”菜单中选中“connect”。

如果串口卡已被成功安装，那么两个串口就可通过两个 TTY 窗口相互收发数据。

4. 同样，您也可以采用以上的方法测试 COM3 和 COM4 的通讯是否正常。

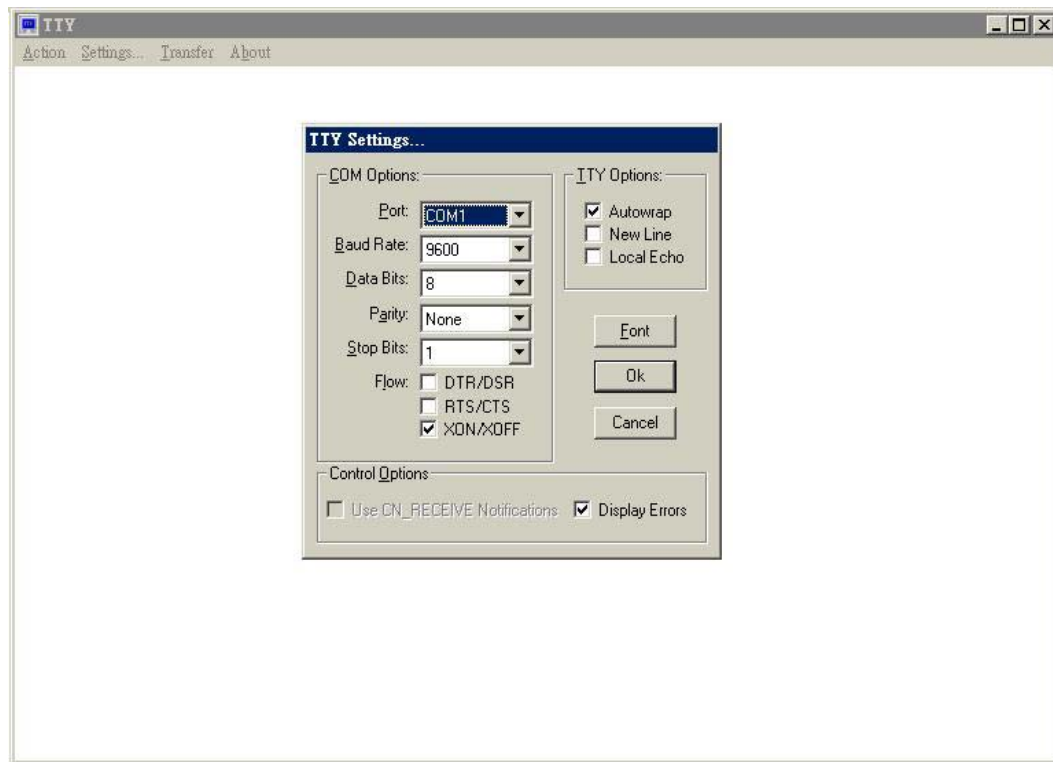


Figure 11 TTY Test Tool

附录三 运动控制卡使用入门

运动控制卡安装在 IPC 上以实现复杂的运动控制功能，这里以 PCI-1240 为例讲解运动控制卡的使用

- 一 PCI-1240 安装
 - 1. 1 PCI-1240 Driver 与 Utility 安装
 - 1. 2 PCI-1240 硬件安装
- 二、 PCI-1240 与驱动器接线
 - 2. 1 PCI-1240 针脚描述
 - 2. 2 PCI-1240 与驱动器连接
- 三、 PCI-1240 测试
 - 3. 1 PCI-1240 Utility 使用
- 四、 PCI-1240 软件编程
 - 4. 1 PCI-1240 软件编程
- 五、 附录
 - 1. PCI-1240 Utility 界面说明：

一、 PCI-1240 安装

1. 1 PCI-1240 Driver 与 Utility 安装

在使用 pci-1240 之前必须安装 pci-1240 驱动，驱动安装步骤：

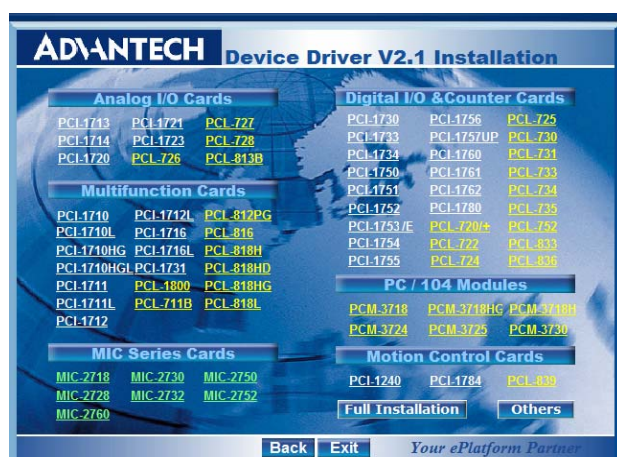
A) 将研华提供的驱动光盘置于光驱中，出现如下画面：



B) 点击 Installation 选项，出现如下画面：



C) 点击 Individual Driver，出现如下画面：



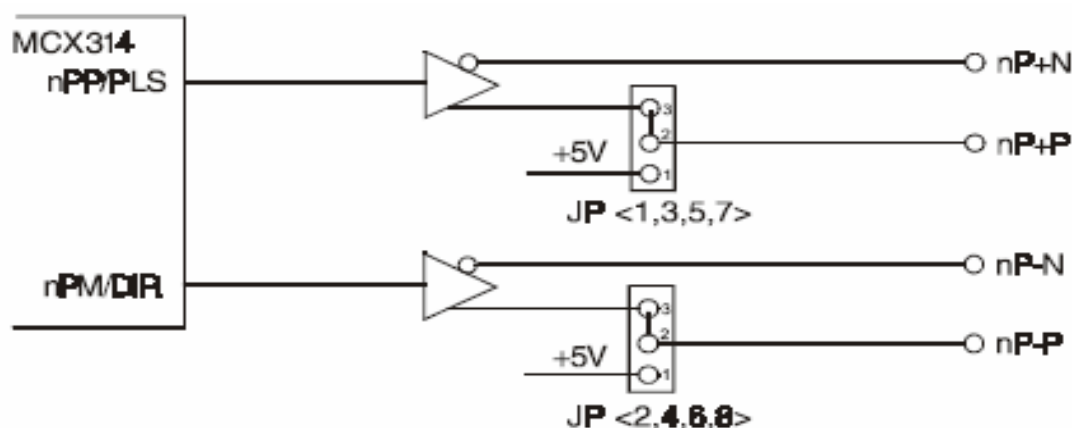
D) 选择 Motion Control Cards 中选项 PCI-1240，点击安装 PCI-1240 驱动；

1. 2 PCI-1240 硬件安装:

1) PCI-1240 跳线设置:

- BoardID 设置: 通过设置板卡上 DIP 开关可以设置 PCI-1240 的 BoardID 从 0—15。
- JP1~8 设置 nP+P, nP+N 和 nP-P, nP-N 输出引脚为 +5v 输出还是差分输出, 缺省设置为差分输出; 如图所示:

注意: 设置为 +5v 单端输出时, 要防止外部噪声窜入 PCI-1240.



III. JP9 Enable/Disable 紧急停止功能, 如图所示:

Jumper	JP9
Emergency stop function enabled when emergency stop signal (EMG) and external GND short	Pin 1 and Pin 2 short (Default)
Emergency stop function enabled when emergency stop signal (EMG) and external GND open	Pin 2 and Pin 3 short

- 2) 单块板卡安装:
 - I. 关闭计算机电源;
 - II. 将 PCI-1240 卡插在计算机的任一 PCI 槽上;
 - III. 重新开启计算机, 系统会自动寻找到 PCI-1240, 根据提示点击 Next 添加 PCI-1240 驱动;
- 3) 多块板卡安装:
 - I. 将板卡的 BoardID DIP 开关设置成不同的值 (不能有重复);
 - II. 先将一块板卡插在一 PCI 槽, 根据单块板卡安装方法, 添加驱动;
 - III. 然后关机, 根据单块板卡安装方法, 依次安装其他板卡。

二、 PCI-1240 与驱动器接线

2. 1 PCI-1240 针脚描述:

YP-N	50	100	UP-N
YP-P	49	99	UP-P
YP+N	48	98	UP+N
YP+P	47	97	UP+P
YOUT7	46	96	UOUT7
YOUT6	48	95	UOUT6
YOUT5	44	94	UOUT5
YOUT4	43	93	UOUT4
GND	42	92	GND
XP-N	41	91	ZP-N
XP-P	40	90	ZP-P
XP+N	39	89	ZP+N
XP+P	38	88	ZP+P
XOUT7	37	87	ZOUT7
XOUT6	36	86	ZOUT6
XOUT5	35	85	ZOUT5
XOUT4	34	84	ZOUT4
GND	33	83	GND
YEXOP-	32	82	UEXOP-
YEXOP+	31	81	UEXOP+
XEXOP-	30	80	ZEXOP-
XEXOP+	29	79	ZEXOP+
YINON	28	78	UINON
YINOP	27	77	UINOP
YECBN	26	76	UECBN
YECBP	25	75	UECBP
YECAN	24	74	UECAN
YECAP	23	73	UECAP
Y_ALARM	22	72	U_ALARM
Y_INPOS	21	71	U_INPOS
XINON	20	70	ZINON
XINOP	19	69	ZINOP
XECBN	18	68	ZECBN
XECBP	17	67	ZECBP
XECAN	16	66	ZECAN
XECAP	15	65	ZECAP
X_ALARM	14	64	Z_ALARM
X_INPOS	13	63	Z_INPOS
Y_IN3	12	62	U_IN3
Y_IN2	11	61	U_IN2
Y_IN1	10	60	U_IN1
YLMT-	9	59	ULMT-
YLMT+	8	58	ULMT+
X_IN3	7	57	Z_IN3
X_IN2	6	56	Z_IN2
X_IN1	5	55	Z_IN1
XLMT-	4	54	ZLMT-
XLMT+	3	53	ZLMT+
EMG	2	52	NC
VEX	1	51	VEX

图 2—1 PCI-1240 针脚定义图
表 2—1 PCI-1240 针脚定义说明

针脚名称	参考地	输入/输出	说明
VEX	*	输入	外部电源（12—24v）
EMG	*	输入	紧急停止信号（所有轴）

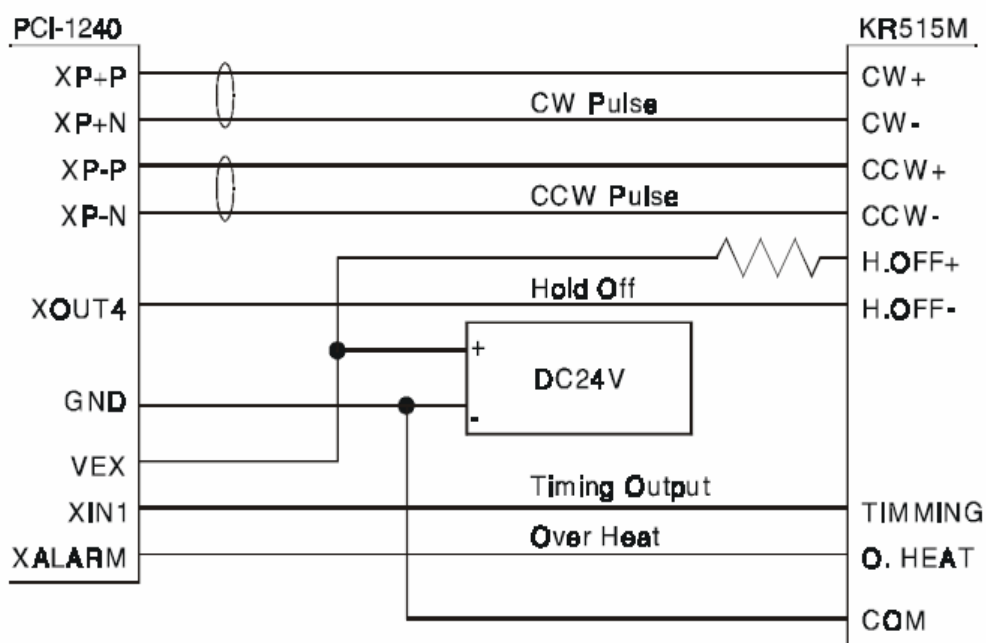
XLMT+	*	输入	X 轴+方向限位控制信号输入
XLMT—	*	输入	X 轴—方向限位控制信号输入
XIN1	*	输入	X 轴减速/立即停止信号
XIN2	*	输入	X 轴减速/立即停止信号
XIN3	*	输入	X 轴减速/立即停止信号
YLMT+	*	输入	Y 轴+方向限位控制信号输入
YLMT—	*	输入	Y 轴—方向限位控制信号输入
YIN1	*	输入	Y 轴减速/立即停止信号
YIN2	*	输入	Y 轴减速/立即停止信号
YIN3	*	输入	Y 轴减速/立即停止信号
XINPOS	*	输入	X 到位输入信号
XALARM	*	输入	X 轴伺服驱动器错误报警信号
XECAP	*	输入	X 轴编码器 A 相+输入
XECAN	*	输入	X 轴编码器 A 相—输入
XECBP	*	输入	X 轴编码器 B 相+输入
XECBN	*	输入	X 轴编码器 B 相—输入
XINOP	*	输入	X 轴编码器 Z 相+输入
XINON	*	输入	X 轴编码器 Z 相—输入
YINPOS	*	输入	Y 到位输入信号
YALARM	*	输入	Y 轴伺服驱动器错误报警信号
YECAP	*	输入	Y 轴编码器 A 相+输入
YECAN	*	输入	Y 轴编码器 A 相—输入
YECBP	*	输入	Y 轴编码器 B 相+输入
YECBN	*	输入	Y 轴编码器 B 相—输入
YINOP	*	输入	Y 轴编码器 Z 相+输入
YINON	*	输入	Y 轴编码器 Z 相—输入
XEXOP+	*	输入	X 轴慢进+向输入
XEXOP-	*	输入	X 轴慢进—向输入
YEXOP+	*	输入	Y 轴慢进+向输入
YEXOP-	*	输入	Y 轴慢进—向输入
GND	*	*	接地脚
XOUT4	GND	输出	X 轴通用输出脚
XOUT5	GND	输出	X 轴通用输出脚
XOUT6	GND	输出	X 轴通用输出脚
XOUT7	GND	输出	X 轴通用输出脚 s
XP+P	GND	输出	X 轴 CW/Pulse + 输出
XP+N	GND	输出	X 轴 CW/Pulse — 输出
XP—P	GND	输出	X 轴 CCW/DIR + 输出
XP—N	GND	输出	X 轴 CCW/DIR — 输出
GND	*	*	接地脚
YOUT4	GND	输出	Y 轴通用输出脚
YOUT5	GND	输出	Y 轴通用输出脚
YOUT6	GND	输出	Y 轴通用输出脚

YOUT7	GND	输出	Y 轴通用输出脚 s
YP+P	GND	输出	Y 轴 CW/Pulse + 输出
YP+N	GND	输出	Y 轴 CW/Pulse - 输出
YP-P	GND	输出	Y 轴 CCW/DIR + 输出
YP-N	GND	输出	Y 轴 CCW/DIR - 输出
VEX	*	输入	外部电源 (12-24v)
ZLMT+	*	输入	Z 轴+方向限位控制信号输入
ZLMT-	*	输入	Z 轴-方向限位控制信号输入
ZIN1	*	输入	Z 轴减速/立即停止信号
ZIN2	*	输入	Z 轴减速/立即停止信号
ZIN3	*	输入	Z 轴减速/立即停止信号
ULMT+	*	输入	U 轴+方向限位控制信号输入
ULMT-	*	输入	U 轴-方向限位控制信号输入
UIN1	*	输入	U 轴减速/立即停止信号
UIN2	*	输入	U 轴减速/立即停止信号
UIN3	*	输入	U 轴减速/立即停止信号
ZINPOS	*	输入	Z 轴到位输入信号
ZALARM	*	输入	Z 轴伺服驱动器错误报警信号
ZECAP	*	输入	Z 轴编码器 A 相+输入
ZECAN	*	输入	Z 轴编码器 A 相-输入
ZECBP	*	输入	Z 轴编码器 B 相+输入
ZECBN	*	输入	Z 轴编码器 B 相-输入
ZINOP	*	输入	Z 轴编码器 Z 相+输入
ZINON	*	输入	Z 轴编码器 Z 相-输入
UINPOS	*	输入	U 轴到位输入信号
UALARM	*	输入	U 轴伺服驱动器错误报警信号
UECAP	*	输入	U 轴编码器 A 相+输入
UECAN	*	输入	U 轴编码器 A 相-输入
UECBP	*	输入	U 轴编码器 B 相+输入
UECBN	*	输入	U 轴编码器 B 相-输入
UINOP	*	输入	U 轴编码器 Z 相+输入
UINON	*	输入	U 轴编码器 Z 相-输入
ZEXOP+	*	输入	Z 轴慢进+向输入
ZEXOP-	*	输入	Z 轴慢进-向输入
UEXOP+	*	输入	U 轴慢进+向输入
UEXOP-	*	输入	U 轴慢进-向输入
GND	*	*	接地脚
ZOUT4	GND	输出	Z 轴通用输出脚
ZOUT5	GND	输出	Z 轴通用输出脚
ZOUT6	GND	输出	Z 轴通用输出脚
ZOUT7	GND	输出	Z 轴通用输出脚 s
ZP+P	GND	输出	Z 轴 CW/Pulse + 输出
ZP+N	GND	输出	Z 轴 CW/Pulse - 输出

ZP-P	GND	输出	Z 轴 CCW/DIR + 输出
ZP-N	GND	输出	Z 轴 CCW/DIR - 输出
GND	*	*	接地脚
UOUT4	GND	输出	U 轴通用输出脚
UOUT5	GND	输出	U 轴通用输出脚
UOUT6	GND	输出	U 轴通用输出脚
UOUT7	GND	输出	U 轴通用输出脚 s
UP+P	GND	输出	U 轴 CW/Pulse + 输出
UP+N	GND	输出	U 轴 CW/Pulse - 输出
UP-P	GND	输出	U 轴 CCW/DIR + 输出
UP-N	GND	输出	U 轴 CCW/DIR - 输出

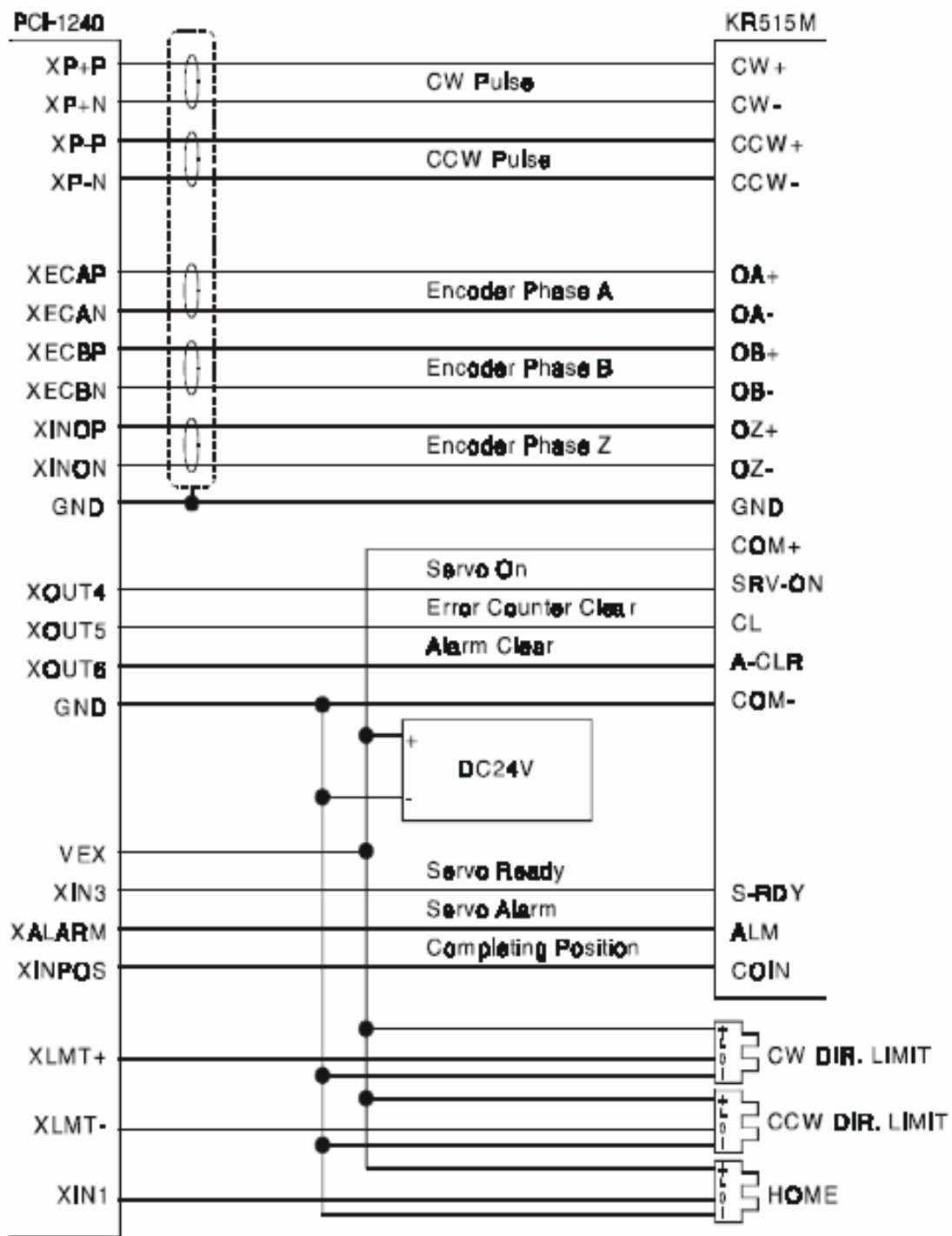
2. 2 PCI-1240 与驱动器连接:

1、与 UPK 步进电机驱动器连接:



说明：外加 DC24v 电源，采用 CW/CCW 驱动方式驱动，温度报警通过 XALARM 引脚输入。

2、与 MINAX X 系列交流伺服电机驱动器连接:



说明：板卡采用 DC24v 供电，采用 CW/CCW 方式驱动。编码器返回信号依次按图示连接到 X 轴的编码器输入脚(XECAP,XECAN,XECBP,XECBN,XINOP,XINON)上。HOME 引脚接 XIN1，限位信号接到 XLMT+,XLMT-上。

接线注意事项：

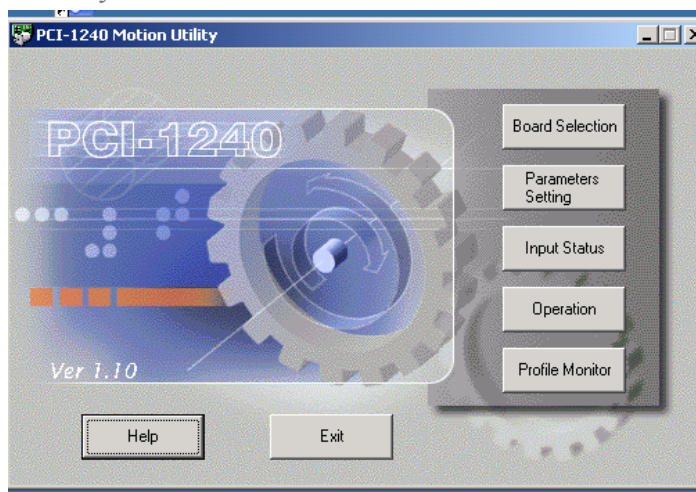
1. 信号线必须远离动力电源线、大电动机、断路器、电焊机等大的电磁源以及视频监控器等；
2. 如果信号线穿过，必须单独隔离，采用屏蔽双绞线作为输入信号线，并且

- 屏蔽双绞线的隔离层只能接一端的地；
3. 信号线和电源线不能布置同一管道中；
 4. 如果信号线和电源线必须平行放置时，两线之间必须保持足够的距离；
 5. 信号的质量与传输信号线有很大的关系，我们推荐使用我们的 PCL-10251 信号线缆；

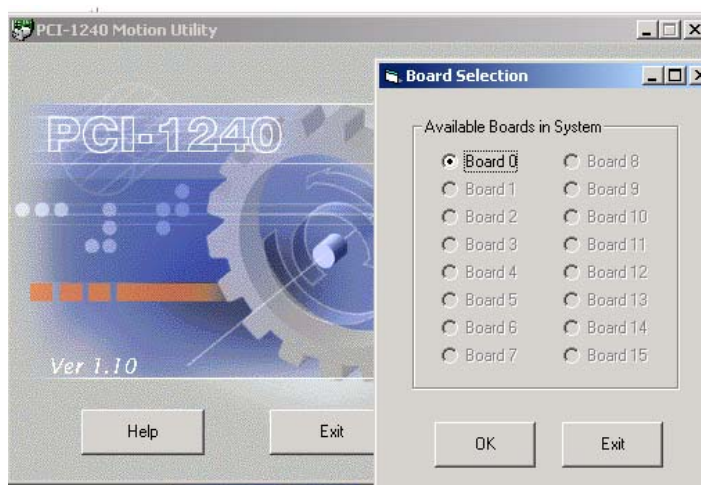
三、 PCI-1240 测试

3. 1 PCI-1240 Utility 使用：

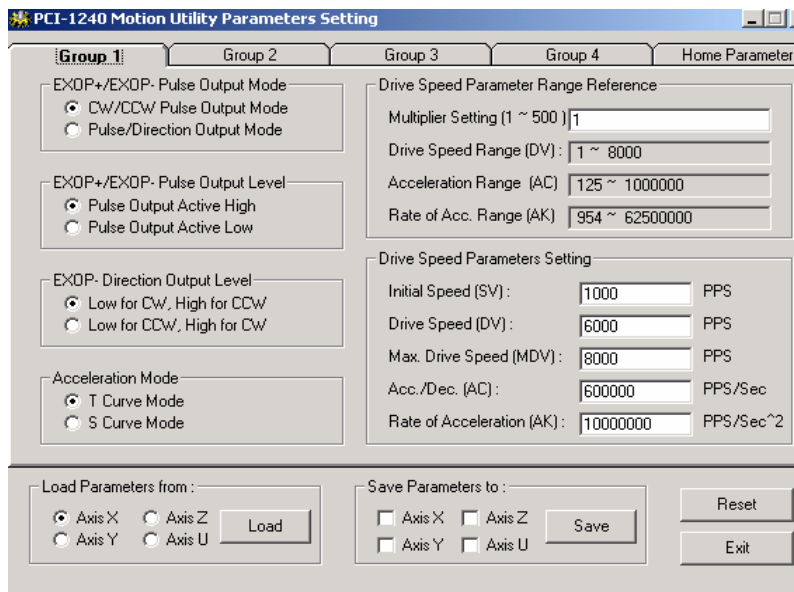
- 1、启动 PCI-1240：开始—>Advantech Automation—>Motion—>PCI-1240 —>PCI-1240 Utility，如图界面：



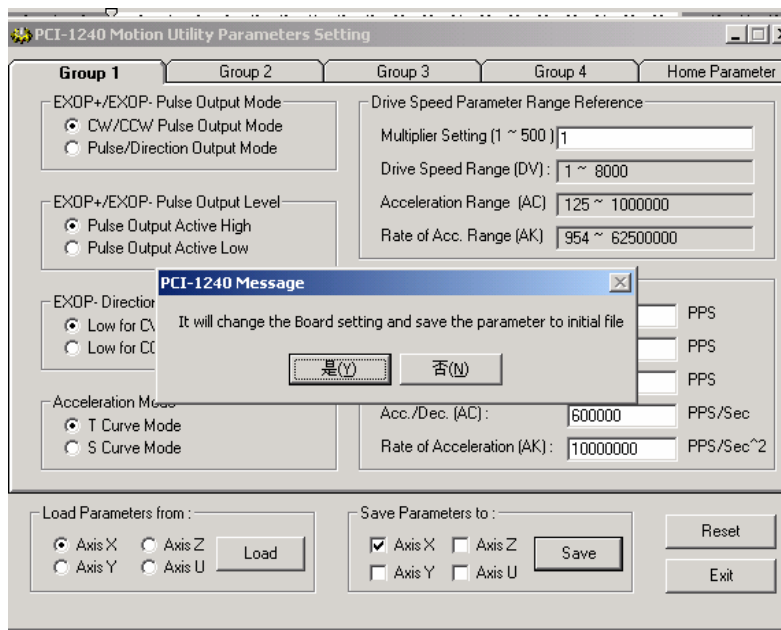
- 2、点击 Board Selection 选择板卡并点击 OK 按钮：



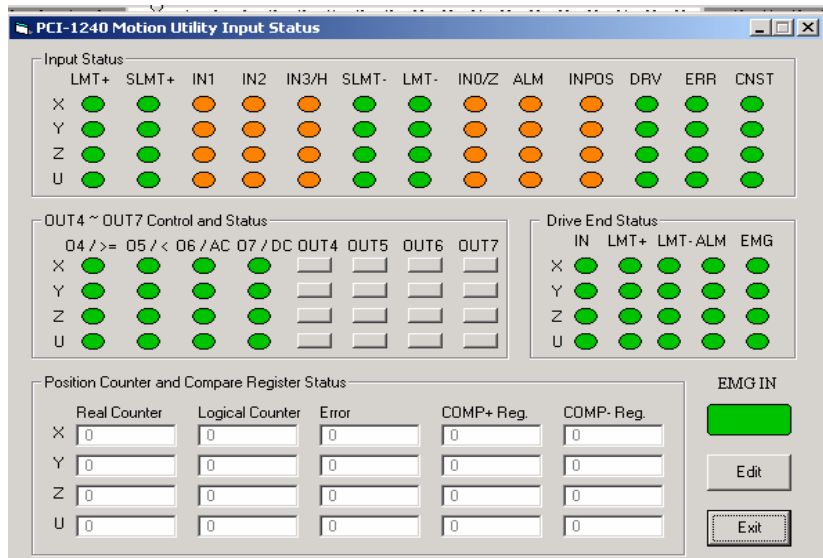
- 4、击 Parameters Setting 按钮，设置运动控制参数（具体设置参照附录参数设置说明：），如图：



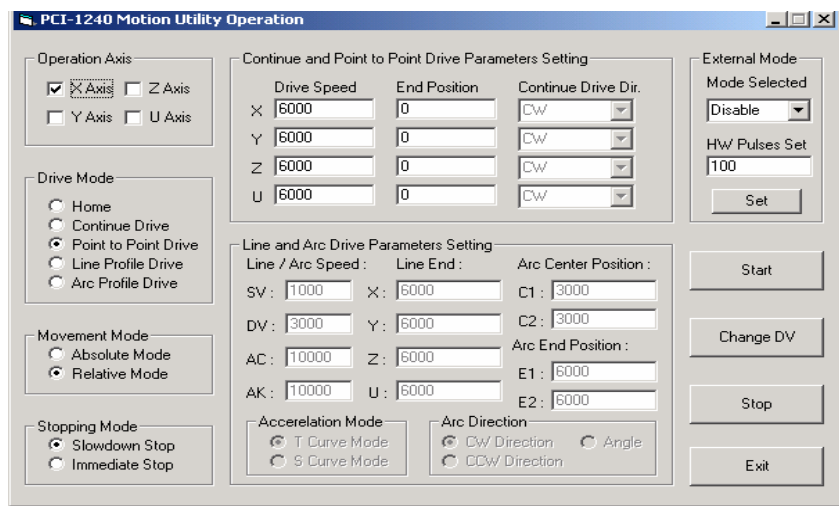
5、设置好参数以后，点击 Save 按钮，出现如下画面，然后点击 Yes:



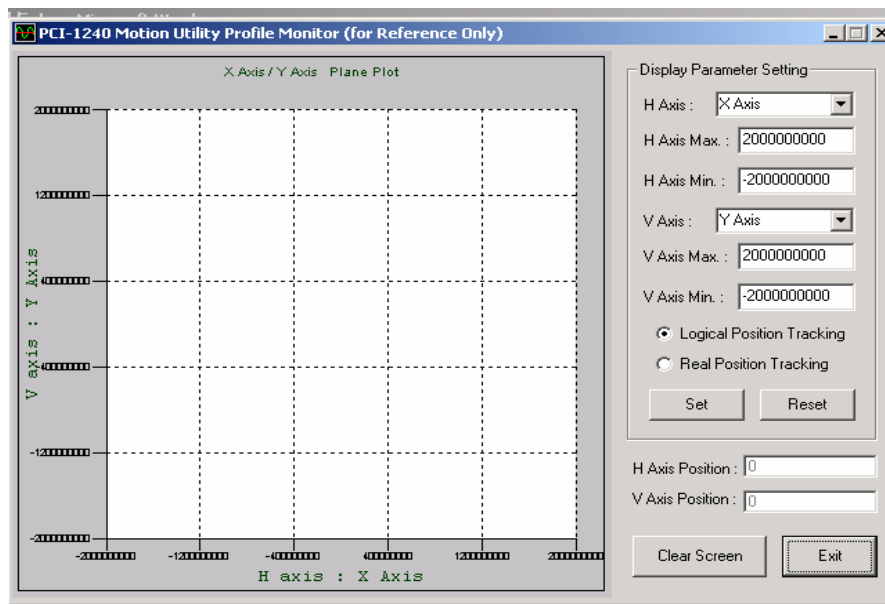
6、点击 Input Status 按钮，显示输入监视画面:

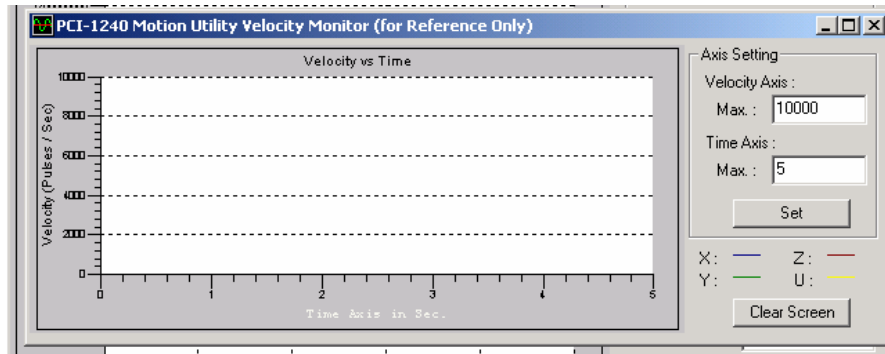


7、点击 Operation 按钮，出现运行截面，如图所示：



7、点击 Profile Monitor 按钮，显示监视运动画面，如图：

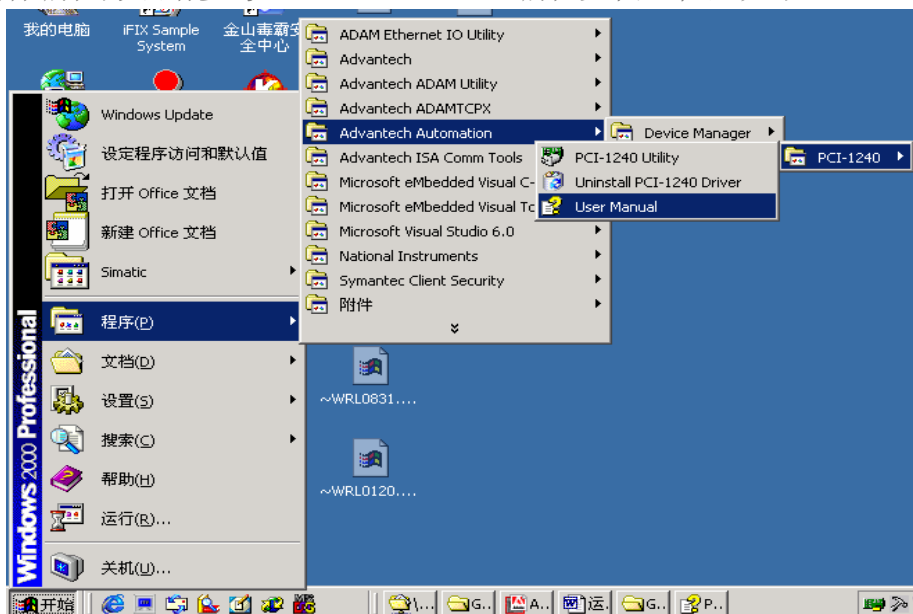




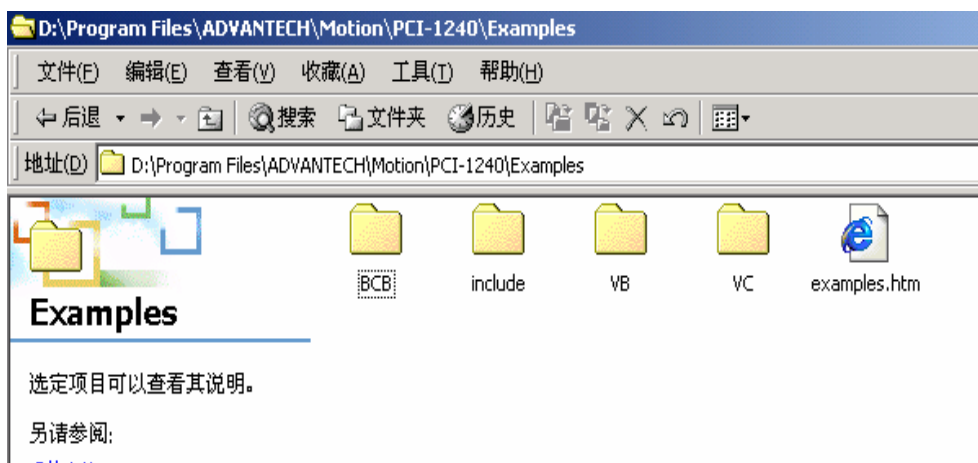
四、 软件编程

4. 1 PCI-1240 软件编程:

研华公司 PCI-1240 运动控制卡提供基于 WIN98/2000 下 32 位 DLL 驱动编程，具体编程方法请参考 PCI-1240 Manual 编程文档，位置如下：



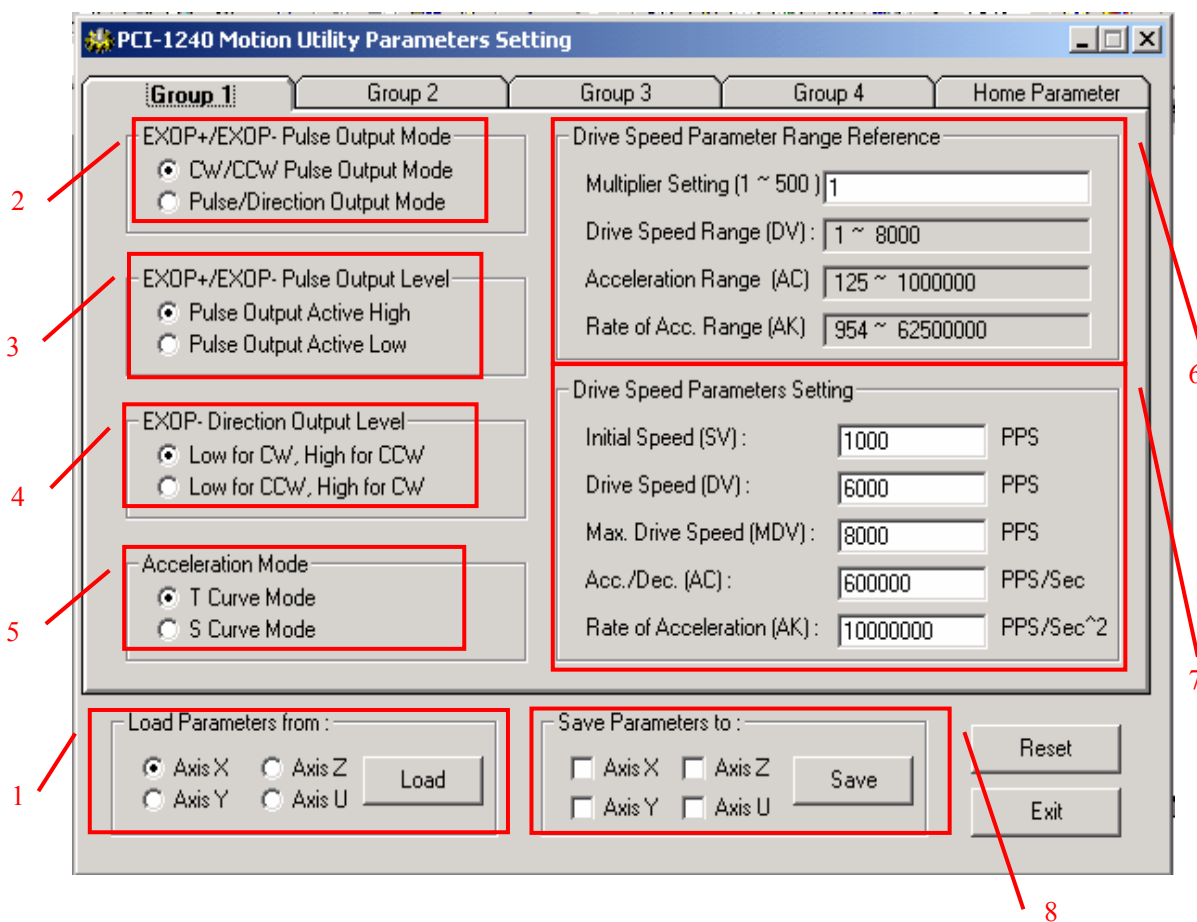
此外，我们提供 VB、VC 例程供参考，路径位于安装目录下（如 D:\Program Files\ADVANTECH\Motion\PCI-1240\Examples）。



附 录

1、PCI-1240 Utility 各界面参数说明：

A. Group1 参数设置说明：



- 1——选择任意轴，点击 Load 按钮，加载该轴上次 Group1 参数设置；
- 2——选择电机驱动器脉冲输出方式：CW/CCW 方式或 Pulse/DIR 方式；
- 3——选择电机驱动器慢进方式（JOG）脉冲输出高低电平，High 或 Low 有效；

4——选择电机驱动器慢进方向（JOG）低电平正转/高电平反转，还是相反；

5——选择加速方式，梯形方式或 S 形方式加速；

6——设置 Multiplier（1—500），其他的参数会相应地改变；

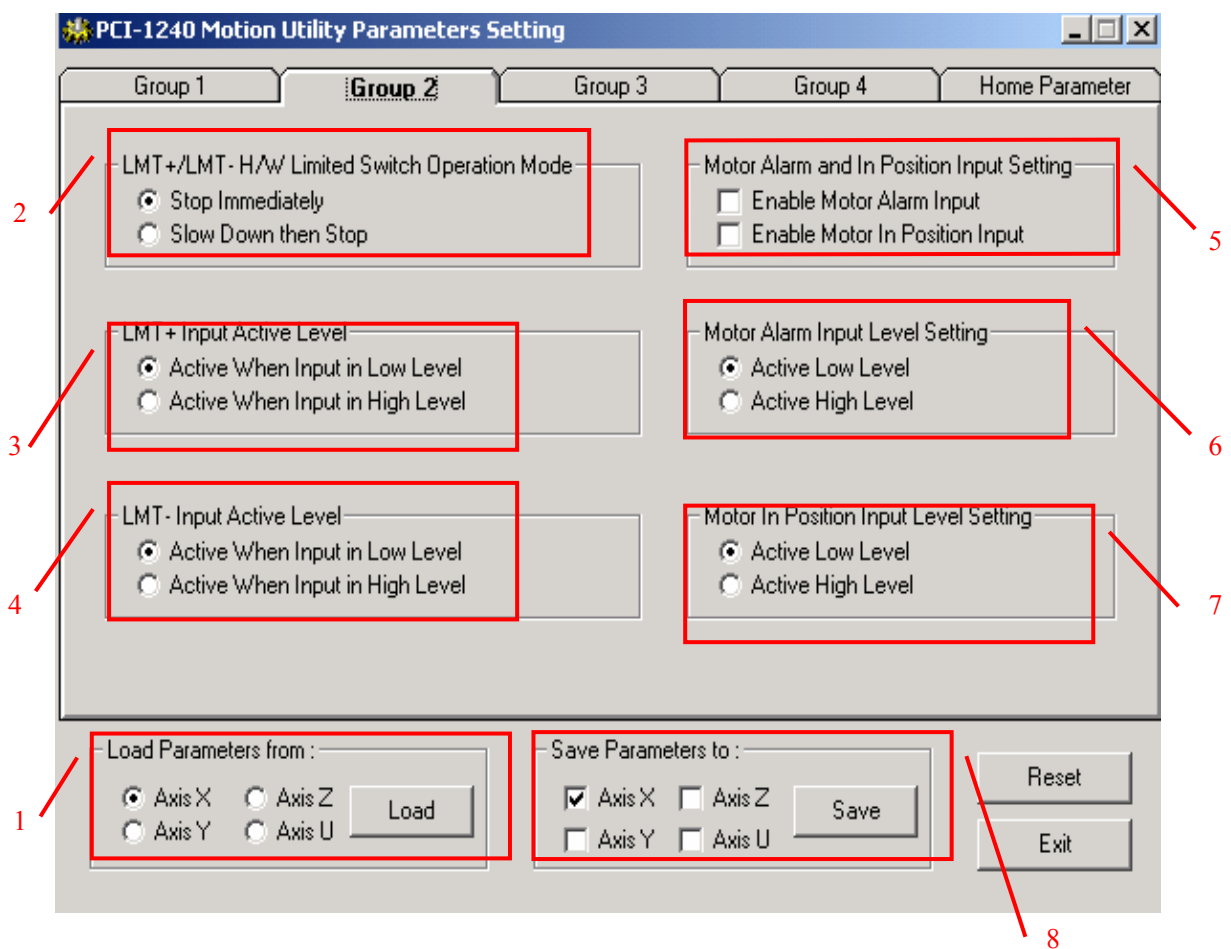
7——设置运行参数：

- Initial Speed——设置初始速度（SV）；
- Driver Speed——设置驱动速度（DV）；
- Max Drive Speed——设置最大驱动速度（MDV）；
- Acc./Dec.——设置速度变化率/加速度（ACC）；
- Rate of Acceleration——设置加速度变化率（AK）；

8——选择任意几轴，点击 Save 保存以上设置参数；

此外 Reset 按钮是复位 PCI-1240 至初始化状态，Exit 按钮退出设置。

B. Group2 参数设置说明：



1——和 Group1 中一样；

2——选择限位开关工作方式（立即停止或减速再停止）；

3——选择正向限位开关输入有效电平，High 或 Low 有效；

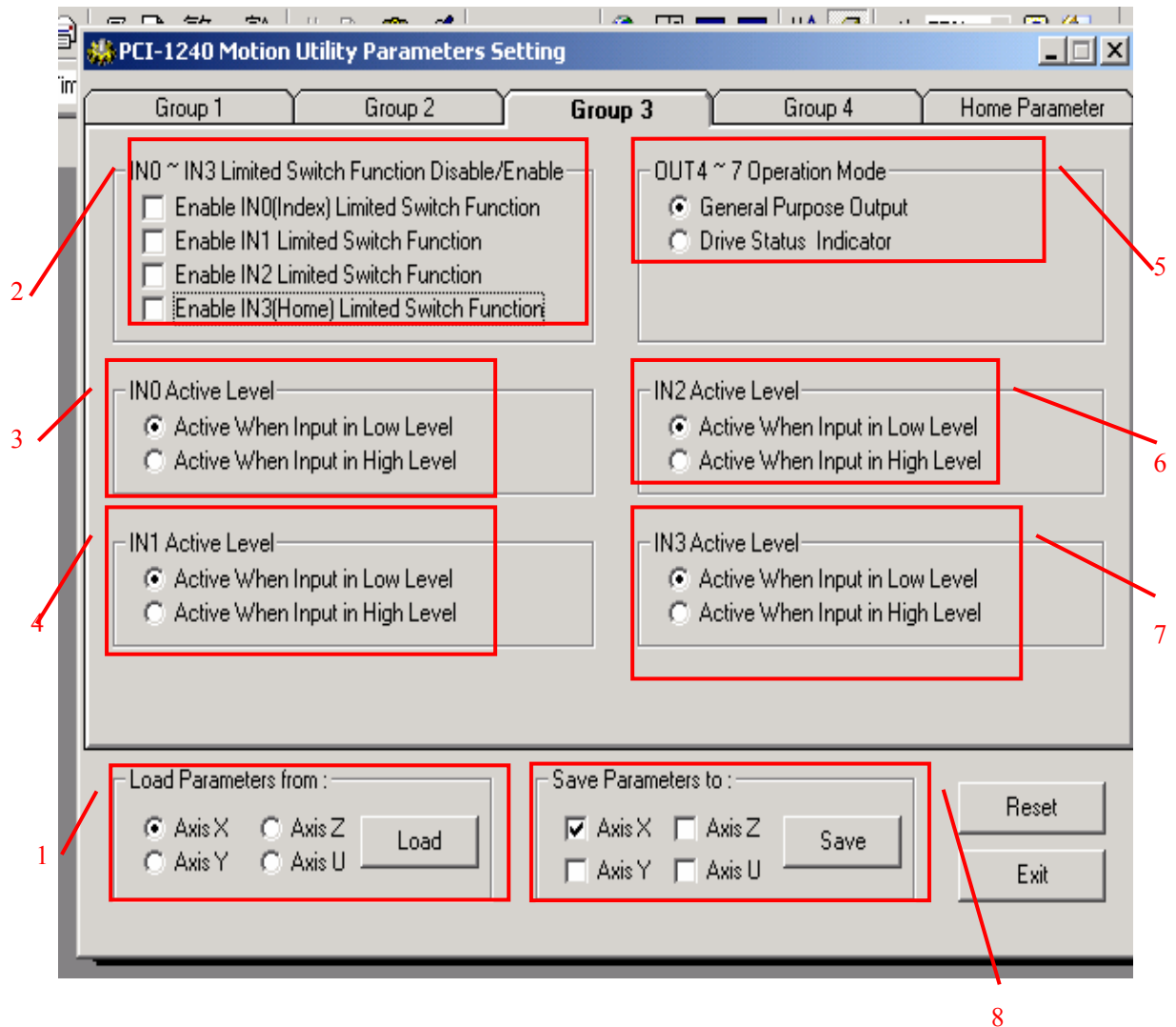
4——选择负向限位开关输入有效电平，High 或 Low 有效；

5——报警（Alarm Input）使能和在位（In Position）使能；

6——选择报警（Alarm Input）输入有效电平，High 或 Low 有效；

- 7——设置在位（In Position）输入有效电平，High 或 Low 有效；
 8——和 Group1 中一样；

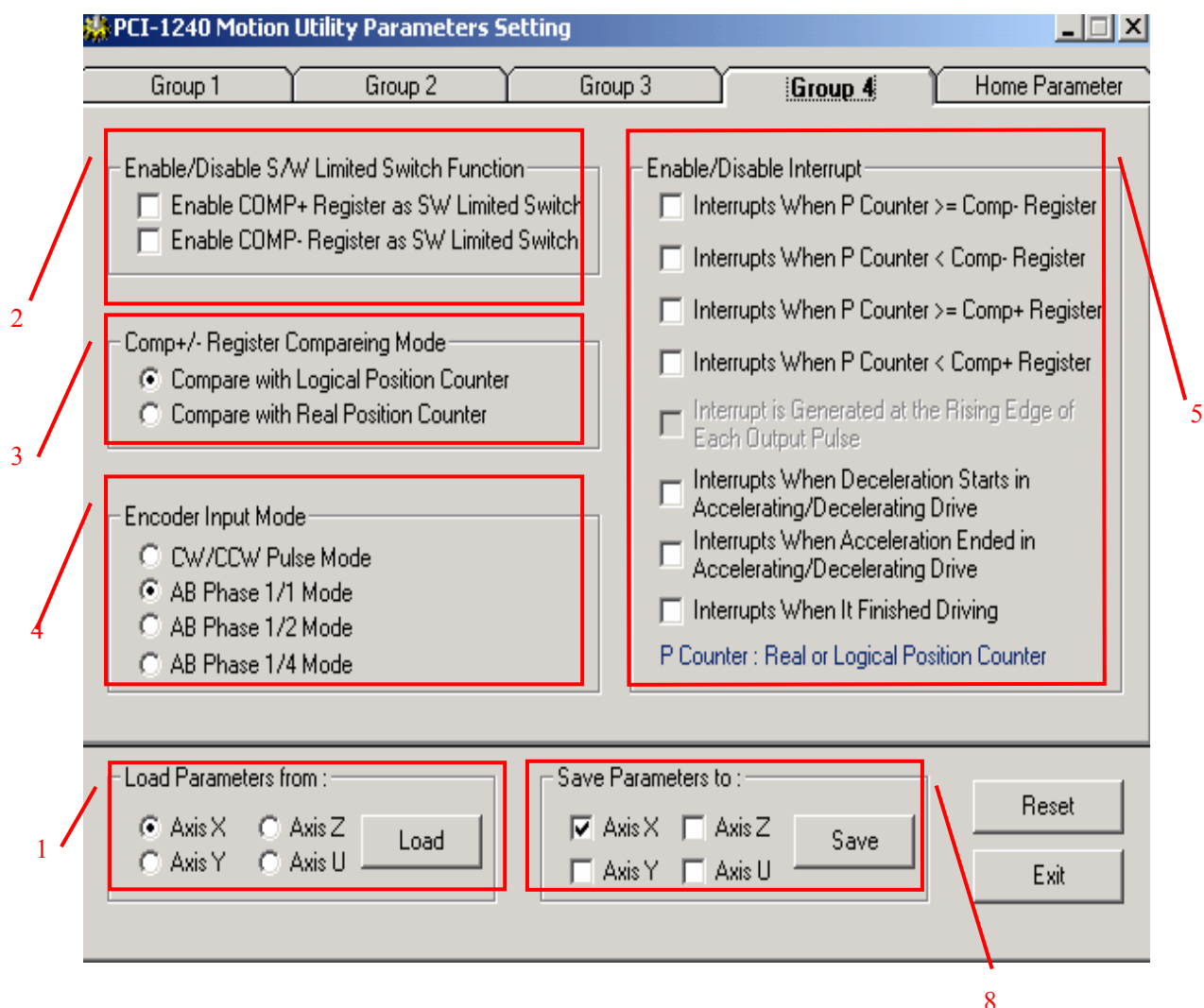
C. Group3 参数设置说明：



- 1——和 Group1 中一样；
 2——IN0-IN3 使能设置；
 ● Enable IN0——（Index In）限位开关输入中断使能；

- Enable IN1——限位开关输入中断使能；
 - Enable IN2——限位开关输入中断使能；
 - Enable IN3——（Home）限位中断功能使能；
- 3——选择 IN0 电平 High 或 Low 有效；
- 4——选择 IN1 电平 High 或 Low 有效；
- 5——设置 OUT4-7 输出模式：通用输出模式或运动状态指示输出；
- 6——选择 IN2 电平 High 或 Low 有效；
- 7——选择 IN3 电平 High 或 Low 有效；
- 8——和 Group1 中一样；

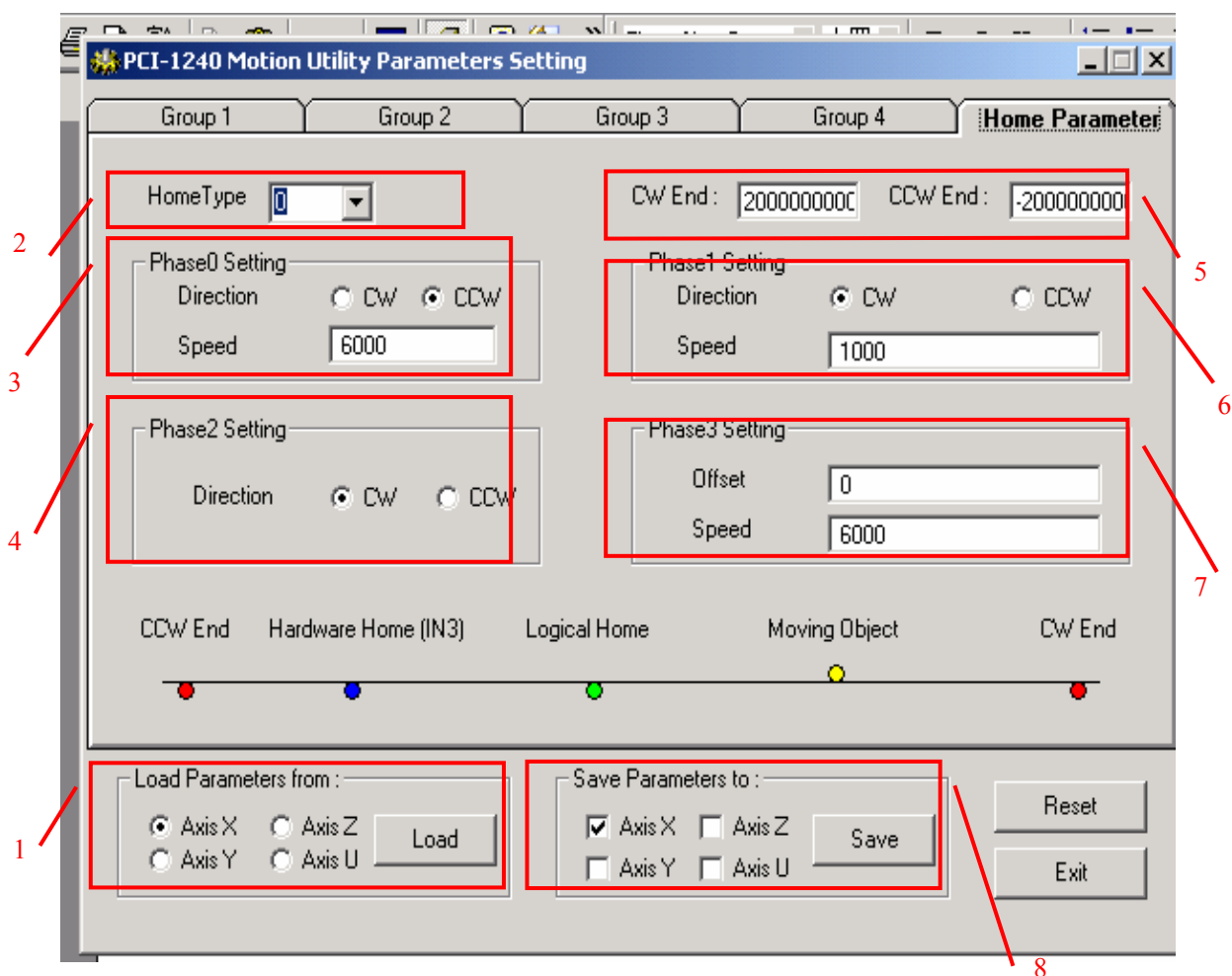
D. Group4 参数设置说明：



- 1——和 Group1 中一样；
- 2——比较寄存器 COMP+, COMP- 限位使能设置；
- 3——COPM+/COMP-比较寄存器（逻辑计数器 Logical 或实际计数器 Real 选择；

- 4——选择编码器输入方式；
- 5——设置中断使能；
- P 计数器与 COMP+/COMP-计数器比较中断使能设置；
 - 减速中断使能设置；
 - 加速结束中断使能设置；
 - 运动结束中断使能设置；
- 6——和 Group1 中一样；

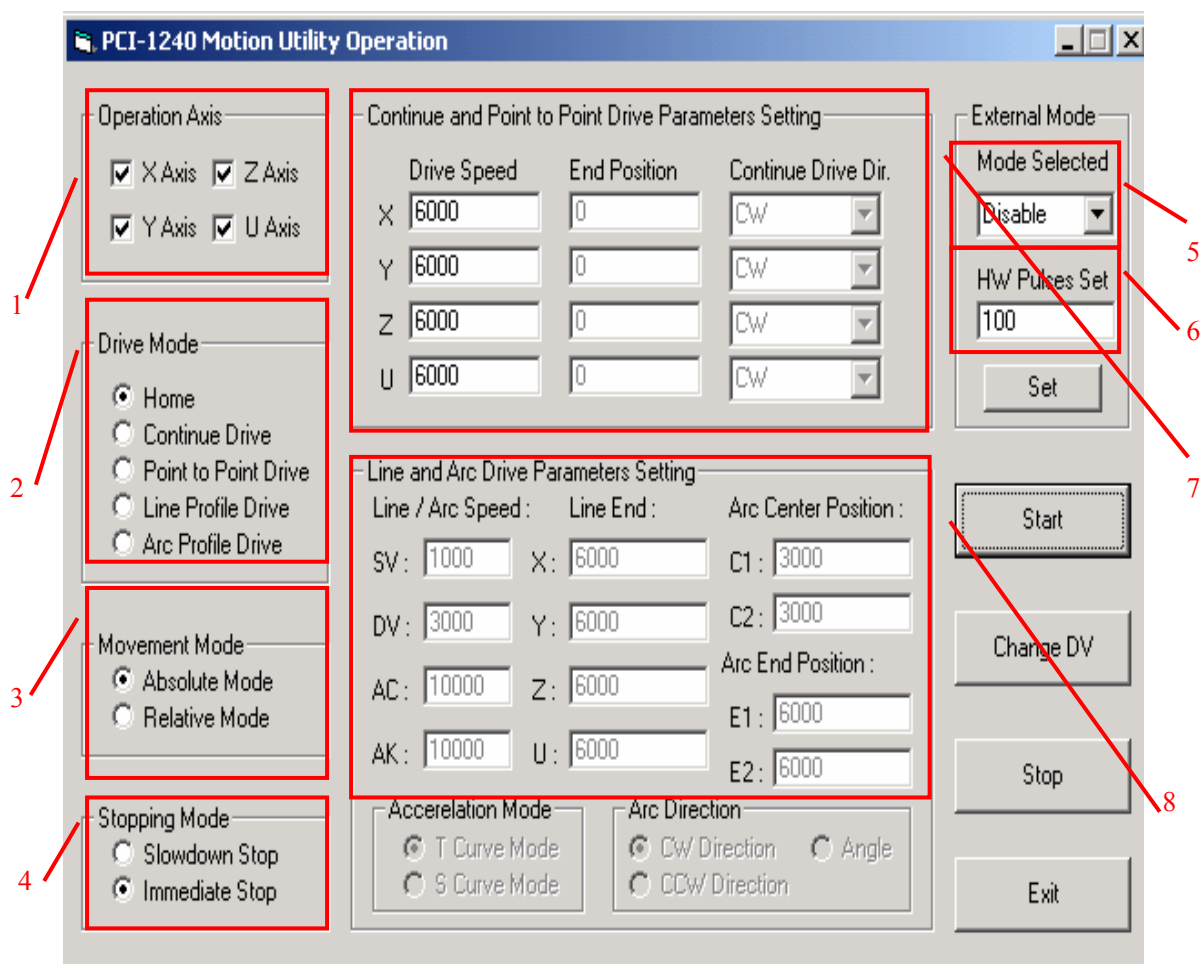
E. Home Parameters 参数设置说明：



- 1——和 Group1 中一样；
- 2——选择 Home 功能类型；
- 3——Home 功能第一阶段方向与驱动速率设置；
- 4——Home 功能第二阶段方向与驱动速率设置；

- 5——CW/CCW 最大脉冲数设置；
- 6——Home 功能第三阶段方向设置；
- 7——Home 功能第四阶段偏移（Offset）与驱动速率设置；
- 8——和 Group1 中一样；

F. 运行（Operation）界面参数设置说明：



- 1——根据选择的 Driver 方式，设置运动轴；
- 2——选择 Driver 方式；
- 3——选择运动方式(绝对运动和相对运动)；
- 4——停止方式：立即停止或缓慢停止；
- 5——拓展模式设置；
- 6——H/W Pulse 设置；
- 7——连续运动和点对点运动参数设置；

8——直线和圆弧运动参数设置;

参考资料:

- 1、PCI-1240 手册;
- 2、PCI-1240 编程文档;
- 3、MCX314 芯片手册;

附录四 数据采集选型指南

Plug-In DA&C Cards

Data Acquisition and Control Tutorial	5-2
Data Acquisition and Control Card Software	5-4
Selection Guide	
Data Acquisition and Control Card	5-6
PCI-bus Data Acquisition and Control Cards	
PCI-bus Multifunction Cards	
PCI-1710/1710L/ 1710HG/1710HGL	100 KS/s, 12-bit, (High-gain), PCI-bus Multifunction Card 5-10
PCI-1711/1711L	100 KS/s, 12-bit, 16-ch S.E. Inputs Low-cost Multifunction Card 5-12
PCI-1712/1712L	1 MS/s, 12-bit, 16-ch High-speed Multifunction Card 5-14
PCI-1716/1716L	250 KS/s, 16-bit, 16-ch High-resolution Multifunction Card 5-16
PCI-bus Analog Input Cards	
PCI-1713	100 KS/s, 12-bit, 32-channel Isolated Analog Input Card 5-18
PCI-1714	30 MS/s Simultaneous 4-ch Analog Input Card 5-19
PCI-bus Analog Output Cards	
PCI-1720	4-channel Isolated Analog Output Card 5-20
PCI-1721	12-bit, 4-ch Advanced Analog Output Card 5-21
PCI-1723	12-bit, 8-ch Isolated Analog Output Card 5-22
PCI-1724U	14-bit, 32-ch Analog Output Card 5-23
PCI-bus Non-Isolated Digital I/O Cards	
PCI-1751	48-bit Digital I/O and Counter Card 5-24
PCI-1753/1753E	96/192-bit Digital I/O Card 5-25
PCI-1755	80 MB/s Ultra high-speed 32-ch Digital I/O Card 5-26
PCI-bus Isolated Digital I/O Cards	
PCI-1730	32-channel Isolated Digital I/O Card 5-28
PCI-1733	32-channel Isolated Digital Input Card 5-28
PCI-1734	32-channel Isolated Digital Output Card 5-28
PCI-bus Isolated Digital I/O Cards	
PCI-1752	64-ch Isolated Digital Output Card 5-30
PCI-1754	64-ch Isolated Digital Input Card 5-30
PCI-1756	64-ch Isolated Digital I/O Card 5-30
PCI-1750	32-channel Isolated Digital I/O and Counter Card 5-32
PCI-1761	8-ch Relay Actuator and 8-ch Isolated D/I Card 5-33
PCI-1760U	8-ch Relay Actuator and Isolated D/I Card 5-34
PCI-1762	16-ch Relay Actuator and 16-ch Isolated D/I Card 5-36
Counter Cards	
PCI-1780	8-ch Counter/ Timer Card 5-37
PCI-1784	4-axis Quadrature Encoder and Counter Card 5-39
ISA-Bus Data Acquisition and Control Cards	
Multifunction Cards	
PCL-1800	330 KS/s High-speed Multifunction Card 5-41
PCL-711B/S	Analog and Digital I/O Card 5-43
PCL-812PG	MultiLab Analog and Digital I/O Card 5-44
PCL-816 / 816-DA	Modular High-resolution Multifunction Card 5-45
2-channel D/A Module	
PCL-818 Series	High-performance Multifunction Card 5-47
PCL-818L/LS/H	40 KS/s Multifunction Card 5-48
General-purpose Multifunction Cards	
PCL-818HD/HG	High-performance Multifunction Card 5-49
Analog Input Cards	
PCL-813B	32-channel S.E. Isolated A/D Card 5-51
Analog Output Cards	
PCL-726	6-channel D/A Output Card 5-52
PCL-727	12-channel D/A Output Card 5-53
PCL-728	Isolated 2-channel D/A Output Card 5-54
Non-Isolated Digital I/O Cards	
PCL-720+	Digital I/O and Counter Card 5-55
PCL-722	144-bit Digital I/O Card 5-56
PCL-724	24-bit Digital I/O Card 5-56
PCL-731	48-bit Digital I/O Card 5-56
Isolated Digital I/O Cards	
PCL-730	32-channel Isolated Digital I/O Card 5-57
PCL-733	32-channel Isolated Digital Input Card 5-57
PCL-734	32-channel Isolated Digital Output Card 5-57
PCL-725	Relay Actuator and Isolated D/I Card 5-59
PCL-735	12-channel Relay Actuator Card 5-60
Counter Cards	
PCL-833	3-axis Quadrature Encoder and Counter Card 5-61
PCL-836	6-channel Counter/Timer Card 5-62
Motion Control Cards	
PCI-1240U	4-axis Stepping/Pulse-type Servo Motor Control Card 5-63
PCI-1241	4-axis Pulse-type Servo Motor Control Card 5-65
PCI-1242	4-axis Voltage-type Servo Motor Control Card 5-67
PCI-1261 (PCI-bus)	6-axis Stepping/Servo Motor Control Card 5-69
PCI-1670	GPIO interface PCI card 5-71
PCI-839+ (ISA-bus)	3-axis Stepping Motor Control Card 5-72
PC/104 I/O Modules	
PCM-3712	12-bit, 2-ch Analog Output Module 5-73
PCM-3718H/HG	100 KS/s, 12-bit, 16-ch Multi-function Module 5-73
PCM-3724	48-ch Digital I/O Module 5-74
PCM-3725	8-ch Isolated Digital Input and 8-ch Relay Output Module 5-74
PCM-3730	32-channel Isolated Digital I/O Module 5-74

Data Acquisition and Control Tutorial

PC-based Data Acquisition System Overview

In the last few years, industrial PC I/O interface products have become increasingly reliable, accurate and affordable. Because of this, PC-based data acquisition and control systems are now widely used in industrial and laboratory applications such as: monitoring, control, data acquisition and automated testing.

Selecting and building a DA&C (Data Acquisition and Control) system that actually does what you want it to do requires some knowledge of electrical and computer engineering. This tutorial gives a brief introduction to what DA&C systems do and how to configure them. It covers:

- Transducers and actuators
- Signal conditioning
- Data acquisition and control hardware
- Getting Started
- Computer systems software

Transducers and Actuators

A transducer converts temperature, pressure, level, length, position, etc. into voltage, current, frequency, pulses or other signals.

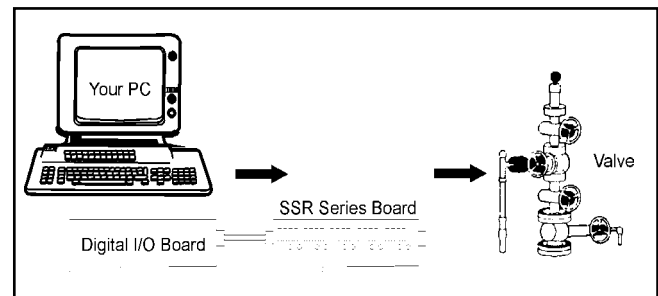
Thermocouples, thermistors and resistance temperature detectors (RTDs) are common transducers for temperature measurements. Other types of transducers include flow sensors, pressure sensors, strain gauges, load cells and LVDTs, which measure flow rate, pressure variances, force or displacement.

An actuator is a device that activates process control equipment by using pneumatic, hydraulic or electrical power. For example, a valve actuator can open and close a valve to control fluid rates.

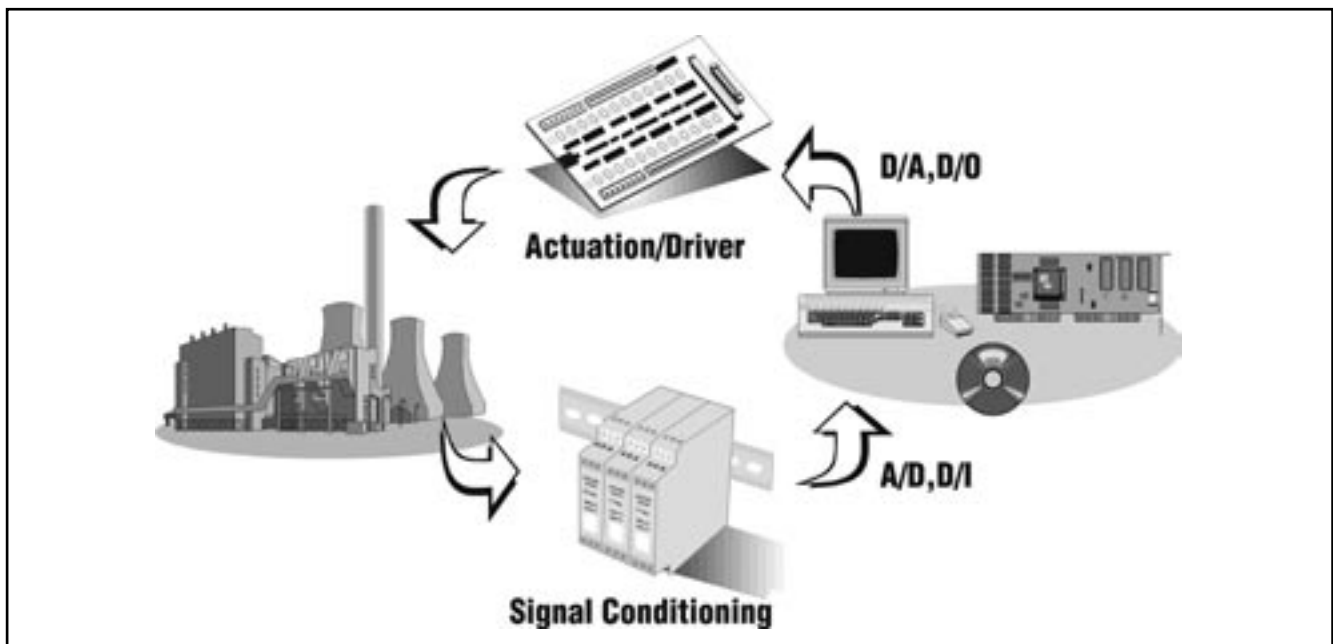
Signal Conditioning

Signal conditioning circuits improve the quality of signals generated by transducers before they are converted into digital signals by the PC's data-acquisition hardware. Examples of signal conditioning are signal scaling, amplification, linearization, cold-junction compensation, filtering, attenuation, excitation, common-mode rejection, and so on.

One of the most common signal conditioning functions is amplification. For maximum resolution, the voltage range of the input signals should be approximately equal to the maximum input range of the A/D converter. Amplification expands the range of the transducer signals so that they match the input range of the A/D converter. For example, a x10 amplifier maps transducer signals that range from 0 to 1 V into the range 0 to 10 V before they go into the A/D converter.



Using digital I/O and SSRs to open and close a valve



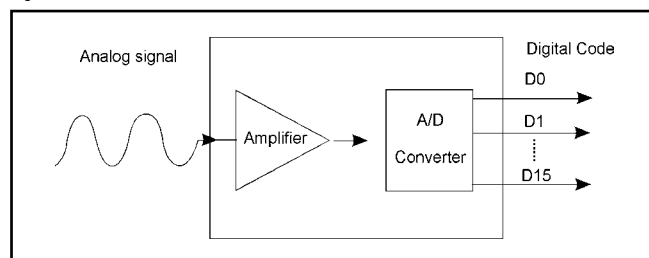
The layout of a typical PC-based data acquisition system

Data Acquisition & Control Hardware

Data acquisition and control hardware generally performs one or more of the following functions: analog input, analog output, digital input, digital output and counter/timer functions. This section will discuss each function and list some considerations that are important when you select a data acquisition and control system.

Analog Inputs (A/D)

Analog to digital (A/D) conversion changes analog voltage or current levels into digital information. The conversion is necessary to enable a computer to process or store the signals.

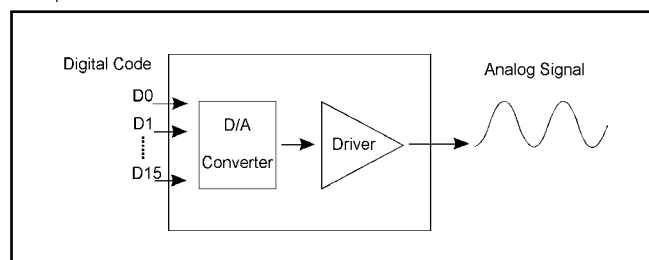


The most significant criteria when selecting A/D hardware are:

1. Number of input channels
2. Single-ended or differential input signals
3. Sampling rate (in samples per second)
4. Resolution (usually measured in bits of resolution)
5. Input range (specified in full-scale volts)
6. Noise and nonlinearity

Analog Outputs (D/A)

The opposite of analog to digital conversion is digital to analog (D/A) conversion. This operation converts digital information into analog voltage or current. D/A devices allow a computer to control real-world events.



Analog output signals may directly control process equipment. The process can give feedback in the form of analog input signals. This is referred to as a closed loop control system with PID control. Analog outputs can also be used to generate waveforms. In this case, the device behaves as a function generator.

Digital Inputs and Outputs

Digital input/output functions are useful in applications such as contact closure and switch status monitoring, industrial On/Off control and digital communications.

Counter/timer

A counter/timer can be used for event counting, flowmeter monitoring, frequency counting, pulse width measurement, time period measurement, and so on.

Getting Started

Advantech: The source for what you need

Advantech manufactures data acquisition hardware and software for measurement, monitoring and applications control. The following guide is provided to help you choose components for your data acquisition system.

Step 1: Know your fundamental goal

Decide whether your DA&C system will be used primarily for measurement, monitoring, control, or analysis. Know the data requirements of your process, and know the number of data collection points in your system. Know the required data collection speed, the sampling rate, the type of measurement, the voltage or current being produced, the desired accuracy and the output resolution at each data collection point. Finally, know the timing of events in your system, and any special environmental conditions that exist.

Step 2: Hardware selection

Select the hardware required to achieve your fundamental goal. Advantech provides plug-in boards for Analog-to-Digital, Digital-to-Analog, Digital I/O, RS-232 or RS-485 needs. Both ISA and PCI bus products are available. Your hardware selection should be based on five major criteria:

1. Number and types of channels
2. Differential or single-ended inputs
3. Resolution
4. Speed
5. Software compatibility with hardware

Step 3: Accessory selection

Most applications require additional accessories which are available as separate items. These include:

1. Expansion peripherals to add channels to your system
2. Cables, signal conditioners and external boxes such as screw terminals or BNC accessories

Step 4: Software selection

More than any other single factor, software will determine your system start-up time, as well as its effectiveness, suitability for your application, and ease of modification.

Three major criteria should determine the choice of software:

1. Operating system used
2. User programming expertise
3. Software compatibility with hardware

1

IPPC & AWS

2

TPC

3

FPM

4

ATM

5

DA&C

6

cPCI

7

ADAM-3000

8

eConnectivity

9

ICOM

10

Software

11

UNO-2000/3000

12

VBOX

13

ADAM-4000

14

ADAM-5000

15

ADAM-6000

16

ADAM-8000

Data Acquisition and Control Card Software

Software

The driving force behind any data acquisition and control system is software control. Programming your data acquisition and control system can be accomplished in the following three ways: hardware-level programming, driver-level programming and package-level programming.

In hardware-level programming, you program your data acquisition hardware's registers directly. To do this, you must determine the control code values that will be written to the hardware's registers, and you must program in a language that offers low-level bit-control functions, such as C or assembly. This is complex and takes a lot of time; time that you may not want to spend. That is why most manufacturers of data acquisition hardware supply their customers with either driver-level or package-level programs.

In driver-level programming, you simply call high-level functions from libraries provided by the card manufacturer. These functions encapsulate complete tasks such as A/D conversion into a single call. You can use high-level programming languages like Microsoft Visual C++, Visual Basic, Inprise Delphi and C++ Builder.

In package-level programming, the application program does the work for you. You simply tell it what to do. Software packages integrate data analysis, presentation and instrument control capabilities into a single interface. These programs offer a multitude of features, such as pull down menus and icons, data logging and analysis, as well as real-time graphic displays.

What is the right software?

As you consider your software options, remember these basic guidelines. The right software should:

1. Include all the components you need to build and execute your applications
2. Fully support your hardware
3. Let your applications change as your needs evolve
4. Be easy to use
5. Provide an user interface that fits your development style

Advantech's complete range of products allows you to select the right software for your needs

Automation Software	Major Features/Benefits	Applications
Driver Software - DLL drivers	- Provides complete DA&C I/O function libraries to utilize H/W functions. - Supports all Windows 95/98/ME/NT/XP/2000 platforms - Programmed with Microsoft® Visual C++, Visual Basic, Inprise® Delphi and C++ Builder - Features lots of programming examples	- Powerful and extensive functions to build DA&C applications - Needs intermediate programming skills - Free and bundled with DA&C devices
Tool Software - ActiveX Controls (OCX)	- Based on Microsoft COM technology. - Features DA&C device I/O functions - Simple tools with dedicated functions to meet end user needs. - Supports general I/O and DMA/Interrupt functions	- Needs low programming skills - Easily integrated with commonly used development packages (such as Visual Basic, Visual C++ and Delphi) to build applications.
DA&C Application Software - Advantech DAQView	- Microsoft Windows-based - Easy to use: just drag and drop to build DA&C applications. - Features flexible toolkit (such as BasicScript) to add custom functions - Supports general I/O functions with polling	- No programming skills needed to build DA&C applications - Cost-effective tools. - Suitable for simple and small-scale DA&C systems
Third Party Software - LabVIEW® drivers	- Supports leading laboratory and industry automation products - Integrates Advantech DA&C devices with leading software to build complex systems. - Supports general I/O and DMA/Interrupt functions	- Total solution for complex or mission critical laboratory or industry automation systems.

Note: The software performance may not be able to match the stated level of hardware performance, and is affected by the performance of the computer and the operating system.

ActiveDAQ: ActiveX Control for Data Acquisition and Control

ActiveDAQ is a collection of add-on controls and libraries that extends Visual Basic from an application front-end to a world-class DA&C development environment. ActiveDAQ includes easy-to-use data acquisition controls for controlling plug-in DAQ boards in Visual Basic.

With DAQ controls, you get a series of OLE controls for controlling Advantech plug-in DA&C cards and remote and distributed DA&C modules. Using DAQ controls, application programming is reduced to setting a collection of DAQ properties. For example, you can drop the analog input control menu and set properties for input channels, acquisition rates, triggering and timing signals through the property sheets.

Windows DLL Drivers

Advantech offers a complete range of Windows DLL drivers to support Windows programming developers, and sells several APIs to give developers maximum flexibility to adapt and customize applications for Windows 95/98/ME/NT/XP/2000. You can apply the Windows DLL drivers to the most popular Windows programming tools, such as Visual C++, Visual Basic, Inprise C++ and Inprise Delphi.

The Advantech DLL for Windows 95/98/ME/NT/XP/2000 drivers are based on the Windows 95/98/ME/NT/XP/2000 kernel technology.

Functionality

The DLL driver facilitates communication between the host computer and Advantech data acquisition and control products. It provides the following services and functions:

1. **Device function:** initializes and configures your hardware and software
2. **Analog input:** converts single and multiple channel A/D
3. **Analog output:** converts a single A/D
4. **Digital I/O:** controls digital I/O for specified channels
5. **Port I/O:** controls port I/O
6. **Counter:** performs event-counting, frequency measurement and pulse output
7. **Temperature function:** measures temperature
8. **Alarm:** operates alarm
9. **Communication port:** operates the communication port
10. **High speed function:** utilizes DMA and Interrupt for data acquisition

Features

- 32-bit ActiveX (OLE) controls for data acquisition
- Compatible with Visual Basic, Visual C++ and Inprise Delphi
- Easy-to-use property sheet interface for configuring controls
- Flexible programming control through properties, events, and methods
- Complete Advantech DA&C controls for analog, digital, and timing I/O operations
- Runs under Windows 95/98/ME/NT/XP/2000

High Speed Functions

The high-speed functions utilize DMA or interrupts for data acquisition. This kind of data transfer is performed in the background, so less CPU time is required and thus the transfer rates improve. These functions are used for large amounts of data that are transferred at high rates. The driver uses double-buffering techniques for continuous, uninterrupted transfer of large amounts of data.



1
IPPC & AWS

2
TPC

3
FPM

4
ATM

5
DA&C

6
cPCI

7
ADAM-3000

8
eConnectivity

9
ICOM

10
Software

11
UNO-2000/3000

12
VBOX

13
ADAM-4000

14
ADAM-5000

15
ADAM-6000

16
ADAM-8000

Data Acquisition and Control Cards

Bus		PCI										
Category		Multifunction					AI		AO			
Model		PCI-1710/1710L	PCI-1710HG/HGL	PCI-1711/1711L	PCI1712/1712L	PCI-1716/1716L	PCI-1713	PCI-1714	PCI-1720	PCI-1721	PCI-1723	PCI-1724
General spec.	Resolution	12 bits	12 bits	12 bits	12 bits	16 bits	12 bits	12 bits	-	-	-	-
	Number of channels	16 SE/8 Diff.	16 SE/8 Diff.	16 SE	16 SE/8 Diff.	16 SE/8 Diff.	32 SE/16 Diff. (Isolation)	4 SE	-	-	-	-
	Onboard FIFO	4K samples	4K samples	1K samples	1K samples	1K samples	4K samples	32 K samples	-	-	-	-
	Sampling rate	100 KS/s	100 KS/s	100 KS/s	1 MS/s	250 KS/s	100 KS/s	30 MS/s	-	-	-	-
	Auto Channel scanning	✓	✓	✓	✓	✓	✓	✓	-	-	-	-
	Input ranges	Unipolar inputs (V)	0 ~ 10, 0 ~ 5, 0 ~ 2.5, 0 ~ 1.25	0 ~ 10, 0 ~ 1, 0 ~ 0.1, 0 ~ 0.01	-	0 ~ 10, 0 ~ 5, 0 ~ 2.5, 0 ~ 1.25	0 ~ 10, 0 ~ 5, 0 ~ 2.5, 0 ~ 1.25	0 ~ 10, 0 ~ 5, 0 ~ 2.5, 0 ~ 1.25				
		Bipolar inputs (V)	±10, 5, 2.5, 1.25, 0.625	±10, 5, 1, 0.5, 0.1, 0.05, 0.01, 0.005	±10, 5, 2.5, 1.25, 0.625	±10, 5, 2.5, 1.25, 0.625	±10, 5, 2.5, 1.25, 0.625	±10, 5, 2.5, 1.25, 0.625	±5, 2.5, 1, 0.5			
		Configurable per-channel	✓	✓	✓	✓	✓	✓	-	-	-	-
	Trigger mode	Pacer/Software/External pulse	✓	✓	✓	✓	✓	✓	-	-	-	-
		Analog slope	-	-	-	-	-	✓	-	-	-	-
		Pretrigger	-	-	-	✓	-	✓	-	-	-	-
		Post-trigger	-	-	-	✓	-	✓	-	-	-	-
	Data transfer mode	About-trigger	-	-	-	✓	-	✓	-	-	-	-
		Software	✓	✓	✓	✓	✓	✓	-	-	-	-
	DMA	-	-	-	Bus-mastering	Bus-mastering	-	Bus-mastering	-	-	-	-
Analog output	Resolution	12 bits	12 bits	12 bits	12 bits	16 bits	-	-	12 bits	12 bits	16 bits	14 bits
	Number of channels	2 (PCI-1710 only)	2 (PCI-1710HG only)	2 (PCI-1711 only)	2 (PCI-1712 only)	2 (PCI-1716 only)	-	-	4 (Isolation)	4 (Waveform Output)	8	24
	On-board FIFO	-	-	-	32K samples	-	-	-	-	1K samples	-	-
	Output range (V)	0 ~ 5, 0 ~ 10	0 ~ 5, 0 ~ 10	0 ~ 5, 0 ~ 10	0 ~ 5, 0 ~ 10, ±5, ±10	0 ~ 5, 0 ~ 10, ±5, ±10	-	-	0 ~ 5, 0 ~ 10, ±5, ±10, 0 ~ 20 mA, 4 ~ 20 mA	0 ~ 5, 0 ~ 10, ±5, ±10, 0 ~ 20 mA, 4 ~ 20 mA	-10 ~ 10 0 ~ 20 mA, 4 ~ 20 mA	±10, 0 ~ 20 mA
	Throughput	38 KS/s Typical**	38 KS/s Typical**	38 KS/s Typical**	1 MS/s	200 KS/s Typical**	-	-	15 KS/s Typical**	10 MHz max. update rate	15 KS/s Typical**	15 KS/s Typical**
	DMA transfer	-	-	-	✓	-	-	-	-	Bus-mastering	-	-
Digital I/O	Input channels	16	16	16	16	16	-	-	-	16	16	-
	Output channels	16	16	16		-	-	-	-		-	-
Timer/counter	Number of channels	1	1	1	3	1	-	1	-	1	-	-
	Resolution	16-bit	16-bit	16-bit	16-bit	16-bit	-	8-bit	-	16-bit	-	-
	Time base	10 MHz	10 MHz	10 MHz	10 MHz	10 MHz	10 MHz	60 MHz	10 MHz	10 MHz	-	-
Isolation Voltage		-	-	-	-	-	2,500 V _{DC}	-	2,500 V _{DC}	-	-	-
Auto Calibration		-	-	-	✓	✓	-	✓	-	✓	✓	-
Board ID		-	-	-	-	✓	-	✓	-	✓	✓	✓
Dimensions (mm)		175 x 100	175 x 100	175 x 100	175 x 100	175 x 100	175 x 100	175 x 100	175 x 100	175 x 100	175 x 100	175 x 100
Connector		68-pin SCSI-II	68-pin SCSI-II	68-pin SCSI-II	68-pin SCSI-II	68-pin SCSI-II	DB-37	4 BNC	DB-37	68-pin SCSI-II	68-pin SCSI-II	DB-62
Windows 95/98/ME/2000/XP DLL Driver		✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Windows 95/98/ME/2000/XP Test Utility		✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
VC++, VB & Delphi Examples		✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Advantech ActiveDAQ		✓	✓	✓	✓	✓	✓	-	✓	✓	✓	✓
LabView I/O Drivers (Ver. 6i and 7.0)		✓	✓	✓	✓	✓	✓	-	✓	✓	✓	-
Advantech Data Acquisition Software		✓	✓	✓	✓	✓	✓	-	✓	✓	✓	✓

* Note: SS = Single DMA channel, Single A/D channel scan SM = Single DMA channel, Multiple A/D channel scan DM = Dual DMA channel, Multiple A/D channel scan

Selection Guide

ISA											
Multifunction								AI	AO		
PCL-711B/S	PCL-812PG	PCL-816/816-DA	PCL-818L/LS	PCL-818H	PCL-818HD	PCL-818HG	PCL-1800	PCL-813B	PCL-726	PCL-727	PCL-728
12 bits	12 bits	16 bits	12 bits	12 bits	12 bits	12 bits	12 bits	12 bits	-	-	-
8 SE	16 SE	16 Diff	16 SE/8 Diff	16 SE/8 Diff	16 SE/8 Diff	16 SE/8 Diff	16 SE/8 Diff	32 SE	-	-	-
-	-	-	-	-	1K samples	1K samples	1K samples	-	-	-	-
25 KS/s	30 KS/s	100 KS/s	40 KS/s	100 KS/s	100 KS/s	100 KS/s	330 KS/s	25 KS/s	-	-	-
-	-	✓	✓	✓	✓	✓	✓	-	-	-	-
-	-	0 ~ 10, 0 ~ 5, 0 ~ 2.5, 0 ~ 1.25	-	0 ~ 10, 0 ~ 5, 0 ~ 2.5, 0 ~ 1.25	0 ~ 10, 0 ~ 5, 0 ~ 2.5, 0 ~ 1.25	0 ~ 10, 0 ~ 1, 0 ~ 0.1, 0 ~ 0.01	0 ~ 10, 0 ~ 5, 0 ~ 2.5, 0 ~ 1.25	0 ~ 10, 0 ~ 5, 0 ~ 2.5, 0 ~ 1.25	-	-	-
±5, 2.5, 1.25, 0.625, 0.3125	±10, 5, 2.5, 1.25, 0.625, 0.3125	±10, 5, 2.5, 1.25	±10, 5, 2.5, 1.25, 0.625	±10, 5, 2.5, 1.25, 0.625	±10, 5, 2.5, 1.25, 0.625	±10, 5, 1, 0.5, 0.1, 0.05, 0.01, 0.005	±10, 5, 2.5, 1.25, 0.625	±10, 5, 2.5, 1.25, 0.625	-	-	-
✓	✓	✓	✓	Fixed polar	✓	✓	✓	✓	-	-	-
✓	✓	✓	✓	✓	✓	✓	✓	Software only	-	-	-
-	-	-	-	-	-	-	✓	-	-	-	-
-	-	-	-	-	-	-	✓	-	-	-	-
-	-	-	-	-	-	-	-	-	-	-	-
-	-	-	-	-	-	-	-	-	-	-	-
✓	✓	✓	✓	✓	✓	✓	✓	Software only	-	-	-
-	SS*	SM* (programmable)	SM*	SM*	SM*	SM*	DM* (programmable)	-	-	-	-
12 bits	12 bits	16 bits	12 bits	12 bits	12 bits	12 bits	12 bits	-	12 bits	12 bits	12 bits
1	2	2 per module	1	1	1	1	2	-	6	12	2 (Isolation)
-	-	-	-	-	-	-	-	-	-	-	-
0 ~ 5, 0 ~ 10	0 ~ 5, 0 ~ 10	16 bit module: ± 10	0 ~ 5, 0 ~ 10	0 ~ 5, 0 ~ 10	0 ~ 5, 0 ~ 10	0 ~ 5, 0 ~ 10	0 ~ 5, 0 ~ 10	-	0 ~ 5, 0 ~ 10, ±5, ±10, 4 ~ 20 mA	-0 ~ 5, 0 ~ 10, ±5, 4 ~ 20 mA	0 ~ 5, 0 ~ 10, ±5, ±10, 0 ~ 20 mA, 4 ~ 20 mA
30 KS/s Typical**	30 KS/s Typical**	16 bit module: 250 KS/s	30 KS/s Typical**	30 KS/s Typical**	30 KS/s Typical**	30 KS/s Typical**	30 KS/s Typical**	-	15 KS/s Typical**	15 KS/s Typical**	17 KS/s Typical**
-	-	✓	-	-	-	-	✓	-	-	-	-
16	16	16	16	16	16	16	16	-	16	16	-
16	16	16	16	16	16	16	16	-	16	16	-
-	1	-	1	1	1	1	1	-	-	-	-
-	16-bit	-	16-bit	16-bit	16-bit	16-bit	16-bit	-	-	-	-
2 MHz	2 MHz	10 MHz	10 MHz	10 MHz	10 MHz	10 MHz	10 MHz	-	-	-	-
-	-	-	-	-	-	-	-	500 V _{DC} min	-	-	500 V _{DC}
-	-	-	-	-	-	-	-	-	-	-	-
-	-	-	-	-	-	-	-	-	-	-	-
175 x 100	185 x 100	337 x 122	155 x 100	185 x 100	185 x 100	185 x 100	218 x 122	219 x 100	337 x 112	337 x 112	185 x 120
2 x 20-pin	2 x 20-pin	DB-37	DB-37	2 x 20-pin	DB-37	DB-37	DB-37	DB-37	4 x 20-pin	2 x 20-pin, DB-37	2 x DB-9
✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓

- 1 IPC & AWS
- 2 TPC
- 3 FPM
- 4 ATM
- 5 DA&C
- 6 cPCI
- 7 ADAM-3000
- 8 eConnectivity
- 9 ICOM
- 10 Software
- 11 UNO-2000/3000
- 12 VBOX
- 13 ADAM-4000
- 14 ADAM-5000
- 15 ADAM-6000
- 16 ADAM-8000

** Note: System-dependent

Data Acquisition and Control Cards

Bus			PCI												
Category			Non-Isolated DI/O			Isolated DI/O									
Model			PCI-1751	PCI-1753/ 1753E	PCI-1755	PCI-1730	PCI-1733	PCI-1734	PCI-1752	PCI-1754	PCI-1756	PCI-1760U	PCI-1761	PCI-1762	PCI-1750
TTL DI/O	Input channels		48	96	32	16	-	-	-	-	-	-	-	-	-
	Output channels					16	-	-	-	-	-	-	-	-	-
	Output channel	Sink current	24 mA @ 0.4 V	24 mA @ 0.44 V	48 mA @ 0.5 V	8 mA @ 0.5 V	-	-	-	-	-	-	-	-	-
		Source current	15 mA @ 2.4 V	24 mA @ 3.76 V	15 mA @ 2.4 V	0.4 mA @ 2.4 V									
Isolated DI/O	Input channel	Number of channels (Input type)	-	-	-	16 (Sink)	32 (Sink)	-	-	64 (Sink)	32 (Sink)	8 (Sink)	8 (Sink)	16 (Sink)	16 (Sink)
		Isolation voltage	-	-	-	2,500 V _{DC}	2,500 V _{DC}	-	-	2,500 V _{DC}	2,500 V _{DC}	2,500 V _{DC}	2,500 V _{DC}	2,500 V _{DC}	2,500 V _{DC}
		Input range	-	-	-	5 ~ 30 V _{DC}	5 ~ 30 V _{DC}	-	-	10 ~ 50 V _{DC}	10 ~ 50 V _{DC}	5 ~ 12 V _{DC}	10 ~ 50 V _{DC}	10 ~ 50 V _{DC}	5 ~ 50 V _{DC}
	Output channel	Number of channels (Output type)	-	-	-	16 (Sink)	-	32 (Sink)	64 (Sink)	-	32 (Sink)	8 X From C	4 X From A 4 X From C	16 X From C	16 (Sink)
		Isolation voltage	-	-	-	2,500 V _{DC}	-	2,500 V _{DC}	2,500 V _{DC}	-	2,500 V _{DC}	2,500 V _{DC}	2,500 V _{DC}	2,500 V _{DC}	2,500 V _{DC}
		Output range	-	-	-	5 ~ 40 V _{DC}	-	5 ~ 40 V _{DC}	5 ~ 40 V _{DC}	-	5 ~ 40 V _{DC}	120 V _{AC} @ 0.5 A 30 V _{DC} @ 1 A	250 V _{AC} @ 3 A 24 V _{DC} @ 3 A	120 V _{AC} @ 0.5 A 30 V _{DC} @ 1 A	5 ~ 40 V _{DC}
		Max. Sink current	-	-	-	200 mA	-	200 mA	200 mA	-	200 mA				200 mA
	Timer/ counter	Number of channels	3	-	3	-	-	-	-	-	-	Up CTR for DI 2 X PWM	-	-	1
		Resolution	16-bit	-	16-bit	-	-	-	-	-	-	16-bit (2,500 Isolation)	-	-	16-bit
		Time base	5 MHz	-	10 MHz	-	-	-	-	-	-	500 Hz for Up CTR	-	-	1 MHz
Advanced Function	Pattern Match		-	✓	✓	-	-	-	-	-	-	✓	-	-	-
	Change of State		-	✓	✓	-	-	-	-	-	-	✓	-	-	-
	Board ID		-	-	✓	✓	-	✓	✓	✓	✓	✓	✓	✓	-
	Channel-Freeze function		-	-	✓	✓	-	✓	✓	✓	✓	-	✓	✓	-
	Output status read back		✓	✓	-	✓	-	✓	✓	-	✓	✓	✓	✓	-
	Dry/Wet Contact		✓	✓	-	✓*	✓*	-	-	-	-	✓	-	-	-
Dimensions (mm)			175 x 100	175 x 100	175 x 100	175 x 100	175 x 100	175 x 100	175 x 100	175 x 100	175 x 100	175 x 100	175 x 100	175 x 100	175 x 100
Connectors			68-pin SCSI-II	100-pin SCSI-II	100-pin SCSI-II	1 x DB-37 4 x 20-pin	1 x DB-37	1 x DB-37	100-pin SCSI-II	100-pin SCSI-II	100-pin SCSI-II	1 X D-37	1 X D-37	1 X D-62	1 X D-37
Windows 95/98/ME/2000/XP DLL Driver			✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Windows 95/98/ME/2000/XP Test Utility			✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
VC++, VB & Delphi Examples			✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Advantech ActiveDAQ			✓	✓	-	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
LabView I/O Drivers (Ver.6i and 7.0)			✓	✓	-	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Advantech Data Acquisition Software			✓	✓	-	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓

* Dry/wet contact can be mixed at the same time within one group.

Selection Guide

ISA									PCI		ISA	
Non-Isolated DI/O				Isolated DI/O					Counter			
PCL-720+	PCL-722	PCL-724	PCL-731	PCL-725	PCL-730	PCL-733	PCL-734	PCL-735	PCI-1784	PCI-1780	PCL-833	PCL-836
32	144	24	48	-	16	-	-	-	-	8	-	16
32				-	16	-	-	-	-	8	-	16
24 mA @ 0.5 V	24 mA @ 0.5 V	24 mA @ 0.4 V	24 mA @ 0.4 V	-	8 mA @ 0.5 V	-	-	-	-	24 mA @ 0.5 V	-	8 mA @ 0.5 V
3 mA @ 2.4 V	15 mA @ 2.4 V	15 mA @ 2.4 V	15 mA @ 2.4 V	-	0.4 mA @ 2.4 V	-	-	-	-	15 mA @ 2.4 V	-	0.4 mA @ 2.4 V
-	-	-	-	8 (Sink)	16 (Sink)	32 (Sink)	-	-	4	-	5	-
-	-	-	-	1,500 V _{DC}	2,500 V _{DC}	2,500 V _{DC}	-	-	2,500 V _{DC}	-	2,500 V _{DC}	-
-	-	-	-	5 ~ 24 V _{DC}	5 ~ 24 V _{DC}	5 ~ 24 V _{DC}	-	-	5 ~ 30 V _{DC}	-	0 ~ 5 V _{DC} (Differential)	-
-	-	-	-	4 X Form A 4 X Form C	16 (Sink)	-	32 (Sink)	12 X Form C	4	-	-	-
-	-	-	-	1,000 V _{DC}	1,000 V _{DC}	-	1,000 V _{DC}	1,000 V _{DC}	2,500 V _{DC}	-	-	-
-	-	-	-	120 V _{AC} @ 0.5 A	5 ~ 40 V _{DC}	-	5 ~ 40 V _{DC}	0.6 A @ 100 V _{DC} 0.6 A @ 125 V _{DC}	0 ~ 5 V _{DC}	-	-	-
-	-	-	-	30 V _{DC} @ 1 A	200 mA	-	200 mA				-	-
3	-	-	-	-	-	-	-	-	4 (2,500 Isolation)	8 X CTR	3 (2,500 Isolation)	6 X CTR 3 X PWM
16-bit	-	-	-	-	-	-	-	-	32-bit	16-bit	24-bit	16-bit
1 MHz	-	-	-	-	-	-	-	-	Encoder: 2 MHz CTR: 8 MHz	20 MHz	Encoder: 1 MHz CTR: 2.4 MHz	10 MHz
-	-	-	-	-	-	-	-	-	✓	-	-	-
-	-	-	-	-	-	-	-	-	✓	-	-	-
-	-	-	-	-	-	-	-	-	✓	✓	-	-
-	-	-	-	-	-	-	-	-	✓	-	-	-
-	-	-	-	-	-	-	-	-	✓	-	-	-
-	-	-	-	-	-	-	-	-	-	-	-	-
185 x 100	334 x 100	125 x 100	185 x 100	147 x 95	185 x 100	185 x 100	185 x 100	155 x 100	175 x 100	175 x 100	185 x 100	185 x 100
5 X 20-pin	6 x 50-pin	1 x 50-pin 2 x 20-pin	2 x 50-pin	1 x DB-37	1 x DB-37 4 x 20-pin	1 x DB-37	1 x DB-37	1 x DB-37	1 x DB-37	68-pin SCSI-II	1 x DB-25	1 x DB-37 2 x 20-pin
✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
✓	✓	✓	✓	✓	✓	✓	✓	✓	-	-	-	-
✓	✓	✓	✓	✓	✓	✓	✓	✓	-	-	✓	✓
✓	✓	✓	✓	✓	✓	✓	✓	✓	-	✓	-	✓

1	IPPC & AWS
2	TPC
3	FPM
4	ATM
5	DA&C
6	cPCI
7	ADAM-3000
8	eConnectivity
9	ICOM
10	Software
11	UNO-2000/3000
12	VBOX
13	ADAM-4000
14	ADAM-5000
15	ADAM-6000
16	ADAM-8000

Industrial Communication

Industrial Communication Selection Guide	9-2
PCI Communication Card Series	9-3
PCI-1601/1602	2-port RS-422/485 PCI Communication Cards 9-4
PCI-1603	PCI bus Isolated Dual-port RS-232/Current-loop Communication Card 9-5
PCI-1610/1612/1612U	4-port RS-232/422/485 PCI Communication Cards 9-6
PCI-1620/1620U	8-port RS-232 PCI Communication Cards 9-7
PCI-1625	8-port Intelligent RS-232/422 PCI Communication Cards 9-8
PCI-1680U	2-port Isolated CAN Interface Universal PCI Communication Card 9-9
PCI-bus Low-profile Cards	
PCI-1602UP/1604UP	2-port Low-profile and Universal PCI Communication Cards 9-10
PCI-1610UP	4-port RS-232 Low-profile and Universal PCI Communication Card 9-11
PCI-1757UP	24-channel Digital Input/Output Card 9-12
ISA-bus Communication Cards	
PCL-740/741	1-port RS-232/422/485/Current-loop Serial Interface Cards 9-13
PCL-743B/745B	2-port RS-422/485 Interface Cards 9-14
PCL-743S/745S	2-port 745S Isolated RS-422/485 Interface Card 9-14
PCL-746+	4-port RS-232/422/485 Interface Cards 9-15
PCL-846	4-port RS-422/485 Interface Card 9-16
PCL-849/849L	4-port RS-232 Interface Card 9-17
PCL-858	8-port RS-232/422 Interface Card 9-18
PCL-841	2-port Isolated CAN-bus Interface Card 9-19
PC/104 Communication Cards	
PCM-3610	Isolated RS-232/422/485 Module 9-20
PCM-3612	Dual-port RS-422/485 Module 9-20
PCM-3614	4-port RS-422/485 High-Speed Module 9-20
PCM-3618	8-port RS-422/485 High-speed Module 9-21
PCM-3640/3641	4-port RS-232 High-speed Module 9-21
PCM-3680	2-port Isolated CAN Interface Module 9-21
PCM-3662	PC/104 plus Ethernet Module 9-22
PCM-3660	Jumperless Ethernet Module 9-22
Fieldbus Communication	
Fieldbus Communication Overview	9-23
AD-CIF50-PB	Profibus DP/FMS Master PCI Card 9-24
AD-CIF104-PB	Profibus DP/FMS Master PC/104 Module 9-25
AD-CIF104-DPS	Profibus DP Slave PC/104 Module 9-25
AD-CIF104P-PB	Profibus DP/FMS Master PC/104+ Module 9-25
AD-CIF50-DNM	DeviceNet Master PCI Card 9-26
AD-CIF104-DNM	DeviceNet Master PC/104 Module 9-27
AD-CIF104-DNS	DeviceNet Slave PC/104 Module 9-27
AD-CIF104P-DNM	DeviceNet Master PC/104+ Module 9-27
AD-CIF50-COM	CANopen Master PCI Card 9-28
AD-CIF104-COM	CANopen Master PC/104 Module 9-29
AD-CIF104-COS	CANopen Slave PC/104 Module 9-29
AD-CIF104P-COM	CANopen Master PC/104+ Module 9-29
AD-SyCon	Fieldbus System Configuration Software 9-30

Industrial Communication Cards Selection Guide

Form Factor	No. of Ports	Communication Interface Support					Protection		Model Name	Page
		Current Loop	RS-232	RS-422	RS-485	CAN	Surge	Isolation		
ISA	1	Yes	Yes	Yes	Yes				PCL-740	9-13
	2	Yes	Yes					2500 V _{DC}	PCL-741	9-13
	2			Yes	Yes				PCL-743B	7-14
	2			Yes	Yes		2500 V _{DC}		PCL-743S	9-14
	2			Yes	Yes		2500 V _{DC}	3000 V _{DC}	PCL-745S	9-14
	2			Yes	Yes			3000 V _{DC}	PCL-745B	9-14
	2					Yes		1000 V _{DC}	PCL-841	9-19
	4		Yes	Yes	Yes				PCL-746B	9-15
	4			Yes	Yes			1000 V _{DC}	PCL-846A-A	9-16
	4			Yes	Yes		2000 V _{DC}	1000 V _{DC}	PCL-846B-A	9-16
	4		Yes						PCL-849A-A	9-17
	4		Yes				3000 V _{DC}		PCL-849B-A	9-17
	4		Yes				3000 V _{DC}		PCL-849+-A	9-17
	4		Yes				2000 V _{DC}		PCL-849L-A	9-17
	8		Yes						PCL-858A-A	9-18
	8		Yes				3000 V _{DC}		PCL-858B-A	9-18
PCI	2			Yes	Yes				PCI-1601A-A	9-4
	2			Yes	Yes		2500 V _{DC}		PCI-1601B-A	9-4
	2			Yes	Yes			3000 V _{DC}	PCI-1602A-A	9-4
	2			Yes	Yes		2500 V _{DC}	3000 V _{DC}	PCI-1602B-A	9-4
	2	Yes	Yes					3000 V _{DC}	PCI-1603-A	9-5
	2							1000 V _{DC}	PCI-1680U-A	9-9
	4		Yes						PCI-1610A-A	9-6
	4		Yes				3000 V _{DC}		PCI-1610B-A	9-6
	4		Yes	Yes	Yes				PCI-1612A-A	9-6
	4		Yes	Yes	Yes		2500 V _{DC}		PCI-1612B-A	9-6
	4		Yes	Yes	Yes		2500 V _{DC}		PCI-1612U-A	9-6
	8		Yes						PCI-1620A-A	9-7
	8		Yes				3000 V _{DC}		PCI-1620B-A	9-7
	8		Yes				3000 V _{DC}		PCI-1620U-A	9-7
	8		Yes						PCI-1625	9-8
Universal Low Profile PCI	2		Yes				2500 V _{DC}	2500 V _{DC}	PCI-1602UP-A	9-10
	2			Yes	Yes		2500 V _{DC}		PCI-1604UP-A	9-10
	4		Yes				2500 V _{DC}		PCI-1610UP-A	9-11
CompactPCI	4		Yes	Yes	Yes		2500 V _{DC}		MIC-3612/3-A	
	4		Yes	Yes	Yes		2500 V _{DC}		MIC-3612/6-A	
	8		Yes						MIC-3620/3-A	
	8		Yes						MIC-3620/6-A	
PC/104	2		Yes	Yes	Yes			1000 V _{DC}	PCM-3610	9-20
	2			Yes	Yes				PCM-3612	9-20
	4			Yes	Yes		1000 V _{DC}		PCM-3614	9-20
	8			Yes	Yes		1000 V _{DC}		PCM-3618	9-21
	4		Yes						PCM-3640/3641	9-21
	2					Yes		1000 V _{DC}	PCM-3680	9-21

Form Factor	Fieldbus Communication Interface Support				Type		Model Name	Page
	Profibus DP	Profibus FMS	DeviceNet	CANopen	Master	Slave		
PCI	Yes	Yes			Yes		AD-CIF50-PB	9-24
			Yes		Yes		AD-CIF50-DNM	9-26
				Yes	Yes		AD-CIF50-COM	9-28
PC/104	Yes	Yes			Yes		AD-CIF104-PB	9-25
	Yes					Yes	AD-CIF104-DPS	9-25
			Yes		Yes		AD-CIF104-DNM	9-27
			Yes			Yes	AD-CIF104-DNS	9-27
				Yes	Yes		AD-CIF104-COM	9-29
				Yes		Yes	AD-CIF104-COS	9-29
PC/104-Plus	Yes	Yes			Yes		AD-CIF104P-PB	9-25
			Yes		Yes		AD-CIF104P-DNM	9-27
				Yes	Yes		AD-CIF104P-COM	9-29